# Atlas 200I A2 加速模块
# 23.0.x(23.0.3 及其系列版本)
# 驱动开发指南

**文档版本** 07
**发布日期** 2025-04-28



**华为技术有限公司**

**版权所有 © 华为技术有限公司 2025。 保留一切权利。**

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

**商标声明**


和其他华为商标均为华为技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

**注意**

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

# 华为技术有限公司

地址： 深圳市龙岗区坂田华为总部办公楼 邮编：518129

网址： https://www.huawei.com

客户服务邮箱： support@huawei.com

客户服务电话： 4008302118

# 安全声明

## 漏洞处理流程

华为公司对产品漏洞管理的规定以“漏洞处理流程”为准，该流程的详细内容请参见如下网址：
https://www.huawei.com/cn/psirt/vul-response-process
如企业客户须获取漏洞信息，请参见如下网址：
https://securitybulletin.huawei.com/enterprise/cn/security-advisory

 版权所有 © 华为技术有限公司 

# 目 录

**1 概述.....1**
1.1 开发场景.....1
1.2 基础概念.....1
1.3 系统架构.....5
**2 用户必读.....6**
**3 开发流程.....7**
**4 准备.....9**
4.1 下载软件包.....10
4.2 准备环境.....12
**5 设备上电并快速启动.....14**
5.1 操作流程.....14
5.2 烧写镜像.....15
5.3 上电和启动设备.....17
**6 制作和烧写启动镜像.....19**
6.1 SD 卡制作和启动系统镜像.....19
6.1.1 概述.....19
6.1.2 SD 卡制作和启动系统镜像（方法一）.....19
6.1.2.1 修改 dts 并编译 DTB.....19
6.1.2.2 （可选）获取内核 Image 文件.....24
6.1.2.3 重构驱动 run 包.....24
6.1.2.4 SD 卡制作系统镜像.....27
6.1.3 SD 卡制作和启动系统镜像（方法二）.....35
6.2 M.2 制作和启动系统镜像.....42
6.3 eMMC 制作和启动系统镜像.....52
**7 接口外设调测参考.....62**
7.1 SerDes（PCIe/SATA/USB/ETH）.....62
7.1.1 调测流程.....62
7.1.2 PCIe 接口调测参考.....63
7.1.3 SATA 接口调测参考.....73
7.1.4 USB 接口调测参考.....83
7.1.5 ETH 接口调测参考.....96

 版权所有 © 华为技术有限公司 

7.2 GPIO ..... 109
7.2.1 概述 ..... 109
7.2.2 寄存器描述 ..... 109
7.2.3 调测参考 ..... 115
7.2.4 Demo 参考 ..... 118
7.3 UART ..... 119
7.3.1 调测流程 ..... 119
7.3.2 寄存器描述 ..... 120
7.3.3 调测参考 ..... 124
7.3.4 Demo 参考 ..... 127
7.4 I2C ..... 129
7.4.1 概述 ..... 129
7.4.2 寄存器描述 ..... 129
7.4.3 调测参考 ..... 148
7.4.4 Demo 参考 ..... 150
7.5 PWM ..... 151
7.5.1 概述 ..... 151
7.5.2 寄存器描述 ..... 151
7.5.3 调测参考 ..... 154
7.5.4 Demo 参考 ..... 154
7.6 CAN ..... 155
7.6.1 调测流程 ..... 155
7.6.2 寄存器描述 ..... 156
7.6.3 调测参考 ..... 158
7.6.4 Demo 参考 ..... 161
7.7 MDIO ..... 162
7.7.1 概述 ..... 162
7.7.2 寄存器描述 ..... 162
7.7.3 调测参考 ..... 167
7.7.3.1 读操作 ..... 167
7.7.3.2 写操作 ..... 169
7.7.4 Demo 参考 ..... 170
7.8 音频 ..... 183
7.8.1 HI-MPI ..... 183
7.8.1.1 Demo 参考 1 ..... 183
7.8.1.2 Demo 参考 2 ..... 185
7.8.1.3 调测参考 ..... 185
7.9 视频 ..... 186
7.9.1 概述 ..... 186
7.9.2 HDMI ..... 186
7.9.2.1 Demo 参考 1 ..... 186
7.9.2.2 Demo 参考 2 ..... 189

 版权所有 © 华为技术有限公司 

7.9.2.3 调测参考 189
7.9.2.4 图形化桌面 189
7.9.3 MIPI-CSI 191
7.9.3.1 Demo 参考 1 191
7.9.3.2 Demo 参考 2 192
7.9.3.3 调测参考 211
7.9.4 MIPI-DSI 211
7.9.4.1 Demo 参考 211
7.9.4.2 调测参考 211
7.10 SPI 211
7.10.1 概述 211
7.10.2 寄存器描述 211
7.10.3 调测参考 215
7.10.4 demo 参考 217
7.11 I2S 219
7.11.1 概述 219
7.11.2 寄存器描述 220
7.11.3 调测参考 222
**8 二次开发场景 227**
8.1 安装工具链 227
8.2 启动流程介绍 228
8.2.1 概述 228
8.2.2 Hboot2 228
8.2.3 用户弹性配置介绍 228
8.2.4 内核介绍 228
8.2.5 DTB 介绍 229
8.2.6 文件系统介绍 231
8.3 源码编译 231
8.3.1 编译内核 231
8.3.1.1 编译并生效内核 Image 文件（Euler） 231
8.3.1.2 编译并生效内核 Image 文件（openEuler-SP1） 234
8.3.1.3 编译并生效内核 DTB 文件 237
8.3.2 编译并生效 userBaseConfig 文件 240
8.3.3 编译驱动 244
8.3.3.1 编译默认驱动 244
8.3.3.2 编译并生效新增用户驱动 245
8.4 Hboot2 源码编译 248
8.4.1 编译 Hboot2 248
8.4.2 生效 Hboot2 249
8.4.3 Demo 参考 250
8.5 驱动自动加载配置 253
8.6 重构驱动包 256

 版权所有 © 华为技术有限公司 

8.7 编辑用户自定义脚本 258
8.8 节能配置 260
**9 驱动与固件升级 264**
9.1 升级前必读 264
9.2 升级准备 265
9.3 升级驱动 265
9.3.1 通过 run 包升级 265
9.3.1.1 升级当前区 265
9.3.1.2 升级备分区 266
9.3.2 通过 tar.gz 包升级 269
9.4 升级固件 270
9.5 版本回退 271
**10 关键特性介绍 272**
10.1 主备分区介绍 272
10.2 安全启动介绍 275
10.3 安全关机介绍 276
10.4 休眠唤醒介绍 277
**11 产品化方案参考 280**
11.1 （可选）MAC 地址烧录 280
**12 设备检查 282**
12.1 查询基本信息 282
12.2 查询详细信息 283
12.3 查询产品类型 284
12.4 查询所有芯片健康状态 285
**13 工具介绍与使用 286**
13.1 系统管理工具介绍 286
13.1.1 npu-smi 工具 286
13.1.2 DCMI 工具 286
13.2 烧写工具介绍 286
13.3 其他工具介绍 287
13.3.1 GPIO 工具介绍 287
13.3.1.1 概述 287
13.3.1.2 使用指南 287
13.3.1.3 使用示例 287
13.3.1.3.1 获取 GPIO 帮助信息 287
13.3.1.3.2 查询 GPIO 管脚值 287
13.3.1.3.3 设置 GPIO 管脚值 288
13.3.1.3.4 查询 GPIO 管脚方向 288
13.3.1.3.5 设置 GPIO 管脚方向 289
13.3.2 vrd-tool 工具介绍 290

 版权所有 © 华为技术有限公司 

13.3.3 boot_tool 工具介绍 292
13.3.4 efuse-tool 工具介绍 296
**14 常用操作 298**
14.1 使用 PuTTY 登录设备（网口方式） 298
14.2 使用 PuTTY 登录设备（串口方式） 300
14.3 使用 WinSCP 传输文件 301
**15 FAQ 304**
15.1 使用 root 用户 SSH 登录服务器出现 Access Denied 错误 304
15.2 驱动升级过程中报错显示无法删除文件夹 305
15.3 emmc-head 工具不可用 305
15.4 更新网络源时签名不可用 306
15.5 制卡过程中出现“[ERROR] Can not get disk, please use fdisk -l to check available disk name!”的报错 308
15.6 运行程序超内存导致设备异常挂死，设备能 ping 通，但 SSH 无法远程连接 309
15.7 系统启动时在加载 PCIe 设备处卡死，无法正常启动 310
15.8 SD 卡作为系统启动介质时，启动过程中显示挂载分区失败 310
**A 附录 312**
A.1 用户信息 312
A.2 自定义配置 user_base_config.xml 文件参数说明 313
A.3 参数说明/常用命令 318
A.4 相关工具 319
A.5 组件分区介绍 322
A.6 定制文件系统 323
A.6.1 定制 OpenEuler 22.03 文件系统 323
A.6.2 定制 Ubuntu 22.04 文件系统 325
A.7 术语&缩略语 328

 版权所有 © 华为技术有限公司 

# 1 概述

1.1 开发场景
1.2 基础概念
1.3 系统架构

## 1.1 开发场景

### 设备启动

Atlas 200I A2 加速模块出厂自带固件，用户完成底板设计后，需要将驱动和系统安装至整机设备。

Atlas 200I A2 加速模块支持将驱动和系统烧录至SD卡、M.2或者eMMC，再从SD卡、M.2或者eMMC启动设备。

### 接口调测

在设备启动之后，Atlas 200I A2 加速模块的UART0为串口，出厂即调通，可直接连接串口登录设备。用户可根据底板接口设计修改对应的userBaseConfig文件和DTB文件，生效后，测试对应的接口，例如PCIe、SATA、USB、ETH和CAN等接口。

## 1.2 基础概念

### 算力档位

算力（也称哈希率）是运算处理能力的一个度量单位，表示处理器计算哈希函数输出的速度。TOPS为算力单位，1TOPS代表处理器每秒可以进行一万亿次操作。

Atlas 200I A2 加速模块支持调整算力档位为20TOPS和8TOPS，适用于不同算力场景，满足用户对于算力的不同需求。

npu-smi工具支持算力档位的查询和设置，详细操作请参考《**npu-smi 命令参考**》的“查询指定芯片的算力档位”和“设置指定芯片的算力档位”章节。

 版权所有 © 华为技术有限公司 

## SerDes 功能

Atlas 200I A2 加速模块提供了灵活的片上高速SerDes组网模块，可以根据不同产品的应用场景，实现GE、USB 3.0、PCIe和SATA的灵活组网。详细请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“SerDes接口”章节。用户可通过userBaseConfig灵活配置SerDes，详细操作请参见**8.3.2 编译并生效userBaseConfig文件**。

## USB Device

Atlas 200I A2 加速模块对外提供四个USB控制器接口，包含Type-C接口类型USB接口，适配USB 3.0（SuperSpeed）通信协议，可使用该Type-C接口实现虚拟网卡功能以及虚拟U盘功能。

## Hboot2

在Atlas 200I A2 加速模块的SFC Flash上预置了Hboot2程序，可根据BOOTSEL真值表选择启动介质，从启动介质中加载Kernel、DTB等程序。BOOTSEL真值表的详细信息请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“启动接口设计”章节。

## 文件系统

烧录工具支持单Root Filesystem和冗余Root Filesystem两种文件系统，可使用boot_tool工具查询文件系统启动的相关信息。例如boot_tool get image_info查询Root Filesystem所在的分区信息。

烧录工具的详细介绍请参见**13.2 烧写工具介绍**。

**图 1-1** 单 Root Filesystem

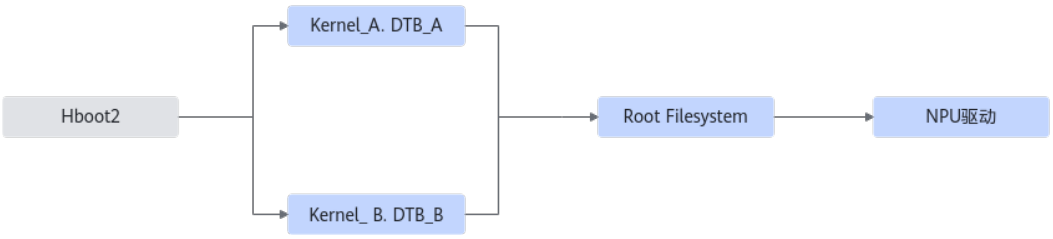
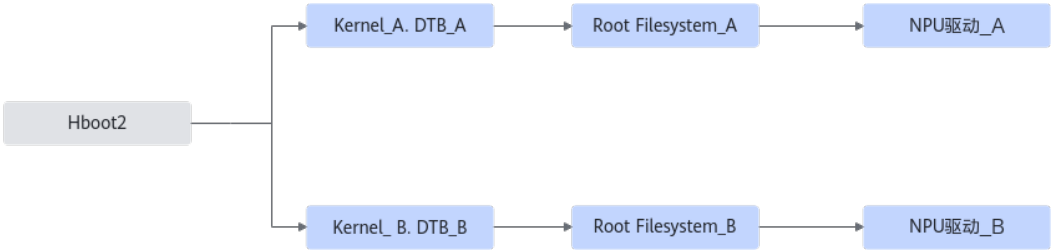


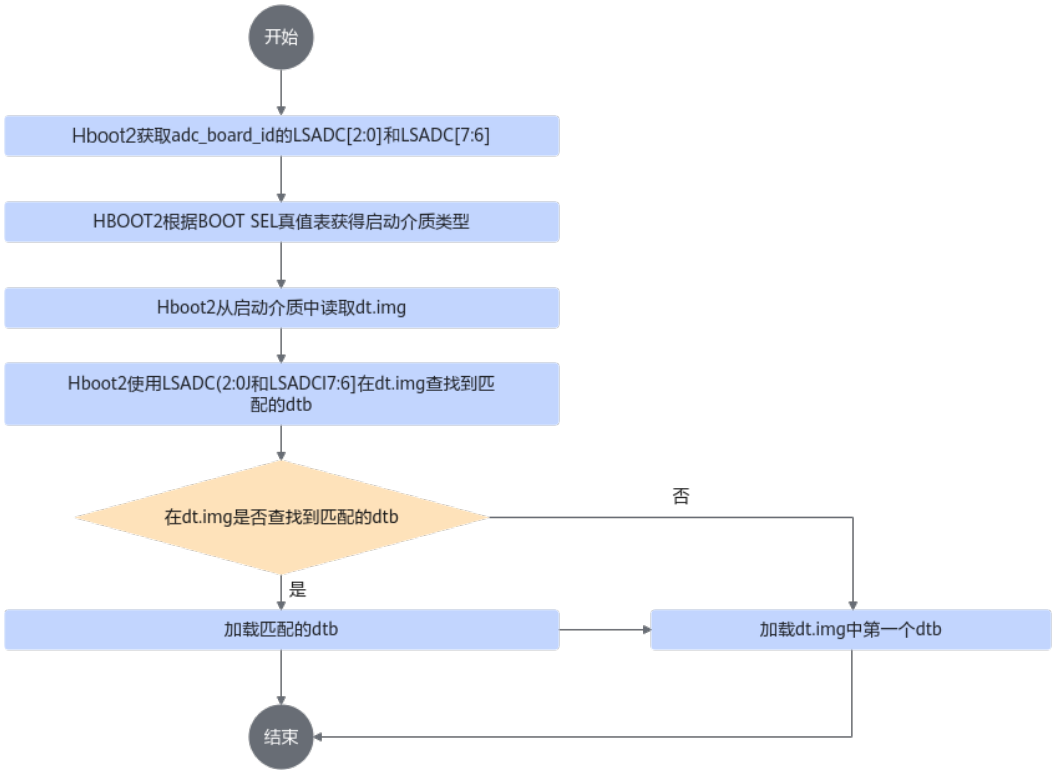

**图 1-2** 冗余 Root Filesystem




## DTB 文件

Atlas 200I A2 加速模块通过提供dt.img镜像集成不同的DTB，Hboot2根据Atlas 200I A2 加速模块的adc_board_id和底板的adc_board_id从dt.img中选择匹配的DTB并加载，当无匹配的DTB时会加载dt.img中的第一个DTB。

 版权所有 © 华为技术有限公司 




用户通过DTS自定义不同的DTB实现丰富的整机硬件接口使能，DTS源码提供了可供整机参考的DTS，详细介绍如下。

| dts Sample | dts域段作用介绍 | 整机示例 |
| --- | --- | --- |
| hi1910B-asic-M150-B50.dts | • M150：表示算力为20T的Atlas 200I A2 加速模块。<br>• B50：表示adc_board_id的LSADC[7:6]为50，LSADC[7:6]用于区分不同的底板，详细请参考《**Atlas 200I A2 加速模块 硬件开发指南**》。<br>• dts名字和dts内容对应的域段：hisi，boardid = <0x0 0x5 0x0 0x1 0x5 0x0>。 | Atlas 500 A2 智能小站 |
| hi1910B-asic-M100-B51.dts | • M100：表示算力为8T的Atlas 200I A2 加速模块。<br>• B51：表示adc_board_id的LSADC[7:6]为51，LSADC[7:6]用于区分不同的底板，详细请参考《**Atlas 200I A2 加速模块 硬件开发指南**》。<br>• dts名字和dts内容对应的域段：hisi，boardid = <0x0 0x5 0x1 0x1 0x0 0x0>。 | Atlas 200I DK A2 开发者套件 |


版权所有 © 华为技术有限公司

| dts Sample | dts域段作用介绍 | 整机示例 |
| --- | --- | --- |
| hi1910B-asic-M150-B51.dts | • M150：表示算力为20T的Atlas 200I A2 加速模块。<br>• B51：表示adc_board_id的LSADC[7:6]为51，LSADC[7:6]用于区分不同的底板，详细请参考《**Atlas 200I A2 加速模块 硬件开发指南**》。<br>• DTS名字和DTS内容对应的域段：hisi，boardid = <0x0 0x5 0x1 0x1 0x5 0x0>。 | 算力为20T的Atlas 200I A2 加速模块+Atlas 200I DK A2 开发者套件的底板 |
| hi1910B-asic-default.dts | Hboot2根据adc_board_id的LSADC[2:0]和LSADC[7:6]在dt.img中无法找到匹配的dtb时，会默认加载dtb中第一个DTB（hi1910B-asic-default.dtb）并启动，默认DTB中删除了PCIe配置。 | NA |

**说明**

源码包Ascend310B-source tar.gz中包含hi1910B-asic-M150-B50.dts、hi1910B-asic-M100-B51.dts、hi1910B-asic-M150-B51.dts和hi1910B-asic-default.dts，详细获取方式请参见**4.1 下载软件包**。

可通过以下两种方式自定义DTS：

- 新增自定义整机的DTS，比如整机底板的LSADC[7:6]为30，搭配20T算力的Atlas 200I A2 加速模块时，DTS可以为hi1910B-asic-M150-B30.dts，同时修改DTS中的boardid字段，如下加粗内容所示。

```
{
compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
hisi,boardid = <0x0 0x3 0x0 0x1 0x5 0x0>;
#address-cells = <0x2>;
#size-cells = <0x2>;
interrupt-parent = <0x1>;
model = "Hisilicon PhosphorHi1910B evb";
```

- 修改默认DTS（hi1910B-asic-default.dts），适配整机的硬件接口。

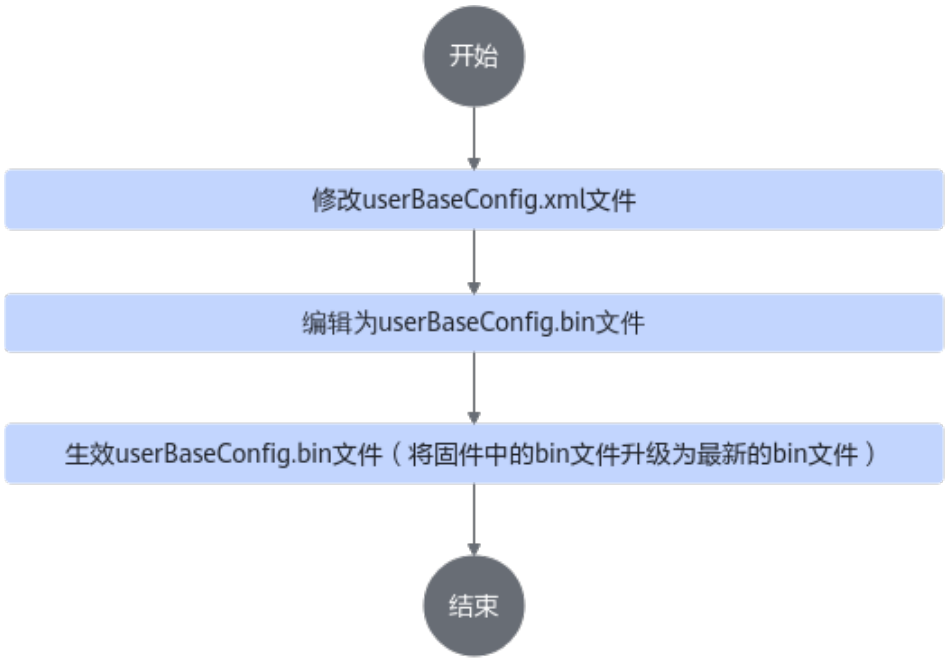
## userBaseConfig 文件

userBaseConfig文件为弹性配置，可通过userBaseConfig文件配置SerDes信息， 将SerDes复用为PCIe、SATA、USB或者GE接口。

Atlas 200I A2 加速模块出厂自带固件（userBaseConfig.bin文件），已默认配置Atlas 200I DK A2 开发者套件、Atlas 500 A2 智能小站的SerDes复用关系，可根据实际的底板硬件设计重新修改userBaseConfig文件进行适配。具体配置流程如**图1-3**所示。

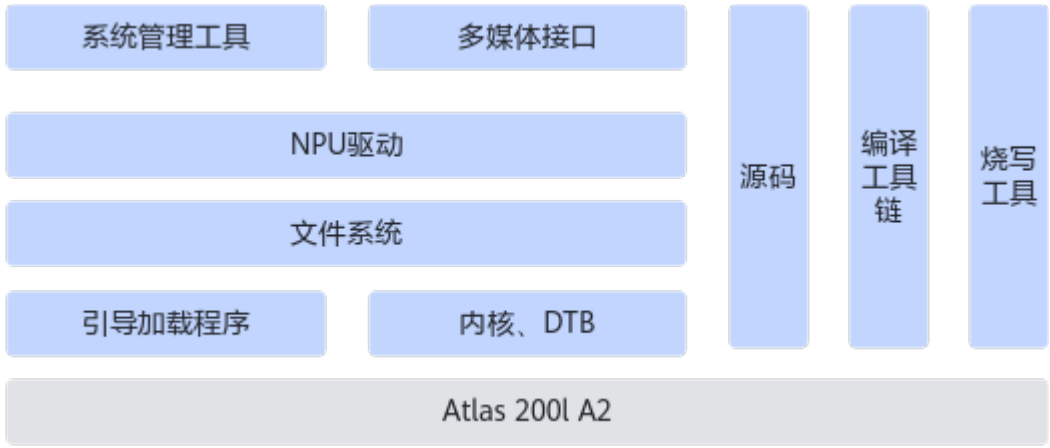

版权所有 © 华为技术有限公司

**图 1-3** 配置 userBaseConfig 文件




# 1.3 系统架构

驱动软件主要包括引导加载程序、内核、DTB、文件系统、NPU驱动、源码、编译工具链以及烧写工具。




- 引导加载程序：Bootloader程序，负责从启动介质中加载内核等程序。
- 内核、DTB：Linux内核。
- 文件系统：支持Ubuntu 22.04、openEuler 22.03 LTS的文件系统。
- NPU驱动：NPU内核态和用户态驱动。
- 源码：内核、DTB、NPU内核态驱动源码。
- 编译工具链：NPU内核态驱动编译工具链，用于编译Linux内核、DTB、NPU内核态驱动。
- 烧写工具：把内核、DTB、文件系统烧写到启动介质的工具。


版权所有 © 华为技术有限公司

# 2 用户必读

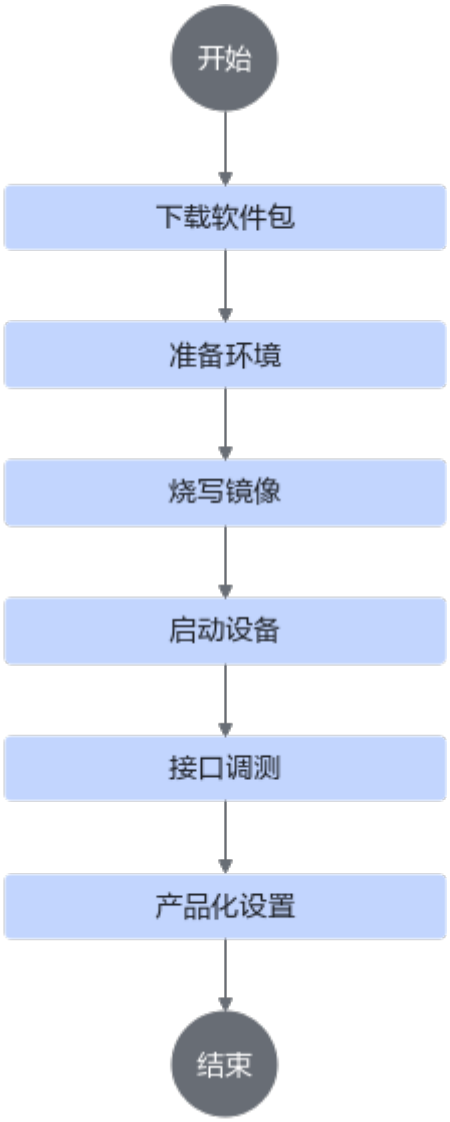
- Atlas 200I A2 加速模块的adc_board_id可通过算力区分不同的硬件类型。当算力为20T时，Atlas 200I A2 加速模块的adc_board_id为150；当算力为8T时，Atlas 200I A2 加速模块的adc_board_id为100。具体信息可参见**DTB文件**。
- 底板的adc_board_id可通过底板的实际硬件电路中的电阻取值获得，具体信息可参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“ADC BOARD Type配置”章节。设备启动成功且登录系统之后，可使用命令**npu-smi info -t board -i** *id*查询底板的Board ID（例如0x21为十六进制数值，转化为十进制数值为33），详细信息请参考《**npu-smi 命令参考**》的“查询详细信息”章节。
- 编译DTB时，需要提前配置dts文件，dts中boardid字段请务必配置为用户实际使用底板的adc_board_id与Atlas 200I A2 加速模块的adc_board_id的组合值，例如底板的adc_board_id为33，Atlas 200I A2 加速模块的adc_board_id为150，则boardid的取值为33150。设备启动成功且登录系统之后，可使用命令**npu-smi info -t board -i** *id* **-c** *chip_id*查询指定NPU芯片的整机Board ID（例如0x817e为十六进制数值，转化为十进制数值为33150），详细信息请参考《**npu-smi 命令参考**》的“查询指定芯片信息”章节。
- 设备快速启动编译DTB时，DTB中PCIe配置已删除，二次开发时若需要使用请参见**7.1 SerDes（PCIe/SATA/USB/ETH）**进行调测。


版权所有 © 华为技术有限公司

# 3 开发流程

Atlas 200I A2 加速模块驱动开发流程如下图所示。

**图 3-1** 开发流程





版权所有 © 华为技术有限公司

**表 3-1** 阶段流程说明

| 阶段流程 | 说明 |
| --- | --- |
| 下载软件包 | 下载开发流程中所需要使用的软件包，具体请参见**4.1 下载软件包**。 |
| 准备环境 | 根据不同开发场景准备对应的环境，具体请参见**4.2 准备环境**。 |
| 烧写镜像 | 为帮助用户快速启动设备，Atlas 200I A2 加速模块提供SD卡启动镜像文件，通过下载启动镜像直接烧写即可，具体请参见**5.2 烧写镜像**。如用户需要自定义SD卡启动镜像文件或从eMMC启动，具体请参见**6 制作和烧写启动镜像**。 |
| 启动设备 | 进入Atlas 200I A2 加速模块系统。具体请参见**5.3 上电和启动设备**。 |
| 接口调测 | 调测外设接口，使外设接口能正常通讯，具体请参见**7 接口外设调测参考**。 |
| 产品化设置 | 定制自己的MAC地址，具体请参见**11.1 （可选）MAC地址烧录**。 |


版权所有 © 华为技术有限公司

# 4 准备

4.1 下载软件包
4.2 准备环境

 版权所有 © 华为技术有限公司 

# 4.1 下载软件包

## 软件包信息

**表 4-1** 软件包使用说明

| 软件包 | 包名称 | 说明 |
| --- | --- | --- |
| 工具包<br>(源码、编译工具链、烧写等工具) | Ascend-hdk-310b-sdk-soc_*<version>*.zip | 解压后获取如下软件包：<br>● 工具链安装包**toolchain.tar.gz**<br>● 源码包**Ascend310B-source.tar.gz**<br>● 制卡脚本工具包**sdtool.tar.gz**<br>● vrd升级工具包**vrdtool.tar.gz**<br>● efuse烧写工具包**efuse-tool.tar.gz**<br>● emmc镜像制作工具包**emmc_burn_tool.tar.gz**<br>● hdmi验证工具包**hdmi_sample.tar.gz**<br>● audio验证工具包**audio_sample.tar.gz**<br>● hdm软件包**A500-A2-hdm_*<version>*.zip**(解压后获取**A500-A2-hdm_*<version>*.tar.gz**)<br>*<version>*表示NPU版本号，具体请根据实际情况进行替换。 |
| HBoot2源码包 | Ascend-hdk-310b-hboot2-sdk-soc_*<version>*.zip | 开源包。<br>解压后获取如下软件包：<br>Ascend310B-hboot2-source.tar.gz<br>*<version>*表示NPU版本号，具体请根据实际情况进行替换。 |


版权所有 © 华为技术有限公司

| 软件包 | 包名称 | 说明 |
| --- | --- | --- |
| 恢复出厂镜像包 | Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.zip | 解压后获取**Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz**，**Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz.cms和Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz.crl。**<br>*<version>*表示NPU版本号，具体请根据实际情况进行替换。 |
| | Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.zip | 解压后获取**Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz**<br>若有软实时OS需求，可使用此驱动包。<br>*<version>*表示NPU版本号，具体请根据实际情况进行替换。 |
| 驱动包 | Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run | *<version>*表示NPU版本号，具体请根据实际情况进行替换。 |
| | Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run | 若有软实时OS需求，可使用此驱动包。<br>*<version>*表示NPU版本号，具体请根据实际情况进行替换。 |
| 固件包 | Ascend-hdk-310b-npu-firmware-soc_*x.x.x.x.X*.run | *x.x.x.x.X*表示NPU固件版本号，具体请根据实际情况进行替换。 |
| 注：具体操作步骤中软件包名称请以客户实际获取的软件包为准。 | | |

## 获取软件包

**步骤1** 登录**Atlas 200I A2 软件**，进入产品包下载。

**步骤2** 单击软件包后面的 下载 ，获取所需软件包。

下载本软件即表示您同意**社区华为软件许可协议**的条款和条件。

**----结束**

## 软件数字签名验证

为了防止软件包在传递过程或存储期间被恶意篡改，下载软件包时需下载对应的数字签名文件用于完整性验证。


版权所有 © 华为技术有限公司

在软件包下载之后，请参考《 OpenPGP签名验证指南 》，对从Support网站下载的软件包进行PGP数字签名校验。如果校验失败，请不要使用该软件包，先联系华为技术支持工程师解决。

使用软件包安装/升级之前，也需要按上述过程先验证软件包的数字签名，确保软件包未被篡改。

运营商客户请访问：**https://support.huawei.com/carrier/digitalSignatureAction**

企业客户请访问：**https://support.huawei.com/enterprise/zh/tool/pgp-verify-TL1000000054**

# 4.2 准备环境

开发Atlas 200I A2 加速模块必须的基础硬件，如下表所示。

| 操作 | 硬件 | 说明 |
| --- | --- | --- |
| 设备上电并快速启动 | Atlas 200I A2 加速模块整机 | Atlas 200I A2 加速模块和用户自主设计的底板。 |
| | Micro SD卡 | • 推荐使用Micro SD卡，容量推荐不小于32G（class10）。<br>• 烧录镜像时会格式化SD卡，建议准备一张Atlas 200I A2 加速模块专用的SD卡。 |
| | Micro SD卡的USB读卡器 | USB接口的读卡器。 |
| | Linux服务器 | 带网口的Linux服务器，可运行的即可，无其他配置要求。 |
| 制作和烧写启动镜像 | Micro SD卡 | 用于SD卡制作和启动系统镜像。<br>• 推荐使用Micro SD卡，容量推荐不小于32G（class10）。<br>• 烧录镜像时会格式化SD卡，建议准备一张Atlas 200I A2 加速模块专用的SD卡。 |
| | Micro SD卡的USB读卡器 | 用于SD卡制作和启动系统镜像。<br>USB接口的读卡器。 |
| | 串口线 | 连接PC和Atlas 200I A2 加速模块底板的串口。 |
| | Atlas 200I A2 加速模块整机 | • Atlas 200I A2 加速模块和用户自主设计的底板，底板有eMMC。<br>• Atlas 200I A2 加速模块金手指“S72”管脚已开发为update按键。 |
| | U盘 | 用于eMMC卡制作和启动系统镜像。<br>容量不小于16G 。 |
| | Linux服务器 | 服务器配置要求如**Linux服务器配置要求**所示。 |


版权所有 © 华为技术有限公司

| 操作 | 硬件 | 说明 |
| --- | --- | --- |
| 接口调测/其他二次开发操作 | 串口线 | 连接PC和Atlas 200I A2 加速模块底板的串口。 |
| | Linux服务器 | 服务器配置要求如**Linux服务器配置要求**所示。 |
| 驱动与固件升级/产品化方案参考 | Atlas 200I A2 加速模块整机 | Atlas 200I A2 加速模块和用户自己设计的底板。 |

## Linux 服务器配置要求

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装如下依赖。

  Python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake、rpm2cpio、qemu-user-static、binfmt-support、gcc-aarch64-linux-gnu、g++-aarch64-linux-gnu、dosfstools、parted、kpartx。

  **说明**

  - 所有的依赖必须用root用户进行安装。
  - 用户可通过如下命令安装上述依赖。

    **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake rpm2cpio qemu-user-static binfmt-support gcc-aarch64-linux-gnu g++-aarch64-linux-gnu dosfstools parted kpartx**

    Ubuntu 22.04系统：“gcc-aarch64-linux-gnu”与“g++-aarch64-linux-gnu”版本要求为11.3.0，其他依赖软件包无版本要求。默认安装的gcc版本为11.3.0。
- 安装交叉编译器，参见**8.1 安装工具链**。
- 建议在制卡环境上预留足够的磁盘空间用于软件镜像制作。
  - 仅安装驱动包，预留空间为：（驱动包大小 + 镜像包大小 + 用户自定义软件包大小）x 2。
  - 安装驱动包和离线推理引擎包，预留空间为：（驱动包大小 + 镜像包大小 + 用户自定义软件包大小 + 离线推理引擎包大小）x 2。


版权所有 © 华为技术有限公司

# 5 设备上电并快速启动

5.1 操作流程
5.2 烧写镜像
5.3 上电和启动设备

## 5.1 操作流程

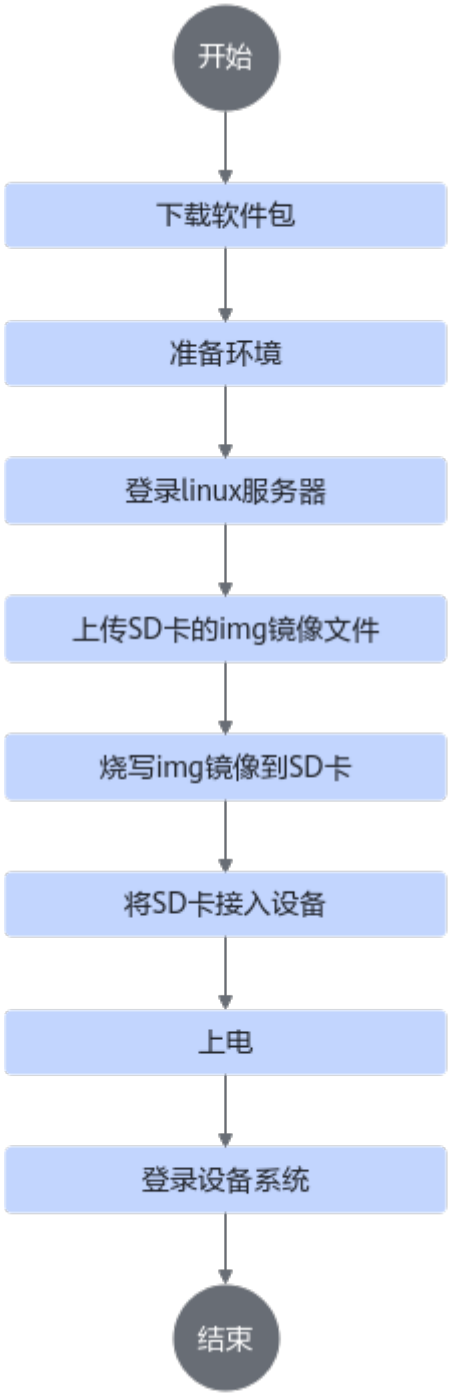
设备快速启动仅支持SD卡启动，如果用户需要从eMMC或M.2启动Atlas 200I A2 加速模块，请参见**6 制作和烧写启动镜像**。

本章节介绍Atlas 200I A2 加速模块的SD卡启动镜像制作以及设备启动的流程。为了帮助用户更高效完成启动镜像制作，建议本章节直接下载SD卡启动镜像文件，如用户需要自定义启动镜像文件，请参见**6 制作和烧写启动镜像**章节制作。

设备快速启动仅支持SD卡启动，具体流程如**图5-1**所示。


版权所有 © 华为技术有限公司

**图 5-1** 操作流程




# 5.2 烧写镜像

## 前提条件

已获取**recoverfs-<os-type>-SD.img**镜像包。

*<os-type>*表示目标镜像的操作系统类型。目前支持Ubuntu 22.04和openEuler 22.03。该镜像只包含基础驱动包，未包含推理软件包。


版权所有 © 华为技术有限公司

## 操作步骤

**步骤1** 将SD卡插入USB读卡器（推荐使用32G (class10)及以上）。

**步骤2** 将USB读卡器插入Linux服务器。

**步骤3** 登录Linux服务器。

**步骤4** 执行如下命令，切换至root用户。

**su - root**

**步骤5** 使用WinSCP工具或其他工具，将recoverfs-*<os-type>*-SD.img镜像包上传至root用户属组目录下，例如“/home/ascend/mksd”。工具使用方法请参见14.3 使用WinSCP传输文件。

**步骤6** 执行如下命令，进入recoverfs-*<os-type>*-SD.img镜像包的上传目录，例如“/home/ascend/mksd”。

**cd** */home/ascend/mksd*

**步骤7** 执行如下命令，查找SD卡所在的USB设备名称，例如“/dev/sdc”。

**fdisk -l**

显示类似信息如下：

```
Disk /dev/sdc: 28.33 GiB, 30416044032 bytes, 59406336 sectors
Disk model: UHSII uSD Reader
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 57C9FC21-66A9-4474-8EEB-DD67845D38CB
```

其中Disk model: UHSII uSD Reader为SD卡所在的USB设备名称，不同的USB设备显示的名称不一样，此处以“UHSII uSD Reader”为例。

**步骤8** 执行如下命令，烧写镜像到SD卡。

**dd if=recoverfs-<os-type>-SD.img of=***/dev/sdc* **status=progress**

出现如下类似信息则代表烧写成功，此处仅为示例，具体烧写花费时间和烧写速率以实际情况为准。

```
4751360+0 records in
4751360+0 records out
2432696320 bytes (2.4 GB, 2.3 GiB) copied, 45.217 s, 53.8 MB/s
```

**说明**

若出现写入数据量不等于4751360，则代表烧写失败，建议重新执行**dd if=recoverfs-<os-type>-SD.img of=**/dev/sdc **status=progress**烧写镜像。

**步骤9** 烧写镜像后会出现1.5G的golden分区，可以通过**mount**命令挂载后查看分区内容。

**mount** */dev/sdc1* **/mnt**

**ll /mnt**

**说明**

查看完文件请执行**umount /mnt**，防止文件损坏。

 版权所有 © 华为技术有限公司 

**步骤10** （可选）为避免出现内存不足导致设备OOM异常挂死问题，请参见**15.6 运行程序超内存导致设备异常挂死，设备能ping通，但SSH无法远程连接**进行预防处理。

**----结束**

# 5.3 上电和启动设备

## 背景信息

Atlas 200I A2 加速模块的UART0为串口，出厂即调通，可直接连接串口登录设备。

## 操作步骤

**步骤1** 准备串口线，连接PC和Atlas 200I A2 加速模块底板串口。

**步骤2** SD卡插入Atlas 200I A2 加速模块底板的SD卡槽。

**步骤3** 连接电源线，上电Atlas 200I A2 加速模块。

若系统启动失败，请参见**15.7 系统启动时在加载PCIe设备处卡死，无法正常启动**或**15.8 SD卡作为系统启动介质时，启动过程中显示挂载分区失败**解决。

**步骤4** 登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

**说明**

- Atlas 200I A2 加速模块（使用非Atlas 200I DK A2 开发者套件底板）USB接口登录的默认IP为192.168.1.2，Atlas 200I DK A2 开发者套件USB接口登录的默认IP为192.168.0.2。
- 若Atlas 200I A2 加速模块上电后网络可以ping通，但无法通过SSH登录，请参见**15.1 使用root用户SSH登录服务器出现Access Denied错误**解决。

**步骤5** （可选）关闭apt自动软件更新功能。若安装的OS为Ubuntu 22.04，由于apt自动软件更新功能会消耗大量资源，建议关闭apt自动软件更新功能。

1. 执行如下命令，切换至root用户。

   **su - root**

2. 执行以下命令，打开20auto-upgrades文件。

   **vim /etc/apt/apt.conf.d/20auto-upgrades**

   将下列加粗字体参数的值修改为0。

   ```
   APT::Periodic::Update-Package-Lists "0";
   APT::Periodic::Unattended-Upgrade "0";
   APT::Periodic::Download-Upgradeable-Packages "0";
   APT::Periodic::AutocleanInterval "0";
   ```

3. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

**步骤6** 执行如下命令，查询设备健康状态。

**npu-smi info -t health -i 0**

```
NPU ID                  : 0
Chip Count              : 1

Chip ID                 : 0
Health                  : OK
```


版权所有 © 华为技术有限公司

Health显示OK，表示正常。

**----结束**

 版权所有 © 华为技术有限公司 

# 6 制作和烧写启动镜像

Atlas 200I AI加速模块作为主/协处理器时，可通过SD卡和eMMC方式制作启动镜像包，容器部署场景请参见《**Atlas 200I A2 加速模块 23.0.0 昇腾软件安装指南**》。

6.1 SD卡制作和启动系统镜像

6.2 M.2制作和启动系统镜像

6.3 eMMC制作和启动系统镜像

## 6.1 SD 卡制作和启动系统镜像

### 6.1.1 概述

Atlas 200I A2 加速模块提供两种方法通过SD卡制作和启动系统镜像。

- 方法一：将驱动和系统直接烧写至SD卡，在将SD卡插入Atlas 200I A2 加速模块整机，SD卡作为启动盘将Atlas 200I A2 加速模块整机启动。
- 方法二：将驱动和系统制作成启动镜像（该镜像默认使能了主备分区，并开启了主备同步功能），在将启动镜像烧写至SD卡，然后将SD卡插入Atlas 200I A2 加速模块整机，SD卡作为启动盘将Atlas 200I A2 加速模块整机启动。

### 6.1.2 SD 卡制作和启动系统镜像（方法一）

#### 6.1.2.1 修改 dts 并编译 DTB

用户需要根据自己的底板适配自己的dt.img文件。Hboot2根据Atlas 200I A2 加速模块的adc_board_id和底板的adc_board_id在dt.img中选择匹配的DTB加载，当无匹配的DTB时会加载默认DTB文件。本节讲述了默认支持最小集启动的DTB文件的步骤。

#### 操作步骤

制作default dts和制作adc_board_id的LSADC[7:6]设置为实际底板的值的dts。

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

 版权所有 © 华为技术有限公司 

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“**Ascend310B-source.tar.gz**”上传至Linux系统root用户属组目录，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“**Ascend310B-source.tar.gz**”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“hi1910BL”目录。

**cd Ascend310B-source/dtb/dts/hi1910b/hi1910BL/**

**步骤7** 默认加载最小集hi1910B-default.dts。

1. 查看boardid字段值和pcie相关dtsi，确认与如下所示回显信息加粗字体保持一致。

```
/ {
        compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
        hisi,boardid = <0x0 0x0 0x0 0x0>;
        #address-cells = <0x2>;
        #size-cells = <0x2>;
        interrupt-parent = <0x1>;
        model = "Hisilicon PhosphorHi1910B evb";

        /include/ "base/hi1910B-asic-kernel-rc.dtsi"
        /include/ "base/hi1910B-commmon-smmu.dtsi"
        /include/ "base/hi1910B-mpam.dtsi"
        /include/ "base/hi1910B-lp-pm.dtsi"
        /include/ "base/hi1910B-lp-devmng.dtsi"
        /include/ "base/hi1910B-mbigen-ao.dtsi"
        /include/ "base/hi1910B-gpio.dtsi"
        /include/ "base/hi1910B-i2c.dtsi"
        /include/ "base/hi1910B-spi.dtsi"
        // /include/ "base/hi1910B-bindcore.dtsi"
        // /include/ "base/hi1910B-can.dtsi"
        /include/ "base/hi1910B-emmc.dtsi"
        /include/ "base/hi1910B-ipcdrv.dtsi"
        /include/ "base/hi1910B-ts-drv.dtsi"
        /include/ "base/hi1910B-tsdrv.dtsi"
        /include/ "base/hi1910B-network-M100-B51.dtsi"
        /include/ "base/hi1910B-hidvpp.dtsi"
        /include/ "base/hi1910B-hdmi.dtsi"
        /include/ "base/hi1910B-audio.dtsi"
        /include/ "base/hi1910B-mipitx.dtsi"
        /include/ "base/hi1910B-vdp.dtsi"
        /include/ "base/hi1910B-isp.dtsi"
        /include/ "base/hi1910B-hilink.dtsi"
        /include/ "base/hi1910B-mntn-milanr3.dtsi"
        /include/ "base/hi1910B-gpu.dtsi"
        /include/ "base/hi1910B-ctl.dtsi"
        /include/ "base/hi1910B-itrustee.dtsi"
        /include/ "base/hi1910B-sata.dtsi"
        // /include/ "product/hi1910B-pcie-rc-M100-B51.dtsi"
        /include/ "product/hi1910B-pinctrl-M100-B51.dtsi"
        /include/ "product/hi1910B-hiusbc-B51.dtsi"
        /include/ "product/hi1910B-usb-B51.dtsi"

        poweroff {
                compatible = "hisi,power_off";
                reg = <0x0 0xc4060000 0x0 0x10000>;
                pinctrl-names = "default", "idle";
                pinctrl-0 = <&poweroff_pmx_func>;
```

 版权所有 © 华为技术有限公司 

```
            pinctrl-1 = <&poweroff_pmx_idle>;
        };
```

2. 执行如下命令，进入“base”目录。

   **cd base**

3. 执行如下命令拷贝hi1910B-emmc.dtsi另存为hi1910B-emmc-default.dtsi。

   **cp hi1910B-emmc.dtsi hi1910B-emmc-default.dtsi**

4. 执行**vim hi1910B-emmc-default.dtsi**命令，按如下回显信息注释掉加粗字体的两行代码，并增加non-removable代码。

```
sdhci1: sdhci1@82010000 {  // sd
        compatible = "hisilicon,sdhci_sd";
        reg = <0x0 0x82010000 0x0 0x10000>;
        interrupt-parent = <&mbigen_sd>;
        interrupts = <917 4>;
        bus-width = <4>;
        host_id = <0>;
        no-sdio;
        no-mmc;
        max-frequency =<200000000>;
        clock-frequency = <1200000000>;
        non-removable;
        // cd-gpio = <&gpio3 8 0>;
       // cd-inverted;
        wp-inverted; /*sd卡需要配置这个 否则会被设置为只读*/
        timing_0_cfg = <4000000 0x6 0x6 0x0 0x0 4000000 0 0x23 0x0 0x44 0x42 0x1 0x1 0x2 0x8>;
        timing_1_cfg = <200000000 0x7 0x6 0xf 0x0 200000000 1 0x23 0x1 0x44 0x44 0x2 0x0 0x0
0x20>;
        timing_2_cfg = <1200000000 0x7 0x6 0x5 0x0 200000000 1 0x23 0x1 0x44 0x44 0x2 0x0 0x0
0x20>;:
        timing_3_cfg = <1200000000 0x7 0x6 0x5 0x0 200000000 1 0x23 0x1 0x44 0x44 0x2 0x0 0x0
0x20>;
        timing_4_cfg = <1200000000 0x7 0x6 0x5 0x0 200000000 1 0x23 0x1 0x44 0x44 0x2 0x0 0x0
0x20>;
        timing_5_cfg = <1200000000 0x6 0x6 0x5 0x1 200000000 0 0x25 0x4 0x48 0x42 0x4 0x0 0x0
0x20>;
```

5. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

6. 执行如下命令，进入“hi1910BL”目录。

   **cd** */opt/***Ascend310B-source/dtb/dts/hi1910b/hi1910BL/**

7. 执行**vim hi1910B-default.dts**命令，按如下加粗字体回显信息注释掉“/include/ "base/hi1910B-emmc.dtsi"”代码，并增加“/include/ "base/hi1910B-emmc-default.dtsi"”代码。

```
/ {
        compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
        hisi,boardid = <0x0 0x0 0x0 0x0>;
        #address-cells = <0x2>;
        #size-cells = <0x2>;
        interrupt-parent = <0x1>;
        model = "Hisilicon PhosphorHi1910B evb";

        /include/ "base/hi1910B-asic-kernel-rc.dtsi"
        /include/ "base/hi1910B-commmon-smmu.dtsi"
        /include/ "base/hi1910B-mpam.dtsi"
        /include/ "base/hi1910B-lp-pm.dtsi"
        /include/ "base/hi1910B-lp-devmng.dtsi"
        /include/ "base/hi1910B-mbigen-ao.dtsi"
        /include/ "base/hi1910B-gpio.dtsi"
        /include/ "base/hi1910B-i2c.dtsi"
        /include/ "base/hi1910B-spi.dtsi"
        // /include/ "base/hi1910B-bindcore.dtsi"
        // /include/ "base/hi1910B-can.dtsi"
        // /include/ "base/hi1910B-emmc.dtsi"
```

 版权所有 © 华为技术有限公司 

```
    /include/ "base/hi1910B-emmc-default.dtsi"
    /include/ "base/hi1910B-ipcdrv.dtsi"
    /include/ "base/hi1910B-ts-drv.dtsi"
    /include/ "base/hi1910B-tsdrv.dtsi"
    /include/ "base/hi1910B-network-M100-B51.dtsi"
    /include/ "base/hi1910B-hidvpp.dtsi"
    /include/ "base/hi1910B-hdmi.dtsi"
    /include/ "base/hi1910B-audio.dtsi"
    /include/ "base/hi1910B-mipitx.dtsi"
    /include/ "base/hi1910B-vdp.dtsi"
    /include/ "base/hi1910B-isp.dtsi"
    /include/ "base/hi1910B-hilink.dtsi"
    /include/ "base/hi1910B-mntn-milanr3.dtsi"
    /include/ "base/hi1910B-gpu.dtsi"
    /include/ "base/hi1910B-ctl.dtsi"
    /include/ "base/hi1910B-itrustee.dtsi"
    /include/ "base/hi1910B-sata.dtsi"
    ///include/ "product/hi1910B-pcie-rc-M100-B51.dtsi"
    /include/ "product/hi1910B-pinctrl-M100-B51.dtsi"
    /include/ "product/hi1910B-hiusbc-B51.dtsi"
    /include/ "product/hi1910B-usb-B51.dtsi"
```

8. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

**步骤8** 添加适配产品boardid的dts文件。

**说明**

系统正常启动后，用户可根据需求自行修改适配产品boardid的dts文件。

以boardid是33150为例，添加适配产品boardid的dts文件。

1. 执行如下命令拷贝hi1910B-default.dts另存为hi1910B-asic-M150-B33.dts。

   **cp hi1910B-default.dts hi1910B-asic-M150-B33.dts**

2. 执行**vim hi1910B-asic-M150-B33.dts**命令，修改boardid字段值，如下回显信息的加粗字体所示。

```
/ {
    compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
    hisi,boardid = <0x0 0x3 0x3 0x1 0x5 0x0>;
    #address-cells = <0x2>;
    #size-cells = <0x2>;
    interrupt-parent = <0x1>;
    model = "Hisilicon PhosphorHi1910B evb";

    /include/ "base/hi1910B-asic-kernel-rc.dtsi"
    /include/ "base/hi1910B-commmon-smmu.dtsi"
    /include/ "base/hi1910B-mpam.dtsi"
    /include/ "base/hi1910B-lp-pm.dtsi"
    /include/ "base/hi1910B-lp-devmng.dtsi"
    /include/ "base/hi1910B-mbigen-ao.dtsi"
    /include/ "base/hi1910B-gpio.dtsi"
    /include/ "base/hi1910B-i2c.dtsi"
    /include/ "base/hi1910B-spi.dtsi"
    // /include/ "base/hi1910B-bindcore.dtsi"
    // /include/ "base/hi1910B-can.dtsi"
    // /include/ "base/hi1910B-emmc.dtsi"
    /include/ "base/hi1910B-emmc-default.dtsi"
    /include/ "base/hi1910B-ipcdrv.dtsi"
    /include/ "base/hi1910B-ts-drv.dtsi"
    /include/ "base/hi1910B-tsdrv.dtsi"
    /include/ "base/hi1910B-network-M100-B51.dtsi"
    /include/ "base/hi1910B-hidvpp.dtsi"
    /include/ "base/hi1910B-hdmi.dtsi"
    /include/ "base/hi1910B-audio.dtsi"
    /include/ "base/hi1910B-mipitx.dtsi"
    /include/ "base/hi1910B-vdp.dtsi"
```

 版权所有 © 华为技术有限公司 

```
/include/ "base/hi1910B-isp.dtsi"
/include/ "base/hi1910B-hilink.dtsi"
/include/ "base/hi1910B-mntn-milanr3.dtsi"
/include/ "base/hi1910B-gpu.dtsi"
/include/ "base/hi1910B-ctl.dtsi"
/include/ "base/hi1910B-itrustee.dtsi"
/include/ "base/hi1910B-sata.dtsi"
// /include/ "product/hi1910B-pcie-rc-M100-B51.dtsi"
/include/ "product/hi1910B-pinctrl-M100-B51.dtsi"
/include/ "product/hi1910B-hiusbc-B51.dtsi"
/include/ "product/hi1910B-usb-B51.dtsi"

poweroff {
        compatible = "hisi,power_off";
        reg = <0x0 0xc4060000 0x0 0x10000>;
        pinctrl-names = "default", "idle";
        pinctrl-0 = <&poweroff_pmx_func>;
        pinctrl-1 = <&poweroff_pmx_idle>;
};
```

3. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

**步骤9** 执行如下命令，进入“dtbtool”目录。

**cd** */opt*/**Ascend310B-source/dtb/dtbtool/**

**步骤10** 执行如下命令，修改CMakelists文件，将所增加的dts添加到编译路径中，如下回显信息的加粗字体所示。

**vim CMakeLists.txt**

```
set(DTS_BASE_DIR ${TOP_DIR}/tools/customize/dts)
if(PRODUCT STREQUAL "ascend310Besl" OR
   PRODUCT STREQUAL "ascend310B" OR
   PRODUCT STREQUAL "ascend310Bemu" OR
   PRODUCT STREQUAL "ascend310Brc" OR
   PRODUCT STREQUAL "ascend310Brcesl" OR
   PRODUCT STREQUAL "ascend310Brcemu")
      # 当前driver跟esl用相同的dts，后期需要更新到单独的目录
      set(DTS_SRC_DIR ${DTS_BASE_DIR}/hi1910b/hi1910BL)
      set(DTS_FILE_LIST hi1910B-default.dts hi1910B-evb-900.dts hi1910B-evb-901.dts hi1910B-evb-902.dts
hi1910B-evb-903.dts hi1910B-evb-905.dts hi1910B-asic-M150-B50.dts hi1910B-asic-M100-B51.dts hi1910B-
asic-M150-B51.dts hi1910B-asic-M151-B50.dts hi1910B-asic-M101-B51.dts hi1910B-asic-M150-B33.dts)
      set(OUT_DT_IMAGE dt.img)
elseif(PRODUCT STREQUAL "helper310p")
      set(DTS_SRC_DIR ${DTS_BASE_DIR}/helper)
   set(DTS_FILE_LIST  helper-evb-925-1p.dts helper-evb-949-1p.dts helper-asic-450-1p.dts helper-
asic-455-1p.dts helper-asic-475-2p.dts helper-evb-900-2p.dts)
```

**说明**

默认dts要添加在第一个编译的位置，在加粗字体段落末尾添加对应产品的dts。

**步骤11** 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

**:wq!**

**步骤12** 重新编译生成dt.img文件。

1. 返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译dtb文件。

   **bash build.sh dtb**

   出现如下回显，且生成“dt.img”文件表示编译内核dtb文件成功。

 版权所有 © 华为技术有限公司 

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

**说明**

编译后的dt.img文件会自动存放于“Ascend310B-source/output”目录下。

把编译生成dt.img文件重构入驱动run包，具体请参见**6.1.2.3 重构驱动run包**。

**----结束**

## 6.1.2.2 （可选）获取内核 Image 文件

当用户有软实时OS的需求时，需使用带-rt参数的驱动包（Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run），具体可参见**4.1 下载软件包**。此驱动包不包含内核Image文件，因此需要用户自行编译内核，具体请参见**8.3.1.2 编译并生效内核Image文件（openEuler-SP1）**。为了便于用户快速使用，也可访问**Atlas 200I A2 加速模块软件包**，下载并解压Ascend-hdk-310b-npu-openEuler-22.03-LTS-SP1-Imgae-soc_linux-arrch64.zip，直接获取Image。待生成或直接获取内核Image文件后，需将其重构加入驱动run包中，具体请参见**6.1.2.3 重构驱动run包**。

## 6.1.2.3 重构驱动 run 包

本节介绍了用户需要新增或替换驱动包中的Image、dt.img、驱动文件（ko文件）或驱动文件的加载脚本等时，如何进行驱动包的重构。

## 操作步骤

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至Linux系统root用户属组目录，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 使用WinSCP工具，将**4.1 下载软件包**获取的驱动包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run”（若有软实时OS需求，使用驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run）上传至**步骤6**的“Ascend310B-source”目录下。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤8** 执行以下命令，为驱动包添加可执行权限。

**chmod u+x Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-aarch64.run**

 版权所有 © 华为技术有限公司 

若有软实时OS需求，该命令为**chmod u+x Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run**。

**步骤9** 屏蔽spi驱动。

1. 使用WinSCP工具，将**4.1 下载软件包**获取的驱动包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run”（若有软实时OS需求，使用驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run）上传至Linux系统root用户属组目录，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。
2. 执行如下命令，进入“opt”目录。

   **cd** */opt*
3. 将驱动包解压至“opt”的子目录“repack”下。

   **bash Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run --noexec --extract=./repack**

   若有软实时OS需求，该命令为**bash Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run --noexec --extract=./repack**。
4. 执行如下命令进入解压目录。

   **cd repack/scripts/**
5. 执行如下命令修改minirc_sys_init.sh文件。

   **vim minirc_sys_init.sh**
6. 屏蔽下图绿色方框中自动加载驱动。

```
load_device_modules()
{
    echo "load device modules"
    ko_path="${targetdir}"/driver

    ko_check ${ko_path}/drv_gpio_uniform.ko
    ko_check ${ko_path}/drv_sys_ctrl_uniform.ko
    ko_check ${ko_path}/drv_i2c_uniform.ko
    # ko_check ${ko_path}/drv_spi_uniform.ko
    ko_check ${ko_path}/drv_usb_uniform.ko

    # support i2c module
    ko_check ${ko_path}/i2c-dev.ko
    # ko_check ${ko_path}/spidev.ko
    # support usb/sata module
    ko_check ${ko_path}/usb-storage.ko
    ko_check ${ko_path}/xhci-hcd.ko
    ko_check ${ko_path}/xhci-plat-hcd.ko
```

7. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

**步骤10** 屏蔽power off驱动。

1. 执行如下命令进入minirc_sys_init.sh文件。

   **vim minirc_sys_init.sh**
2. 屏蔽下图绿色方框中自动加载驱动。

 版权所有 © 华为技术有限公司 

```
    ko_check ${ko_path}/hsm_ras.ko

    # ko_check ${ko_path}/kbox_init.ko
    # kbox_ko_check

    echo "finish load device modules"
    # /var/device_sys_init_ext.sh
}
```

3. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

4. 执行如下命令进入minirc_sys_init_ext.sh文件。

   **vim minirc_sys_init_ext.sh**

5. 屏蔽下图绿色方框中自动加载驱动。

```
load_device_modules()
{
    echo "load device ext modules"
    ko_path="${targetdir}"/driver
    ko_check ${ko_path}/pwm.ko
    ko_check ${ko_path}/drv_ascend_devmng.ko
    ko_check ${ko_path}/drv_mem_ctrl.ko
    # ko_check ${ko_path}/drv_power_off.ko

    echo "finish load device ext modules"
}
```

6. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

**步骤11** （可选）新增或替换文件。若用户需要新增或替换驱动包中的Image（若有软实时OS需求，请参见**6.1.2.2 （可选）获取内核Image文件**下载）、dt.img、驱动文件（ko文件）或驱动文件的加载脚本等，需执行以下操作。

1. 执行以下命令，在“Ascend310B-source”目录下新建“repack”目录并进入。若已创建，则无需重复创建。

   **mkdir repack**

   **cd repack**

2. 执行以下命令，在“repack”目录下新建“firmware”目录，用于替换Image或dt.img文件。新建“drivers”目录，用于替换驱动文件（ko文件）。新建“scripts”目录，用于替换驱动文件的加载脚本。

   **mkdir firmware**

   **mkdir drivers**

   **mkdir scripts**

3. 执行以下命令，进入“firmware”目录。

   **cd firmware**

   上传Image或dt.img文件，文件生成方式请参见**8.3.1 编译内核**，文件传输方式请参见**14.3 使用WinSCP传输文件**。

4. 执行以下命令，查看上传的Image或dt.img文件。

   **ls**


版权所有 © 华为技术有限公司

5. 执行以下命令回到“repack”目录，再进入“drivers”目录。

   **cd** */opt*/**Ascend310B-source/repack**

   **cd drivers**

6. 上传新的驱动文件（ko文件），文件生成方式请参见**8.3.3 编译驱动**，文件传输方式请参见**14.3 使用WinSCP传输文件**。
7. 执行以下命令，查看上传的驱动文件（ko文件）。

   **ls**

8. 执行以下命令回到“repack”目录，再进入“scripts”目录。

   **cd** */opt*/**Ascend310B-source/repack**

   **cd scripts**

9. 上传驱动自动加载的配置文件11-atlas.conf或filelist.csv与minirc_sys_init_ext.sh，文件生成方式请参见**8.5 驱动自动加载配置**，文件传输方式请参见**14.3 使用WinSCP传输文件**。
10. 上传**步骤10**修改后的minirc_sys_init.sh和minirc_sys_init_ext.sh。
11. 执行以下命令，查看上传的驱动文件的加载脚本。

    **ls**

12. 执行如下命令，回到“Ascend310B-source”目录。

    **cd** */opt*/**Ascend310B-source**

**步骤12** 执行如下命令，编译驱动。

**bash build.sh repack ./Ascend-hdk-310b-npu-driver-soc_**<*version*>**_linux-aarch64.run**

若有软实时OS需求，该命令为**bash build.sh repack ./Ascend-hdk-310b-npu-driver-soc_**<*version*>**_linux-rt-aarch64.run**。

出现如下回显，表示驱动重打包成功。

```
repack success!
```

**说明**

- 编译后的Ascend-hdk-310b-npu-driver-soc_<*version*>_linux-aarch64-repack.run（若有软实时OS需求，该驱动包为Ascend-hdk-310b-npu-driver-soc_<*version*>_linux-rt-aarch64-repack.run）驱动文件会自动存放于“Ascend310B-source/”目录下。
- <*version*>表示软件包版本号。

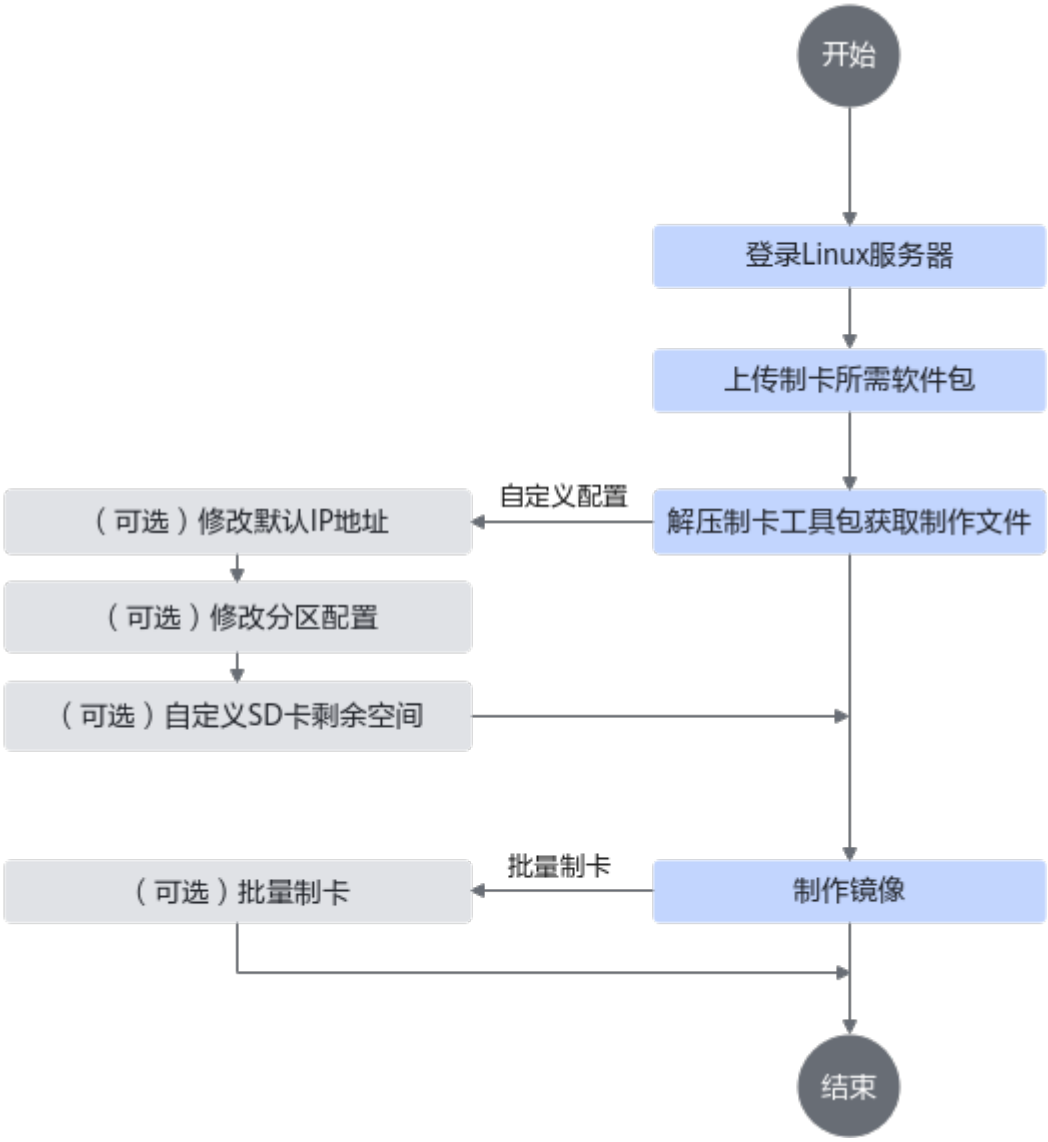
**----结束**

## 6.1.2.4 SD 卡制作系统镜像

本章节指导用户如何烧写SD卡启动盘。通过本章节将SD卡烧写成功后，可直接作为启动盘将Atlas 200I A2 加速模块整机启动。

 版权所有 © 华为技术有限公司 

**图 6-1** SD 卡制作系统镜像操作流程




## 前提条件

已获取表6-1中对应的ISO和文件系统镜像。

**表 6-1** 镜像获取

| 镜像 | 目标OS | 支持的版本 | 获取方式 | 说明 |
| --- | --- | --- | --- | --- |
| ISO | Ubuntu | 22.04 | 单击**ubuntu 22.04 ISO镜像**下载。 | - |


版权所有 © 华为技术有限公司

<table>
<tr><th>镜像</th><th>目标OS</th><th>支持的版本</th><th>获取方式</th><th>说明</th></tr>
<tr><td></td><td>openEuler</td><td>openEuler-22.03-LTS<br>openEuler-22.03-LTS-SP1[a]</td><td>• Windows环境<br>– openEuler-22.03-LTS：单击openEuler-22.03-LTS ISO镜像下载。<br>– openEuler-22.03-LTS-SP1：单击openEuler-22.03-LTS-SP1 ISO镜像下载。<br>• Linux环境<br>同Windows方式获取iso，执行命令wget iso镜像链接。</td><td>若下载ISO速度过慢，可参考如下方式获取。<br>1. 访问镜像链接。<br>2. 客户自行选择相近的Location。<br>3. 下载OS镜像。<br>• openEuler-22.03-LTS：进入“openEuler-22.03-LTS/ISO/aarch64”目录，下载OS镜像“openEuler-22.03-LTS-aarch64-dvd.iso”。<br>• openEuler-22.03-LTS-SP1：进入“openEuler-22.03-LTS-SP1/ISO/aarch64”目录，下载OS镜像“openEuler-22.03-LTS-SP1-aarch64-dvd.iso”。</td></tr>
<tr><td>文件系统</td><td>Ubuntu</td><td>22.04</td><td>访问Atlas 200I A2 加速模块软件包，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img。</td><td rowspan="2">若用户需要自定义文件系统，请参见A.6 定制文件系统。</td></tr>
<tr><td>openEuler</td><td>openEuler-22.03-LTS<br>openEuler-22.03-LTS-SP1[a]</td><td>访问Atlas 200I A2 加速模块软件包，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img或Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img。</td></tr>
<tr><td colspan="5">a：若有软实时OS需求时，需使用的OS为openEuler-22.03-LTS-SP1，否则需使用openEuler-22.03-LTS或Ubuntu。</td></tr>
</table>

 版权所有 © 华为技术有限公司 

> **说明**
>
> 社区版软件不需要申请下载权限可以直接下载，但软件不能用于商业用途， 详细请参见**社区华为软件许可协议**；如果您需要用于商业用途，参见**A.6 定制文件系统**的操作步骤进行制作。因此如果您下载软件仅用于调试验证，推荐下载社区版软件。

## 操作步骤

**步骤1** 将SD卡插入USB读卡器（推荐使用32G (class10)及以上）。

**步骤2** 将USB读卡器插入Linux服务器。

**步骤3** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤4** 执行如下命令，切换至root用户。

**su - root**

**步骤5** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的制卡所需软件包sdtool.tar.gz、驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run（若有软实时OS需求，不能直接使用驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run，该驱动包需要用户自行编译生成内核Image文件后进行重构，用户使用重构后的驱动包）和**前提条件**中获取的ISO与文件系统镜像上传至root用户属组目录下，例如“/home/ascend/mksd”。工具使用方法请参见**14.3 使用WinSCP传输文件**。

> **说明**
>
> - 对于openEuler操作系统，需要将ISO与定制后的文件系统（请参考**A.6 定制文件系统**章节）一并上传至root用户属组目录下，例如“/home/ascend/mksd”。需注意“openEuler-22.03-LTS-aarch64-dvd.iso”镜像配套“Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img”使用，“openEuler-22.03-LTS-SP1-aarch64-dvd.iso”镜像配套“Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img”使用。
> - 当前目录只允许存放一个版本的软件包。
> - 对于Ubuntu 22.04系统，需要将“ubuntu-22.04-live-server-arm64.iso”镜像与定制后的Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img（请参考**A.6 定制文件系统**章节）一并上传至root用户属组目录下，例如“/home/ascend/mksd”。
> - USB读卡器方式制卡时会自动创建用户名和属组都为HwHiAiUser的默认用户，该用户用来运行应用程序，同时该用户还有用户态AI驱动的管理权限。
> - 不支持将用户态AI驱动的管理权限切换到其他用户。请不要删除HwHiAiUser用户，以免导致系统异常。如果系统使用过程中有删除、重新增加HwHiAiUser用户的操作，需要重新启动设备。

**步骤6** 执行如下命令，进入sdtool.tar.gz制卡脚本工具包的上传目录，例如“/home/ascend/mksd”。

**cd** */home/ascend/mksd*

**步骤7** 执行如下命令，解压sdtool.tar.gz制卡脚本工具包。

**tar -xzvf sdtool.tar.gz**

**步骤8** （可选）用户需要预置自定义软件包时，请执行如下步骤，修改自定义预置软件包脚本。

 版权所有 © 华为技术有限公司 

1. 执行如下命令，编辑minirc_install_hook.sh脚本。

   **vim minirc_install_hook.sh**

2. 参考**8.7 编辑用户自定义脚本**，编辑用户自定义脚本。
3. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

**步骤9** 执行如下命令，将解压后的文件拷贝至制卡脚本工具包所在目录，例如“/home/ascend/mksd”。

**cp -arf sdtool/* ./**

执行如下命令，查看文件是否拷贝至制卡脚本工具包所在目录。

**ll**

类似回显信息如下：

```
total 2109376
drwxr-xr-x  5 root  root        4096 Jun 21 02:52 ./
drwxr-xr-x 16 root  root        4096 Jun 21 02:03 ../
-rw-r--r--  1 root  root   180094149 Jun 21 02:11 Ascend-hdk-310b-npu-driver-soc_23.0.rc1.1_linux-aarch64-repack.run
-rw-r--r--  1 root  root   550127845 Jun 21 02:14 Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip
-rw-rw-r--  1 20001 20001          0 Jun 21 02:52 boot_image_info
-rwxrwxr-x  1 20001 20001      55328 Apr 21 16:00 emmc-head*
-rwxrwxr-x  1 20001 20001       2464 Apr 21 16:00 install_hook_demo.sh*
-rwxrwxr-x  1 20001 20001      48760 Apr 21 16:00 make_os_recover.sh*
-rwxrwxr-x  1 20001 20001      62383 Apr 21 16:00 make_os_sd.sh*
-rwxrwxr-x  1 20001 20001      12777 Apr 21 16:00 make_sd_card.py*
-rw-r--r--  1 root  root          0 Jun 21 02:19 minirc_install_hook.sh
-rw-rw-r--  1 20001 20001        110 Apr 21 16:00 mksd.conf
-rw-rw-r--  1 20001 20001       1024 Jun 21 02:52 parttion_head_info
-rwxrwxr-x  1 20001 20001       7195 Apr 21 16:00 preconfig.sh*
drwxr-xr-x  4 root  root        4096 Jun 21 02:15 sdtool/
-rw-r--r--  1 root  root      380372 Jun 21 02:14 sdtool.tar.gz
drwxrwxr-x  2 20001 20001       4096 Apr 21 16:00 synctool/
drwxrwxr-x  3 20001 20001       4096 Jun 21 02:21 tools/
-rw-r--r--  1 root  root  1429155840 Jun 21 02:10 ubuntu-22.04-live-server-arm64.iso
```

**步骤10** 执行如下命令，检查emmc-head工具是否可用。

**./emmc-head --help**

```
Usages: emmc-head firmware_path boot_a_devname boot_b_devname [force_recover]
The following files must be contained in firmware_path:
Image, itrustee.img, dt.img, initrd.
boot_a_devname: A Partition boot device name, for example, eMMC:mmcblk0p2, SD:mmcblk1p2
boot_b_devname: B Partition boot device name, for example, eMMC:mmcblk0p3, SD:mmcblk1p3
force_recover: force recover flag.
        Example: /var/davinci/driver/emmc-head ./firmware /dev/mmcblk0p2 /dev/mmcblk0p3
```

**说明**

- emmc-head工具用于更新镜像启动信息。
- 执行命令后有帮助回显信息则表示可用，若回显信息出现-bash: ./emmc-head: No such file or directory的错误提示，请参见**15.3 emmc-head工具不可用**。

**步骤11** （可选）更改网线和USB的默认IP地址。

1. 执行如下命令，进入制卡脚本工具包所在目录，例如“”/home/ascend/mksd。

   **cd** */home/ascend/mksd*

2. 执行如下命令，更改网线和USB的默认IP地址。

 版权所有 © 华为技术有限公司 

**vim make_sd_card.py**

需分别修改“make_sd_card.py”脚本中的“USB_CARD_DEFAULT_IP”、“NETWORK_CARD_DEFAULT_IP”、“NETWORK_CIDR_PREFIX”、“NETWORK_DEFAULT_NETMASK”和“NETWORK_DEFAULT_GATEWAY”的参数值。

**说明**

- USB连接方式的默认IP地址为192.168.1.2，网线连接方式的默认IP地址192.168.0.2。
- USB_CARD_DEFAULT_IP：表示USB连接方式的IP地址。例如192.168.3.5。
- NETWORK_CARD_DEFAULT_IP：表示网线连接方式的IP地址，例如192.168.2.8。
- NETWORK_CIDR_PREFIX：表示网线连接方式的子网掩码中bit是1的个数，例如CIDR PREFIX=24，对应子网掩码为255.255.255.0。
- NETWORK_DEFAULT_NETMASK：表示网线连接方式的子网掩码，例如255.255.255.0。
- NETWORK_DEFAULT_GATEWAY：表示网线连接方式的网关，例如192.168.2.1。

**步骤12** 执行如下命令，查找SD卡所在的USB设备名称，例如“/dev/sdb”。

**fdisk -l**

显示类似信息如下：

```
Disk /dev/sdb: 28.33 GiB, 30416044032 bytes, 59406336 sectors
Disk model: UHSII uSD Reader
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 57C9FC21-66A9-4474-8EEB-DD67845D38CB
```

其中Disk model: UHSII uSD Reader为SD卡所在的USB设备名称，不同的USB设备显示的名称不一样，此处以“UHSII uSD Reader”为例。

**步骤13** （可选）修改镜像包配置文件。

1. 执行如下命令，进行重新配置。

   **bash preconfig.sh reconfigure**

   显示信息如下：

   ```
   [INFO]mksd.conf is exist, reconfiguration will overwrite mksd.conf!
   [INFO]The old mksd.conf will be rename to mksd.conf.bak, do you want to continue (y/n)?
   ```

2. 根据界面提示选择是否覆盖原有mksd.conf文件。
   - y：覆盖原有mksd.conf文件。
   - n：不覆盖原有mksd.conf文件。
3. 根据界面提示选择是否使用默认配置。

   显示信息如下：

   ```
   [INFO]Do you want to use default configuration (y/n)?
   ```

   - y：保留默认配置。
   - n：进行手工配置。
4. 根据界面提示选择是否使能主备分区功能。

   显示信息如下：

   ```
   [INFO]Set filesystem backup enable type (on/off) (default off):
   ```

   - on：使能主备分区功能。
   - off/回车键：不使能主备分区功能。

 版权所有 © 华为技术有限公司 

5. 根据界面提示选择是否分配系统分区的存储容量。分区关系表可参考**10.1 主备分区介绍**。

   显示信息如下：

   ```
   [INFO]Set rootfs partitation size, enter a number(MB) (default 6144):
   ```

   - 输入目标容量（MB）。
   - 回车键：使用默认项。

6. 根据界面提示选择是否分配日志分区的存储容量。

   显示信息如下：

   ```
   [INFO]Set log partitation size, enter a number(MB) (default 1024):
   ```

   - 输入目标容量（MB）。
   - 回车键：使用默认项。

7. 根据界面提示选择是否分配用户数据分区的存储容量。

   显示信息如下：

   ```
   [INFO]Set home data size, enter a number(MB) (default 1024):
   ```

   - 输入目标容量（MB）。
   - 回车键：使用默认项。

**步骤14** （可选）自定义SD卡剩余空间。

1. 执行以下命令，打开mksd.conf文件。

   **vim mksd.conf**

   mksd.conf文件示例如下所示。

   ```
   #!/bin/bash
   FS_BACKUP_FLAG=on
   FAST_BOOT_FLAG=off
   ROOT_PART_SIZE=6144
   LOG_PART_SIZE=1024
   HOME_DATA_SIZE=1024
   ```

   **说明**

   - 当**步骤13**使能了主备分区时，FS_BACKUP_FLAG的值为on，否则为off。
   - 当**步骤13**没有更改ROOT_PART_SIZE、LOG_PART_SIZE与HOME_DATA_SIZE的值，则此处为默认大小。

2. 在mksd.conf文件末尾添加分区配置，以默认分区大小为例。

   - 未使能主备分区，则需要预留9216M空间，在mksd.conf文件末尾添加以下加粗字体内容，如下所示：

     ```
     #!/bin/bash
     FS_BACKUP_FLAG=off
     FAST_BOOT_FLAG=off
     ROOT_PART_SIZE=6144
     LOG_PART_SIZE=1024
     HOME_DATA_SIZE=1024
     OFFSIZE=9216 END=13312
     OFFSIZE=13312 END=19456
     OFFSIZE=19456 END=30412
     ```

   - 使能主备分区，则需要预留15360M空间，在mksd.conf文件末尾添加以下加粗字体内容，如下所示：

     ```
     #!/bin/bash
     FS_BACKUP_FLAG=on
     FAST_BOOT_FLAG=off
     ROOT_PART_SIZE=6144
     LOG_PART_SIZE=1024
     HOME_DATA_SIZE=1024
     OFFSIZE=15360 END=18332
     ```

 版权所有 © 华为技术有限公司 

**说明**

- 自定义分区时需要预留一定空间，预留大小为ROOT_PART_SIZE(开启备区时需要*2)+LOG_PART_SIZE+HOME_DATA_SIZE+固件烧写预留大小（1024MB）。
- 配置格式如**未使能主备分区的mksd.conf文件示例**所示，OFFSIZE=9216为预留空间的大小，也是自定义分区的起始地址。END=OFFSIZE+1024*3=13312，即第一个自定义分区大小为3GB。
- 最后一个分区END的大小不能超过SD卡容量最大值，可使用**fdisk -l**可以查询SD卡容量大小。

**步骤15** 执行如下命令，运行启动镜像脚本“make_sd_card.py”，制作SD卡。例如：SD卡所在的USB设备名称为“/dev/sdb”。

**python3 make_sd_card.py local** */dev/sdb*

显示如下信息，表示开始制卡。

```
root@ubuntu:/home/ascend/mksd# python3 make_sd_card.py local /dev/sdb
Begin to make SD Card...
FS_BACKUP_FLAG on
FAST_BOOT_FLAG off
ROOT_PART_SIZE 6144(MB)
LOG_PART_SIZE 1024(MB)
HOME_DATA_SIZE 1024(MB)
preconfig success.
Please make sure you have installed dependency packages:
        apt-get install -y qemu-user-static binfmt-support gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
dosfstools parted kpartx
Please input Y: continue, other to install them:
```

**步骤16** 输入“Y”。等待约15分钟。显示如下信息，表示制卡成功。

```
Make Card successfully!
```

**步骤17** （可选）查看分区信息。

若执行**步骤14**自定义SD卡剩余空间，可执行以下命令查看分区信息。

**fdisk -l**

**图 6-2** 分区信息示例

```
Disk /dev/sda: 29.7 GiB, 31914983424 bytes, 62333952 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: E82EC7E2-0AAC-4FA1-A3B3-C7B7E4CAD334

Device        Start      End  Sectors  Size Type
/dev/sda1   1605633  1810432   204800  100M Linux filesystem
/dev/sda2   1810433 14393344 12582912    6G Linux filesystem
/dev/sda3  14393345 26976256 12582912    6G Linux filesystem
/dev/sda4  26976257 29073408  2097152    1G Linux filesystem
/dev/sda5  29073409 31170560  2097152    1G Linux filesystem
/dev/sda6  31252480 37543935  6291456    3G Linux filesystem
```

图中/dev/sda6是使能备区后的分区信息。

**步骤18** （可选）配置IP文件。

1. 执行如下命令，挂载SD卡主区（例如：/dev/sdb2）到“/mnt”目录。

   **mount** */dev/sdb2* **/mnt**

 版权所有 © 华为技术有限公司 

2. 执行以下命令，编辑ifcfg-eth0配置文件。用户可根据现场网络规划配置IP与网关。

   **vim /mnt/etc/sysconfig/network-scripts/ifcfg-eth0**

3. 执行以下命令解挂载SD卡。

   **umount /mnt**

**说明**

- 挂载SD卡后，不同文件系统的网络配置文件路径与名称不同。
  - openEuler网络配置文件路径与名称：/mnt/etc/sysconfig/network-scripts/ifcfg-eth0
  - Ubuntu22.04网络配置文件路径与名称：/mnt/etc/netplan/01-netcfg.yaml
- 用户配置完网络后，须对“/mnt”目录进行解挂载，否则可能会导致SD卡内的文件系统损坏。

**步骤19** （可选）批量制卡。

**说明**

批量制卡需保证复制卡与母卡型号和规格完全一致。

- 方法1：通过内存卡拷贝机直接进行SD卡复制。
- 方法2：将制作好OS的SD卡使用dd命令导出镜像，进行批量烧录制卡。
  a. 将SD插入USB读卡器，并将USB读卡器插入Linux服务器。
  b. 导出镜像，请执行如下命令。详细请自行查询Linux dd命令使用方法。
     命令：**dd if=***母盘盘符* **of=***导出镜像名*
     示例：**dd if=/dev/sda of=/root/tmp/sd.image**
  c. 将镜像烧录至待复制子卡，请执行如下命令。
     命令：**dd if=***镜像名* **of=***待复制子盘盘符*
     示例：**dd if=/root/tmp/sd.image of=/dev/sdb**

**步骤20** （可选）为避免出现内存不足导致设备OOM异常挂死问题，请参考**15.6 运行程序超内存导致设备异常挂死，设备能ping通，但SSH无法远程连接**解决。

**步骤21** 上电和启动设备，具体请参见**5.3 上电和启动设备**。

**----结束**

## 6.1.3 SD 卡制作和启动系统镜像（方法二）

本章节用于制作从SD卡启动需要的镜像，该镜像默认使能了主备分区，并开启了主备同步功能。

### 前提条件

已获取**表6-2**中对应的ISO和文件系统镜像。


版权所有 © 华为技术有限公司

**表 6-2** 镜像获取

| 镜像 | 目标OS | 支持的版本 | 获取方式 | 说明 |
| --- | --- | --- | --- | --- |
| ISO | Ubuntu | 22.04 | 单击**ubuntu 22.04 ISO镜像**下载。 | - |
| | openEuler | openEuler-22.03-LTS | • Windows环境<br>– openEuler-22.03-LTS：单击**openEuler-22.03-LTS ISO镜像**下载。<br>– openEuler-22.03-LTS-SP1：单击**openEuler-22.03-LTS-SP1 ISO镜像**下载。<br>• Linux环境<br>同Windows方式获取iso，执行命令**wget** *iso镜像链接*。 | 若下载ISO速度过慢，可参考如下方式获取。<br>1. 访问**镜像链接**。<br>2. 客户自行选择相近的Location。<br>3. 下载OS镜像。<br>• openEuler-22.03-LTS：进入“openEuler-22.03-LTS/ISO/aarch64”目录，下载OS镜像“openEuler-22.03-LTS-aarch64-dvd.iso”。<br>• openEuler-22.03-LTS-SP1：进入“openEuler-22.03-LTS-SP1/ISO/aarch64”目录，下载OS镜像“openEuler-22.03-LTS-SP1-aarch64-dvd.iso”。 |
| | | openEuler-22.03-LTS-SP1[a] | | |
| 文件系统 | Ubuntu | 22.04 | 访问**Atlas 200I A2 加速模块软件包**，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img。 | 若用户需要自定义文件系统，请参见**A.6 定制文件系统**。 |


版权所有 © 华为技术有限公司

| 镜像 | 目标OS | 支持的版本 | 获取方式 | 说明 |
| --- | --- | --- | --- | --- |
| | openEuler | openEuler-22.03-LTS | 访问**Atlas 200I A2 加速模块软件包**，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img或Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img。 | |
| | | openEuler-22.03-LTS-SP1[a] | | |
| a：若有软实时OS需求时，需使用的OS为openEuler-22.03-LTS-SP1，否则需使用openEuler-22.03-LTS或Ubuntu。 | | | | |

**说明**

社区版软件不需要申请下载权限可以直接下载，但软件不能用于商业用途， 详细请参见**社区华为软件许可协议**；如果您需要用于商业用途，参见**A.6 定制文件系统**的操作步骤进行制作。因此如果您下载软件仅用于调试验证，推荐下载社区版软件。

## 制作及烧写镜像

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的制卡所需软件包sdtool.tar.gz、驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run（若有软实时OS需求，不能直接使用驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run，该驱动包需要用户自行编译生成内核Image文件后进行重构，用户使用重构后的驱动包）和**前提条件**中获取的ISO与文件系统镜像上传至root用户属组目录下，例如“/home/ascend/mksd”。

**说明**

- 对于openEuler操作系统，需要将“openEuler-22.03-LTS-aarch64-dvd.iso”镜像与定制后的Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img（请参考**A.6 定制文件系统**章节）一并上传至root用户属组目录下，例如“/home/ascend/mksd”。
- 当前目录只允许存放一个版本的软件包。
- 对于Ubuntu 22.04系统，需要将“ubuntu-22.04-live-server-arm64.iso”镜像与定制后的Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img（请参考**A.6 定制文件系统**章节）一并上传至root用户属组目录下，例如“/home/ascend/mksd。”

**步骤4** 执行如下命令，进入sdtool.tar.gz制卡脚本工具包的上传目录，例如“/home/ascend/mksd”。

**cd** */home/ascend/mksd*


版权所有 © 华为技术有限公司

**步骤5** 在制卡脚本工具包的上传目录下（例如“/home/ascend/mksd”），执行如下命令解压sdtool.tar.gz制卡脚本工具包。

**tar -xzvf sdtool.tar.gz**

**步骤6** 执行如下命令，将解压后的文件拷贝至制卡脚本工具包所在目录，例如“/home/ascend/mksd”。

**cp -arf sdtool/* .**

**步骤7** 执行如下命令，进入“recovertool”目录。

**cd** */home/ascend/mksd*/**recovertool**

**步骤8** 使用WinSCP工具或其他工具，根据实际需求将对应软件包上传至“recovertool”目录下，制卡所需软件包请参见**表4-1**。工具使用方法请参见**14.3 使用WinSCP传输文件**。

- 非软实时OS需求，将“Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz”软件包上传至“recovertool”目录下。
- 软实时OS需求，将“Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz”软件包上传至“recovertool”目录下。

  上传“Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz”软件包后，需执行如下命令进行软件包重构。

  **mkdir rt && tar -zxvf Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz -C ./rt && rm -rf rt/Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run && cp ../Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run rt/ && cd rt && tar zcvf Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz ./* && cp Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz ../ && cd .. && rm -rf rt**

  **说明**

  *<version>*表示NPU版本号，具体请根据实际情况进行替换。

**步骤9** （可选）更改网线和USB的默认IP地址。

1. 执行如下命令，进入制卡脚本工具包所在目录，例如“/home/ascend/mksd”。

   **cd** */home/ascend/mksd*
2. 执行如下命令，更改网线的默认IP地址。

   **vi make_sd_card.py**

   需分别修改“make_sd_card.py”脚本中的“USB_CARD_DEFAULT_IP”、“NETWORK_CARD_DEFAULT_IP”、“NETWORK_CIDR_PREFIX”、“NETWORK_DEFAULT_NETMASK”和“NETWORK_DEFAULT_GATEWAY”的参数值。

   **说明**

   - USB连接方式的默认IP地址为192.168.1.2，网线连接方式的默认IP地址192.168.0.2。
   - USB_CARD_DEFAULT_IP：表示USB连接方式的IP地址，例如192.168.3.5。
   - NETWORK_CARD_DEFAULT_IP：表示IP地址，例如192.168.1.8。
   - NETWORK_CIDR_PREFIX：表示子网掩码中bit是1的个数，例如CIDR PREFIX=24，对应子网掩码为255.255.255.0。
   - NETWORK_DEFAULT_NETMASK：表示子网掩码，例如255.255.255.0。
   - NETWORK_DEFAULT_GATEWAY：表示网关，例如192.168.1.1。

 版权所有 © 华为技术有限公司 

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** （可选）若用户需要自定义分区的配置，请执行该步骤。

1. 在驱动包所在目录（例如“/home/ascend/mksd/”）执行如下命令，解压驱动run包，获取“restore_factory.sh”脚本。

   **bash Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-aarch64.run --noexec --extract=.***/tmp**

   若有软实时OS需求，请执行如下命令。

   **bash Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-rt-aarch64-repack.run --noexec --extract=.***/tmp**

2. 拷贝“restore_factory.sh”脚本到“recovertool”目录。

   **cp** */home/ascend/mksd*/**tmp/scripts/restore_factory.sh** */home/ascend/mksd*/**recovertool/**

   **说明**

   tmp为临时解压目录，若不需要使用可以删除。

3. 修改分区配置。

   用户可自定义修改p2及之后的分区，p1不可修改。

   **vim ./recovertool/restore_factory.sh**

   修改如图6-3和图6-4红框中所示的配置。

**图 6-3** restore_factory.sh 脚本配置 1

```
#!/bin/bash
#
# Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.
#
# Entry script for restoring factory settings.
# This script is raised by user root, so can't not open other scripts here.
# Because if using other script here, they also has root authority,this takes secuity risk.
# Create: 2022-9-21

P_MAX_PN=8
#SIZE_GiB must be int
P2_SIZE_GiB=6
P3_SIZE_GiB=6
P4_SIZE_GiB=1
P5_SIZE_GiB=1
P6_SIZE_GiB=2
P7_SIZE_GiB=3
P8_SIZE_GiB=7
```

 版权所有 © 华为技术有限公司 

**图 6-4** restore_factory.sh 脚本配置 2

```
CREATE_NEW_PFLAG=no
function add_new_part()
{
    add_p_n=$P_MAX_PN
    tmp_add_size=0
    cat > mkpart_tmp.conf << EOF
#write partition size below, one size occupies one row

EOF

    while read line
    do
        if [[ "$line" -gt "0" ]];then
            add_p_n=$[add_p_n+1]
            add_off=$[sectorEnd8+tmp_add_size]
            add_end=$[line*1024+add_off]
            add_list+="mkpart p${add_p_n} ${add_off}MiB ${add_end}MiB "
            tmp_add_size=$[tmp_add_size+line*1024]
        fi
    done < ./mkpart_tmp.conf
    P_MAX_PN=$add_p_n

    rm -f ./mkpart_tmp.conf
}
```

a. 修改分区大小。

   直接修改各个分区PX_SIZE_GiB的值，如修改P8分区大小为20G，则P8_SIZE_GiB=20。

b. 增加新分区。

   修改CREATE_NEW_PFLAG=yes

   找到add_new_part函数，在#write partition size below, one size occupies one row下面写入分区大小配置，增加几个分区就写几行。

   如：增加三个分区，每个分区大小为20G

```
......
cat > mkpart_tmp.conf << EOF
#write partition size below, one size occupies one row
20
20
20
EOF
......
```

**须知**

- 为了保证文件系统的正常使用，修改p2~p8分区大小时不能小于原配置大小。
- 分区大小的赋值需要为大于0的整数，不支持配置为小数。
- 修改分区大小或增加新分区后，总大小不能超过SD的最大容量。

  总大小=p2~p8分区大小+新增分区大小+2.3G，其中2.3G为烧写的镜像包大小，需要保留。

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤11** 在脚本工具包所在目录（例如“/home/ascend/mksd/”）执行如下命令，检查emmc-head工具是否可用。

```
./emmc-head --help
```

 版权所有 © 华为技术有限公司 

执行命令后有帮助回显信息则表示可用；若回显信息出现-bash: ./emmc-head: No such file or directory的错误提示，请参见15.3 emmc-head工具不可用。

```
Usages: emmc-head firmware path boot_a_devname boot_b_devname [force_recover]
the three files must be included in the firmware path:
Image, itrustee.img, dt.img.
force_recover: force recover flag.
   Example: emmc-head ./firmware /dev/mmcblk1p1 /dev/mmcblk1p2
```

**步骤12** 在脚本工具包所在目录（例如“/home/ascend/mksd/”）执行命令启动镜像脚本“make_sd_card.py”，制作烧写到SD的系统镜像。

**python3 make_sd_card.py mkrecoverimg SD**

显示如下类似信息，表示开始制卡，此处仅为示例，具体请以实际情况为准。

```
Begin to make Image package for restoring factory settings...
Please make sure you have installed dependency packages:
  apt-get install -y qemu-user-static binfmt-support gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
dosfstools parted kpartx
Please input Y: continue, other to install them:
```

**步骤13** 根据打印提示信息输入“Y”，等待约10分钟（具体等待时间以实际环境为准）。显示类似如下信息，表示制卡成功。

```
Make Card successfully!
```

执行**ls -l**查看当前目录下生成的镜像文件为recoverfs-<os-type>-SD.img，其中<os-type>表示目标镜像的操作系统类型，例如：Ubuntu或者openEuler。

**说明**

若出现“[ERROR] Making M.2 SATA SSD or eMMC or USB or SD failed”类似回显信息，则代表制卡失败，建议用户按照日志错误排查，重新执行**python3 make_sd_card.py mkrecoverimg SD**命令制作镜像。

**步骤14** 烧写镜像到SD卡。

1. 将SD卡插入USB读卡器（推荐使用32G (class10)及以上）。
2. 将USB读卡器插入Linux服务器。
3. 执行**fdisk -l**查询SD卡所在的USB设备名称，例如“/dev/sdb”。

   显示类似信息如下：

   ```
   Disk /dev/sdb: 28.33 GiB, 30416044032 bytes, 59406336 sectors
   Disk model: UHSII uSD Reader
   Units: sectors of 1 * 512 = 512 bytes
   Sector size (logical/physical): 512 bytes / 512 bytes
   I/O size (minimum/optimal): 512 bytes / 512 bytes
   Disklabel type: gpt
   Disk identifier: 57C9FC21-66A9-4474-8EEB-DD67845D38CB
   ```

   其中Disk model: UHSII uSD Reader为SD卡所在的USB设备名称，不同的USB设备显示的名称不一样，此处以“UHSII uSD Reader”为例。
4. 执行**mkfs.ext4 -F** /dev/sdb命令格式化SD，出现如下类似关键回显信息表明格式化成功。

   ```
   Writing superblocks and filesystem accounting information: done
   ```
5. 在镜像recoverfs-<os-type>-SD.img上传路径下（例如“/home/ascend/mksd”），执行**dd if=recoverfs-<os-type>-SD.img of=**/dev/sdb **status=progress**，烧写镜像到SD，等待约3分钟，出现如下类似信息则代表烧写成功，此处仅为示例，具体烧写花费时间和烧写速率以实际情况为准。

   ```
   4751360+0 records in
   4751360+0 records out
   2432696320 bytes (2.4 GB, 2.3 GiB) copied, 45.217 s, 53.8 MB/s
   ```

 版权所有 © 华为技术有限公司 

**说明**

若出现写入数据量不等于4751360，则代表烧写失败，建议重新执行**dd if=recoverfs-<os-type>-SD.img of=**/dev/sda **status=progress**烧写镜像。

6. 烧写镜像后会出现1.5G的golden分区，可以通过**mount**命令挂载后查看分区内容。

   **mount** */dev/sdb1* **/mnt**

   **ll /mnt**

   **说明**

   查看完文件请执行**umount /mnt**，防止文件损坏。

**步骤15** 上电和启动设备，具体请参见**5.3 上电和启动设备**。

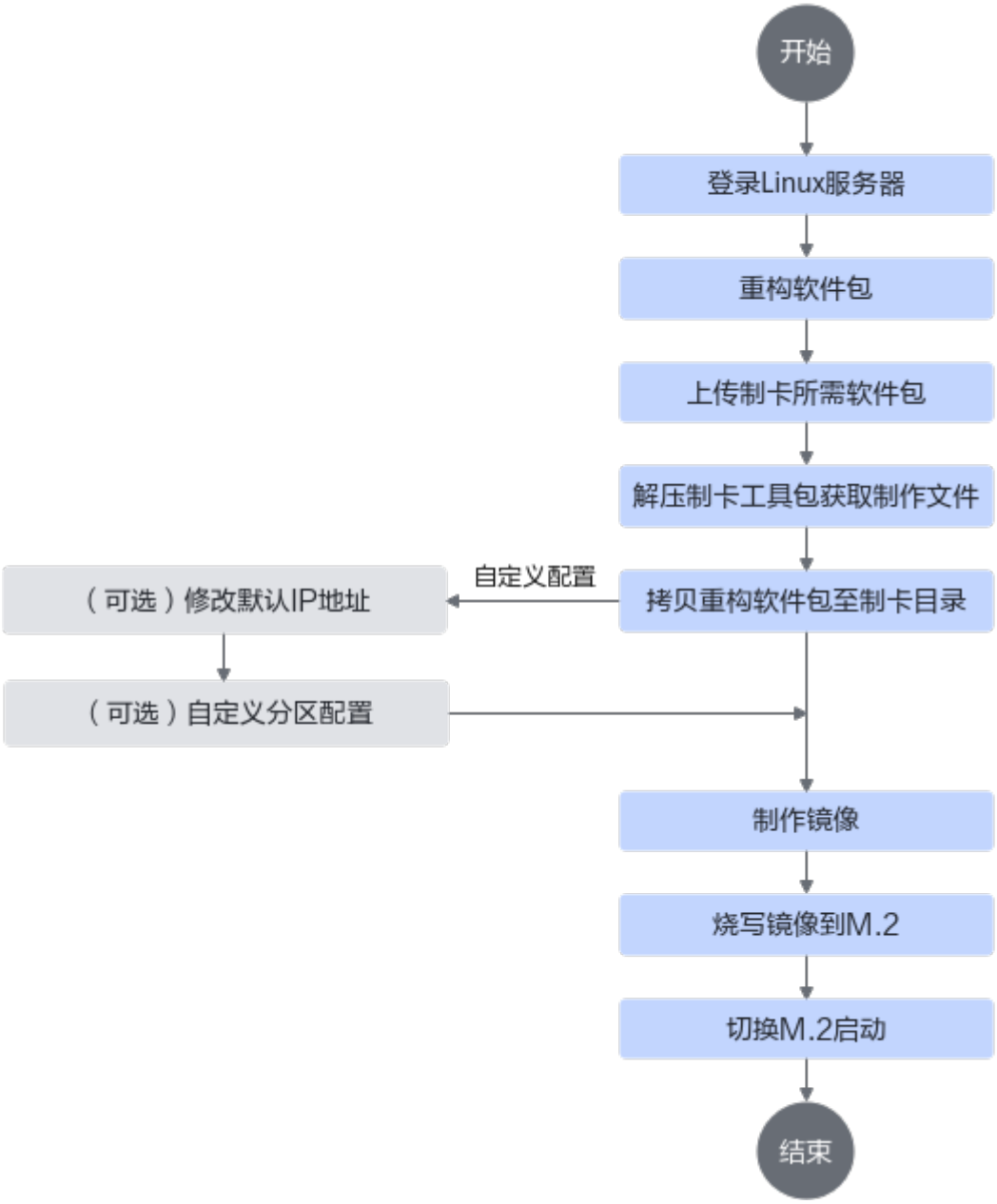
**----结束**

# 6.2 M.2 制作和启动系统镜像

本章节用于制作从M.2启动需要的镜像，该镜像默认使能了主备分区，并开启了主备同步功能。M.2制作和启动系统镜像的整体流程如下所示。

 版权所有 © 华为技术有限公司 

**图 6-5** M.2 制作和启动系统镜像流程




**须知**

系统从M.2启动时，需要用户根据Atlas 200I A2 加速模块底板设计配置userBaseConfig文件使能SATA接口。在任何介质下，直接下载软件包升级驱动和固件重启后，系统会提示找不到M.2 SATA的启动介质，导致M.2 SATA介质的OS启动失败，需要重新构建新的镜像包，通过镜像安装的方式进行升级。

## 前提条件

已获取**表6-3**中对应的ISO和文件系统镜像。


版权所有 © 华为技术有限公司

**表 6-3** 镜像获取

| 镜像 | 目标OS | 支持的版本 | 获取方式 | 说明 |
| --- | --- | --- | --- | --- |
| ISO | Ubuntu | 22.04 | 单击**ubuntu 22.04 ISO镜像**下载。 | - |
| | openEuler | openEuler-22.03-LTS | • Windows环境<br>– openEuler-22.03-LTS：单击**openEuler-22.03-LTS ISO镜像**下载。<br>– openEuler-22.03-LTS-SP1：单击**openEuler-22.03-LTS-SP1 ISO镜像**下载。<br>• Linux环境<br>同Windows方式获取iso，执行命令**wget** *iso镜像链接*。 | 若下载ISO速度过慢，可参考如下方式获取。<br>1. 访问**镜像链接**。<br>2. 客户自行选择相近的Location。<br>3. 下载OS镜像。<br>• openEuler-22.03-LTS：进入“openEuler-22.03-LTS/ISO/aarch64”目录，下载OS镜像“openEuler-22.03-LTS-aarch64-dvd.iso”。<br>• openEuler-22.03-LTS-SP1：进入“openEuler-22.03-LTS-SP1/ISO/aarch64”目录，下载OS镜像“openEuler-22.03-LTS-SP1-aarch64-dvd.iso”。 |
| | | openEuler-22.03-LTS-SP1[a] | | |
| 文件系统 | Ubuntu | 22.04 | 访问**Atlas 200I A2 加速模块软件包**，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img。 | 若用户需要自定义文件系统，请参见**A.6 定制文件系统**。 |


版权所有 © 华为技术有限公司

| 镜像 | 目标OS | 支持的版本 | 获取方式 | 说明 |
| --- | --- | --- | --- | --- |
| | openEuler | openEuler-22.03-LTS | 访问**Atlas 200I A2 加速模块软件包**，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img或Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img。 | |
| | | openEuler-22.03-LTS-SP1[a] | | |
| a：若有软实时OS需求时，需使用的OS为openEuler-22.03-LTS-SP1，否则需使用openEuler-22.03-LTS或Ubuntu。 | | | | |

**说明**

社区版软件不需要申请下载权限可以直接下载，但软件不能用于商业用途， 详细请参见**社区华为软件许可协议**；如果您需要用于商业用途，参见**A.6 定制文件系统**的操作步骤进行制作。因此如果您下载软件仅用于调试验证，推荐下载社区版软件。

## 制作及烧写镜像

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 修改userBaseConfig文件，重新生成新的“Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz”和“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64-repack.run”。（如有软实时OS需求，重新生成新的“Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz”和“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run”）

1. 参考**7.1.3 SATA接口调测参考**，根据Atlas 200I A2 加速模块底板设计修改user_base_config.xml文件并编译，生成“userBaseConfig.bin”文件。
2. 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”和驱动包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run”（若有软实时OS需求，使用驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run）上传至Linux系统root用户属组目录，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。
3. 执行如下命令，进入源码包所在目录“/opt”。

   **cd** */opt*
4. 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

   **tar -xzvf Ascend310B-source.tar.gz**

 版权所有 © 华为技术有限公司 

**说明**

如果已解压过源码包，可跳过此步骤。

5. 执行如下命令，进入“Ascend310B-source”目录。

   **cd Ascend310B-source**

6. 执行以下命令，为驱动包添加可执行权限。

   **chmod u+x *[/opt](/opt)*/Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run**

   若有软实时OS需求，请执行如下命令。

   **chmod u+x *[/opt](/opt)*/Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run**

7. 执行如下命令，将驱动包解压至当前文件夹下。

   **bash** *[/opt](/opt)*/**Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-aarch64.run --noexec --extract=./repack**

   若有软实时OS需求，请执行如下命令。

   **bash** *[/opt](/opt)*/**Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-rt-aarch64.run --noexec --extract=./repack**

8. 执行以下命令，进入“/opt/Ascend310B-source/repack/firmware”目录。

   **cd** *[/opt](/opt)*/**Ascend310B-source/repack/firmware**

9. 执行如下命令，将**步骤3.1**生成的“userBaseConfig.bin”文件到拷贝当前文件夹下。

   **cp** *[/opt](/opt)*/**Ascend310B-source/output/userBaseConfig.bin** *[/opt](/opt)*/**Ascend310B-source/repack/firmware**

10. （可选）如有软实时需求，参考**6.1.2.2 （可选）获取内核Image文件**，将Image文件使用WinSCP工具或其他工具拷贝到当前所在的“firmware”目录下。

11. 执行如下命令，返回“Ascend310B-source”目录。

    **cd** *[/opt](/opt)*/**Ascend310B-source**

12. 执行如下命令，编译驱动。

    **bash build.sh repack ../Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-aarch64.run**

    编译生成文件在当前路径下，名称为“**Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-aarch64-repack.run**”。

    若有软实时OS需求，请执行如下命令。

    **bash build.sh repack ../Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-rt-aarch64.run**

    编译生成文件在当前路径下，名称为“**Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-rt-aarch64-repack.run**”。

    出现如下类似回显信息，表示驱动重打包成功。

    ```
    repack success!
    ```

13. 执行如下命令，新建打包目录并进入该目录。

    **mkdir -p** *[/opt](/opt)*/**tar**

    **cd** *[/opt](/opt)*/**tar**

14. 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的“Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz”软件包上传至“/opt/tar”目录下。若有软

 版权所有 © 华为技术有限公司 

实时OS需求拷贝名为“Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz”的软件包。

15. 执行如下命令，解压tar.gz包。

    **tar -zxvf Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz**

    若有软实时OS需求，请执行如下命令。

    **tar -zxvf Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz**

16. 执行如下命令，删除旧文件。

    **rm Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run**

    若有软实时OS需求，请执行如下命令。

    **rm Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run**

17. 执行如下命令，拷贝**步骤3.12**生成的run包到当前文件夹下。

    **cp** */opt*/**Ascend310B-source/Ascend-hdk-310b-npu-driver-soc_***<version>**_linux-aarch64-repack.run** */opt*/**tar**

    若有软实时OS需求，请执行如下命令。

    **cp** */opt*/**Ascend310B-source/Ascend-hdk-310b-npu-driver-soc_***<version>**_linux-rt-aarch64-repack.run** */opt*/**tar**

18. 执行如下命令，重新打包生成tar.gz。

    **tar -zcvf Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz** */opt*/**tar/***

    若有软实时OS需求，请执行如下命令。

    **tar -zcvf Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz** */opt*/**tar/***

**步骤4** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的制卡所需软件包“sdtool.tar.gz”和**前提条件**中获取的ISO与文件系统镜像上传至root用户属组目录下，例如“/home/ascend/mksd”。

**说明**

- 对于openEuler操作系统，需要将ISO与定制后的文件系统（请参考**A.6 定制文件系统**章节）一并上传至root用户属组目录下，例如“/home/ascend/mksd”。需注意“openEuler-22.03-LTS-aarch64-dvd.iso”镜像配套“Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img”使用，“openEuler-22.03-LTS-SP1-aarch64-dvd.iso”镜像配套“Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img”使用。
- 当前目录只允许存放一个版本的软件包。
- 对于Ubuntu 22.04系统，需要将“ubuntu-22.04-live-server-arm64.iso”镜像与定制后的Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img（请参考**A.6 定制文件系统**章节）一并上传至root用户属组目录下，例如“/home/ascend/mksd。”

**步骤5** 执行如下命令，进入sdtool.tar.gz制卡脚本工具包的上传目录，例如“/home/ascend/mksd”。

**cd** */home/ascend/mksd*

**步骤6** 在制卡脚本工具包的上传目录下（例如“/home/ascend/mksd”），执行如下命令解压sdtool.tar.gz制卡脚本工具包。

**tar -xzvf sdtool.tar.gz**

 版权所有 © 华为技术有限公司 

**步骤7** 执行如下命令，将解压后的文件拷贝至制卡脚本工具包所在目录，例如“/home/ascend/mksd”。

**cp -arf sdtool/* .**

**步骤8** 执行如下命令，进入“recovertool”目录。

**cd** */home/ascend/mksd***/recovertool**

**步骤9** 执行如下命令，拷贝构建依赖的驱动文件。

**cp** */opt***/Ascend310B-source/Ascend-hdk-310b-npu-driver-soc_***<*version*>_linux-aarch64-repack.run** */home/ascend/mksd*

**cp** */opt***/tar/Ascend-hdk-310b-npu-soc_***<*version*>_linux-aarch64.tar.gz** */home/ascend/mksd***/recovertool**

若有软实时OS需求，请执行如下命令。

**cp** */opt***/Ascend310B-source/Ascend-hdk-310b-npu-driver-soc_***<*version*>_linux-rt-aarch64-repack.run** */home/ascend/mksd*

**cp** /opt**/tar/Ascend-hdk-310b-npu-soc_***<*version*>_linux-rt-aarch64.tar.gz** */home/ascend/mksd***/recovertool**

**步骤10** （可选）更改网线和USB的默认IP地址。

1. 执行如下命令，进入制卡脚本工具包所在目录，例如“/home/ascend/mksd”。

   **cd** */home/ascend/mksd*

2. 执行如下命令，更改网线的默认IP地址。

   **vi make_sd_card.py**

   需分别修改“make_sd_card.py”脚本中的“USB_CARD_DEFAULT_IP”、“NETWORK_CARD_DEFAULT_IP”、“NETWORK_CIDR_PREFIX”、“NETWORK_DEFAULT_NETMASK”和“NETWORK_DEFAULT_GATEWAY”的参数值。

   **说明**

   - USB连接方式的默认IP地址为192.168.1.2，网线连接方式的默认IP地址192.168.0.2。
   - USB_CARD_DEFAULT_IP：表示USB连接方式的IP地址，例如192.168.3.5。
   - NETWORK_CARD_DEFAULT_IP：表示IP地址，例如192.168.1.8。
   - NETWORK_CIDR_PREFIX：表示子网掩码中bit是1的个数，例如CIDR PREFIX=24，对应子网掩码为255.255.255.0。
   - NETWORK_DEFAULT_NETMASK：表示子网掩码，例如255.255.255.0。
   - NETWORK_DEFAULT_GATEWAY：表示网关，例如192.168.1.1。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤11** （可选）若用户需要自定义分区的配置，请执行该步骤。

1. 在驱动包所在目录（例如“/home/ascend/mksd/”）执行如下命令，解压驱动run包，获取“restore_factory.sh”脚本。

   **bash Ascend-hdk-310b-npu-driver-soc_***<version>*_linux-aarch64-repack.run --noexec --extract=***./tmp***

   若有软实时OS需求，请执行如下命令。

   **bash Ascend-hdk-310b-npu-driver-soc_***<version>*_linux-rt-aarch64-repack.run --noexec --extract=***./tmp***

 版权所有 © 华为技术有限公司 

2. 拷贝“restore_factory.sh”脚本到“recovertool”目录。

   **cp** */home/ascend/mksd*/**tmp/scripts/restore_factory.sh** */home/ascend/mksd*/**recovertool/**

   **说明**

   tmp为临时解压目录，若不需要使用可以删除。

3. 修改分区配置。

   用户可自定义修改p2及之后的分区，p1不可修改。

   **vim ./recovertool/restore_factory.sh**

   修改如图6-6和图6-7红框中所示的配置。

**图 6-6** restore_factory.sh 脚本配置 1

```
#!/bin/bash
#
# Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.
#
# Entry script for restoring factory settings.
# This script is raised by user root, so can't not open other scripts here.
# Because if using other script here, they also has root authority,this takes secuity risk.
# Create: 2022-9-21

P_MAX_PN=8
#SIZE_GiB must be int
P2_SIZE_GiB=6
P3_SIZE_GiB=6
P4_SIZE_GiB=1
P5_SIZE_GiB=1
P6_SIZE_GiB=2
P7_SIZE_GiB=3
P8_SIZE_GiB=7
```

**图 6-7** restore_factory.sh 脚本配置 2

```
CREATE_NEW_PFLAG=no
function add_new_part()
{
    add_p_n=$P_MAX_PN
    tmp_add_size=0
    cat > mkpart_tmp.conf << EOF
#write partition size below, one size occupies one row

EOF

    while read line
    do
        if [[ "$line" -gt "0" ]];then
            add_p_n=$[add_p_n+1]
            add_off=$[sectorEnd8+tmp_add_size]
            add_end=$[line*1024+add_off]
            add_list+="mkpart p${add_p_n} ${add_off}MiB ${add_end}MiB "
            tmp_add_size=$[tmp_add_size+line*1024]
        fi
    done < ./mkpart_tmp.conf
    P_MAX_PN=$add_p_n

    rm -f ./mkpart_tmp.conf
}
```

a. 修改分区大小。

   直接修改各个分区PX_SIZE_GiB的值，如修改P8分区大小为20G，则P8_SIZE_GiB=20。

b. 增加新分区。

   修改CREATE_NEW_PFLAG=yes

 版权所有 © 华为技术有限公司 

找到add_new_part函数，在#write partition size below, one size occupies one row下面写入分区大小配置，增加几个分区就写几行。

如：增加三个分区，每个分区大小为20G

```
......
cat > mkpart_tmp.conf << EOF
#write partition size below, one size occupies one row
20
20
20
EOF
......
```

**须知**

- 为了保证文件系统的正常使用，修改p2~p8分区大小时不能小于原配置大小。
- 分区大小的赋值需要为大于0的整数，不支持配置为小数。
- 修改分区大小或增加新分区后，总大小不能超过M.2的最大容量。

  总大小=p2~p8分区大小+新增分区大小+2.3G，其中2.3G为烧写的镜像包大小，需要保留。

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤12** 在脚本工具包所在目录（例如“/home/ascend/mksd/”）执行如下命令，检查emmc-head工具是否可用。

**./emmc-head --help**

执行命令后有帮助回显信息则表示可用；若回显信息出现-bash: ./emmc-head: No such file or directory的错误提示，请参见**15.3 emmc-head工具不可用**。

```
Usages: emmc-head firmware_path boot_a_devname boot_b_devname [force_recover]
The following files must be contained in firmware_path:
Image, itrustee.img, dt.img, initrd.
boot_a_devname: A Partition boot device name, for example, eMMC:mmcblk0p2, SD:mmcblk1p2
boot_b_devname: B Partition boot device name, for example, eMMC:mmcblk0p3, SD:mmcblk1p3
force_recover: force recover flag.
        Example: /var/davinci/driver/emmc-head ./firmware /dev/mmcblk0p2 /dev/mmcblk0p3
```

**步骤13** 在脚本工具包所在目录（例如“/home/ascend/mksd/”）执行命令启动镜像脚本“make_sd_card.py”，制作烧写到M.2的系统镜像。

**python3 make_sd_card.py mkrecoverimg M.2**

显示如下类似信息，表示开始制卡，此处仅为示例，具体请以实际情况为准。

```
Begin to make Image package for restoring factory settings...
Please make sure you have installed dependency packages:
   apt-get install -y qemu-user-static binfmt-support gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
dosfstools parted kpartx
Please input Y: continue, other to install them:
```

**步骤14** 根据打印提示信息输入“Y”，等待约10分钟（具体等待时间以实际环境为准）。显示如下信息，表示制卡成功。

```
Step: Start to make M.2 SATA SSD or eMMC. It need some time, please wait...
Make Card successfully!
```

 版权所有 © 华为技术有限公司 

执行**ls -l**查看当前目录下生成的镜像文件为recoverfs-<os-type>-M.2.img，其中<os-type>表示目标镜像的操作系统类型，例如：Ubuntu或者openEuler。

**说明**

若出现“[ERROR] Making M.2 SATA SSD failed”类似回显信息，则代表制卡失败，建议用户按照日志错误排查，重新执行**python3 make_sd_card.py mkrecoverimg M.2**命令制作镜像。

**步骤15** 烧写镜像到M.2。

1. 将Atlas 200I A2 加速模块整机下电。
2. 将M.2插入Atlas 200I A2 加速模块整机并上电。
3. 使用WinSCP工具或其他工具上传镜像recoverfs-<os-type>-M.2.img（文件大小为2.3G）到Atlas 200I A2 加速模块整机（例如“/home/test”）。

   **说明**

   - 其中<os-type>表示目标镜像的操作系统类型，例如：Ubuntu或者openEuler。
   - 也可以通过转换器将M.2插入制作镜像的服务器，将镜像recoverfs-<os-type>-M.2.img（文件大小为2.3G）烧录至M.2后再将M.2插入Atlas 200I A2 加速模块整机。

4. 执行**fdisk -l**查询M.2盘符，如下述内容中粗体所示为/dev/sda，具体盘符请以实际环境为准。如查询不到盘符，请参考**7.1.3 SATA接口调测参考**解决。

   ```
   Disk /dev/sda: 111.79 GiB, 120034123776 bytes, 234441648 sectors
   Disk model: Netac SSD 120GB
   Units: sectors of 1 * 512 = 512 bytes
   Sector size (logical/physical): 512 bytes / 512 bytes
   I/O size (minimum/optimal): 512 bytes / 512 bytes
   Disklabel type: gpt
   Disk identifier: 86CA1C8B-D6CC-4761-AEAC-2A953681B164
   ```

5. 执行**mkfs.ext4 -F** /dev/sda命令格式化M.2，出现如下类似关键回显信息表明格式化成功。

   ```
   Writing superblocks and filesystem accounting information: done
   ```

6. 在镜像recoverfs-<os-type>-M.2.img上传路径下（例如“/home/test”），执行**dd if=recoverfs-<os-type>-M.2.img of=**/dev/sda **status=progress**，烧写镜像到M.2，等待约3分钟，出现如下类似信息则代表烧写成功，此处仅为示例，具体烧写花费时间和烧写速率以实际情况为准。

   ```
   4751360+0 records in
   4751360+0 records out
   2432696320 bytes (2.4 GB, 2.3 GiB) copied, 45.217 s, 53.8 MB/s
   ```

   **说明**

   若出现写入数据量不等于4751360，则代表烧写失败，建议重新执行**dd if=recoverfs-<os-type>-M.2.img of=**/dev/sda **status=progress**烧写镜像。

7. 烧写镜像后会出现1.5G的golden分区，可以通过**mount**命令挂载后查看分区内容。

   **mount** */dev/sda1* **/mnt**

   **ll /mnt**

   **说明**

   查看完文件请执行**umount /mnt**，防止文件损坏。

**----结束**

**说明**

若M.2插入服务器，则烧写镜像到M.2后，需将M.2安装在Atlas 200I A2 加速模块整机上。

 版权所有 © 华为技术有限公司 

### 切换启动介质至 M.2 设备

**步骤1** 将启动介质修改为M.2 SATA SSD硬盘，具体请参考《**Atlas 200I A2 加速模块 硬件开发指南**》中的“启动方式选择”章节。

**步骤2** Atlas 200I A2 加速模块整机通过插拔电源插头进行强制上下电。

**步骤3** 使能SSH服务。

制作Ubuntu 22.04系统的镜像，在烧录镜像启动后如需使用root用户登录，需要执行该步骤。openEuler 22.03系统为默认开启允许root远程登录，请跳过此步骤。

1. 使用HwHiAiUser登录Atlas 200I A2 加速模块整机OS。详细信息请参见**14.1 使用PuTTY登录设备（网口方式）**。

   默认密码请参见《**Atlas 系列硬件产品 账户清单**》。

2. 执行如下命令，切换到root用户。

   **su root**

3. 执行如下命令，修改“PermitRootLogin”选项为“yes”。

   **vi /etc/ssh/sshd_config**

   ```
   # Authentication:

   #LoginGraceTime 2m
   PermitRootLogin yes
   #StrictModes yes
   #MaxAuthTries 6
   #MaxSessions 10
   ```

4. 执行如下命令，重启SSH服务。

   **/etc/init.d/ssh restart**

5. 打开新的连接终端，重新使用SSH远程连接。

**步骤4** 登录Atlas 200I A2 加速模块整机OS。详细信息请参见**14.1 使用PuTTY登录设备（网口方式）**。

- 首次登录时，先使用HwHiAiUser登录，修改HwHiAiUser用户的初始密码，再执行**su - root**切换为root用户，并修改root用户的初始密码。
- 非首次登录，可使用HwHiAiUser或root用户登录。

**说明**

默认密码请参见《**Atlas 系列硬件产品 账户清单**》。

**步骤5** （可选）自定义同步主备文件系统关键数据，详细请参考**10.1 主备分区介绍**。

**----结束**

## 6.3 eMMC 制作和启动系统镜像

用于制作Atlas 200I A2 加速模块从eMMC启动需要的镜像，用户可基于业务需要选择是否开启快速启动配置并制作快速启动所需要的镜像。该镜像裁剪了部分系统服务，保证eMMC作为存储介质启动时，直连网络15秒内通信正常。


版权所有 © 华为技术有限公司

## 前提条件

已获取表6-4中对应的ISO和文件系统镜像。

**表 6-4** 镜像获取

| 镜像 | 目标OS | 支持的版本 | 获取方式 | 说明 |
| --- | --- | --- | --- | --- |
| ISO | Ubuntu | 22.04 | 单击**ubuntu 22.04 ISO镜像**下载。 | - |
| | openEuler | openEuler-22.03-LTS | • Windows环境<br>– openEuler-22.03-LTS：单击**openEuler-22.03-LTS ISO镜像**下载。<br>– openEuler-22.03-LTS-SP1：单击**openEuler-22.03-LTS-SP1 ISO镜像**下载。<br>• Linux环境<br>同Windows方式获取iso，执行命令**wget** *iso镜像链接*。 | 若下载ISO速度过慢，可参考如下方式获取。<br>1. 访问**镜像链接**。<br>2. 客户自行选择相近的Location。<br>3. 下载OS镜像。<br>• openEuler-22.03-LTS：进入“openEuler-22.03-LTS/ISO/aarch64”目录，下载OS镜像“openEuler-22.03-LTS-aarch64-dvd.iso”。<br>• openEuler-22.03-LTS-SP1：进入“openEuler-22.03-LTS-SP1/ISO/aarch64”目录，下载OS镜像“openEuler-22.03-LTS-SP1-aarch64-dvd.iso”。 |
| | | openEuler-22.03-LTS-SP1[a] | | |
| 文件系统 | Ubuntu | 22.04 | 访问**Atlas 200I A2 加速模块软件包**，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img。 | 若用户需要自定义文件系统，请参见**A.6 定制文件系统**。 |


版权所有 © 华为技术有限公司

| 镜像 | 目标OS | 支持的版本 | 获取方式 | 说明 |
| --- | --- | --- | --- | --- |
| | openEuler | openEuler-22.03-LTS | 访问**Atlas 200I A2 加速模块软件包**，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img或Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img。 | |
| | | openEuler-22.03-LTS-SP1[a] | | |
| a：若有软实时OS需求时，需使用的OS为openEuler-22.03-LTS-SP1，否则需使用openEuler-22.03-LTS或Ubuntu。 | | | | |

**说明**

社区版软件不需要申请下载权限可以直接下载，但软件不能用于商业用途， 详细请参见**社区华为软件许可协议**；如果您需要用于商业用途，参见**A.6 定制文件系统**的操作步骤进行制作。因此如果您下载软件仅用于调试验证，推荐下载社区版软件。

## 制作镜像

**步骤1** 将U盘（使用16G及以上）插入Linux服务器。

**步骤2** 登录Linux服务器。

**步骤3** 执行如下命令，切换至root用户。

```
su - root
```

**步骤4** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的制卡所需软件包sdtool.tar.gz、驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run（若有软实时OS需求，不能直接使用驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run，该驱动包需要用户自行编译生成内核Image文件后进行重构，用户使用重构后的驱动包，具体操作请参见**6.1.2.3 重构驱动run包**）和**前提条件**中获取的ISO与文件系统镜像上传至root用户属组目录下，例如“/home/ascend/mksd”。工具使用方法请参见**14.3 使用WinSCP传输文件**。

**说明**

- 对于openEuler操作系统，需要将ISO与定制后的文件系统（请参考**A.6 定制文件系统**章节）一并上传至root用户属组目录下，例如“/home/ascend/mksd”。需注意“openEuler-22.03-LTS-aarch64-dvd.iso”镜像配套“Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img”使用，“openEuler-22.03-LTS-SP1-aarch64-dvd.iso”镜像配套“Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img”使用。
- 当前目录只允许存放一个版本的软件包。
- 对于Ubuntu 22.04系统，需要将“ubuntu-22.04-live-server-arm64.iso”镜像与定制后的Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img（请参考**A.6 定制文件系统**章节）一并上传至root用户属组目录下，例如“/home/ascend/mksd”。


版权所有 © 华为技术有限公司

**步骤5** 执行如下命令，进入sdtool.tar.gz制卡脚本工具包的上传目录，例如“/home/ascend/mksd”。

**cd** */home/ascend/mksd*

**步骤6** 执行如下命令，解压sdtool.tar.gz制卡脚本工具包。

**tar -xzvf sdtool.tar.gz**

**步骤7** 执行如下命令，将解压后的文件拷贝至制卡脚本工具包所在目录，例如“/home/ascend/mksd”。

**cp -arf sdtool/* ./**

**步骤8** 执行如下命令，进入“recovertool”目录。

**cd ./recovertool**

**步骤9** 使用WinSCP工具或其他工具，根据实际需求将对应软件包上传至“recovertool”目录下，制卡所需软件包请参见**表4-1**。工具使用方法请参见**14.3 使用WinSCP传输文件**。

- 非软实时OS需求，将“Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz”软件包上传至“recovertool”目录下。
- 软实时OS需求，将“Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz”软件包上传至“recovertool”目录下。

  上传“Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz”软件包后，需执行如下命令进行软件包重构。

  **mkdir rt && tar -zxvf Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz -C ./rt && rm -rf rt/Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run && cp ../Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run rt/ && cd rt && tar zcvf Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz ./* && cp Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz ../ && cd .. && rm -rf rt**

  **说明**

  *<version>*表示NPU版本号，具体请根据实际情况进行替换。

**步骤10** （可选）更改网线和USB的默认IP地址。

1. 执行如下命令，进入制卡脚本工具包所在目录，例如“/home/ascend/mksd”。

   **cd** */home/ascend/mksd*

2. 执行如下命令，更改网线和USB的默认IP地址。

   **vim make_sd_card.py**

   需分别修改“make_sd_card.py”脚本中的“USB_CARD_DEFAULT_IP”、“NETWORK_CARD_DEFAULT_IP”、“NETWORK_CIDR_PREFIX”、“NETWORK_DEFAULT_NETMASK”和“NETWORK_DEFAULT_GATEWAY”的参数值。

 版权所有 © 华为技术有限公司 

**说明**

- USB连接方式的默认IP地址为192.168.1.2，网线连接方式的默认IP地址192.168.0.2。
- USB_CARD_DEFAULT_IP：表示USB连接方式的IP地址。例如192.168.3.5。
- NETWORK_CARD_DEFAULT_IP：表示网线连接方式的IP地址，例如192.168.2.8。
- NETWORK_CIDR_PREFIX：表示网线连接方式的子网掩码中bit是1的个数，例如CIDR PREFIX=24，对应子网掩码为255.255.255.0。
- NETWORK_DEFAULT_NETMASK：表示网线连接方式的子网掩码，例如255.255.255.0。
- NETWORK_DEFAULT_GATEWAY：表示网线连接方式的网关，例如192.168.2.1。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤11** （可选）执行如下命令，修改“mksd.conf”中的“FAST_BOOT_FLAG”的值为“on”，打开快速启动配置。

**vim** */home/ascend/mksd*/**mksd.conf**

```
FS_BACKUP_FLAG=off
FAST_BOOT_FLAG=on
ROOT_PART_SIZE=6144
LOG_PART_SIZE=1024
HOME_DATA_SIZE=1024
```

配置完成后，按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

**:wq!**

**说明**

启动镜像默认使能了主备分区，开启了主备同步功能。若开启快速启动，会导致主备同步功能失效，用户需根据实际使用场景配置。主备分区介绍可参考**10.1 主备分区介绍**。

**步骤12** （可选）若用户需要自定义分区的配置，请执行该步骤。

1. 在驱动包所在目录（例如“/home/ascend/mksd/”）执行如下命令，解压驱动run包，获取“restore_factory.sh”脚本。

   **bash Ascend-hdk-310b-npu-driver-soc**_*<version>*_**linux-aarch64.run --noexec --extract=***./tmp*

   若有软实时OS需求，请执行如下命令。

   **bash Ascend-hdk-310b-npu-driver-soc**_*<version>*_**linux-rt-aarch64-repack.run --noexec --extract=***./tmp*

2. 拷贝“restore_factory.sh”脚本到“recovertool”目录。

   **cp ./tmp/scripts/restore_factory.sh ./recovertool/**

   **说明**

   tmp为临时解压目录，若不使用时可以删除。

3. 修改分区配置。

   用户可自定义修改p2及之后的分区，p1不可修改。

   **vim ./recovertool/restore_factory.sh**

   修改如**图6-8**和**图6-9**红框中所示的配置。

 版权所有 © 华为技术有限公司 

**图 6-8** restore_factory.sh 脚本配置 1

```
#!/bin/bash
#
# Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.
#
# Entry script for restoring factory settings.
# This script is raised by user root, so can't not open other scripts here.
# Because if using other script here, they also has root authority,this takes secuity risk.
# Create: 2022-9-21

P_MAX_PN=8
#SIZE_GiB must be int
P2_SIZE_GiB=6
P3_SIZE_GiB=6
P4_SIZE_GiB=1
P5_SIZE_GiB=1
P6_SIZE_GiB=2
P7_SIZE_GiB=3
P8_SIZE_GiB=7
```

**图 6-9** restore_factory.sh 脚本配置 2

```
CREATE_NEW_PFLAG=no
function add_new_part()
{
    add_p_n=$P_MAX_PN
    tmp_add_size=0
    cat > mkpart_tmp.conf << EOF
#write partition size below, one size occupies one row

EOF

    while read line
    do
        if [[ "$line" -gt "0" ]];then
            add_p_n=$[add_p_n+1]
            add_off=$[sectorEnd8+tmp_add_size]
            add_end=$[line*1024+add_off]
            add_list+="mkpart p${add_p_n} ${add_off}MiB ${add_end}MiB "
            tmp_add_size=$[tmp_add_size+line*1024]
        fi
    done < ./mkpart_tmp.conf
    P_MAX_PN=$add_p_n

    rm -f ./mkpart_tmp.conf
}
```

a. 修改分区大小。

   直接修改各个分区PX_SIZE_GiB的值，如修改P8分区大小为20G，则P8_SIZE_GiB=20。

b. 增加新分区。

   修改CREATE_NEW_PFLAG=yes

   找到add_new_part函数，在#write partition size below, one size occupies one row下面写入分区大小配置，增加几个分区就写几行。

   如：增加三个分区，每个分区大小为20G

```
......
cat > mkpart_tmp.conf << EOF
#write partition size below, one size occupies one row
20
20
20
EOF
......
```

 版权所有 © 华为技术有限公司 

**须知**

- 为了保证文件系统的正常使用，修改p2~p8分区大小时不能小于原配置大小。
- 分区大小的赋值需要为大于0的整数，不支持配置为小数。
- 修改分区大小或增加新分区后，总大小不能超过eMMC的最大容量。
  总大小=p2~p8分区大小+新增分区大小+2.3G，其中2.3G为烧写的镜像包大小，需要保留。

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤13** 在脚本工具包所在目录（例如“/home/ascend/mksd/”）执行如下命令，检查emmc-head工具是否可用。

**./emmc-head --help**

```
Usages: emmc-head firmware_path boot_a_devname boot_b_devname [force_recover]
The following files must be contained in firmware_path:
Image, itrustee.img, dt.img, initrd.
boot_a_devname: A Partition boot device name, for example, eMMC:mmcblk0p2, SD:mmcblk1p2
boot_b_devname: B Partition boot device name, for example, eMMC:mmcblk0p3, SD:mmcblk1p3
force_recover: force recover flag.
        Example: /var/davinci/driver/emmc-head ./firmware /dev/mmcblk0p2 /dev/mmcblk0p3
```

**说明**

- emmc-head工具用于更新镜像启动信息。
- 执行命令后有帮助回显信息则表示可用，若回显信息出现-bash: ./emmc-head: No such file or directory的错误提示，请参见**15.3 emmc-head工具不可用**。

**步骤14** 执行如下命令，启动镜像脚本“make_sd_card.py”，制作烧写到eMMC的恢复出厂镜像。

**python3 make_sd_card.py mkrecoverimg eMMC**

显示如下类似信息，表示开始制卡。

```
Begin to make Image package for restoring factory settings...
FS_BACKUP_FLAG off
FAST_BOOT_FLAG on
ROOT_PART_SIZE 6144(MB)
LOG_PART_SIZE 1024(MB)
HOME_DATA_SIZE 1024(MB)
preconfig success.
Please make sure you have installed dependency packages:
   apt-get install -y qemu-user-static binfmt-support gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
dosfstools parted kpartx
Please input Y: continue, other to install them:
```

**步骤15** 根据打印提示信息输入“Y”。

显示如下类似信息，表示制卡成功。

```
Make Card successfully!
```

执行**ls -l**查看当前目录下生成的镜像文件为“recoverfs-*<os-type>*-eMMC.img”，其中<os-type>表示目标镜像的操作系统类型。

**步骤16** 替换软实时OS需求的驱动包、ISO和文件系统镜像。

 版权所有 © 华为技术有限公司 

**说明**

仅当有软实时OS需求时需要执行此操作。

1. 执行如下命令，进入驱动包、ISO和文件系统所在目录，例如“/home/ascend/mksd”。

   **cd** */home/ascend/mksd*

2. 执行如下命令删除驱动包、ISO和文件系统镜像。

   **rm -rf Ascend-hdk-310b-npu-driver-soc_**<*version*>**_linux-rt-aarch64-repack.run**

   **rm -rf openEuler-22.03-LTS-SP1-aarch64-dvd.iso**

   **rm -rf Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img**

3. 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的驱动包Ascend-hdk-310b-npu-driver-soc_<*version*>_linux-aarch64.run和**前提条件**中获取的openEuler-22.03-LTS ISO（openEuler-22.03-LTS-aarch64-dvd.iso）与openEuler-22.03-LTS文件系统（Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img）上传至当前目录下，例如“/home/ascend/mksd”。

**步骤17** 执行如下命令，查找USB设备名称，例如“/dev/sdc”。

**fdisk -l**

类似回显信息如下。

```
Disk /dev/sdc: 29.14 GiB, 31293702144 bytes, 61120512 sectors
Disk model: STORAGE DEVICE
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: E66417A5-1553-4BA1-A3E5-939DCE8C52AB
```

其中“Disk model: STORAGE DEVICE”为USB设备名称，不同的USB设备显示的名称不一样，此处以“STORAGE DEVICE”为例。

**步骤18** 执行如下命令，启动镜像脚本“make_sd_card.py”，制作U盘。

此处以“/dev/sdc”USB设备名称为例。

**python3 make_sd_card.py local** */dev/sdc* **USB**

显示如下类似信息，表示开始制卡。

```
root@ubuntu:/home/ascend/mksd# python3 make_sd_card.py local /dev/sdc USB
Begin to make USB...
FS_BACKUP_FLAG off
FAST_BOOT_FLAG on
ROOT_PART_SIZE 6144(MB)
LOG_PART_SIZE 1024(MB)
HOME_DATA_SIZE 1024(MB)
preconfig success.
Please make sure you have installed dependency packages:
        apt-get install -y qemu-user-static binfmt-support gcc-aarch64-linux-gnu        g++-aarch64-linux-gnu
dosfstools parted kpartx
Please input Y: continue, other to install them:Y
```

**步骤19** 根据打印提示信息输入“Y”，等待约15分钟（具体等待时间以实际环境为准）。

```
Make Card successfully!
```

显示如上类似回显信息，表示制卡成功。


版权所有 © 华为技术有限公司

**步骤20** （可选）执行如下命令查看分区信息。

**fdisk -l**

```
Device       Start      End  Sectors  Size Type
/dev/sdc1   1605633  1810432   204800  100M Linux filesystem
/dev/sdc2   1810433 14393344 12582912    6G Linux filesystem
/dev/sdc3  14393345 14598144   204800  100M Linux filesystem
/dev/sdc4  14598145 16695296  2097152    1G Linux filesystem
/dev/sdc5  16695297 18792448  2097152    1G Linux filesystem
```

**步骤21** （可选）为避免出现内存不足导致设备OOM异常挂死问题，请参见**15.6 运行程序超内存导致设备异常挂死，设备能ping通，但SSH无法远程连接**解决。

**----结束**

## 烧写镜像和启动设备

**步骤1** 确认Atlas 200I A2 加速模块金手指“S72”管脚已开发为update按键，具体开发请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“连接器功能”章节。

**步骤2** 将U盘插入Atlas 200I A2 加速模块底板。

**步骤3** 准备串口线，连接PC和Atlas 200I A2 加速模块底板串口。

**步骤4** 上电Atlas 200I A2 加速模块。

**步骤5** 通过串口登录Atlas 200I A2 加速模块，用于查看启动日志信息，具体请参见**14.2 使用PuTTY登录设备（串口方式）**。

**步骤6** 上电Atlas 200I A2 加速模块，同时按下update按键直到进入U盘启动模式。

串口打印显示类似如下信息表明进入U盘启动模式。

```
Usb Boot Entry.
Success open USB device: 2
```

**步骤7** 等待eMMC烧写。

串口打印显示如下类似信息表示eMMC正在烧写。

```
1229762560 bytes (1.2 GB, 1.1 GiB) copied, 86 s, 14.3 MB/s
```

烧写完成Atlas 200I A2 加速模块自动重启。

**步骤8** 登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

**说明**

- Atlas 200I A2 加速模块（使用非Atlas 200I DK A2 开发者套件底板）USB接口登录的默认IP为192.168.1.2，Atlas 200I DK A2 开发者套件USB接口登录的默认IP为192.168.0.2。
- 若Atlas 200I A2 加速模块上电后网络可以ping通，但无法通过SSH登录，请参见**15.1 使用root用户SSH登录服务器出现Access Denied错误**解决。

**步骤9** （可选）关闭apt自动软件更新。若安装的OS为Ubuntu 22.04，由于apt自动软件更新会消耗很多资源，建议关闭apt自动软件更新。

1. 执行如下命令，切换至root用户。

   **su - root**

2. 执行以下命令，打开20auto-upgrades文件。

   **vim /etc/apt/apt.conf.d/20auto-upgrades**

 版权所有 © 华为技术有限公司 

将下列参数的值修改为0。

```
APT::Periodic::Update-Package-Lists "0";
APT::Periodic::Unattended-Upgrade "0";
APT::Periodic::Download-Upgradeable-Packages "0";
APT::Periodic::AutocleanInterval "0";
```

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** 执行如下命令，查询设备健康状态。

**npu-smi info -t health -i 0**

```
NPU ID                  : 0
Chip Count              : 1

Chip ID                 : 0
Health                  : OK
```

Health显示OK，表示正常。

**----结束**

 版权所有 © 华为技术有限公司 

# 7 接口外设调测参考

7.1 SerDes（PCIe/SATA/USB/ETH）
7.2 GPIO
7.3 UART
7.4 I2C
7.5 PWM
7.6 CAN
7.7 MDIO
7.8 音频
7.9 视频
7.10 SPI
7.11 I2S

## 7.1 SerDes（PCIe/SATA/USB/ETH）

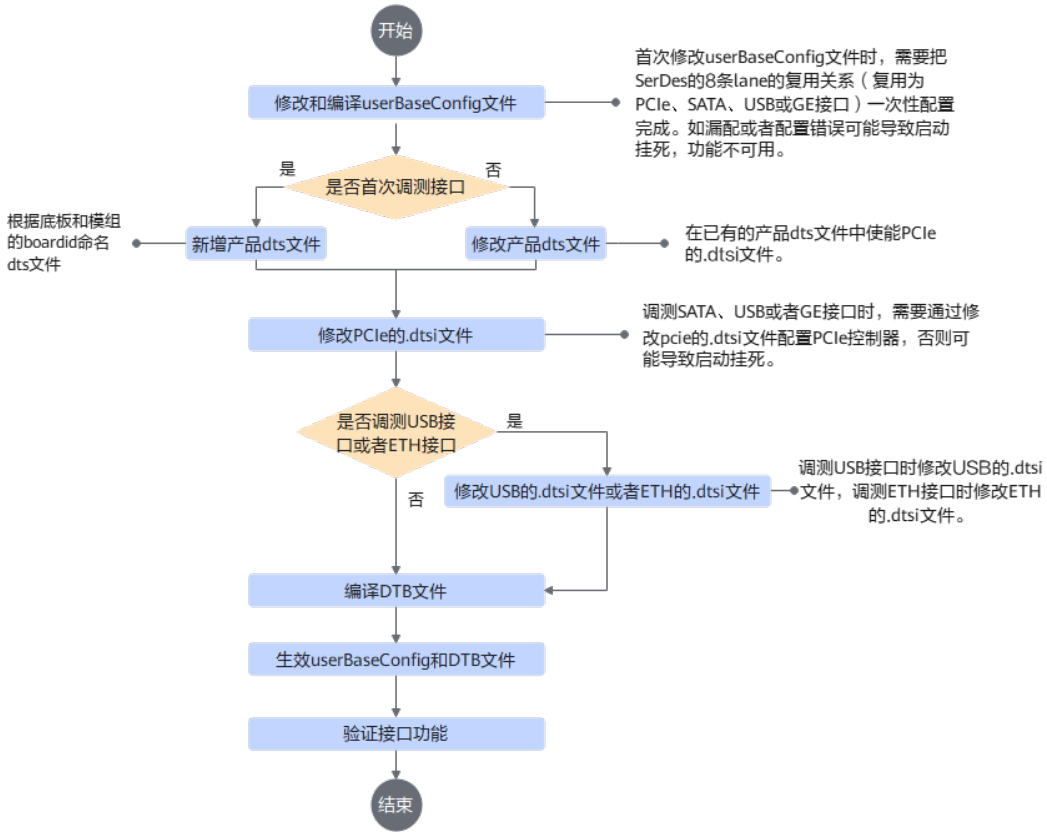
### 7.1.1 调测流程

SerDes接口提供8条lane，可通过配置userBaseConfig中lane属性将SerDes复用为PCIe、SATA、USB或者ETH接口。

Atlas 200I A2 加速模块出厂自带固件（userBaseConfig.bin文件），需要用户根据底板设计修改对应的user_base_config.xml文件和DTB文件，具体流程如**图7-1**所示。


版权所有 © 华为技术有限公司

**图 7-1** SerDes 调测流程




## 7.1.2 PCIe 接口调测参考

本章节以其中一个不同于Atlas 200I DK A2 开发者套件、Atlas 500 A2 智能小站的用户底板设计为例，根据底板的设计来配置对应PCIe接口的userBaseConfig文件和DTB文件。

### 前提条件

已获取设备boardid，本章节以33150为例。具体请参见**2 用户必读**。

用户底板的adc_board_id为33，Atlas 200I A2 加速模块的算力为20T（adc_board_id为150），用户整机（Atlas 200I A2 加速模块+底板）boardid为33150（对应十六进制817e）。

### 注意事项

- 如果已经调测过其他接口，直接在已修改好的产品dts和对应的.dtsi文件中，修改对应内容。
- 由于调测其他接口可能会在产品dts中注释掉对应PCIe的.dtsi文件，所以需要将产品dts文件中对应PCIe的.dtsi文件取消注释，使能功能。

### 修改和编译 userBaseConfig 和 DTB 文件

本操作以首次调测接口为例介绍。


版权所有 © 华为技术有限公司

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 修改“user_base_config.xml”配置文件。

1. 执行如下命令，进入配置文件所在目录。

   **cd config/adaptive/adaptive_config/**

2. 执行如下命令，打开“user_base_config.xml”文件。

   **vim user_base_config.xml**

3. 在“user_base_config.xml”中配置用户整机对应的**board_id**、**chip0_strategy**、**max_power**和**serdes_info**。具体参数含义请参见**表7-1**和**表7-2**。

```
<item name="product_strategy_info"><!--产品板策略ID-->
<!--33150 对应十六进制817e-->
<param board_id ="817e" phy_num = "1" chip0_strategy="0" max_power="0"  equal="0"/>
</item>

<item name="evb_strategy_info">
</item>

<item name="strategy_info_array"><!--启动策略矩阵-->
<subitem subname="sub_strategy0">
<module class="serdes">
<submodule subclass="serdes_info">
<param lan_index="0" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="2"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="1" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0"     port_index = "0"/>
<param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="4" serdes_type="ETH" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff"  align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="5" serdes_type="ETH"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff" align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
<param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
</submodule>
</module>
```

   - 第一处加粗字体修改为用户产品的实际boardid以及算力等级，各参数含义如下。

 版权所有 © 华为技术有限公司 

**表 7-1** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| board_id | 如33150对应十六进制817e。 | 十六进制的boardid。<br>boardid为用户实际使用底板的adc_board_id与Atlas 200I A2 加速模块的adc_board_id的组合值。 |
| phy_num | 1。 | 多p系统才会使用，固定为1。 |
| chip0_strategy | 0~n。 | 策略索引号。对应代码中的sub_strategy0和sub_strategy1。 |
| max_power | ▪ 0表示20T。<br>▪ 2表示8T。 | 最大算力档位。 |
| euqal | 0-n。 | 匹配DTB时等效board_id使用，保持默认配置0即可。 |

- 第二处加粗字体修改为用户产品的实际SerDes配置，具体规则如下。
  - PCIe控制器0（lane0~lane1）支持RC/EP模式，其他PCIe控制器只支持RC模式。
  - Macro0（lane0~lane3）中若存在PCIe和其他协议（SATA）共存，则PCIe只能支持到PCIE_GEN2；若只存在PCIe协议，则PCIe支持到PCIE_GEN3。
  - PCIe支持降lane应用，如PCIe x4降lane到PCIe x2/x1。
  - Macro0（lane0~lane3）支持PCIE和SATA协议，Macro1（lane4~lane7）支持PCIe、USB、ETH协议，每个Macro同时最多只能配置两种协议。
  - SATA只能在Macro0（lane0~lane3）使用。
  - USB控制器0（lane4）只支持HOST，USB控制器1~USB控制器3（lane5~lane7）支持HOST+DEVICE。
  - USB只能在Macro1（lane4~lane7）使用。
  - 若为系统直出的USB 2.0，则无需配置SerDes。

  各参数含义如下。

 版权所有 © 华为技术有限公司 

**表 7-2** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| lan_index | 0~7。 | lane编号，Atlas 200I A2 加速模块一共有8条serdes lane。 |
| serdes_type | Atlas 200I A2 加速模块支持PCIE、SATA、USB、ETH。 | 该条lane使用的协议。 |
| ssc_enable | ■ 0：否。<br>■ 1：是。 | 是否开展频[a]，只有用作PCIe和SATA时才能打开。 |
| polarity_tx | ■ 0：否。<br>■ 1：是。 | tx极性是否翻转。 |
| polarity_rx | ■ 0：否。<br>■ 1：是。 | rx极性是否翻转。 |
| lan_order | ■ 0：否。<br>■ 1：是。 | lane序是否反转。只有用作ETH时需要修改，具体请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“以太网接口”章节。 |
| bandwidth | ■ 1表示x1。<br>■ 2表示x2。<br>■ 4表示x4。<br>■ ff表示不配置。 | 用作PCIe时，表示位宽x4/x2/x1，其他协议不使用。<br>■ x4：使用四条lane。<br>■ x2：使用两条lane。<br>■ x1：使用一条lane。 |
| align_mode | 0。 | 该lane是master还是slave，Atlas 200I A2 加速模块不使用该参数。 |
| frequency | ■ PCIE_GEN2。<br>■ PCIE_GEN3。<br>■ 1GE。<br>■ 2.5GE。<br>■ USB3.0。<br>■ SATA3.0。 | 使用的频点[b]，主要用来区分PCIE GEN2/3，和ETH的1GE/2.5GE。 |


版权所有 © 华为技术有限公司

| param | 配置值 | 说明 |
| --- | --- | --- |
| port_index | 0~n。 | 用作ETH时，表示lane的网口端口号，其他协议不使用。不使用时设置为0。 |
| a：时钟展频通过频率调制的手段将集中在窄频带范围内的能量分散到设定的宽频带范围，通过降低时钟在基频和奇次谐波频率的幅度（能量），达到降低系统电磁辐射峰值的目的。<br>b：PCIe的频点是指数据在总线上传输的速率，也称为数据传输速度。 | | |

**须知**

弹性配置中的SerDes复用关系参数必须与实际硬件单板保持一致，SerDes错误配置可能导致启动挂死，功能不可用。

根据表7-3中用户产品的实际serdes配置，为使能接口功能，user_base_config.xml文件中serdes_info配置的示例如下。

**表 7-3** SerDes 接口复用关系

| lane编号 | 协议 | 是否开展频 | tx极性是否翻转 | rx极性是否翻转 | lane序是否反转 | 位宽 | 频点 |
| --- | --- | --- | --- | --- | --- | --- | --- |
| lane0 | PCIE | 否 | 否 | 否 | 否 | x2 | PCIE_GEN2 |
| lane1 | PCIE | 否 | 否 | 否 | 否 | 不配置[a] | PCIE_GEN2 |
| lane2 | SATA | 是 | 否 | 否 | 否 | 不配置 | SATA3.0 |
| lane3 | PCIE | 否 | 否 | 否 | 否 | x1 | PCIE_GEN2 |
| lane4 | ETH | 否 | 否 | 否 | 是 | 不配置 | 1GE |
| lane5 | ETH | 否 | 否 | 否 | 是 | 不配置 | 1GE |
| lane6 | USB | 否 | 否 | 否 | 否 | 不配置 | USB3.0 |
| lane7 | USB | 否 | 否 | 否 | 否 | 不配置 | USB3.0 |
| a：lane0的PCIe位宽为x2，需要使用两条lane，所以lane1不设置位宽。 | | | | | | | |


版权所有 © 华为技术有限公司

user_base_config.xml文件中serdes_info配置如下。

```
<submodule subclass="serdes_info">
<param lan_index="0" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="2"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="1" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0"   port_index = "0"/>
<param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="4" serdes_type="ETH" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff"  align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="5" serdes_type="ETH"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff" align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
<param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
</submodule>
```

**说明**

位宽只能配置为x4/x2/x1。

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤8** 编译userBaseConfig文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译userBaseConfig文件。

   **bash build.sh usrBaseConfig**

   出现如下回显，表示编译内核userBaseConfig文件成功。

   ```
   generate /opt/Ascend310B-source/output/userBaseConfig.bin success!
   sign /opt/Ascend310B-source/output/userBaseConfig.bin success!
   ```

   **说明**

   编译后的“userBaseConfig.bin”文件会自动存放于“Ascend310B-source/output”目录下。

**步骤9** 新增并修改整机dts文件。

以用户整机的boardid为33150为例，故dts文件名称为“hi1910B-asic-M150-B33.dts”。具体请参见**DTB文件**。

1. 执行如下命令，进入dts文件目录。

   **cd** */opt*/**Ascend310B-source/dtb/dts/hi1910b/hi1910BL**

2. 执行如下命令，新增适配boardid33150的dts文件。

   **cp hi1910B-asic-M150-B50.dts** *hi1910B-asic-M150-B33.dts*

   将“hi1910B-asic-M150-B50.dts”另存为boardid33150的dts文件“hi1910B-asic-M150-B33.dts”。

3. 执行以下命令修改dts文件，配置用户产品的实际整机boardid，确认所使用的PCIe的.dtsi文件及路径，具体内容如下加粗字体所示。

   **vim hi1910B-asic-M150-B33.dts**

   ```
   compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
   hisi,boardid = <0x0 0x3 0x3 0x1 0x5 0x0>;  //修改为实际整机boardid
   #address-cells = <0x2>;
   #size-cells = <0x2>;
   ```

 版权所有 © 华为技术有限公司 

```
interrupt-parent = <0x1>;
model = "Hisilicon PhosphorHi1910B evb";

/include/ "base/hi1910B-asic-kernel-rc.dtsi"
/include/ "base/hi1910B-commmon-smmu.dtsi"
/include/ "base/hi1910B-mpam.dtsi"
/include/ "base/hi1910B-lp-pm.dtsi"
/include/ "base/hi1910B-lp-devmng.dtsi"
/include/ "base/hi1910B-mbigen-ao.dtsi"
/include/ "base/hi1910B-gpio.dtsi"
/include/ "base/hi1910B-ipcdrv.dtsi"
/include/ "base/hi1910B-ts-drv.dtsi"
/include/ "base/hi1910B-tsdrv.dtsi"
/include/ "base/hi1910B-network-M150-B50.dtsi"
/include/ "base/hi1910B-hidvpp.dtsi"
/include/ "base/hi1910B-hdmi.dtsi"
/include/ "product/hi1910B-audio.dtsi"
/include/ "base/hi1910B-mipitx.dtsi"
/include/ "base/hi1910B-vdp.dtsi"
/include/ "base/hi1910B-isp.dtsi"
/include/ "base/hi1910B-hilink.dtsi"
/include/ "base/hi1910B-mntn-milanr3.dtsi"
/include/ "base/hi1910B-gpu.dtsi"
/include/ "base/hi1910B-ctl.dtsi"
/include/ "base/hi1910B-itrustee.dtsi"
/include/ "base/hi1910B-sata.dtsi"
/include/ "product/hi1910B-pcie-rc-M150-B50.dtsi" //确认所使用的PCIe的.dtsi文件及路径
/include/ "product/hi1910B-mdio-M150-B50.dtsi"
/include/ "product/hi1910B-pinctrl-M150-B50.dtsi"
/include/ "product/hi1910B-can-M150-B50.dtsi"
/include/ "product/hi1910B-hiusbc-B50.dtsi"
/include/ "product/hi1910B-emmc-M150-B50.dtsi"
/include/ "product/hi1910B-i2c-M150-B50.dtsi"
/include/ "product/hi1910B-spi-M150-B50.dtsi"
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** 修改PCIe的.dtsi文件，配置PCIe控制器。

1. 执行如下命令进入“product”目录，并修改PCIe的.dtsi文件。

   **cd product**

   **vim** *hi1910B-pcie-rc-M150-B50.dtsi*

   根据**步骤7.3**中配置的**serdes_info**选择需要开放的PCIe控制器，lan_index与PCIe控制器的对应关系如下。

| lan_index | 控制器 |
| --- | --- |
| lane0~lane1 | PCIe控制器0 |
| lane2~lane3 | PCIe控制器1 |
| lane4~lane5 | PCIe控制器2 |
| lane6~lane7 | PCIe控制器3 |

   以**步骤7.3**中**serdes_info**配置为例，共使用了PCIe控制器0、PCIe控制器1，则在PCIe的.dtsi文件中保留PCIe控制器0、PCIe控制器1两个控制器的配置，将PCIe控制器2、PCIe控制器3的配置代码注释即可，其中**/***和***/**代表注释代码段，具体修改如下所示。

```
pcie@0x820000000 {
compatible = "pci-host-ecam-generic";
// ECAM 0x820000000 pci host bridge0 //
```

 版权所有 © 华为技术有限公司 

```
reg = <0x8 0x20000000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x0 0x3>;
msi-map = <0x0 &its 0x0 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M //
ranges = <0x02000000 0x0 0xB0000000 0x0 0xB0000000 0x0 0x04000000
0x43000000 0x8 0x30000000 0x8 0x30000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xf800 0 0 0x7>;
// INTX //
interrupt-map = <0x4000 0 0 1 &mbigen_pcie0_local 704 4
0x4000 0 0 2 &mbigen_pcie0_local 705 4      //PCIe控制器0
0x4000 0 0 3 &mbigen_pcie0_local 706 4
0x4000 0 0 4 &mbigen_pcie0_local 707 4>;
iommu-map = <0 &io_smmu 0 0x400>;
status = "ok";
};

pcie@0x840000000 {
compatible = "pci-host-ecam-generic";
pcie-core-id = <1>;
// ECAM 0x840000000 pci host bridge0
reg = <0x8 0x40400000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x4 0x7>;
msi-map = <0x400 &its 0x400 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M
ranges = <0x02000000 0x0 0xB4000000 0x0 0xB4000000 0x0 0x04000000
0x43000000 0x8 0x50000000 0x8 0x50000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xff0000 0 0 0x7>;
// INTX
interrupt-map = <0x40000 0 0 1 &mbigen_pcie1_local 712 4
0x40000 0 0 1 &mbigen_pcie1_local 713 4      //PCIe控制器1
0x40000 0 0 1 &mbigen_pcie1_local 714 4
0x40000 0 0 1 &mbigen_pcie1_local 715 4>;
iommu-map = <0x400 &io_smmu 0x400 0x400>;
status = "ok";
};

/*
pcie@0x860000000 {
compatible = "pci-host-ecam-generic";
// ECAM 0x860000000 pci host bridge0 //
reg = <0x8 0x60800000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x8 0xb>;
msi-map = <0x800 &its 0x800 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M //
ranges = <0x02000000 0x0 0xB8000000 0x0 0xB8000000 0x0 0x04000000
0x43000000 0x8 0x70000000 0x8 0x70000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xf800 0 0 0x7>;
// INTX //
```

 版权所有 © 华为技术有限公司 

```
interrupt-map = <0x4000 0 0 1 &mbigen_pcie2_local 720 4
0x4000 0 0 2 &mbigen_pcie2_local 721 4      //PCIe控制器2
0x4000 0 0 3 &mbigen_pcie2_local 722 4
0x4000 0 0 4 &mbigen_pcie2_local 723 4>;
iommu-map = <0x800 &io_smmu 0x800 0x400>;
status = "ok";
};
*/

/*
pcie@0x900000000 {
compatible = "pci-host-ecam-generic";
pcie-core-id = <3>;
// ECAM 0x900000000 pci host bridge0
reg = <0x9 0x00c00000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0xc 0xf>;
msi-map = <0xc00 &its 0xc00 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M
ranges = <0x02000000 0x0 0xBC000000 0x0 0xBC000000 0x0 0x04000000
0x43000000 0x9 0x10000000 0x9 0x10000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xff0000 0 0 0x7>;
// INTX
interrupt-map = <0xc0000 0 0 1 &mbigen_pcie3_local 728 4
0xc0000 0 0 1 &mbigen_pcie3_local 729 4       //PCIe控制器3
0xc0000 0 0 1 &mbigen_pcie3_local 730 4
0xc0000 0 0 1 &mbigen_pcie3_local 731 4>;
iommu-map = <0xc00 &io_smmu 0xc00 0x400>;
status = "ok";
};
*/
```

**说明**

- 若配置PCIe位宽为x4，例如lane0配置为x4，则只需要配置PCIe控制器0即可，无需配置PCIe控制器1。
- **/***和***/**代表注释代码段，注释成功后，部分软件可能不显示注释格式，如MobaXterm，但不影响注释结果。

2. 配置完成后按“Esc”键，再执行**:wq!**命令保存修改，并按“Enter”键退出。

**步骤11** 将新增的整机dts文件添加到编译路径中。

1. 执行如下命令，进入“dtbtool”目录。

   **cd** */opt*/**Ascend310B-source/dtb/dtbtool/**

2. 执行如下命令，修改CMakelists文件，将所增加的dts文件添加到编译路径中，具体修改如下加粗字体所示。

   **vim CMakeLists.txt**

```
set(DTS_BASE_DIR ${TOP_DIR}/tools/customize/dts)
if(PRODUCT STREQUAL "ascend310Besl" OR
   PRODUCT STREQUAL "ascend310B" OR
   PRODUCT STREQUAL "ascend310Bemu" OR
   PRODUCT STREQUAL "ascend310Brc" OR
   PRODUCT STREQUAL "ascend310Brcesl" OR
   PRODUCT STREQUAL "ascend310Brcemu")
    # 当前driver跟esl用相同的dts，后期需要更新到单独的目录
    set(DTS_SRC_DIR ${DTS_BASE_DIR}/hi1910b/hi1910BL)
    set(DTS_FILE_LIST hi1910B-default.dts unreleased/hi1911-esl/hi1911-esl-b600.dts unreleased/
hi1911-esl/hi1911-esl-rc-b600.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0.dts unreleased/
hi1911-fpga/hi1911-fpga-b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0_b2.dts
```

 版权所有 © 华为技术有限公司 

```
unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0.dts unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0_b2.dts hi1910B-evb-900.dts hi1910B-evb-901.dts hi1910B-evb-902.dts hi1910B-evb-903.dts hi1910B-evb-905.dts hi1910B-asic-M150-B50.dts hi1910B-asic-M100-B51.dts hi1910B-asic-M150-B51.dts hi1910B-asic-M160-B51.dts hi1910B-asic-M100-B00.dts hi1910B-asic-M101-B00.dts hi1910B-asic-M150-B00.dts hi1910B-asic-M151-B00.dts hi1910B-asic-M150-B33.dts)
```

**说明**

新增加的dts只能在加粗字体段落末尾添加。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤12** 编译DTB文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译DTB文件。

   **bash build.sh dtb**

   出现如下回显，且生成“dt.img”文件表示编译内核userBaseConfig文件成功。

   ```
   generate /opt/Ascend310B-source/output/dt.img success!
   sign /opt/Ascend310B-source/output/dt.img success!
   ```

   **说明**

   编译后的“dt.img”文件会自动存放于“Ascend310B-source/output”目录下。

**----结束**

## 生效 userBaseConfig 和 DTB 文件

**步骤1** 准备网线，连接PC和Atlas 200 AI加速模块底板网口，登录Atlas 200I A2 加速模块整机，具体操作请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用“WinSCP”，将编译后的“userBaseConfig.bin”和“dt.img”文件上传至任意目录下，例如“/run”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 升级“userBaseConfig.bin”文件。

1. 执行如下命令，进入“/run”目录。

   **cd** */run*

2. 执行如下命令，升级“userBaseConfig.bin”文件。

   **/var/davinci/driver/upgrade-tool --device_index -1 --component Usr_Base_Config --path userBaseConfig.bin**

   出现如下回显，表示升级userBaseConfig.bin文件成功。

   ```
   {"device": 0, "succeed"}
   ```

**步骤5** 升级“dt.img”文件。具体请参见**生效DTB文件**。

**步骤6** 升级完成后重启生效。

**reboot**

建议查看串口日志确认升级重启进展。

**----结束**

 版权所有 © 华为技术有限公司 

### 验证接口功能

以连接NVMe硬盘为例，将配套的NVMe硬盘连接到Atlas 200I A2 加速模块的底板上。

**步骤1** 将配套的NVMe硬盘连接到Atlas 200I A2 加速模块的底板上。

**步骤2** 准备串口线，连接PC和Atlas 200I A2 加速模块底板串口。

**步骤3** 登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

**说明**

- Atlas 200I A2 加速模块（使用非Atlas 200I DK A2 开发者套件底板）USB接口登录的默认IP为192.168.1.2，Atlas 200I DK A2 开发者套件USB接口登录的默认IP为192.168.0.2。
- 若Atlas 200I A2 加速模块上电后网络可以ping通，但无法通过SSH登录，请参见**15.1 使用root用户SSH登录服务器出现Access Denied错误**解决。

**步骤4** 执行如下命令，查询硬盘是否在位。

**lsblk**

显示硬盘详细信息，如显示类似容量、厂家信息等即表示硬盘在位。PCIe接口通信正常。

**----结束**

## 7.1.3 SATA 接口调测参考

本章节以其中一个不同于Atlas 200I DK A2 开发者套件、Atlas 500 A2 智能小站的用户底板设计为例，根据底板的设计来配置对应SATA接口的userBaseConfig文件和DTB文件。

### 前提条件

已获取设备boardid，本章节以33150为例。具体请参见**2 用户必读**。

用户底板的adc_board_id为33，Atlas 200I A2 加速模块的算力为20T（adc_board_id为150），用户整机（Atlas 200I A2 加速模块+底板）boardid为33150（对应十六进制817e）。

### 注意事项

- 如果已经调测过其他接口，直接在已修改好的产品dts和对应的.dtsi文件中，修改对应内容。
- 由于调测其他接口可能会在产品dts中注释掉对应PCIe的.dtsi文件，所以需要将产品dts文件中对应PCIe的.dtsi文件取消注释，使能功能。

### 修改和编译 userBaseConfig 和 DTB 文件

本操作以首次调测接口为例介绍。

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**


版权所有 © 华为技术有限公司

**步骤3** 使用WinSCP工具，将4.1 下载软件包获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见14.3 使用WinSCP传输文件。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 修改“user_base_config.xml”配置文件。

1. 执行如下命令，进入配置文件所在目录。

   **cd config/adaptive/adaptive_config/**

2. 执行如下命令，打开“user_base_config.xml”文件。

   **vim user_base_config.xml**

3. 在“user_base_config.xml”中配置用户整机对应的**board_id**、**chip0_strategy**、**max_power**和**serdes_info**。具体参数含义请参见表7-4和表7-5。

```
<item name="product_strategy_info"><!--产品板策略ID-->
<!--33150 对应十六进制817e-->
<param board_id ="817e" phy_num = "1" chip0_strategy="0" max_power="0"  equal="0"/>
</item>

<item name="evb_strategy_info">
</item>

<item name="strategy_info_array"><!--启动策略矩阵-->
<subitem subname="sub_strategy0">
<module class="serdes">
<submodule subclass="serdes_info">
<param lan_index="0" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="2"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="1" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0"     port_index = "0"/>
<param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="4" serdes_type="ETH" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff"  align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="5" serdes_type="ETH"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff" align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
<param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
</submodule>
</module>
```

   - 第一处加粗字体修改为用户产品的实际boardid以及算力等级，各参数含义如下。

 版权所有 © 华为技术有限公司 

**表 7-4** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| board_id | 如33150对应十六进制817e。 | 十六进制的boardid。<br>boardid为用户实际使用底板的adc_board_id与Atlas 200I A2 加速模块的adc_board_id的组合值。 |
| phy_num | 1。 | 多p系统才会使用，固定为1。 |
| chip0_strategy | 0~n。 | 策略索引号。对应代码中的sub_strategy0和sub_strategy1。 |
| max_power | ▪ 0表示20T。<br>▪ 2表示8T。 | 最大算力档位。 |
| euqal | 0-n。 | 匹配DTB时等效board_id使用，保持默认配置0即可。 |

- 第二处加粗字体修改为用户产品的实际SerDes配置，具体规则如下。
  - PCIe控制器0（lane0~lane1）支持RC/EP模式，其他PCIe控制器只支持RC模式。
  - Macro0（lane0~lane3）中若存在PCIe和其他协议（SATA）共存，则PCIe只能支持到PCIE_GEN2；若只存在PCIe协议，则PCIe支持到PCIE_GEN3。
  - PCIe支持降lane应用，如PCIe x4降lane到PCIe x2/x1。
  - Macro0（lane0~lane3）支持PCIE和SATA协议，Macro1（lane4~lane7）支持PCIe、USB、ETH协议，每个Macro同时最多只能配置两种协议。
  - SATA只能在Macro0（lane0~lane3）使用。
  - USB控制器0（lane4）只支持HOST，USB控制器1~USB控制器3（lane5~lane7）支持HOST+DEVICE。
  - USB只能在Macro1（lane4~lane7）使用。
  - 若为系统直出的USB 2.0，则无需配置SerDes。

  各参数含义如下。

 版权所有 © 华为技术有限公司 

**表 7-5** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| lan_index | 0~7。 | lane编号，Atlas 200I A2 加速模块一共有8条serdes lane。 |
| serdes_type | Atlas 200I A2 加速模块支持PCIE、SATA、USB、ETH。 | 该条lane使用的协议。 |
| ssc_enable | ■ 0：否。<br>■ 1：是。 | 是否开展频[a]，只有用作PCIe和SATA时才能打开。 |
| polarity_tx | ■ 0：否。<br>■ 1：是。 | tx极性是否翻转。 |
| polarity_rx | ■ 0：否。<br>■ 1：是。 | rx极性是否翻转。 |
| lan_order | ■ 0：否。<br>■ 1：是。 | lane序是否反转。只有用作ETH时需要修改，具体请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“以太网接口”章节。 |
| bandwidth | ■ 1表示x1。<br>■ 2表示x2。<br>■ 4表示x4。<br>■ ff表示不配置。 | 用作PCIe时，表示位宽x4/x2/x1，其他协议不使用。<br>■ x4：使用四条lane。<br>■ x2：使用两条lane。<br>■ x1：使用一条lane。 |
| align_mode | 0。 | 该lane是master还是slave，Atlas 200I A2 加速模块不使用该参数。 |
| frequency | ■ PCIE_GEN2。<br>■ PCIE_GEN3。<br>■ 1GE。<br>■ 2.5GE。<br>■ USB3.0。<br>■ SATA3.0。 | 使用的频点[b]，主要用来区分PCIE GEN2/3，和ETH的1GE/2.5GE。 |


版权所有 © 华为技术有限公司

| param | 配置值 | 说明 |
| --- | --- | --- |
| port_index | 0~n。 | 用作ETH时，表示lane的网口端口号，其他协议不使用。不使用时设置为0。 |
| a：时钟展频通过频率调制的手段将集中在窄频带范围内的能量分散到设定的宽频带范围，通过降低时钟在基频和奇次谐波频率的幅度（能量），达到降低系统电磁辐射峰值的目的。<br>b：PCIe的频点是指数据在总线上传输的速率，也称为数据传输速度。 | | |

**须知**

弹性配置中的SerDes复用关系参数必须与实际硬件单板保持一致，SerDes错误配置可能导致启动挂死，功能不可用。

根据表7-6中用户产品的实际serdes配置，为使能接口功能，user_base_config.xml文件中serdes_info配置的示例如下。

**表 7-6** SerDes 接口复用关系

| lane编号 | 协议 | 是否开展频 | tx极性是否翻转 | rx极性是否翻转 | lane序是否反转 | 位宽 | 频点 |
| --- | --- | --- | --- | --- | --- | --- | --- |
| lane0 | PCIE | 否 | 否 | 否 | 否 | x2 | PCIE_GEN2 |
| lane1 | PCIE | 否 | 否 | 否 | 否 | 不配置[a] | PCIE_GEN2 |
| lane2 | SATA | 是 | 否 | 否 | 否 | 不配置 | SATA3.0 |
| lane3 | PCIE | 否 | 否 | 否 | 否 | x1 | PCIE_GEN2 |
| lane4 | ETH | 否 | 否 | 否 | 是 | 不配置 | 1GE |
| lane5 | ETH | 否 | 否 | 否 | 是 | 不配置 | 1GE |
| lane6 | USB | 否 | 否 | 否 | 否 | 不配置 | USB3.0 |
| lane7 | USB | 否 | 否 | 否 | 否 | 不配置 | USB3.0 |
| a：lane0的PCIe位宽为x2，需要使用两条lane，所以lane1不设置位宽。 | | | | | | | |


版权所有 © 华为技术有限公司

user_base_config.xml文件中serdes_info配置如下。

```
<submodule subclass="serdes_info">
<param lan_index="0" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="2"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="1" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0"   port_index = "0"/>
<param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="4" serdes_type="ETH" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff"  align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="5" serdes_type="ETH"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff" align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
<param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
</submodule>
```

**说明**

位宽只能配置为x4/x2/x1。

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤8** 编译userBaseConfig文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译userBaseConfig文件。

   **bash build.sh usrBaseConfig**

   出现如下回显，表示编译内核userBaseConfig文件成功。

   ```
   generate /opt/Ascend310B-source/output/userBaseConfig.bin success!
   sign /opt/Ascend310B-source/output/userBaseConfig.bin success!
   ```

   **说明**

   编译后的“userBaseConfig.bin”文件会自动存放于“Ascend310B-source/output”目录下。

**步骤9** 新增并修改整机dts文件。

以用户整机的boardid为33150为例，故dts文件名称为“hi1910B-asic-M150-B33.dts”。具体请参见**DTB文件**。

1. 执行如下命令，进入dts文件目录。

   **cd** */opt*/**Ascend310B-source/dtb/dts/hi1910b/hi1910BL**

2. 执行如下命令，新增适配boardid33150的dts文件。

   **cp hi1910B-asic-M150-B50.dts** *hi1910B-asic-M150-B33.dts*

   将“hi1910B-asic-M150-B50.dts”另存为boardid33150的dts文件“hi1910B-asic-M150-B33.dts”。

3. 执行以下命令修改dts文件，配置用户产品的实际整机boardid，确认所使用的PCIe的.dtsi文件及路径，具体内容如下加粗字体所示。

   **vim hi1910B-asic-M150-B33.dts**

   ```
   compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
   hisi,boardid = <0x0 0x3 0x3 0x1 0x5 0x0>;  //修改为实际整机boardid
   #address-cells = <0x2>;
   #size-cells = <0x2>;
   ```

 版权所有 © 华为技术有限公司 

```
interrupt-parent = <0x1>;
model = "Hisilicon PhosphorHi1910B evb";

/include/ "base/hi1910B-asic-kernel-rc.dtsi"
/include/ "base/hi1910B-commmon-smmu.dtsi"
/include/ "base/hi1910B-mpam.dtsi"
/include/ "base/hi1910B-lp-pm.dtsi"
/include/ "base/hi1910B-lp-devmng.dtsi"
/include/ "base/hi1910B-mbigen-ao.dtsi"
/include/ "base/hi1910B-gpio.dtsi"
/include/ "base/hi1910B-ipcdrv.dtsi"
/include/ "base/hi1910B-ts-drv.dtsi"
/include/ "base/hi1910B-tsdrv.dtsi"
/include/ "base/hi1910B-network-M150-B50.dtsi"
/include/ "base/hi1910B-hidvpp.dtsi"
/include/ "base/hi1910B-hdmi.dtsi"
/include/ "product/hi1910B-audio.dtsi"
/include/ "base/hi1910B-mipitx.dtsi"
/include/ "base/hi1910B-vdp.dtsi"
/include/ "base/hi1910B-isp.dtsi"
/include/ "base/hi1910B-hilink.dtsi"
/include/ "base/hi1910B-mntn-milanr3.dtsi"
/include/ "base/hi1910B-gpu.dtsi"
/include/ "base/hi1910B-ctl.dtsi"
/include/ "base/hi1910B-itrustee.dtsi"
/include/ "base/hi1910B-sata.dtsi"
/include/ "product/hi1910B-pcie-rc-M150-B50.dtsi" //确认所使用的PCIe的.dtsi文件及路径
/include/ "product/hi1910B-mdio-M150-B50.dtsi"
/include/ "product/hi1910B-pinctrl-M150-B50.dtsi"
/include/ "product/hi1910B-can-M150-B50.dtsi"
/include/ "product/hi1910B-hiusbc-B50.dtsi"
/include/ "product/hi1910B-emmc-M150-B50.dtsi"
/include/ "product/hi1910B-i2c-M150-B50.dtsi"
/include/ "product/hi1910B-spi-M150-B50.dtsi"
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** 修改PCIe的.dtsi文件，配置PCIe控制器。

1. 执行如下命令进入“product”目录，并修改PCIe的.dtsi文件。

   **cd product**

   **vim** *hi1910B-pcie-rc-M150-B50.dtsi*

   根据**步骤7.3**中配置的**serdes_info**选择需要开放的PCIe控制器，lan_index与PCIe控制器的对应关系如下。

| lan_index | 控制器 |
| --- | --- |
| lane0~lane1 | PCIe控制器0 |
| lane2~lane3 | PCIe控制器1 |
| lane4~lane5 | PCIe控制器2 |
| lane6~lane7 | PCIe控制器3 |

   以**步骤7.3**中**serdes_info**配置为例，共使用了PCIe控制器0、PCIe控制器1，则在PCIe的.dtsi文件中保留PCIe控制器0、PCIe控制器1两个控制器的配置，将PCIe控制器2、PCIe控制器3的配置代码注释即可，其中**/***和***/**代表注释代码段，具体修改如下所示。

```
pcie@0x820000000 {
compatible = "pci-host-ecam-generic";
// ECAM 0x820000000 pci host bridge0 //
```

 版权所有 © 华为技术有限公司 

```
reg = <0x8 0x20000000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x0 0x3>;
msi-map = <0x0 &its 0x0 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M //
ranges = <0x02000000 0x0 0xB0000000 0x0 0xB0000000 0x0 0x04000000
0x43000000 0x8 0x30000000 0x8 0x30000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xf800 0 0 0x7>;
// INTX //
interrupt-map = <0x4000 0 0 1 &mbigen_pcie0_local 704 4
0x4000 0 0 2 &mbigen_pcie0_local 705 4      //PCIe控制器0
0x4000 0 0 3 &mbigen_pcie0_local 706 4
0x4000 0 0 4 &mbigen_pcie0_local 707 4>;
iommu-map = <0 &io_smmu 0 0x400>;
status = "ok";
};

pcie@0x840000000 {
compatible = "pci-host-ecam-generic";
pcie-core-id = <1>;
// ECAM 0x840000000 pci host bridge0
reg = <0x8 0x40400000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x4 0x7>;
msi-map = <0x400 &its 0x400 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M
ranges = <0x02000000 0x0 0xB4000000 0x0 0xB4000000 0x0 0x04000000
0x43000000 0x8 0x50000000 0x8 0x50000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xff0000 0 0 0x7>;
// INTX
interrupt-map = <0x40000 0 0 1 &mbigen_pcie1_local 712 4
0x40000 0 0 1 &mbigen_pcie1_local 713 4      //PCIe控制器1
0x40000 0 0 1 &mbigen_pcie1_local 714 4
0x40000 0 0 1 &mbigen_pcie1_local 715 4>;
iommu-map = <0x400 &io_smmu 0x400 0x400>;
status = "ok";
};

/*
pcie@0x860000000 {
compatible = "pci-host-ecam-generic";
// ECAM 0x860000000 pci host bridge0 //
reg = <0x8 0x60800000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x8 0xb>;
msi-map = <0x800 &its 0x800 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M //
ranges = <0x02000000 0x0 0xB8000000 0x0 0xB8000000 0x0 0x04000000
0x43000000 0x8 0x70000000 0x8 0x70000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xf800 0 0 0x7>;
// INTX //
```

 版权所有 © 华为技术有限公司 

```
interrupt-map = <0x4000 0 0 1 &mbigen_pcie2_local 720 4
0x4000 0 0 2 &mbigen_pcie2_local 721 4      //PCIe控制器2
0x4000 0 0 3 &mbigen_pcie2_local 722 4
0x4000 0 0 4 &mbigen_pcie2_local 723 4>;
iommu-map = <0x800 &io_smmu 0x800 0x400>;
status = "ok";
};
*/

/*
pcie@0x900000000 {
compatible = "pci-host-ecam-generic";
pcie-core-id = <3>;
// ECAM 0x900000000 pci host bridge0
reg = <0x9 0x00c00000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0xc 0xf>;
msi-map = <0xc00 &its 0xc00 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M
ranges = <0x02000000 0x0 0xBC000000 0x0 0xBC000000 0x0 0x04000000
0x43000000 0x9 0x10000000 0x9 0x10000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xff0000 0 0 0x7>;
// INTX
interrupt-map = <0xc0000 0 0 1 &mbigen_pcie3_local 728 4
0xc0000 0 0 1 &mbigen_pcie3_local 729 4       //PCIe控制器3
0xc0000 0 0 1 &mbigen_pcie3_local 730 4
0xc0000 0 0 1 &mbigen_pcie3_local 731 4>;
iommu-map = <0xc00 &io_smmu 0xc00 0x400>;
status = "ok";
};
*/
```

**说明**

- 若配置PCIe位宽为x4，例如lane0配置为x4，则只需要配置PCIe控制器0即可，无需配置PCIe控制器1。
- **/***和***/**代表注释代码段，注释成功后，部分软件可能不显示注释格式，如MobaXterm，但不影响注释结果。

2. 配置完成后按“Esc”键，再执行**:wq!**命令保存修改，并按“Enter”键退出。

**步骤11** 将新增的整机dts文件添加到编译路径中。

1. 执行如下命令，进入“dtbtool”目录。

   **cd** */opt*/**Ascend310B-source/dtb/dtbtool/**

2. 执行如下命令，修改CMakelists文件，将所增加的dts文件添加到编译路径中，具体修改如下加粗字体所示。

   **vim CMakeLists.txt**

```
set(DTS_BASE_DIR ${TOP_DIR}/tools/customize/dts)
if(PRODUCT STREQUAL "ascend310Besl" OR
   PRODUCT STREQUAL "ascend310B" OR
   PRODUCT STREQUAL "ascend310Bemu" OR
   PRODUCT STREQUAL "ascend310Brc" OR
   PRODUCT STREQUAL "ascend310Brcesl" OR
   PRODUCT STREQUAL "ascend310Brcemu")
    # 当前driver跟esl用相同的dts，后期需要更新到单独的目录
    set(DTS_SRC_DIR ${DTS_BASE_DIR}/hi1910b/hi1910BL)
    set(DTS_FILE_LIST hi1910B-default.dts unreleased/hi1911-esl/hi1911-esl-b600.dts unreleased/
hi1911-esl/hi1911-esl-rc-b600.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0.dts unreleased/
hi1911-fpga/hi1911-fpga-b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0_b2.dts
```

 版权所有 © 华为技术有限公司 

```
unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0.dts unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0_b2.dts hi1910B-evb-900.dts hi1910B-evb-901.dts hi1910B-evb-902.dts hi1910B-evb-903.dts hi1910B-evb-905.dts hi1910B-asic-M150-B50.dts hi1910B-asic-M100-B51.dts hi1910B-asic-M150-B51.dts hi1910B-asic-M160-B51.dts hi1910B-asic-M100-B00.dts hi1910B-asic-M101-B00.dts hi1910B-asic-M150-B00.dts hi1910B-asic-M151-B00.dts hi1910B-asic-M150-B33.dts)
```

**说明**

新增加的dts只能在加粗字体段落末尾添加。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤12** 编译DTB文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译DTB文件。

   **bash build.sh dtb**

   出现如下回显，且生成“dt.img”文件表示编译内核userBaseConfig文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

**说明**

编译后的“dt.img”文件会自动存放于“Ascend310B-source/output”目录下。

**----结束**

## 生效 userBaseConfig 和 DTB 文件

**步骤1** 准备网线，连接PC和Atlas 200I A2 加速模块底板网口，登录Atlas 200I A2 加速模块整机，具体操作请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤2** 插拔电源，查看串口日志，保障当前启动区域为主区。

具体串口日志如下图所示。

```
current[4], last[99]
[0.151] [BOOT_TRACE]Load main userbasecfg
non-secure vef success!
Cfg type-1 no cfg info.
No info in userBaseConfig
current[5], last[99]
[0.163] [BOOT_TRACE]Load main sysbasecfg
SyscfgProc [350|0|1]
Update bitmap: 0xf 0x1 0x1 0x0 0x3 0x3 0x0 0x0 0x0 0x0 0x0 0x0 0x1 0x7 0xf 0x3f
bitmap info update load success
SyscfgProc [352|0|0]
write share info ok
```

**步骤3** 执行如下命令，切换至root用户。

**su - root**

**步骤4** 使用“WinSCP”，将编译后的“userBaseConfig.bin”和“dt.img”文件上传至任意目录下，例如“/run”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤5** 升级“userBaseConfig.bin”文件。

1. 执行如下命令，进入“/run”目录。

 版权所有 © 华为技术有限公司 

**cd** */run*

2. 执行如下命令，升级“userBaseConfig.bin”文件。

   **/var/davinci/driver/upgrade-tool --device_index -1 --component Usr_Base_Config --path userBaseConfig.bin**

   出现如下回显，表示升级userBaseConfig.bin文件成功。

   ```
   {"device": 0, "succeed"}
   ```

**步骤6** 升级“dt.img”文件。具体请参见**生效DTB文件**。

**步骤7** 升级完成后重启生效。

**reboot**

建议查看串口日志确认升级重启进展。

**----结束**

## 验证接口功能

以连接SATA硬盘为例，将配套的SATA硬盘连接到Atlas 200I A2 加速模块的底板上。

**步骤1** 将配套的SATA硬盘连接到Atlas 200I A2 加速模块的底板上。

**步骤2** 准备串口线，连接PC和Atlas 200I A2 加速模块底板串口。

**步骤3** 登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

**说明**

- Atlas 200I A2 加速模块（使用非Atlas 200I DK A2 开发者套件底板）USB接口登录的默认IP为192.168.1.2，Atlas 200I DK A2 开发者套件USB接口登录的默认IP为192.168.0.2。
- 若Atlas 200I A2 加速模块上电后网络可以ping通，但无法通过SSH登录，请参见**15.1 使用root用户SSH登录服务器出现Access Denied错误**解决。

**步骤4** 执行如下命令，查询硬盘是否在位。

**lsblk**

显示硬盘详细信息，如显示类似容量、厂家信息等即表示硬盘在位。SATA接口通信正常。

**----结束**

# 7.1.4 USB 接口调测参考

本章节以其中一个不同于Atlas 200I DK A2 开发者套件、Atlas 500 A2 智能小站的用户底板设计为例，根据底板的设计来配置对应USB接口的userBaseConfig文件和DTB文件。

## 背景信息

调测USB接口需要修改PCIe的.dtsi文件，配置PCIe控制器，否则可能导致启动挂死。

## 前提条件

已获取设备boardid，本章节以33150为例。具体请参见**2 用户必读**。

 版权所有 © 华为技术有限公司 

用户底板的adc_board_id为33，Atlas 200I A2 加速模块的算力为20T（adc_board_id为150），用户整机（Atlas 200I A2 加速模块+底板）boardid为33150（对应十六进制817e）。

## 修改和编译 userBaseConfig 和 DTB 文件

本操作以首次调测接口为例介绍。

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 修改“user_base_config.xml”配置文件。

1. 执行如下命令，进入配置文件所在目录。

   **cd config/adaptive/adaptive_config/**

2. 执行如下命令，打开“user_base_config.xml”文件。

   **vim user_base_config.xml**

3. 在“user_base_config.xml”中配置用户整机对应的**board_id**、**chip0_strategy**、**max_power**和**serdes_info**。具体参数含义请参见**表7-7**和**表7-8**。

```
<item name="product_strategy_info"><!--产品板策略ID-->
<!--33150 对应十六进制817e-->
<param board_id ="817e" phy_num = "1" chip0_strategy="0" max_power="0"  equal="0"/>
</item>

<item name="evb_strategy_info">
</item>

<item name="strategy_info_array"><!--启动策略矩阵-->
<subitem subname="sub_strategy0">
<module class="serdes">
<submodule subclass="serdes_info">
<param lan_index="0" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="2"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="1" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0"     port_index = "0"/>
<param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="4" serdes_type="ETH" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff"  align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="5" serdes_type="ETH"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff" align_mode="0" frequency="1GE" port_index = "0"/>
```

 版权所有 © 华为技术有限公司 

```
<param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
<param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
</submodule>
</module>
```

- 第一处加粗字体修改为用户产品的实际boardid以及算力等级，各参数含义如下。

  **表 7-7** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| board_id | 如33150对应十六进制817e。 | 十六进制的boardid。<br>boardid为用户实际使用底板的adc_board_id与Atlas 200I A2 加速模块的adc_board_id的组合值。 |
| phy_num | 1。 | 多p系统才会使用，固定为1。 |
| chip0_strategy | 0~n。 | 策略索引号。对应代码中的sub_strategy0和sub_strategy1。 |
| max_power | ▪ 0表示20T。<br>▪ 2表示8T。 | 最大算力档位。 |
| euqal | 0-n。 | 匹配DTB时等效board_id使用，保持默认配置0即可。 |

- 第二处加粗字体修改为用户产品的实际SerDes配置，具体规则如下。
  - PCIe控制器0（lane0~lane1）支持RC/EP模式，其他PCIe控制器只支持RC模式。
  - Macro0（lane0~lane3）中若存在PCIe和其他协议（SATA）共存，则PCIe只能支持到PCIE_GEN2；若只存在PCIe协议，则PCIe支持到PCIE_GEN3。
  - PCIe支持降lane应用，如PCIe x4降lane到PCIe x2/x1。
  - Macro0（lane0~lane3）支持PCIE和SATA协议，Macro1（lane4~lane7）支持PCIe、USB、GE协议，每个Macro同时最多只能配置两种协议。
  - SATA只能在Macro0（lane0~lane3）使用。
  - USB控制器0（lane4）只支持HOST，USB控制器1~USB控制器3（lane5~lane7）支持HOST+DEVICE。
  - USB只能在Macro1（lane4~lane7）使用。

 版权所有 © 华为技术有限公司 

- 若为系统直出的USB 2.0，则无需配置SerDes。

各参数含义如下。

**表 7-8** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| lan_index | 0~7。 | lane编号，Atlas 200I A2 加速模块一共有8条serdes lane。 |
| serdes_type | Atlas 200I A2 加速模块支持PCIE、SATA、USB、ETH。 | 该条lane使用的协议。 |
| ssc_enable | ■ 0：否。<br>■ 1：是。 | 是否开展频[a]，只有用作PCIe和SATA时才能打开。 |
| polarity_tx | ■ 0：否。<br>■ 1：是。 | tx极性是否翻转。 |
| polarity_rx | ■ 0：否。<br>■ 1：是。 | rx极性是否翻转。 |
| lan_order | ■ 0：否。<br>■ 1：是。 | lane序是否反转。只有用作ETH时需要修改，具体请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“以太网接口”章节。 |
| bandwidth | ■ 1表示x1。<br>■ 2表示x2。<br>■ 4表示x4。<br>■ ff表示不配置。 | 用作PCIe时，表示位宽x4/x2/x1，其他协议不使用。<br>■ x4：使用四条lane。<br>■ x2：使用两条lane。<br>■ x1：使用一条lane。 |
| align_mode | 0。 | 该lane是master还是slave，Atlas 200I A2 加速模块不使用该参数。 |

 版权所有 © 华为技术有限公司 

| param | 配置值 | 说明 |
| --- | --- | --- |
| frequency | ■ PCIE_GEN2。<br>■ PCIE_GEN3。<br>■ 1GE。<br>■ 2.5GE。<br>■ USB3.0。<br>■ SATA3.0。 | 使用的频点[b]，主要用来区分PCIE GEN2/3，和ETH的1GE/2.5GE。 |
| port_index | 0~n。 | 用作ETH时，表示lane的网口端口号，其他协议不使用。不使用时设置为0。 |
| a：时钟展频通过频率调制的手段将集中在窄频带范围内的能量分散到设定的宽频带范围，通过降低时钟在基频和奇次谐波频率的幅度（能量），达到降低系统电磁辐射峰值的目的。<br>b：PCIe的频点是指数据在总线上传输的速率，也称为数据传输速度。 | | |

**须知**

弹性配置中的SerDes复用关系参数必须与实际硬件单板保持一致，SerDes错误配置可能导致启动挂死，功能不可用。

根据表7-9中用户产品的实际serdes配置，为使能接口功能，user_base_config.xml文件中serdes_info配置的示例如下。

**表 7-9** SerDes 接口复用关系

| lane编号 | 协议 | 是否开展频 | tx极性是否翻转 | rx极性是否翻转 | lane序是否反转 | 位宽 | 频点 |
| --- | --- | --- | --- | --- | --- | --- | --- |
| lane0 | PCIE | 否 | 否 | 否 | 否 | x2 | PCIE_GEN2 |
| lane1 | PCIE | 否 | 否 | 否 | 否 | 不配置[a] | PCIE_GEN2 |
| lane2 | SATA | 是 | 否 | 否 | 否 | 不配置 | SATA3.0 |
| lane3 | PCIE | 否 | 否 | 否 | 否 | x1 | PCIE_GEN2 |
| lane4 | ETH | 否 | 否 | 否 | 是 | 不配置 | 1GE |


版权所有 © 华为技术有限公司

| lan编号 | 协议 | 是否开展频 | tx极性是否翻转 | rx极性是否翻转 | lane序是否反转 | 位宽 | 频点 |
|---|---|---|---|---|---|---|---|
| lane5 | ETH | 否 | 否 | 否 | 是 | 不配置 | 1GE |
| lane6 | USB | 否 | 否 | 否 | 否 | 不配置 | USB3.0 |
| lane7 | USB | 否 | 否 | 否 | 否 | 不配置 | USB3.0 |
| a：lane0的PCIe位宽为x2，需要使用两条lane，所以lane1不设置位宽。 | | | | | | | |

user_base_config.xml文件中serdes_info配置如下。

```
<submodule subclass="serdes_info">
<param lan_index="0" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="2"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="1" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0"   port_index = "0"/>
<param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="4" serdes_type="ETH" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff"  align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="5" serdes_type="ETH"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff" align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
<param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
</submodule>
```

**说明**

位宽只能配置为x4/x2/x1。

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤8** 编译userBaseConfig文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译userBaseConfig文件。

   **bash build.sh usrBaseConfig**

   出现如下回显，表示编译内核userBaseConfig文件成功。

   ```
   generate /opt/Ascend310B-source/output/userBaseConfig.bin success!
   sign /opt/Ascend310B-source/output/userBaseConfig.bin success!
   ```

   **说明**

   编译后的“userBaseConfig.bin”文件会自动存放于“Ascend310B-source/output”目录下。

**步骤9** 新增并修改整机dts文件。

 版权所有 © 华为技术有限公司 

用户整机的boardid为33150，故dts文件名称为“hi1910B-asic-M150-B33.dts”。具体请参见**DTB文件**。

1. 执行如下命令，进入dts文件目录。

   **cd** */opt*/**Ascend310B-source/dtb/dts/hi1910b/hi1910BL**

2. 执行如下命令，新增适配boardid33150的dts文件。

   **cp hi1910B-asic-M150-B50.dts** *hi1910B-asic-M150-B33.dts*

   将“hi1910B-asic-M150-B50.dts”另存为boardid33150的dts文件“hi1910B-asic-M150-B33.dts”。

3. 执行以下命令修改dts文件，配置用户产品的实际整机boardid，确认所使用的PCIe的.dtsi文件和USB的.dtsi文件及路径，具体内容如下加粗字体所示。

   **vim hi1910B-asic-M150-B33.dts**

   ```
   compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
   hisi,boardid = <0x0 0x3 0x3 0x1 0x5 0x0>;  //修改为实际整机boardid
   #address-cells = <0x2>;
   #size-cells = <0x2>;
   interrupt-parent = <0x1>;
   model = "Hisilicon PhosphorHi1910B evb";

   /include/ "base/hi1910B-asic-kernel-rc.dtsi"
   /include/ "base/hi1910B-commmon-smmu.dtsi"
   /include/ "base/hi1910B-mpam.dtsi"
   /include/ "base/hi1910B-lp-pm.dtsi"
   /include/ "base/hi1910B-lp-devmng.dtsi"
   /include/ "base/hi1910B-mbigen-ao.dtsi"
   /include/ "base/hi1910B-gpio.dtsi"
   /include/ "base/hi1910B-ipcdrv.dtsi"
   /include/ "base/hi1910B-ts-drv.dtsi"
   /include/ "base/hi1910B-tsdrv.dtsi"
   /include/ "base/hi1910B-network-M150-B50.dtsi"
   /include/ "base/hi1910B-hidvpp.dtsi"
   /include/ "base/hi1910B-hdmi.dtsi"
   /include/ "product/hi1910B-audio.dtsi"
   /include/ "base/hi1910B-mipitx.dtsi"
   /include/ "base/hi1910B-vdp.dtsi"
   /include/ "base/hi1910B-isp.dtsi"
   /include/ "base/hi1910B-hilink.dtsi"
   /include/ "base/hi1910B-mntn-milanr3.dtsi"
   /include/ "base/hi1910B-gpu.dtsi"
   /include/ "base/hi1910B-ctl.dtsi"
   /include/ "base/hi1910B-itrustee.dtsi"
   /include/ "base/hi1910B-sata.dtsi"
   /include/ "product/hi1910B-pcie-rc-M150-B50.dtsi" //确认所使用的PCIe的.dtsi文件及路径
   /include/ "product/hi1910B-mdio-M150-B50.dtsi"
   /include/ "product/hi1910B-pinctrl-M150-B50.dtsi"
   /include/ "product/hi1910B-can-M150-B50.dtsi"
   /include/ "product/hi1910B-hiusbc-B50.dtsi" //确认所使用的USB的.dtsi文件及路径
   /include/ "product/hi1910B-emmc-M150-B50.dtsi"
   /include/ "product/hi1910B-i2c-M150-B50.dtsi"
   /include/ "product/hi1910B-spi-M150-B50.dtsi"
   ```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** 修改PCIe的.dtsi文件，配置PCIe控制器。

1. 执行如下命令进入“product”目录，并修改PCIe的.dtsi文件。

   **cd product**

   **vim** *hi1910B-pcie-rc-M150-B50.dtsi*

   根据**步骤7.3**中配置的**serdes_info**选择需要开放的PCIe控制器，lan_index与PCIe控制器的对应关系如下。

 版权所有 © 华为技术有限公司 

| lan_index | 控制器 |
| --- | --- |
| lane0~lane1 | PCIe控制器0 |
| lane2~lane3 | PCIe控制器1 |
| lane4~lane5 | PCIe控制器2 |
| lane6~lane7 | PCIe控制器3 |

以**步骤7.3**中**serdes_info**配置为例，共使用了PCIe控制器0、PCIe控制器1，则在PCIe的.dtsi文件中保留PCIe控制器0、PCIe控制器1两个控制器的配置，将PCIe控制器2、PCIe控制器3的配置代码注释即可，其中/*和*/代表注释代码段，具体修改如下所示。

```
pcie@0x820000000 {
compatible = "pci-host-ecam-generic";
// ECAM 0x820000000 pci host bridge0 //
reg = <0x8 0x20000000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x0 0x3>;
msi-map = <0x0 &its 0x0 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M //
ranges = <0x02000000 0x0 0xB0000000 0x0 0xB0000000 0x0 0x04000000
0x43000000 0x8 0x30000000 0x8 0x30000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xf800 0 0 0x7>;
// INTX //
interrupt-map = <0x4000 0 0 1 &mbigen_pcie0_local 704 4
0x4000 0 0 2 &mbigen_pcie0_local 705 4    //PCIe控制器0
0x4000 0 0 3 &mbigen_pcie0_local 706 4
0x4000 0 0 4 &mbigen_pcie0_local 707 4>;
iommu-map = <0 &io_smmu 0 0x400>;
status = "ok";
};

pcie@0x840000000 {
compatible = "pci-host-ecam-generic";
pcie-core-id = <1>;
// ECAM 0x840000000 pci host bridge0
reg = <0x8 0x40400000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x4 0x7>;
msi-map = <0x400 &its 0x400 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M
ranges = <0x02000000 0x0 0xB4000000 0x0 0xB4000000 0x0 0x04000000
0x43000000 0x8 0x50000000 0x8 0x50000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xff0000 0 0 0x7>;
// INTX
interrupt-map = <0x40000 0 0 1 &mbigen_pcie1_local 712 4
0x40000 0 0 1 &mbigen_pcie1_local 713 4    //PCIe控制器1
0x40000 0 0 1 &mbigen_pcie1_local 714 4
0x40000 0 0 1 &mbigen_pcie1_local 715 4>;
iommu-map = <0x400 &io_smmu 0x400 0x400>;
```


版权所有 © 华为技术有限公司

```
status = "ok";
};

/*
pcie@0x860000000 {
compatible = "pci-host-ecam-generic";
// ECAM 0x860000000 pci host bridge0 //
reg = <0x8 0x60800000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x8 0xb>;
msi-map = <0x800 &its 0x800 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M //
ranges = <0x02000000 0x0 0xB8000000 0x0 0xB8000000 0x0 0x04000000
0x43000000 0x8 0x70000000 0x8 0x70000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xf800 0 0 0x7>;
// INTX //
interrupt-map = <0x4000 0 0 1 &mbigen_pcie2_local 720 4
0x4000 0 0 2 &mbigen_pcie2_local 721 4      //PCIe控制器2
0x4000 0 0 3 &mbigen_pcie2_local 722 4
0x4000 0 0 4 &mbigen_pcie2_local 723 4>;
iommu-map = <0x800 &io_smmu 0x800 0x400>;
status = "ok";
};
*/

/*
pcie@0x900000000 {
compatible = "pci-host-ecam-generic";
pcie-core-id = <3>;
// ECAM 0x900000000 pci host bridge0
reg = <0x9 0x00c00000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0xc 0xf>;
msi-map = <0xc00 &its 0xc00 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M
ranges = <0x02000000 0x0 0xBC000000 0x0 0xBC000000 0x0 0x04000000
0x43000000 0x9 0x10000000 0x9 0x10000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xff0000 0 0 0x7>;
// INTX
interrupt-map = <0xc0000 0 0 1 &mbigen_pcie3_local 728 4
0xc0000 0 0 1 &mbigen_pcie3_local 729 4    //PCIe控制器3
0xc0000 0 0 1 &mbigen_pcie3_local 730 4
0xc0000 0 0 1 &mbigen_pcie3_local 731 4>;
iommu-map = <0xc00 &io_smmu 0xc00 0x400>;
status = "ok";
};
*/
```

**说明**

- 若配置PCIe位宽为x4，例如lane0配置为x4，则只需要配置PCIe控制器0即可，无需配置PCIe控制器1。
- **/***和***/**代表注释代码段，注释成功后，部分软件可能不显示注释格式，如MobaXterm，但不影响注释结果。

2. 配置完成后按“Esc”键，再执行**:wq!**命令保存修改，并按“Enter”键退出。

 版权所有 © 华为技术有限公司 

**步骤11** 修改USB的.dtsi文件。

1. 执行如下命令进入“product”目录，并修改USB的dtsi文件。

   **cd product**

   **vim** *hi1910B-hiusbc-B50.dtsi*

   根据步骤7.3中配置的**serdes_info**选择需要开放的USB控制器，lan_index与USB控制器的对应关系如下。

| lan_index | 控制器 |
| --- | --- |
| lane4 | USB控制器0 |
| lane5 | USB控制器1 |
| lane6 | USB控制器2 |
| lane7 | USB控制器3 |

   **说明**

   - 若产品存在系统直出的USB 2.0接口，该接口不占用SerDes的lane，但需要根据硬件设计开放对应的USB控制器，并将**u2-only**参数修改为1。
   - 本章以无系统直出的USB 2.0接口为例。

   以步骤7.3中**serdes_info**配置为例，共使用了USB控制器2和USB控制器3，则在dtsi文件中保留USB控制器2和USB控制器3的配置，将USB控制器0和USB控制器1的配置代码注释即可，其中**/***和***/**代表注释代码段，具体修改如下所示。

```
/*
hiusbc0@A5000000 {
compatible = "hisilicon,hiusbc";
reg = <0x0 0xA5000000 0x0 0x80000>;    // USB_CTRL0 BASE
interrupt-parent = <&mbigen_usb0>;
interrupts = <918 4>;
maximum-speed = "super-speed";
dma-coherent;
mode = <1>;                            // 1:host 2:device
usb-id = <0>;                          // USB Controller id
u2-only = <0>;
io-subctrl-base = <0x0 0xA0120000 0x0 0x10000>;
pcs-base = <0x0 0xA0800000 0x0 0x100>;
}
*/

/*
hiusbc1@A5080000 {
compatible = "hisilicon,hiusbc";
reg = <0x0 0xA5080000 0x0 0x80000>;    // USB_CTRL1 BASE
interrupt-parent = <&mbigen_usb1>;
interrupts = <922 4>;
maximum-speed = "super-speed";
dma-coherent;
mode = <2>;                            // 1:host 2:device
usb-id = <1>;                          // USB Controller id
u2-only = <0>;
io-subctrl-base = <0x0 0xA0120000 0x0 0x10000>;
pcs-base = <0x0 0xA0804000 0x0 0x100>;

phy {
phy_base = <0x0 0xa0810000 0x0 0x200>;
cfg_off = <0x14>;
cfg_mask = <0x3FFC0000>;
```

 版权所有 © 华为技术有限公司 

```
cfg_value = <0x4350040>;
};
};
*/

hiusbc2@A5100000 {
compatible = "hisilicon,hiusbc";
reg = <0x0 0xA5100000 0x0 0x80000>;    // USB_CTRL2 BASE
interrupt-parent = <&mbigen_usb2>;
interrupts = <926 4>;
maximum-speed = "super-speed";
dma-coherent;
mode = <1>;                            // 1:host 2:device
usb-id = <2>;                          // USB Controller id
u2-only = <0>;
io-subctrl-base = <0x0 0xA0120000 0x0 0x10000>;
pcs-base = <0x0 0xA0808000 0x0 0x100>;

phy {
phy_base = <0x0 0xa0810200 0x0 0x200>;
cfg_off = <0x14>;
cfg_mask = <0x3FFC0000>;
cfg_value = <0xC310040>;
};
};

hiusbc3@A5180000 {
compatible = "hisilicon,hiusbc";
reg = <0x0 0xA5180000 0x0 0x80000>;    // USB_CTRL3 BASE
interrupt-parent = <&mbigen_usb3>;
interrupts = <930 4>;
maximum-speed = "super-speed";
dma-coherent;
mode = <1>;                            // 1:host 2:device
usb-id = <3>;                          // USB Controller id
u2-only = <0>;
io-subctrl-base = <0x0 0xA0120000 0x0 0x10000>;
pcs-base = <0x0 0xA080C000 0x0 0x100>;

phy {
phy_base = <0x0 0xa0810400 0x0 0x200>;
cfg_off = <0x14>;
cfg_mask = <0x3FFC0000>;
cfg_value = <0x5350040>;
};
};
```

再根据具体功能要求修改USB控制器的配置参数，需要修改的参数如**表7-10**所示，此处以USB控制器1为例，需要配置的参数如下内容中加粗字体所示。

**表 7-10** 控制器参数

| 参数 | 配置值 | 说明 |
| --- | --- | --- |
| mode | 1，2 | 支持host与device切换，其中1为host配置，2为device配置。 |
| u2-only | 0，1 | 支持USB 3.0与USB 2.0切换，其中0为USB 3.0配置，1为USB 2.0配置。当该控制器只支持USB 2.0时，需要将此参数修改为1。 |

 版权所有 © 华为技术有限公司 

**说明**

在计算机领域中，host通常指的是一台计算机或服务器，它可以提供服务或资源给其他计算机或设备使用。host通常是指网络中的主机，也可以是一个操作系统中的主机。

而device通常指的是一种计算机外围设备，例如打印机、扫描仪、摄像头、键盘、鼠标等。这些设备通常需要连接到计算机或服务器上才能使用。在移动设备领域中，device通常指的是智能手机、平板电脑、手持设备等。

```
hiusbc1@A5080000 {
compatible = "hisilicon,hiusbc";
reg = <0x0 0xA5080000 0x0 0x80000>;    // USB_CTRL1 BASE
interrupt-parent = <&mbigen_usb1>;
interrupts = <922 4>;
maximum-speed = "super-speed";
dma-coherent;
mode = <2>;                            // 1:host 2:device
usb-id = <1>;                         // USB Controller id
u2-only = <0>;
io-subctrl-base = <0x0 0xA0120000 0x0 0x10000>;
pcs-base = <0x0 0xA0804000 0x0 0x100>;

phy {
phy_base = <0x0 0xa0810000 0x0 0x200>;
cfg_off = <0x14>;
cfg_mask = <0x3FFC0000>;
cfg_value = <0x4350040>;
};
};
```

**说明**

USB控制器0只支持host，不支持device。

2. 按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤12** 将新增的整机dts文件添加到编译路径中。

1. 执行如下命令，进入“dtbtool”目录。

   **cd** */opt*/**Ascend310B-source/dtb/dtbtool/**

2. 执行如下命令，修改CMakelists文件，将所增加的dts文件添加到编译路径中，具体修改如下加粗字体所示。

   **vim CMakeLists.txt**

```
set(DTS_BASE_DIR ${TOP_DIR}/tools/customize/dts)
if(PRODUCT STREQUAL "ascend310Besl" OR
   PRODUCT STREQUAL "ascend310B" OR
   PRODUCT STREQUAL "ascend310Bemu" OR
   PRODUCT STREQUAL "ascend310Brc" OR
   PRODUCT STREQUAL "ascend310Brcesl" OR
   PRODUCT STREQUAL "ascend310Brcemu")
    # 当前driver跟esl用相同的dts，后期需要更新到单独的目录
    set(DTS_SRC_DIR ${DTS_BASE_DIR}/hi1910b/hi1910BL)
    set(DTS_FILE_LIST hi1910B-default.dts unreleased/hi1911-esl/hi1911-esl-b600.dts unreleased/
hi1911-esl/hi1911-esl-rc-b600.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0.dts unreleased/
hi1911-fpga/hi1911-fpga-b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0_b2.dts
unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0.dts unreleased/hi1911-fpga/hi1911-fpga-rc-
b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0_b2.dts hi1910B-evb-900.dts hi1910B-
evb-901.dts hi1910B-evb-902.dts hi1910B-evb-903.dts hi1910B-evb-905.dts hi1910B-asic-M150-
B50.dts hi1910B-asic-M100-B51.dts hi1910B-asic-M150-B51.dts hi1910B-asic-M160-B51.dts hi1910B-
asic-M100-B00.dts hi1910B-asic-M101-B00.dts hi1910B-asic-M150-B00.dts hi1910B-asic-M151-
B00.dts  hi1910B-asic-M150-B33.dts)
```

**说明**

新增加的dts只能在加粗字体段落末尾添加。

 版权所有 © 华为技术有限公司 

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤13** 编译DTB文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译DTB文件。

   **bash build.sh dtb**

   出现如下回显，且生成“dt.img”文件表示编译内核userBaseConfig文件成功。

   ```
   generate /opt/Ascend310B-source/output/dt.img success!
   sign /opt/Ascend310B-source/output/dt.img success!
   ```

   **说明**

   编译后的“dt.img”文件会自动存放于“Ascend310B-source/output”目录下。

**----结束**

## 生效 userBaseConfig 和 DTB 文件

**步骤1** 准备网线，连接PC和Atlas 200 AI加速模块底板网口，登录Atlas 200I A2 加速模块整机，具体操作请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用“WinSCP”，将编译后的“userBaseConfig.bin”和“dt.img”文件上传至任意目录下，例如“/run”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 升级“userBaseConfig.bin”文件。

1. 执行如下命令，进入“/run”目录。

   **cd** */run*

2. 执行如下命令，升级“userBaseConfig.bin”文件。

   **/var/davinci/driver/upgrade-tool --device_index -1 --component Usr_Base_Config --path userBaseConfig.bin**

   出现如下回显，表示升级userBaseConfig.bin文件成功。

   ```
   {"device": 0, "succeed"}
   ```

**步骤5** 升级“dt.img”文件。具体请参见**生效DTB文件**。

**步骤6** 升级完成后重启生效。

**reboot**

建议查看串口日志确认升级重启进展。

**----结束**

## 验证接口功能

以连接U盘为例，将配套的U盘连接到Atlas 200I A2 加速模块的底板上。

**步骤1** 将配套的U盘连接到Atlas 200I A2 加速模块的底板上。

 版权所有 © 华为技术有限公司 

**步骤2** 准备串口线，连接PC和Atlas 200I A2 加速模块底板串口。

**步骤3** 登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

**说明**

- Atlas 200I A2 加速模块（使用非Atlas 200I DK A2 开发者套件底板）USB接口登录的默认IP为192.168.1.2，Atlas 200I DK A2 开发者套件USB接口登录的默认IP为192.168.0.2。
- 若Atlas 200I A2 加速模块上电后网络可以ping通，但无法通过SSH登录，请参见**15.1 使用root用户SSH登录服务器出现Access Denied错误**解决。

**步骤4** 执行如下命令，查询硬盘是否在位。

**lsusb**

显示硬盘详细信息，如显示类似厂家信息即表示U盘在位。USB接口通信正常。

**----结束**

## 7.1.5 ETH 接口调测参考

本章节以其中一个不同于Atlas 200I DK A2 开发者套件、Atlas 500 A2 智能小站的用户底板设计为例，根据底板的设计来配置对应ETH接口的userBaseConfig文件和DTB文件。

### 背景信息

- 调测ETH接口需要修改PCIe的.dtsi文件，配置PCIe控制器，否则可能导致启动挂死。
- ETH接口底板设计请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“以太网接口”章节。

### 前提条件

已获取设备boardid，本章节以33150为例。具体请参见**2 用户必读**。

用户底板的adc_board_id为33，Atlas 200I A2 加速模块的算力为20T（adc_board_id为150），用户整机（Atlas 200I A2 加速模块+底板）boardid为33150（对应十六进制817e）。

### 修改和编译 userBaseConfig 和 DTB 文件

本操作以首次调测接口为例介绍。

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*


版权所有 © 华为技术有限公司

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 修改“user_base_config.xml”配置文件。

1. 执行如下命令，进入配置文件所在目录。
   **cd config/adaptive/adaptive_config/**
2. 执行如下命令，打开“user_base_config.xml”文件。
   **vim user_base_config.xml**
3. 在“user_base_config.xml”中配置用户整机对应的**board_id**、**chip0_strategy**、**max_power**和**serdes_info**。具体参数含义请参见表7-11和表7-12。

```
<item name="product_strategy_info"><!--产品板策略ID-->
<!--33150 对应十六进制817e-->
<param board_id ="817e" phy_num = "1" chip0_strategy="0" max_power="0"  equal="0"/>
</item>

<item name="evb_strategy_info">
</item>

<item name="strategy_info_array"><!--启动策略矩阵-->
<subitem subname="sub_strategy0">
<module class="serdes">
<submodule subclass="serdes_info">
<param lan_index="0" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="2"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="1" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0"     port_index = "0"/>
<param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="4" serdes_type="ETH" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff"  align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="5" serdes_type="ETH"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff" align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
<param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
</submodule>
</module>
```

   - 第一处加粗字体修改为用户产品的实际boardid以及算力等级，各参数含义如下。

**表 7-11** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| board_id | 如33150对应十六进制817e。 | 十六进制的boardid。<br>boardid为用户实际使用底板的adc_board_id与Atlas 200I A2 加速模块的adc_board_id的组合值。 |
| phy_num | 1。 | 多p系统才会使用，固定为1。 |


版权所有 © 华为技术有限公司

| param | 配置值 | 说明 |
| --- | --- | --- |
| chip0_strategy | 0~n。 | 策略索引号。对应代码中的sub_strategy0和sub_strategy1。 |
| max_power | ▪ 0表示20T。<br>▪ 2表示8T。 | 最大算力档位。 |

- 第二处加粗字体修改为用户产品的实际SerDes配置，具体规则如下。
  - PCIe控制器0（lane0~lane1）支持RC/EP模式，其他PCIe控制器只支持RC模式。
  - Macro0（lane0~lane3）中若存在PCIe和其他协议（SATA）共存，则PCIe只能支持到PCIE_GEN2；若只存在PCIe协议，则PCIe支持到PCIE_GEN3。
  - PCIe支持降lane应用，如PCIe x4降lane到PCIe x2/x1。
  - Macro0（lane0~lane3）支持PCIE和SATA协议，Macro1（lane4~lane7）支持PCIe、USB、GE协议，每个Macro同时最多只能配置两种协议。
  - SATA只能在Macro0（lane0~lane3）使用。
  - USB控制器0（lane4）只支持HOST，USB控制器1~USB控制器3（lane5~lane7）支持HOST+DEVICE。
  - USB只能在Macro1（lane4~lane7）使用。
  - 若为系统直出的USB 2.0，则无需配置SerDes。

  各参数含义如下。

**表 7-12** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| lan_index | 0~7。 | lane编号，Atlas 200I A2 加速模块一共有8条serdes lane。 |
| serdes_type | Atlas 200I A2 加速模块支持PCIE、SATA、USB、ETH。 | 该条lane使用的协议。 |
| ssc_enable | ▪ 0：否。<br>▪ 1：是。 | 是否开展频，只有用作PCIe和SATA时才能打开。 |


版权所有 © 华为技术有限公司

| param | 配置值 | 说明 |
| --- | --- | --- |
| polarity_tx | ■ 0：否。<br>■ 1：是。 | tx极性是否翻转。 |
| polarity_rx | ■ 0：否。<br>■ 1：是。 | rx极性是否翻转。 |
| lan_order | ■ 0：否。<br>■ 1：是。 | lane序是否反转。 |
| bandwidth | ■ 1表示x1。<br>■ 2表示x2。<br>■ 4表示x4。<br>■ ff表示不配置。 | 用作PCIe时，表示位宽x4/x2/x1，其他协议不使用。<br>■ x4：使用四条lane。<br>■ x2：使用两条lane。<br>■ x1：使用一条lane。 |
| align_mode | 0。 | 该lane是master还是slave，Atlas 200I A2 加速模块不使用该参数。 |
| frequency | ■ PCIE_GEN2。<br>■ PCIE_GEN3。<br>■ 1GE。<br>■ 2.5GE。<br>■ USB3.0<br>■ SATA3.0 | 使用的频点，主要用来区分PCIE GEN2/3，和ETH的1GE/2.5GE。 |
| port_index | 0~n。 | 用作ETH时，表示lane的网口端口号，其他协议不使用。 |

**须知**

弹性配置中的SerDes复用关系参数必须与实际硬件单板保持一致，SerDes错误配置可能导致启动挂死，功能不可用。

根据表7-13中用户产品的实际serdes配置，为使能ETH功能，user_base_config.xml文件中serdes_info配置的示例如下。


版权所有 © 华为技术有限公司

**表 7-13** SerDes 接口复用关系

| lan编号 | 协议 | 是否开展频 | tx极性是否翻转 | rx极性是否翻转 | lane序是否反转 | 位宽 | 频点 |
|---|---|---|---|---|---|---|---|
| lane0 | PCIE | 否 | 否 | 否 | 否 | x2 | PCIE_GEN2 |
| lane1 | PCIE | 否 | 否 | 否 | 否 | 不配置[a] | PCIE_GEN2 |
| lane2 | SATA | 是 | 否 | 否 | 否 | 不配置 | SATA3.0 |
| lane3 | PCIE | 否 | 否 | 否 | 否 | x1 | PCIE_GEN2 |
| lane4 | ETH | 否 | 否 | 否 | 是 | 不配置 | 1GE |
| lane5 | ETH | 否 | 否 | 否 | 是 | 不配置 | 1GE |
| lane6 | USB | 否 | 否 | 否 | 否 | 不配置 | USB3.0 |
| lane7 | USB | 否 | 否 | 否 | 否 | 不配置 | USB3.0 |
| a：lane0的PCIe位宽为x2，需要使用两条lane，所以lane1不设置位宽。 | | | | | | | |

user_base_config.xml文件中serdes_info配置如下。

```
<submodule subclass="serdes_info">
<param lan_index="0" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="2"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="1" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0"   port_index = "0"/>
<param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
<param lan_index="4" serdes_type="ETH" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff"  align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="5" serdes_type="ETH"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff" align_mode="0" frequency="1GE" port_index = "0"/>
<param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
<param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
</submodule>
```

**说明**

位宽只能配置为x4/x2/x1。

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤8** 编译userBaseConfig文件。

 版权所有 © 华为技术有限公司 

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译userBaseConfig文件。

   **bash build.sh usrBaseConfig**

   出现如下回显，表示编译内核userBaseConfig文件成功。

   ```
   generate /opt/Ascend310B-source/output/userBaseConfig.bin success!
   sign /opt/Ascend310B-source/output/userBaseConfig.bin success!
   ```

   **说明**

   编译后的“userBaseConfig.bin”文件会自动存放于“Ascend310B-source/output”目录下。

**步骤9** 新增并修改整机dts文件。

用户整机的boardid为33150，故dts文件名称为“hi1910B-asic-M150-B33.dts”。具体请参见**DTB文件**。

1. 执行如下命令，进入配置文件所在目录。

   **cd dtb/dts/hi1910b/hi1910BL**

2. 执行如下命令，新增适配boardid33150的dts文件。

   **cp hi1910B-asic-M150-B50.dts** *hi1910B-asic-M150-B33.dts*

   将“hi1910B-asic-M150-B50.dts”另存为boardid33150的dts文件“hi1910B-asic-M150-B33.dts”。

3. 执行以下命令修改dts文件，配置用户产品的实际整机boardid，确认所使用的PCIe的.dtsi文件和ETH的.dtsi文件及路径，具体内容如下加粗字体所示。

   **vim hi1910B-asic-M150-B33.dts**

   ```
   compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
   hisi,boardid = <0x0 0x3 0x3 0x1 0x5 0x0>;  //修改为实际整机boardid
   #address-cells = <0x2>;
   #size-cells = <0x2>;
   interrupt-parent = <0x1>;
   model = "Hisilicon PhosphorHi1910B evb";

   /include/ "base/hi1910B-asic-kernel-rc.dtsi"
   /include/ "base/hi1910B-commmon-smmu.dtsi"
   /include/ "base/hi1910B-mpam.dtsi"
   /include/ "base/hi1910B-lp-pm.dtsi"
   /include/ "base/hi1910B-lp-devmng.dtsi"
   /include/ "base/hi1910B-mbigen-ao.dtsi"
   /include/ "base/hi1910B-gpio.dtsi"
   /include/ "base/hi1910B-ipcdrv.dtsi"
   /include/ "base/hi1910B-ts-drv.dtsi"
   /include/ "base/hi1910B-tsdrv.dtsi"
   /include/ "base/hi1910B-network-M150-B50.dtsi" //确认所使用的ETH的.dtsi文件及路径
   /include/ "base/hi1910B-hidvpp.dtsi"
   /include/ "base/hi1910B-hdmi.dtsi"
   /include/ "product/hi1910B-audio.dtsi"
   /include/ "base/hi1910B-mipitx.dtsi"
   /include/ "base/hi1910B-vdp.dtsi"
   /include/ "base/hi1910B-isp.dtsi"
   /include/ "base/hi1910B-hilink.dtsi"
   /include/ "base/hi1910B-mntn-milanr3.dtsi"
   /include/ "base/hi1910B-gpu.dtsi"
   /include/ "base/hi1910B-ctl.dtsi"
   /include/ "base/hi1910B-itrustee.dtsi"
   /include/ "base/hi1910B-sata.dtsi"
   /include/ "product/hi1910B-pcie-rc-M150-B50.dtsi" //确认所使用的PCIe的.dtsi文件及路径
   /include/ "product/hi1910B-mdio-M150-B50.dtsi"
   /include/ "product/hi1910B-pinctrl-M150-B50.dtsi"
   ```

 版权所有 © 华为技术有限公司 

```
/include/ "product/hi1910B-can-M150-B50.dtsi"
/include/ "product/hi1910B-hiusbc-B50.dtsi"
/include/ "product/hi1910B-emmc-M150-B50.dtsi"
/include/ "product/hi1910B-i2c-M150-B50.dtsi"
/include/ "product/hi1910B-spi-M150-B50.dtsi"
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** 修改PCIe的.dtsi文件，配置PCIe控制器。

1. 执行如下命令进入“product”目录，并修改PCIe的.dtsi文件。

   **cd product**

   **vim** *hi1910B-pcie-rc-M150-B50.dtsi*

   根据**步骤7.3**中配置的**serdes_info**选择需要开放的PCIe控制器，lan_index与PCIe控制器的对应关系如下。

| lan_index | 控制器 |
| --- | --- |
| lane0~lane1 | PCIe控制器0 |
| lane2~lane3 | PCIe控制器1 |
| lane4~lane5 | PCIe控制器2 |
| lane6~lane7 | PCIe控制器3 |

   以**步骤7.3**中**serdes_info**配置为例，共使用了PCIe控制器0、PCIe控制器1，则在PCIe的.dtsi文件中保留PCIe控制器0、PCIe控制器1两个控制器的配置，将PCIe控制器2、PCIe控制器3的配置代码注释即可，其中**/***和***/**代表注释代码段，具体修改如下所示。

```
pcie@0x820000000 {
compatible = "pci-host-ecam-generic";
// ECAM 0x820000000 pci host bridge0 //
reg = <0x8 0x20000000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x0 0x3>;
msi-map = <0x0 &its 0x0 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M //
ranges = <0x02000000 0x0 0xB0000000 0x0 0xB0000000 0x0 0x04000000
0x43000000 0x8 0x30000000 0x8 0x30000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xf800 0 0 0x7>;
// INTX //
interrupt-map = <0x4000 0 0 1 &mbigen_pcie0_local 704 4
0x4000 0 0 2 &mbigen_pcie0_local 705 4
0x4000 0 0 3 &mbigen_pcie0_local 706 4
0x4000 0 0 4 &mbigen_pcie0_local 707 4>;
iommu-map = <0 &io_smmu 0 0x400>;
status = "ok";
};

pcie@0x840000000 {
compatible = "pci-host-ecam-generic";
pcie-core-id = <1>;
// ECAM 0x840000000 pci host bridge0
reg = <0x8 0x40400000 0 0x00400000>;
reg-names = "ecam-cfg";
```


版权所有 © 华为技术有限公司

```
bus-range = <0x4 0x7>;
msi-map = <0x400 &its 0x400 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M
ranges = <0x02000000 0x0 0xB4000000 0x0 0xB4000000 0x0 0x04000000
0x43000000 0x8 0x50000000 0x8 0x50000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xff0000 0 0 0x7>;
// INTX
interrupt-map = <0x40000 0 0 1 &mbigen_pcie1_local 712 4
0x40000 0 0 1 &mbigen_pcie1_local 713 4
0x40000 0 0 1 &mbigen_pcie1_local 714 4
0x40000 0 0 1 &mbigen_pcie1_local 715 4>;
iommu-map = <0x400 &io_smmu 0x400 0x400>;
status = "ok";
};

/*
pcie@0x860000000 {
compatible = "pci-host-ecam-generic";
// ECAM 0x860000000 pci host bridge0 //
reg = <0x8 0x60800000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0x8 0xb>;
msi-map = <0x800 &its 0x800 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M //
ranges = <0x02000000 0x0 0xB8000000 0x0 0xB8000000 0x0 0x04000000
0x43000000 0x8 0x70000000 0x8 0x70000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xf800 0 0 0x7>;
// INTX //
interrupt-map = <0x4000 0 0 1 &mbigen_pcie2_local 720 4
0x4000 0 0 2 &mbigen_pcie2_local 721 4
0x4000 0 0 3 &mbigen_pcie2_local 722 4
0x4000 0 0 4 &mbigen_pcie2_local 723 4>;
iommu-map = <0x800 &io_smmu 0x800 0x400>;
status = "ok";
};
*/

/*
pcie@0x900000000 {
compatible = "pci-host-ecam-generic";
pcie-core-id = <3>;
// ECAM 0x900000000 pci host bridge0
reg = <0x9 0x00c00000 0 0x00400000>;
reg-names = "ecam-cfg";
bus-range = <0xc 0xf>;
msi-map = <0xc00 &its 0xc00 0x2000>;
msi-map-mask = <0xffff>;
#address-cells = <3>;
#size-cells = <2>;
device_type = "pci";
dma-coherent;
// BAR MAP: 32bit mem 64M, 64bit mem 256M
ranges = <0x02000000 0x0 0xBC000000 0x0 0xBC000000 0x0 0x04000000
0x43000000 0x9 0x10000000 0x9 0x10000000 0x0 0x10000000>;
#interrupt-cells = <1>;
interrupt-map-mask = <0xff0000 0 0 0x7>;
// INTX
```

 版权所有 © 华为技术有限公司 

```
interrupt-map = <0xc0000 0 0 1 &mbigen_pcie3_local 728 4
0xc0000 0 0 1 &mbigen_pcie3_local 729 4
0xc0000 0 0 1 &mbigen_pcie3_local 730 4
0xc0000 0 0 1 &mbigen_pcie3_local 731 4>;
iommu-map = <0xc00 &io_smmu 0xc00 0x400>;
status = "ok";
};
*/
```

**说明**

- 若配置PCIe位宽为x4，例如lane0配置为x4，则只需要配置PCIe0控制器即可，无需配置PCIe1控制器。
- **/***和***/**代表注释代码段，注释成功后，部分软件可能不显示注释格式，如MobaXterm，但不影响注释结果。

2. 配置完成后按“Esc”键，再执行**:wq!**命令保存修改，并按“Enter”键退出。

**步骤11** 修改ETH的.dtsi文件。

1. 执行如下命令进入“base”目录，并修改ETH的.dtsi文件。

   **cd base**

   **vim** *hi1910B-network-M150-B50.dtsi*

   此处以上述产品的配置为例。

```
eth0:xge0@a7100000 {
    compatible = "hisilicon,hns3-platform-device";
    id = <0>;
    reg = <0x0 0xa7100000 0x0 0x10000>,
          <0x0 0xa7000000 0x0 0x800000>,
          <0x0 0xa0120000 0x0 0x10000>,
          <0x0 0xa0180000 0x0 0x10000>,
          <0x0 0xa0160000 0x0 0x10000>,
          <0x0 0xa0150000 0x0 0x10000>,
          <0x0 0xa0140000 0x0 0x10000>;
    reg-names = "network_base";
    //249 +#253 - 32 = 0xdd
    interrupts = <
    0x0 0xd9 0x1
    0x0 0xda 0x1
    0x0 0xdb 0x1
    0x0 0xdc 0x1
    0x0 0xdd 0x1
    0x0 0xde 0x1
    0x0 0xdf 0x1
    0x0 0xe0 0x1
    0x0 0xe1 0x1
    0x0 0xe2 0x1
    0x0 0xe3 0x1
    0x0 0xe4 0x1
    0x0 0xe5 0x1
    0x0 0xe6 0x1
    0x0 0xe7 0x1
    0x0 0xe8 0x1
    0x0 0xe9 0x4
    0x0 0xed 0x4
    0x0 0xd1 0x4>;
    status = "ok";
    tc_num = <4>;
    queue_num = <4>;
    gpio_index = <6>;
    phy_addr = <2>;
    phy_mode = <1>; // 0:sgmii 1:rgmii 2:sfp
    port_speed = <1000>; // 10: 10M, 100: 100M, 1000: 1G; 2500: 2.5G(only phy_mode sfp support)
    ds_index = <2>;
    phy_gpio_index = <12>;
};
```

 版权所有 © 华为技术有限公司 

```
eth1:xge1@a7200000 {
    compatible = "hisilicon,hns3-platform-device";
    id = <1>;
    reg = <0x0 0xa7200000 0x0 0x10000>,
          <0x0 0xa7000000 0x0 0x800000>,
          <0x0 0xa0120000 0x0 0x10000>,
          <0x0 0xa0190000 0x0 0x10000>,
          <0x0 0xa0160000 0x0 0x10000>,
          <0x0 0xa0150000 0x0 0x10000>,
          <0x0 0xa0140000 0x0 0x10000>;
    reg-names = "network_base";
    //249 +#253 - 32 = 0xdd
    interrupts = <
    0x0 0xd9 0x1
    0x0 0xda 0x1
    0x0 0xdb 0x1
    0x0 0xdc 0x1
    0x0 0xdd 0x1
    0x0 0xde 0x1
    0x0 0xdf 0x1
    0x0 0xe0 0x1
    0x0 0xe1 0x1
    0x0 0xe2 0x1
    0x0 0xe3 0x1
    0x0 0xe4 0x1
    0x0 0xe5 0x1
    0x0 0xe6 0x1
    0x0 0xe7 0x1
    0x0 0xe8 0x1
    0x0 0xea 0x4
    0x0 0xee 0x4
    0x0 0xd2 0x4>;
    status = "ok";
    tc_num = <4>;
    queue_num = <4>;
    gpio_index = <9>;
    phy_addr = <2>;
    phy_mode = <1>; // 0:sgmii 1:rgmii 2:sfp
    port_speed = <1000>; // 10: 10M, 100: 100M, 1000: 1G; 2500: 2.5G(only phy_mode sfp support)
    ds_index = <3>;
    phy_gpio_index = <25>;
};

xge2@a7300000 {
    compatible = "hisilicon,hns3-platform-device";
    id = <2>;
    reg = <0x0 0xa7300000 0x0 0x10000>,
          <0x0 0xa7000000 0x0 0x800000>,
          <0x0 0xa0120000 0x0 0x10000>,
          <0x0 0xa0180000 0x0 0x10000>,
          <0x0 0xa0160000 0x0 0x10000>,
          <0x0 0xa0150000 0x0 0x10000>,
          <0x0 0xa0140000 0x0 0x10000>;
    reg-names = "network_base";
    //249 +#253 - 32 = 0xdd
    interrupts = <
    0x0 0xd9 0x1
    0x0 0xda 0x1
    0x0 0xdb 0x1
    0x0 0xdc 0x1
    0x0 0xdd 0x1
    0x0 0xde 0x1
    0x0 0xdf 0x1
    0x0 0xe0 0x1
    0x0 0xe1 0x1
    0x0 0xe2 0x1
    0x0 0xe3 0x1
    0x0 0xe4 0x1
```

 版权所有 © 华为技术有限公司 

```
        0x0 0xe5 0x1
        0x0 0xe6 0x1
        0x0 0xe7 0x1
        0x0 0xe8 0x1
        0x0 0xeb 0x4
        0x0 0xef 0x4
        0x0 0xd3 0x4>;
        status = "ok";
        tc_num = <4>;
        queue_num = <4>;
        gpio_index = <6>;
        phy_addr = <4>;
        phy_mode = <0>; // 0:sgmii 1:rgmii 2:sfp
        port_speed = <1000>; // 10: 10M, 100: 100M, 1000: 1G; 2500: 2.5G(only phy_mode sfp support)
        ds_index = <0>;
        phy_gpio_index = <12>;
    };

    xge3@a7400000 {
        compatible = "hisilicon,hns3-platform-device";
        id = <3>;
        reg = <0x0 0xa7400000 0x0 0x10000>,
              <0x0 0xa7000000 0x0 0x800000>,
              <0x0 0xa0120000 0x0 0x10000>,
              <0x0 0xa0190000 0x0 0x10000>,
              <0x0 0xa0160000 0x0 0x10000>,
              <0x0 0xa0150000 0x0 0x10000>,
              <0x0 0xa0140000 0x0 0x10000>;
        reg-names = "network_base";
        //249 +#253 - 32 = 0xdd
        interrupts = <
        0x0 0xd9 0x1
        0x0 0xda 0x1
        0x0 0xdb 0x1
        0x0 0xdc 0x1
        0x0 0xdd 0x1
        0x0 0xde 0x1
        0x0 0xdf 0x1
        0x0 0xe0 0x1
        0x0 0xe1 0x1
        0x0 0xe2 0x1
        0x0 0xe3 0x1
        0x0 0xe4 0x1
        0x0 0xe5 0x1
        0x0 0xe6 0x1
        0x0 0xe7 0x1
        0x0 0xe8 0x1
        0x0 0xec 0x4
        0x0 0xf0 0x4
        0x0 0xd4 0x4>;
        status = "ok";
        tc_num = <4>;
        queue_num = <4>;
        gpio_index = <9>;
        phy_addr = <4>;
        phy_mode = <0>; // 0:sgmii 1:rgmii 2:sfp
        port_speed = <1000>; // 10: 10M, 100: 100M, 1000: 1G; 2500: 2.5G(only phy_mode sfp support)
        ds_index = <1>;
        phy_gpio_index = <12>;
    };
```

- **phy_addr**为底板设计的phy地址（PHYADD），例如ETH设计的phy的地址为1（PHYADD1），则**phy_addr**配置为1。
- **phy_mode**为底板设计时MAC和phy的连接通道，例如ETH使用rgmii作为MAC和phy的连接通道，**则phy_mode**配置为1。

  使用sgmii作为MAC和phy的连接通道，**则phy_mode**配置为0，使用sfp作为MAC和phy的连接通道，**则phy_mode**配置为2。

 版权所有 © 华为技术有限公司 

- **phy_gpio_index**为底板设计时复位phy芯片的管脚，此ETH使用gpio6_25作为复位phy芯片的管脚，则**phy_gpio_index**配置为25。管脚复用关系请参见**表7-15**。
- Atlas 200I A2 加速模块提供了两个MDIO的控制器，寄存器基址分别为(**MDIO0**: 0xA0180000)和(**MDIO1**: 0xA0190000)，用户应根据具体硬件配置选择。MDIO的信息请参考**7.7 MDIO**。
- 源码文件中默认未开放xge2、xge3编译，用户需要使用时需要将**compatible**参数修改为**hisilicon,hns3-platform-device**。
- **tc_num**参数负责划分流量等级使用，最多支持8个tc。
- **queue_num**为队列数，是硬件上的一种资源，最多支持16个，一个端口分配的队列数量不能少于tc_num数量。

2. 按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤12** 将新增的整机dts文件添加到编译路径中。

1. 执行如下命令，进入“dtbtool”目录。

   **cd** */opt*/**Ascend310B-source/dtb/dtbtool/**

2. 执行如下命令，修改CMakelists文件，将所增加的dts文件添加到编译路径中，具体修改如下加粗字体所示。

   **vim CMakeLists.txt**

```
set(DTS_BASE_DIR ${TOP_DIR}/tools/customize/dts)
if(PRODUCT STREQUAL "ascend310Besl" OR
   PRODUCT STREQUAL "ascend310B" OR
   PRODUCT STREQUAL "ascend310Bemu" OR
   PRODUCT STREQUAL "ascend310Brc" OR
   PRODUCT STREQUAL "ascend310Brcesl" OR
   PRODUCT STREQUAL "ascend310Brcemu")
    # 当前driver跟esl用相同的dts，后期需要更新到单独的目录
    set(DTS_SRC_DIR ${DTS_BASE_DIR}/hi1910b/hi1910BL)
    set(DTS_FILE_LIST hi1910B-default.dts unreleased/hi1911-esl/hi1911-esl-b600.dts unreleased/
hi1911-esl/hi1911-esl-rc-b600.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0.dts unreleased/
hi1911-fpga/hi1911-fpga-b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0_b2.dts
unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0.dts unreleased/hi1911-fpga/hi1911-fpga-rc-
b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0_b2.dts hi1910B-evb-900.dts hi1910B-
evb-901.dts hi1910B-evb-902.dts hi1910B-evb-903.dts hi1910B-evb-905.dts hi1910B-asic-M150-
B50.dts hi1910B-asic-M100-B51.dts hi1910B-asic-M150-B51.dts hi1910B-asic-M160-B51.dts hi1910B-
asic-M100-B00.dts hi1910B-asic-M101-B00.dts hi1910B-asic-M150-B00.dts hi1910B-asic-M151-
B00.dts  hi1910B-asic-M150-B33.dts)
```

   **说明**

   新增加的dts只能在加粗字体段落末尾添加。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤13** 编译DTB文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译DTB文件。

   **bash build.sh dtb**

   出现如下回显，且生成“dt.img”文件表示编译内核userBaseConfig文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

 版权所有 © 华为技术有限公司 

**说明**

编译后的“dt.img”文件会自动存放于“Ascend310B-source/output”目录下。

**----结束**

## 生效 userBaseConfig 和 DTB 文件

**步骤1** 准备网线，连接PC和Atlas 200 AI加速模块底板网口，登录Atlas 200I A2 加速模块整机，具体操作请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用“WinSCP”，将编译后的“userBaseConfig.bin”和“dt.img”文件上传至任意目录下，例如“/run”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 升级“userBaseConfig.bin”文件。

1. 执行如下命令，进入“/run”目录。

   **cd** */run*

2. 执行如下命令，升级“userBaseConfig.bin”文件。

   **/var/davinci/driver/upgrade-tool --device_index -1 --component Usr_Base_Config --path userBaseConfig.bin**

   出现如下回显，表示升级userBaseConfig.bin文件成功。

   ```
   {"device": 0, "succeed"}
   ```

**步骤5** 升级“dt.img”文件。具体请参见**生效DTB文件**。

**步骤6** 升级完成后重启生效。

**reboot**

建议查看串口日志确认升级重启进展。

**----结束**

## 验证接口功能

此处以eth2为例。

**步骤1** 准备串口线，连接PC和Atlas 200I A2 加速模块底板串口。

**步骤2** 登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

**说明**

- Atlas 200I A2 加速模块（使用非Atlas 200I DK A2 开发者套件底板）USB接口登录的默认IP为192.168.1.2，Atlas 200I DK A2 开发者套件USB接口登录的默认IP为192.168.0.2。
- 若Atlas 200I A2 加速模块上电后网络可以ping通，但无法通过SSH登录，请参见**15.1 使用root用户SSH登录服务器出现Access Denied错误**解决。

**步骤3** 执行**ifconfig**命令配置eth2网口ip。

**步骤4** 准备网线，连接PC和Atlas 200I A2 加速模块底板eth2网口。

 版权所有 © 华为技术有限公司 

**步骤5** 通过网线登录Atlas 200I A2 加速模块，详细信息请参见**14.1 使用PuTTY登录设备（网口方式）**

正常登录。eth2接口通信正常。

**----结束**

# 7.2 GPIO

## 7.2.1 概述

GPIO（General Purpose Input/Output）为通用输入输出接口。Atlas 200I A2 加速模块总共有9组GPIO，每组提供32个可编程的输入输出管脚，每个管脚可以配置为输入或者输出方向，用于生成或采样特定应用的输出或输入信号。

## 7.2.2 寄存器描述

**表 7-14** GPIO 寄存器配置说明

| GROUP | IOMUX_BASE | GPIO_BASE | DIRECTION_OFFSET | SET_VALUE_OFFSET | GET_VALUE_OFFSET |
|---|---|---|---|---|---|
| 0 | 0xC4000000 | 0xC4040000 | 0x4 | 0x0 | 0x50 |
| 1 | 0xC4000000 | 0xC4050000 | 0x4 | 0x0 | 0x50 |
| 2 | 0x82320000 | 0x820F0000 | 0x4 | 0x0 | 0x50 |
| 3 | 0x82320000 | 0x82100000 | 0x4 | 0x0 | 0x50 |
| 4 | 0xA0140000 | 0xA0160000 | 0x4 | 0x0 | 0x50 |
| 5 | 0x3001D000 | 0x30016000 | 0x4 | 0x0 | 0x50 |
| 7 | 0x40014000 | 0x40015000 | 0x4 | 0x0 | 0x50 |


版权所有 © 华为技术有限公司

| GROUP | IOMUX_BASE | GPIO_BASE | DIRECTION_OFFSET | SET_VALUE_OFFSET | GET_VALUE_OFFSET |
|---|---|---|---|---|---|
| • GROUP：为GPIO组号，Atlas 200I A2 加速模块当前不支持查询和设置第6组的GPIO管脚；第8组管脚为休眠唤醒与安全关机特性使用管脚，不支持寄存器配置。<br>• IOMUX_BASE：GPIO复用关系配置基地址，每个管脚对应的复用关系配置偏移地址不同，详细复用关系请参考**表7-15**。<br>• GPIO_BASE：GPIO寄存器基地址，基地址+偏移地址存储的值为一个32位的数值，该数值的每一位对应一个管脚的配置。<br>• DIRECTION_OFFSET：GPIO管脚方向配置偏移地址，配置GPIO_BASE+DIRECTION_OFFSET即可配置GPIO方向。<br>• SET_VALUE_OFFSET：GPIO管脚值设置偏移地址，配置GPIO_BASE+SET_VALUE_OFFSET即可配置GPIO管脚值。<br>• GET_VALUE_OFFSET：GPIO管脚值查询偏移地址，查询GPIO_BASE+GET_VALUE_OFFSET即可查询GPIO管脚值。 | | | | | |

**表 7-15** 管脚复用关系说明

| GROUP | PIN | IOMUX_BASE | RESET_OFFSET | RESET_GPIO |
|---|---|---|---|---|
| 0 | 0 | 0xC4000000 | 0x010 | 0x3 |
| 0 | 1 | 0xC4000000 | 0X014 | 0x3 |
| 0 | 2 | 0xC4000000 | 0x018 | 0x3 |
| 0 | 3 | 0xC4000000 | 0x01C | 0x3 |
| 0 | 4 | 0xC4000000 | 0x020 | 0x0 |
| 0 | 5 | 0xC4000000 | 0x024 | 0x0 |
| 0 | 6 | 0xC4000000 | 0x028 | 0x0 |
| 0 | 7 | 0xC4000000 | 0x02c | 0x0 |
| 0 | 8 | 0xC4000000 | 0x030 | 0x0 |
| 0 | 9 | 0xC4000000 | 0x034 | 0x0 |
| 0 | 10 | 0xC4000000 | 0x038 | 0x0 |
| 0 | 11 | 0xC4000000 | 0x03C | 0x0 |
| 0 | 12 | 0xC4000000 | 0x0AC | 0x3 |
| 0 | 13 | 0xC4000000 | 0x0B0 | 0x3 |
| 0 | 14 | 0xC4000000 | 0x0B4 | 0x3 |
| 0 | 15 | 0xC4000000 | 0x0B8 | 0x3 |


版权所有 © 华为技术有限公司

| GROUP | PIN | IOMUX_BASE | RESET_OFFSET | RESET_GPIO |
|---|---|---|---|---|
| 0 | 16 | 0xC4000000 | 0x0BC | 0x3 |
| 0 | 17 | 0xC4000000 | 0x0C0 | 0x3 |
| 0 | 18 | 0xC4000000 | 0x0C4 | 0x3 |
| 0 | 19 | 0xC4000000 | 0x0C8 | 0x3 |
| 0 | 20 | 0xC4000000 | 0x0D4 | 0x3 |
| 0 | 21 | 0xC4000000 | 0x0D8 | 0x3 |
| 0 | 22 | 0xC4000000 | 0x0DC | 0x3 |
| 0 | 23 | 0xC4000000 | 0x0E0 | 0x3 |
| 0 | 24 | 0xC4000000 | 0x0FC | 0x3 |
| 0 | 25 | 0xC4000000 | 0x100 | 0x3 |
| 0 | 26 | 0xC4000000 | 0x114 | 0x0 |
| 0 | 27 | 0xC4000000 | 0x118 | 0x0 |
| 1 | 0 | 0xC4000000 | 0x0CC | 0x3 |
| 1 | 1 | 0xC4000000 | 0x0D0 | 0x3 |
| 1 | 2 | 0xC4000000 | 0x0E4 | 0x3 |
| 1 | 3 | 0xC4000000 | 0x0E8 | 0x3 |
| 1 | 4 | 0xC4000000 | 0x0EC | 0x3 |
| 1 | 5 | 0xC4000000 | 0x0F0 | 0x3 |
| 1 | 6 | 0xC4000000 | 0x0F4 | 0x3 |
| 1 | 7 | 0xC4000000 | 0x0F8 | 0x3 |
| 1 | 8 | 0xC4000000 | 0x11C | 0x0 |
| 1 | 9 | 0xC4000000 | 0x120 | 0x0 |
| 1 | 10 | 0xC4000000 | 0x124 | 0x3 |
| 1 | 11 | 0xC4000000 | 0x128 | 0x3 |
| 2 | 9 | 0x82320000 | 0x000 | 0x3 |
| 2 | 10 | 0x82320000 | 0x004 | 0x3 |
| 2 | 11 | 0x82320000 | 0x028 | 0x3 |
| 2 | 12 | 0x82320000 | 0x02C | 0x3 |
| 2 | 13 | 0x82320000 | 0x030 | 0x3 |


版权所有 © 华为技术有限公司

| GROUP | PIN | IOMUX_BASE | RESET_OFFSET | RESET_GPIO |
|---|---|---|---|---|
| 2 | 14 | 0x82320000 | 0x034 | 0x3 |
| 2 | 15 | 0x82320000 | 0x038 | 0x3 |
| 2 | 16 | 0x82320000 | 0x03C | 0x3 |
| 2 | 17 | 0x82320000 | 0x040 | 0x3 |
| 2 | 18 | 0x82320000 | 0x044 | 0x3 |
| 2 | 19 | 0x82320000 | 0x048 | 0x3 |
| 2 | 20 | 0x82320000 | 0x04C | 0x3 |
| 2 | 21 | 0x82320000 | 0x050 | 0x3 |
| 2 | 22 | 0x82320000 | 0x054 | 0x3 |
| 2 | 23 | 0x82320000 | 0x058 | 0x3 |
| 2 | 24 | 0x82320000 | 0x05C | 0x3 |
| 2 | 25 | 0x82320000 | 0x060 | 0x3 |
| 2 | 26 | 0x82320000 | 0x064 | 0x3 |
| 2 | 27 | 0x82320000 | 0x068 | 0x3 |
| 2 | 28 | 0x82320000 | 0x06C | 0x3 |
| 2 | 29 | 0x82320000 | 0x070 | 0x3 |
| 2 | 30 | 0x82320000 | 0x074 | 0x3 |
| 2 | 31 | 0x82320000 | 0x078 | 0x3 |
| 3 | 0 | 0x82320000 | 0x07C | 0x3 |
| 3 | 1 | 0x82320000 | 0x080 | 0x3 |
| 3 | 2 | 0x82320000 | 0x084 | 0x3 |
| 3 | 3 | 0x82320000 | 0x088 | 0x3 |
| 3 | 4 | 0x82320000 | 0x08C | 0x3 |
| 3 | 5 | 0x82320000 | 0x090 | 0x3 |
| 3 | 6 | 0x82320000 | 0x094 | 0x3 |
| 3 | 7 | 0x82320000 | 0x098 | 0x3 |
| 4 | 0 | 0xA0140000 | 0x0BC | 0x0 |
| 4 | 1 | 0xA0140000 | 0x0C0 | 0x0 |
| 4 | 2 | 0xA0140000 | 0x068 | 0x3 |


版权所有 © 华为技术有限公司

| GROUP | PIN | IOMUX_BASE | RESET_OFFSET | RESET_GPIO |
|---|---|---|---|---|
| 4 | 3 | 0xA0140000 | 0x06C | 0x3 |
| 4 | 4 | 0xA0140000 | 0x070 | 0x3 |
| 4 | 5 | 0xA0140000 | 0x074 | 0x3 |
| 4 | 6 | 0xA0140000 | 0x078 | 0x3 |
| 4 | 7 | 0xA0140000 | 0x07C | 0x3 |
| 4 | 8 | 0xA0140000 | 0x080 | 0x3 |
| 4 | 9 | 0xA0140000 | 0x084 | 0x3 |
| 4 | 10 | 0xA0140000 | 0x088 | 0x3 |
| 4 | 11 | 0xA0140000 | 0x08C | 0x3 |
| 4 | 12 | 0xA0140000 | 0x090 | 0x3 |
| 4 | 13 | 0xA0140000 | 0x094 | 0x3 |
| 4 | 14 | 0xA0140000 | 0x098 | 0x3 |
| 4 | 15 | 0xA0140000 | 0x09C | 0x3 |
| 4 | 16 | 0xA0140000 | 0x0A0 | 0x3 |
| 4 | 17 | 0xA0140000 | 0x0A4 | 0x3 |
| 4 | 18 | 0xA0140000 | 0x0A8 | 0x3 |
| 4 | 19 | 0xA0140000 | 0x0AC | 0x3 |
| 4 | 21 | 0xA0140000 | 0x0B4 | 0x3 |
| 4 | 22 | 0xA0140000 | 0x0B8 | 0x3 |
| 4 | 23 | 0xA0140000 | 0x0C4 | 0x3 |
| 4 | 24 | 0xA0140000 | 0x0C8 | 0x3 |
| 5 | 0 | 0x3001D0000 | 0x000 | 0x3 |
| 5 | 1 | 0x3001D0000 | 0x004 | 0x3 |
| 5 | 2 | 0x3001D0000 | 0x008 | 0x3 |
| 5 | 3 | 0x3001D0000 | 0x00C | 0x3 |
| 5 | 4 | 0x3001D0000 | 0x010 | 0x3 |
| 5 | 5 | 0x3001D0000 | 0x014 | 0x3 |
| 5 | 6 | 0x3001D0000 | 0x018 | 0x3 |
| 5 | 7 | 0x3001D0000 | 0x01C | 0x3 |


版权所有 © 华为技术有限公司

| GROUP | PIN | IOMUX_BASE | RESET_OFFSET | RESET_GPIO |
|---|---|---|---|---|
| 5 | 8 | 0x3001D0000 | 0x020 | 0x3 |
| 5 | 9 | 0x3001D0000 | 0x024 | 0x3 |
| 5 | 10 | 0x3001D0000 | 0x028 | 0x3 |
| 5 | 11 | 0x3001D0000 | 0x02C | 0x3 |
| 5 | 12 | 0x3001D0000 | 0x030 | 0x3 |
| 5 | 13 | 0x3001D0000 | 0x034 | 0x3 |
| 5 | 14 | 0x3001D0000 | 0x038 | 0x3 |
| 5 | 15 | 0x3001D0000 | 0x03C | 0x3 |
| 5 | 16 | 0x3001D0000 | 0x040 | 0x3 |
| 5 | 17 | 0x3001D0000 | 0x044 | 0x3 |
| 5 | 18 | 0x3001D0000 | 0x048 | 0x3 |
| 5 | 19 | 0x3001D0000 | 0x04C | 0x3 |
| 7 | 1 | 0x400140000 | 0x000 | 0x3 |
| 7 | 2 | 0x400140000 | 0x030 | 0x3 |
| 7 | 3 | 0x400140000 | 0x034 | 0x3 |
| 7 | 4 | 0x400140000 | 0x038 | 0x3 |
| 7 | 5 | 0x400140000 | 0x03C | 0x3 |
| 7 | 6 | 0x400140000 | 0x040 | 0x3 |
| 7 | 7 | 0x400140000 | 0x044 | 0x3 |
| 7 | 8 | 0x400140000 | 0x048 | 0x3 |
| 7 | 9 | 0x400140000 | 0x04C | 0x3 |
| 7 | 10 | 0x400140000 | 0x050 | 0x3 |
| 7 | 11 | 0x400140000 | 0x054 | 0x3 |
| 7 | 12 | 0x400140000 | 0x058 | 0x3 |
| 7 | 13 | 0x400140000 | 0x05C | 0x3 |

- 查询IOMUX_BASE+RESET_OFFSET的值。
  - 若查询值与RESET_GPIO相等，则说明当前管脚已经被复用为GPIO。
  - 若查询值与RESET_GPIO不相等，将IOMUX_BASE+RESET_OFFSET的值设为RESET_GPIO。
- 表中未体现的管脚表明无复用关系，可直接作为GPIO管脚使用。详细使用方法请参考**13.3.1 GPIO工具介绍**。


版权所有 © 华为技术有限公司

**表 7-16** GPIO 管脚编号映射表

| GROUP | PIN_RANGE | START_ID | END_ID |
| --- | --- | --- | --- |
| 0 | 0~31 | 0 | 31 |
| 1 | 0~31 | 32 | 63 |
| 2 | 0~31 | 64 | 95 |
| 3 | 0~31 | 476 | 507 |
| 4 | 0~31 | 128 | 159 |
| 5 | 0~31 | 160 | 191 |
| 7 | 0~31 | 224 | 255 |
| 此表说明GPIO管脚与其编号的对应关系。比如第N组第M个管脚的编号GPIO_ID为第N组的START_ID+M。 | | | |

## 7.2.3 调测参考

部分管脚存在复用关系，在使用时需确保当前管脚已经被复用为GPIO。

示例：第7组第2个管脚存在复用关系，配置寄存器IOMUX（基地址为0x400140000，偏移地址为0x30）为3，将此管脚复用为GPIO。

若当前管脚已为GPIO，可通过如下方法进行调测。通过DTB设备树方式配置GPIO为永久生效，其余方式配置GPIO为临时生效。

### 通过 DTB 设备树方式

通过在dts文件中配置第7组第2个GPIO管脚，生成DTB文件，具体操作步骤如下，此处以Atlas 200I DK A2 开发者套件为例进行文件配置。

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在的“/opt”目录。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**


版权所有 © 华为技术有限公司

**步骤6** 执行如下命令，进入“Ascend310B-source/dtb/dts/hi1910b/hi1910BL”目录，查看底板的dts文件，如图7-2所示。

**cd Ascend310B-source/dtb/dts/hi1910b/hi1910BL**

**图 7-2** 底板 dts 文件

```
drwxrwxr-x 4 20001 20001 4096 Apr  9 20:56 ./
drwxrwxr-x 3 20001 20001 4096 Apr  9 20:56 ../
drwxrwxr-x 2 20001 20001 4096 Apr  9 20:56 base/
-rwxrwxr-x 1 20001 20001 4304 Apr  9 20:56 hi1910B-asic-M100-B51.dts*
-rwxrwxr-x 1 20001 20001 3931 Apr  9 20:56 hi1910B-asic-M101-B51.dts*
-rwxrwxr-x 1 20001 20001 5260 Apr  9 20:56 hi1910B-asic-M150-B50.dts*
-rwxrwxr-x 1 20001 20001 4302 Apr  9 20:56 hi1910B-asic-M150-B51.dts*
-rwxrwxr-x 1 20001 20001 5286 Apr  9 20:56 hi1910B-asic-M151-B50.dts*
-rwxrwxr-x 1 20001 20001 1806 Apr  9 20:56 hi1910B-evb-900.dts*
-rwxrwxr-x 1 20001 20001 1729 Apr  9 20:56 hi1910B-evb-901.dts*
-rwxrwxr-x 1 20001 20001  731 Apr  9 20:56 hi1910B-evb-902.dts*
-rwxrwxr-x 1 20001 20001 1137 Apr  9 20:56 hi1910B-evb-903.dts*
-rwxrwxr-x 1 20001 20001 1758 Apr  9 20:56 hi1910B-evb-905.dts*
drwxrwxr-x 2 20001 20001 4096 Apr  9 20:56 product/
```

**步骤7** 执行如下命令，进入“product”目录。

**cd product**

**步骤8** 执行如下命令，打开模块的“hi1910B-pinctrl-M100-B51.dtsi”文件。

**vim hi1910B-pinctrl-M100-B51.dtsi**

将基地址为0x400140000，偏移地址为0x30的寄存器配置值修改为0x3，如图7-3所示。

**图 7-3** 修改文件信息示例

```
pmx_media: pinmux@400140000 {
        compatible = "pinctrl-single", "pinctrl-single1";
        reg = <0x4 0x140000 0x0 0x60>;
        #gpio-range-cells = <3>;
        pinctrl-single,register-width = <32>;
        pinctrl-single,function-mask = <7>;

        pinctrl-names = "default", "default", "default";
        pinctrl-0 = <&gpclk0_pmx_func &gpio5_pmx_func>;
        pinctrl-1 = <&i2s0_bclk_tx_pmx_func &i2s0_ws_tx_pmx_func>;
        pinctrl-2 = <&i2s0_sd_tx_pmx_func &i2s0_sd_rx_pmx_func>;

        gpclk0_pmx_func: gpclk0_pmx_func {
                pinctrl-single,pins = <0x30    0x3>;
        };

        gpclk0_pmx_idle: gpclk0_pmx_idle {
                pinctrl-single,pins = <0x30    0x3>;
        };

        i2s0_bclk_tx_pmx_func: i2s0_bclk_tx_pmx_func {
                pinctrl-single,pins = <0x34    0x0>;
        };

        i2s0_bclk_tx_pmx_idle: i2s0_bclk_tx_pmx_idle {
                pinctrl-single,pins = <0x34    0x0>;
        };
```


版权所有 © 华为技术有限公司

修改完成后保存并退出。

**步骤9** 执行如下命令，进入“Ascend310B-source”目录。

**cd** */opt*/**Ascend310B-source**

**步骤10** 执行如下命令，编译OS内核。

**bash build.sh dtb**

出现如下回显，且生成“dt.img”文件表示编译内核userBaseConfig文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

**说明**

编译后的“dt.img”文件会自动存放于“Ascend310B-source/output”目录下。

**步骤11** 参考生效DTB文件章节，使配置生效。

**----结束**

## 通过 gpio_operate 工具

通过gpio_operate工具进行操作，详细使用方法请参考13.3.1 GPIO工具介绍。

## 通过 devmem 工具

通过devmem工具进行操作。配置第7组第2个GPIO管脚说明如下。

**步骤1** 查询IOMUX寄存器（基地址为0x400140000，偏移地址为0x30），查询第7组第2个管脚的复用关系。

**步骤2** 配置IOMUX寄存器（基地址为0x400140000，偏移地址为0x30）为3，将第7组第2个管脚复用为GPIO。

**步骤3** 查询GPIO寄存器（基地址为0x400150000，偏移地址为0x4），第2位为第7组第2个GPIO管脚的方向。

**步骤4** 配置GPIO寄存器（基地址为0x400150000，偏移地址为0x4）的第2位为1，可配置第7组第2个GPIO管脚方向为输出方向。

**步骤5** 查询GPIO寄存器（基地址为0x400150000，偏移地址为0x50），查询第7组第2个GPIO管脚值。

**步骤6** 配置GPIO寄存器（基地址为0x400150000，偏移地址为0x0）的第2位为1，可配置第7组第2个GPIO管脚值为1。

**----结束**

## 通过 Sysfs 方式

通过Sysfs方式控制GPIO。配置第7组第2个GPIO管脚说明如下。

**说明**

*GPIO_ID* = START_ID+M = 224 + 2 = 226

**步骤1** 导出GPIO管脚编号。

 版权所有 © 华为技术有限公司 

**echo** *GPIO_ID* **> /sys/class/gpio/export**

**步骤2** 查询GPIO管脚方向。

**cat /sys/class/gpio/gpio***GPIO_ID***/direction**

**步骤3** 配置GPIO管脚方向为输出方向。

**echo out > /sys/class/gpio/gpio***GPIO_ID***/direction**

**步骤4** 查询GPIO管脚值。

**cat /sys/class/gpio/gpio***GPIO_ID***/value**

**步骤5** 配置GPIO管脚值为1。

**echo 1 > /sys/class/gpio/gpio***GPIO_ID***/value**

**----结束**

## 7.2.4 Demo 参考

本章节主要使用devmem工具配置寄存器，devmem是一个LINUX开发中使用的开源调试工具，可基于开源代码编译此工具，从而进行寄存器配置。用户需自行下载devmem工具，参见**8.3.1.1 编译并生效内核Image文件（Euler）**配置内核编译选项，支持devmem工具进行寄存器配置（若有软实时OS需求，参见**8.3.1.2 编译并生效内核Image文件（openEuler-SP1）**配置内核编译选项）。

### 查询管脚复用关系

查询第7组第2个管脚的复用关系。

命令：**devmem** *IOMUX_BASE+RESET_OFFSET*

示例：**devmem 0x400140030**

### 设置管脚复用关系

将第7组第2个管脚复用为GPIO。

命令：**devmem** *IOMUX_BASE+RESET_OFFSET* **32** *RESET_GPIO*

示例：**devmem 0x400140030 32 0x3**

### 查询 GPIO 管脚方向

查询第7组第2个管脚的方向，

命令：**devmem** *GPIO_BASE+DIRECTION_OFFSET*

示例：**devmem 0x400150004**

回显值第2位为0表示当前方向为输入方向，为1表示当前方向为输出方向。

### 设置 GPIO 管脚方向

设置第7组第2个管脚的方向为输出方向。

命令：**devmem** *GPIO_BASE+DIRECTION_OFFSET* **32** *VALUE*


版权所有 © 华为技术有限公司

示例：**devmem 0x400150004 32 0x4**

### 查询 GPIO 管脚值

查询第7组第2个管脚的值。

命令：**devmem** *GPIO_BASE+GET_VALUE_OFFSET*

示例：**devmem 0x400150050**

### 设置 GPIO 管脚值

设置第7组第2个管脚的值为1。

命令：**devmem** *GPIO_BASE+SET_VALUE_OFFSET* **32** *VALUE*

示例：**devmem 0x400150000 32 0x4**

**说明**

设置GPIO管脚方向以及管脚值时，仅需要操作寄存器存储的值的对应位。

## 7.3 UART

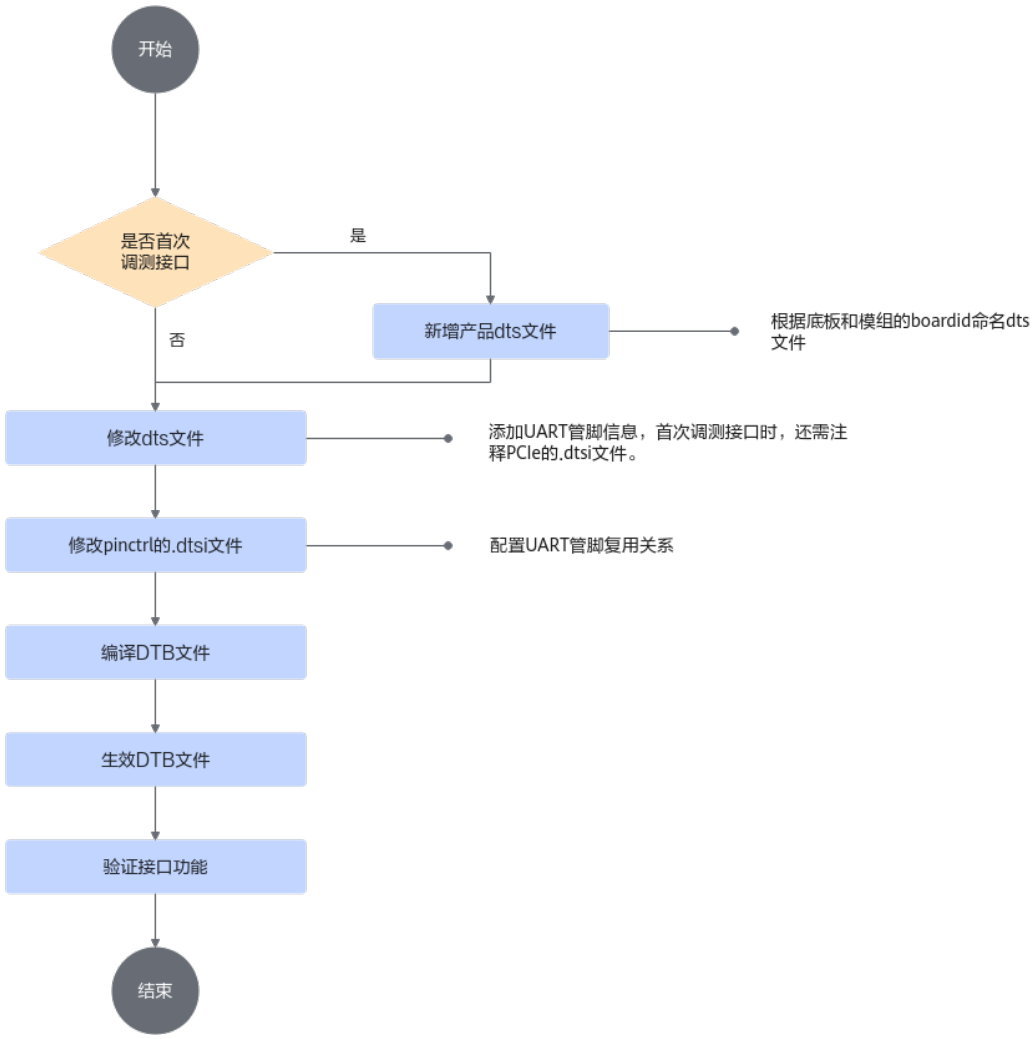
### 7.3.1 调测流程

UART（universal asynchronous receiver/transmitter），通用异步接收发送器，是一种串行、异步、全双工的通信协议。UART将来自CPU等控制设备的并行数据转换为串行数据发送出去，同时可将接收的串行数据转换回接收设备的并行数据，从而实现数据的传输。

Atlas 200I A2 加速模块可提供7路UART接口，UART0为调试串口，出厂即调通，本章节以UART3调测为例，具体调测流程如下图所示。

 版权所有 © 华为技术有限公司 

**图 7-4** UART 接口调测流程




## 7.3.2 寄存器描述

**表 7-17** UART 基地址和中断号

| UART名称 | 基地址 | 中断号 |
| --- | --- | --- |
| UART0 | 0x00C4010000 | 0xa9 |
| UART1 | 0x0082220000 | 0xaa |
| UART2 | 0x0082230000 | 0xab |
| UART3 | 0x0082240000 | 0xac |
| UART6 | 0x0082250000 | 0xad |
| UART7 | 0x0401080000 | 0x11b |


版权所有 © 华为技术有限公司

**表 7-18** UART 管脚复用控制寄存器

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| UTXD1 | 0x0082320000 | 0x038 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_utxd1；<br>011：pad_gpio2_15；<br>111：pad_prb_e[4]；<br>其它：保留。 |
| URXD1 | 0x0082320000 | 0x03C | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_urxd1；<br>011：pad_gpio2_16；<br>111：pad_prb_e[5]；<br>其它：保留。 |
| UTXD2 | 0x0082320000 | 0x040 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_utxd2；<br>001：pad_can_tx3；<br>011：pad_gpio2_17；<br>111：pad_prb_e[6]；<br>其它：保留。 |
| URXD2 | 0x0082320000 | 0x044 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_urxd2；<br>001：pad_can_rx3；<br>011：pad_gpio2_18；<br>111：pad_prb_e[7]；<br>其它：保留。 |


版权所有 © 华为技术有限公司

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| URTS2 | 0x0082320000 | 0x048 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_urts2；<br>001：pad_can_rx2；<br>011：pad_gpio2_19；<br>111：pad_prb_e[8]；<br>其它：保留。 |
| UCTS2 | 0x0082320000 | 0x04C | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_ucts2；<br>001：pad_can_tx2；<br>011：pad_gpio2_20；<br>111：pad_prb_e[9]； |
| UTXD3 | 0x0082320000 | 0x050 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_utxd3；<br>001：pad_can_tx1；<br>011：pad_gpio2_21；<br>111：pad_prb_e[10]； |
| URXD3 | 0x0082320000 | 0x054 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_urxd3；<br>001：pad_can_rx1；<br>011：pad_gpio2_22；<br>111：pad_prb_e[11]； |


版权所有 © 华为技术有限公司

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| URTS3 | 0x0082320000 | 0x058 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_urts3；<br>001：pad_can_rx0；<br>010：pad_urxd6；<br>011：pad_gpio2_23；<br>111：pad_prb_e[12]； |
| UCTS3 | 0x0082320000 | 0x05C | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_ucts3；<br>001：pad_can_tx0；<br>010：pad_utxd6；<br>011：pad_gpio2_24；<br>111：pad_prb_e[13]； |
| I2S0_MCLK | 0x0400140000 | 0x030 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_i2s0_mclk；<br>010：pad_gpclk0；<br>011：pad_gpio7_02；<br>100：pad_utxd7；<br>101：pad_spi7_csn；<br>111：pad_prb_d[2]； |


版权所有 © 华为技术有限公司

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| I2S1_MCLK | 0x0400140000 | 0x044 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_i2s1_mclk；<br>001：pad_i2s_mclk_dbg；<br>010：pad_gpclk1；<br>011：pad_gpio7_07；<br>100：pad_urxd7；<br>101：pad_spi7_sdi；<br>111：pad_prb_d[7]； |

**说明**

- UART6的urxd6、utxd6和UART3的URTS3、UCTS3为复用关系，可以通过UCTS3、URTS3的复用控制寄存器来配置。
- URXD：UART_RXD，接收。
- UTXD：UART_TXD，发送。

## 7.3.3 调测参考

本章节以UART3使用作为参考举例。其他UART例如UART6的配置，可以参考源码包中提供的“hi1910B-asic-M150-B50.dts”文件进行配置。

### 操作步骤

本操作以首次调测为例。

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

 版权所有 © 华为技术有限公司 

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 新增整机dts文件。

例如用户整机的boardid为33150，故dts文件名称为“hi1910B-asic-M150-B33.dts”。具体请参见**DTB文件**。

1. 执行如下命令，进入dts文件目录。

   **cd Ascend310B-source/dtb/dts/hi1910b/hi1910BL**

2. 执行如下命令，新增适配boardid33150的dts文件。

   **cp hi1910B-asic-M150-B50.dts** *hi1910B-asic-M150-B33.dts*

   将“hi1910B-asic-M150-B50.dts”另存为boardid33150的dts文件“hi1910B-asic-M150-B33.dts”。

**步骤7** 修改整机dts文件。

1. 执行以下命令进入dts文件编辑模式。

   **vim hi1910B-asic-M150-B33.dts**

2. 在dts文件中，分别配置用户产品的实际整机boardid，注释PCIe的.dtsi文件和确认所使用的pinctrl的.dtsi文件及路径，具体内容如下加粗字体所示。

```
compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
hisi,boardid = <0x0 0x3 0x3 0x1 0x5 0x0>;  //修改为实际整机boardid
#address-cells = <0x2>;
#size-cells = <0x2>;
interrupt-parent = <0x1>;
model = "Hisilicon PhosphorHi1910B evb";

/include/ "base/hi1910B-asic-kernel-rc.dtsi"
/include/ "base/hi1910B-commmon-smmu.dtsi"
/include/ "base/hi1910B-mpam.dtsi"
/include/ "base/hi1910B-lp-pm.dtsi"
/include/ "base/hi1910B-lp-devmng.dtsi"
/include/ "base/hi1910B-mbigen-ao.dtsi"
/include/ "base/hi1910B-gpio.dtsi"
/include/ "base/hi1910B-ipcdrv.dtsi"
/include/ "base/hi1910B-ts-drv.dtsi"
/include/ "base/hi1910B-tsdrv.dtsi"
/include/ "base/hi1910B-network-M150-B50.dtsi"
/include/ "base/hi1910B-hidvpp.dtsi"
/include/ "base/hi1910B-hdmi.dtsi"
/include/ "product/hi1910B-audio.dtsi"
/include/ "base/hi1910B-mipitx.dtsi"
/include/ "base/hi1910B-vdp.dtsi"
/include/ "base/hi1910B-isp.dtsi"
/include/ "base/hi1910B-hilink.dtsi"
/include/ "base/hi1910B-mntn-milanr3.dtsi"
/include/ "base/hi1910B-gpu.dtsi"
/include/ "base/hi1910B-ctl.dtsi"
/include/ "base/hi1910B-itrustee.dtsi"
/include/ "base/hi1910B-sata.dtsi"
///include/ "product/hi1910B-pcie-rc-M150-B50.dtsi" //注释掉PCIe的.dtsi文件
/include/ "product/hi1910B-mdio-M150-B50.dtsi"
/include/ "product/hi1910B-pinctrl-M150-B50.dtsi"  //确认所使用的pinctrl的.dtsi文件及路径
/include/ "product/hi1910B-can-M150-B50.dtsi"
/include/ "product/hi1910B-hiusbc-B50.dtsi"
/include/ "product/hi1910B-emmc-M150-B50.dtsi"
/include/ "product/hi1910B-i2c-M150-B50.dtsi"
/include/ "product/hi1910B-spi-M150-B50.dtsi"
```

 版权所有 © 华为技术有限公司 

**说明**

如果已调测SerDes接口，直接在已修改的产品dts文件中确认pinctrl的.dtsi文件及路径即可，无需修改boardid和注释PCIe的.dtsi文件。

3. 在dts中配置UART3以及UART3管脚复用为串口管脚，具体内容如下加粗字体所示。

```
uart3: uart@82240000 {
compatible = "arm,pl011", "arm,primecell";
reg = < 0x00 0x82240000 0x00 0x1000 >;  //配置为UART基地址
interrupts = < 0x00 0xac 0x04 >;         //配置UART中断号
current-speed = < 0x1c200 >;
arm,primecell-periphid = <0x00241011>;
clocks = <&refclk150M>;                  //配置UART时钟频率
clock-names = "apb_pclk";
pinctrl-names = "default", "idle";
pinctrl-0 = <&uart3_tx_pmx_func &uart3_rx_pmx_func>;   //工作状态
pinctrl-1 = <&uart3_tx_pmx_idle &uart3_rx_pmx_idle>;   //休眠状态
};
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤8** 修改pinctrl的.dtsi文件

1. 执行如下命令进入“product”目录，进入pinctrl的.dtsi文件的编辑模式。

   **cd product**

   **vim** *hi1910B-pinctrl-M150-B50.dtsi*

2. 在pinctrl的.dtsi文件中配置utxd3、urxd3的复用关系，具体内容如下加粗字体所示。

```
pmx_peri: pinmux@82320000 {
compatible = "pinctrl-single", "pinctrl-single1";
reg = <0x0 0x82320000 0x0 0x9c>; //寄存器中utxd3、urxd3的MUX Base Address。
#gpio-range-cells = <3>;
pinctrl-single,register-width = <32>;
pinctrl-single,function-mask = <7>;
uart3_tx_pmx_func: uart3_tx_pmx_func {
pinctrl-single,pins = <0x50    0x0>;  //0x50为寄存器中管脚的Offset Address，0x0为二进制的管脚复用关系000（pad_utxd3）发送
};
uart3_tx_pmx_idle: uart3_tx_pmx_idle {
pinctrl-single,pins = <0x50    0x0>;  //0x50为寄存器中管脚的Offset Address，0x0为二进制的管脚复用关系000（pad_utxd3）发送
};
uart3_rx_pmx_func:uart3_rx_pmx_func {
pinctrl-single,pins = <0x54    0x0>; //0x54为寄存器中管脚的Offset Address，0x0为二进制的管脚复用关系000（pad_urxd3）接收
};
uart3_rx_pmx_idle: uart3_rx_pmx_idle {
pinctrl-single,pins = <0x54    0x0>; //0x54为寄存器中管脚的Offset Address，0x0为二进制的管脚复用关系000（pad_urxd3）接收
};
};
```

**须知**

同一个管脚只能复用为一种功能，如uart3或can，不可同时复用为多个功能。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤9** 将新增的整机dts文件添加到编译路径中。

1. 执行如下命令，进入“dtbtool”目录。

   **cd** */opt*/**Ascend310B-source/dtb/dtbtool/**

 版权所有 © 华为技术有限公司 

2. 执行如下命令，修改CMakelists文件，将所增加的dts文件添加到编译路径中，具体修改如下加粗字体所示。

   **vim CMakeLists.txt**

```
set(DTS_BASE_DIR ${TOP_DIR}/tools/customize/dts)
if(PRODUCT STREQUAL "ascend310Besl" OR
   PRODUCT STREQUAL "ascend310B" OR
   PRODUCT STREQUAL "ascend310Bemu" OR
   PRODUCT STREQUAL "ascend310Brc" OR
   PRODUCT STREQUAL "ascend310Brcesl" OR
   PRODUCT STREQUAL "ascend310Brcemu")
    # 当前driver跟esl用相同的dts，后期需要更新到单独的目录
    set(DTS_SRC_DIR ${DTS_BASE_DIR}/hi1910b/hi1910BL)
    set(DTS_FILE_LIST hi1910B-default.dts unreleased/hi1911-esl/hi1911-esl-b600.dts unreleased/
hi1911-esl/hi1911-esl-rc-b600.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0.dts unreleased/
hi1911-fpga/hi1911-fpga-b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0_b2.dts
unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0.dts unreleased/hi1911-fpga/hi1911-fpga-rc-
b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0_b2.dts hi1910B-evb-900.dts hi1910B-
evb-901.dts hi1910B-evb-902.dts hi1910B-evb-903.dts hi1910B-evb-905.dts hi1910B-asic-M150-
B50.dts hi1910B-asic-M100-B51.dts hi1910B-asic-M150-B51.dts hi1910B-asic-M160-B51.dts hi1910B-
asic-M100-B00.dts hi1910B-asic-M101-B00.dts hi1910B-asic-M150-B00.dts hi1910B-asic-M151-
B00.dts  hi1910B-asic-M150-B33.dts)
```

   **说明**

   新增加的dts只能在加粗字体段落末尾添加。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** 编译DTB文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译DTB文件。

   **bash build.sh dtb**

   出现如下回显，且生成“dt.img”文件表示编译内核userBaseConfig文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

   **说明**

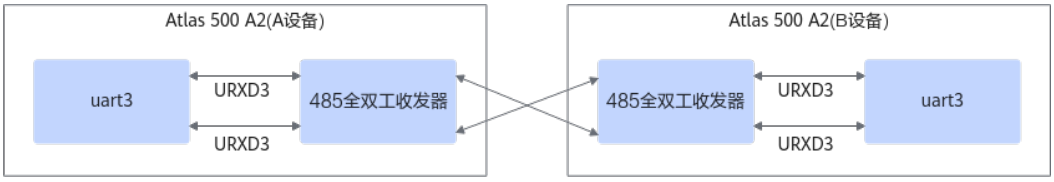
   编译后的“dt.img”文件会自动存放于“Ascend310B-source/output”目录下。

**步骤11** 生效DTB文件，具体请参见**生效DTB文件**。

**----结束**

## 7.3.4 Demo 参考

以Atlas 500 A2 智能小站的UART3调试作为参考，采用两台Atlas 500 A2 智能小站的对接测试UART3串口收发功能。具体请参见《**Atlas 500 A2 智能小站 用户指南**》的“RS485全双工接口使用指导”或“RS485半双工接口使用指导”章节。




在Atlas 500 A2 智能小站（A设备）设备上执行如下命令，通过uart3发送数据包。

**echo test > /dev/ttyAMA2**


版权所有 © 华为技术有限公司

在Atlas 500 A2 智能小站（B设备）设备上执行如下命令，通过uart3接收数据包。

**cat /dev/ttyAMA2**

**test**

## 注意事项

Atlas 500 A2 智能小站的RS485半双工接口调试中，会使用到Atlas 500 A2 智能小站“/usr/local/scripts”路径下自带的“485_work_mode_switch.sh”文件。Atlas 200I A2 加速模块环境需要通过如下操作增加“485_work_mode_switch.sh”文件。

**步骤1** 登录Atlas 200I A2 加速模块OS。详细信息请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤2** 执行如下命令，进入“/usr/local/scripts”路径。

**cd /usr/local/scripts**

**步骤3** 执行如下命令，增加“485_work_mode_switch.sh”文件。

**vim 485_work_mode_switch.sh**

添加如下代码，完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

```
#! /bin/bash
# set 485 work mode switch
current_time=`date +"%b  %-d %H:%M:%S"`
if [ "$1" == "send" ];then
    gpio_operate set_direction 4 19 1 > /dev/null
    gpio_operate set_value 4 19 1 > /dev/null
    if [ $? -eq 0 ];then
        echo "Setting to send mode succeeded."
        echo "$current_time Euler 485_work_mode_switch.sh[8]: Setting to send mode succeeded."
>> /var/log/messages
    else
        echo "Setting to send mode failed."
        echo "$current_time Euler 485_work_mode_switch.sh[11]: Setting to send mode failed." >> /var/log/
messages
    fi
elif [ "$1" == "recv" ];then
    gpio_operate set_direction 4 19 1 > /dev/null
    gpio_operate set_value 4 19 0 > /dev/null
    if [ $? -eq 0 ];then
        echo "Setting to receive mode succeeded."
        echo "$current_time Euler 485_work_mode_switch.sh[18]: Setting to receive mode succeeded."
>> /var/log/messages
    else
        echo "Setting to receive mode failed."
        echo "$current_time Euler 485_work_mode_switch.sh[21]: Setting to receive mode failed." >> /var/log/
messages
    fi
else
    echo "invalid parameter"
fi
```

**步骤4** 执行如下命令，增加“485_work_mode_switch.sh”文件执行权限。

**chmod u+x 485_work_mode_switch.sh**

**----结束**

 版权所有 © 华为技术有限公司 

# 7.4 I2C

## 7.4.1 概述

I²C (Inter-Integrated Circuit Bus) 总线是一种简单、双向二线制同步串行总线。它只需要两根线即可在连接于总线上的器件之间传送信息。

## 7.4.2 寄存器描述

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_CTRL | I²C帧控制寄存器 | 0x0000 | 32 | RW | 0x00000019 | reserved | 31:7 | - | 0x0000000 | reserved |
| | | | | | | hs_mstcode_addr | 6:4 | RW | 0x1 | I²C高速模式下master code地址 |
| | | | | | | restart_en | 3 | RW | 0x1 | 是否允许主设备产生“重新开始”条件。<br>0：不允许；<br>1：允许。 |
| | | | | | | slv_addr_10bit | 2 | RW | 0x0 | 作为主设备时发出的地址位数。<br>0：7位地址；<br>1：10位地址。 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | speed_mode | 1:0 | RW | 0x1 | I2C设备的操作速度。<br>00：标准速度；<br>01：快速速度；<br>10：高速速度；<br>11：reserved。 |
| SLV_ADDR | I²C访问的目标设备地址寄存器 | 0x0004 | 32 | RW | 0x00000055 | reserved | 31:12 | - | 0x00000 | reserved |
| | | | | | | gc_sbyte_en | 11 | RW | 0x0 | 是否执行general call和start byte功能。<br>0：不执行；<br>1：执行。 |
| | | | | | | gc_sbyte_mode | 10 | RW | 0x0 | 当执行general call和start byte的功能时。<br>0：General call；<br>1：Start byte。 |
| | | | | | | slv_addr | 9:0 | RW | 0x055 | Master访问的slave地址<br>7bit地址时：slv_addr[6:0]有效<br>10bit地址时：slv_addr[9:0]有效 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| CMD_TXDATA | $I^2C$设备的命令及TX数据控制寄存器 | 0x0008 | 32 | RW | 0x00000000 | reserved | 31:11 | - | 0x000000 | reserved |
| | | | | | | sr_en | 10 | RW | 0x0 | 该bit控制在1 byte数据被发送或接收前是否发送restart，restart_en为1是该位才有效。<br>0：如果restart_en为1，只有当传输方向从上一个命令改变时，才下发RESTART；当restart_en为0时，下发STOP +START。<br>1：如果restart_en为1，不管传输方向是否从上一个命令改变，则在发送/接收数据之前（根据CMD的值）发出RESTART，；如果restart_en为0，则发出一个STOP后接着一个START。 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | p_en | 9 | RW | 0x0 | 该bit控制在1 byte数据被发送或接收后是否产生stop。<br>0：在该byte之后不发送stop；<br>1：在该byte之后发送stop； |
| | | | | | | rnw | 8 | RW | 0x0 | 读/写控制位。<br>0：写，表示I2C设备将要向I2C总线发送数据。此时低8bit是I2C设备要向I2C总线发送的数据；<br>1：读，表示I2C设备将要从I2C总线读回数据。 |
| | | | | | | tx_data | 7:0 | RW | 0x00 | 将要在I2C总线上发送的数据。 |
| RX DATA | $I^2C$设备接收数据接口 | 0x000C | 32 | RO | 0x00000000 | reserved | 31:8 | - | 0x00000000 | reserved |
| | | | | | | rx_data | 7:0 | RO | 0x00 | 接收数据端口 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| SS_SCL_HCNT | $I^2C$设备的分频数值寄存器。SS模式下SCL高电平周期数配置寄存器 | 0x0010 | 32 | RW | 0x00000010 | reserved | 31:16 | - | 0x0000 | reserved |
| | | | | | | ss_scl_hcnt | 15:0 | RW | 0x0010 | 标准速度下的SCL信号高电平的时钟周期数（core_clk周期）。对应产生的高电平周期数为ss_scl_hcnt + fs_spk_len + 7 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| SS_SCL_LCNT | $I^2C$设备的分频数值寄存器。SS模式下SCL低电平周期数配置寄存器 | 0x0014 | 32 | RW | 0x00001D6 | reserved | 31:16 | - | 0x0000 | reserved |
| | | | | | | ss_scl_lcnt | 15:0 | RW | 0x01D6 | 标准速度下的SCL信号低电平的时钟周期数（core_clk周期）。对应产生的低电平周期数为ss_scl_lcnt + 1 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| FS_SCL_HCNT | $I^2C$设备的分频数值寄存器。FS模式下SCL高电平周期数配置寄存器 | 0x0018 | 32 | RW | 0x00000006 | reserved | 31:16 | - | 0x0000 | reserved |
| | | | | | | fs_scl_hcnt | 15:0 | RW | 0x0006 | 快速速度下的SCL信号高电平的时钟周期数（core_clk周期）。对应产生的高电平周期数为fs_scl_hcnt + fs_spk_len + 7 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| FS_SCL_LCNT | I²C设备的分频数值寄存器。FS模式下SCL低电平周期数配置寄存器 | 0x001C | 32 | RW | 0x00000082 | reserved | 31:16 | - | 0x0000 | reserved |
| | | | | | | fs_scl_lcnt | 15:0 | RW | 0x0082 | 快速速度下的SCL信号低电平的时钟周期数（core_clk周期）。对应产生的低电平周期数为fs_scl_lcnt + 1 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| HS_SCL_HCNT | $I^2C$设备的分频数值寄存器。HS模式下SCL高电平周期数配置寄存器 | 0x0020 | 32 | RW | 0x00000006 | reserved | 31:16 | - | 0x0000 | reserved |
| | | | | | | hs_scl_hcnt | 15:0 | RW | 0x0006 | 高速速度下的SCL信号高电平的时钟周期数（core_clk周期）。对应产生的高电平周期数为hs_scl_hcnt + hs_spk_len + 7 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| HS_SCL_LCNT | I²C设备的分频数值寄存器。HS模式下SCL低电平周期数配置寄存器 | 0x0024 | 32 | RW | 0x00000010 | reserved | 31:16 | - | 0x0000 | reserved |
| | | | | | | hs_scl_lcnt | 15:0 | RW | 0x0010 | 高速速度下的SCL信号低电平的时钟周期数（core_clk周期）。对应产生的低电平周期数为hs_scl_lcnt + 1 |
| FIFO_CTRL | FIFO控制寄存器 | 0x0028 | 32 | RW | 0x00000004F0 | reserved | 31:14 | - | 0x00000 | reserved |
| | | | | | | tx_fifo_ae_thre | 13:8 | RW | 0x04 | tx fifo将空阈值 |
| | | | | | | rx_fifo_af_thre | 7:2 | RW | 0x3C | rx fifo将满阈值 |
| | | | | | | tx_fifo_clr | 1 | RW | 0x0 | tx_fifo清空寄存器，1有效，需配1再配0 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | rx_fifo_clr | 0 | RW | 0x0 | rx_fifo清空寄存器，1有效，需配1再配0 |
| FIFO_STATE | FIFO状态寄存器 | 0x002C | 32 | RO | 0x000030C | fifo_state | 31:0 | RO | 0x00000030C | fifo状态<br>bit31~12：reserved<br>bit11：tx_fifo_full<br>bit10：tx_fifo_afull<br>bit9：tx_fifo_empty<br>bit8：tx_fifo_aempty<br>bit7：tx_fifo_werror<br>bit6：tx_fifo_rerror<br>bit5：rx_fifo_full<br>bit4：rx_fifo_afull<br>bit3：rx_fifo_empty<br>bit2：rx_fifo_aempty<br>bit1：rx_fifo_werror<br>bit0：rx_fifo_rerror |

 版权所有 © 华为技术有限公司 

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| SDA_HOLD | I2C SDA HOLD时间控制寄存器 | 0x0030 | 32 | RW | 0x00000001 | reserved | 31:24 | - | 0x00 | reserved |
| | | | | | | sda_hold_rx | 23:16 | RW | 0x00 | 作为接收时，设置要求的SDA保持时间，单位为core_clk周期 |
| | | | | | | sda_hold_tx | 15:0 | RW | 0x0001 | 作为发送时，设置要求的SDA保持时间，以core_clk周期为单位 |
| DMA_CTRL | DMA模式控制寄存器 | 0x0034 | 32 | RW | 0x00000000 | reserved | 31:1 | - | 0x00000000 | reserved |
| | | | | | | dma_en | 0 | RW | 0x0 | DMA模式使能控制<br>0：DMA不使能<br>1：DMA使能 |
| FS_SPK_LEN | 在快速模式下，滤毛刺长度寄存器 | 0x0038 | 32 | RW | 0x00000005 | reserved | 31:8 | - | 0x000000 | reserved |
| | | | | | | fs_spk_len | 7:0 | RW | 0x05 | 以core_clk的cycle为准，Fast Speed模式下设置SCL和SDA中可被过滤毛刺的最长长度，配置为0时不滤波，支持最大为50ns |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| HS_SPK_LEN | 在高速模式下，滤毛刺长度寄存器 | 0x003C | 32 | RW | 0x00000001 | reserved | 31:8 | - | 0x000000 | reserved |
| | | | | | | hs_spk_len | 7:0 | RW | 0x01 | 以core_clk的cycle为准，High Speed模式下设置SCL和SDA中可被过滤毛刺的最长长度，配置为0时不滤波，支持最大为10ns |
| INT_CLR | 中断清除寄存器 | 0x0040 | 32 | WC | 0x00000000 | reserved | 31:1 | - | 0x0000000 | reserved |
| | | | | | | tx_aempty_int | 0 | WC | 0x0 | tx fifo将空中断清除:<br>1：写1时清除中断；<br>0：中断不清楚。 |
| INT_MSTAT | 中断屏蔽状态寄存器 | 0x0044 | 32 | RO | 0x00000000 | reserved | 31:5 | - | 0x0000000 | reserved |
| | | | | | | tx_aempty_int_mstat | 4 | RO | 0x0 | tx fifo将空屏蔽后的中断状态:<br>1'b1：有中断;<br>1'b0：无中断; |
| | | | | | | rx_afull_int_mstat | 3 | RO | 0x0 | rx fifo将满屏蔽后的中断状态:<br>1'b1：有中断;<br>1'b0：无中断; |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | fifo_err_int_mstat | 2 | RO | 0x0 | fifo溢出屏蔽后的中断状态:<br>1'b1：有中断;<br>1'b0：无中断; |
| | | | | | | trans_err_int_mstat | 1 | RO | 0x0 | 传输错误屏蔽后的中断状态:<br>1'b1：有中断;<br>1'b0：无中断; |
| | | | | | | trans_cmplt_int_mstat | 0 | RO | 0x0 | 传输完成屏蔽后的中断状态:<br>1'b1：有中断;<br>1'b0：无中断; |
| INT_CLR | 中断清除寄存器 | 0x0048 | 32 | WC | 0x00000000 | reserved | 31:4 | - | 0x00000000 | reserved |
| | | | | | | rx_afull_int | 3 | WC | 0x0 | rx fifo将满中断清除:<br>1：写1时清除中断;<br>0：中断不清楚。 |
| | | | | | | fifo_err_int | 2 | WC | 0x0 | fifo溢出中断清除:<br>1：写1时清除中断;<br>0：中断不清楚。 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | trans_err_int | 1 | WC | 0x0 | 传输错误中断清除:<br>1：写1时清除中断；<br>0：中断不清楚。 |
| | | | | | | trans_cmplt_int | 0 | WC | 0x0 | 传输完成中断清除:<br>1：写1时清除中断；<br>0：中断不清楚。 |
| INT_MASK | 中断屏蔽寄存器 | 0x004C | 32 | RW | 0x00000001 | reserved | 31:5 | - | 0x0000000 | reserved |
| | | | | | | tx_aempty_int_mask | 4 | RW | 0x0 | tx fifo将空中断屏蔽位:<br>1'b0：中断屏蔽;<br>1'b1：中断不屏蔽; |
| | | | | | | rx_afull_int_mask | 3 | RW | 0x0 | rx fifo将满中断屏蔽位:<br>1'b0：中断屏蔽;<br>1'b1：中断不屏蔽; |
| | | | | | | fifo_err_int_mask | 2 | RW | 0x0 | fifo溢出中断屏蔽位:<br>1'b0：中断屏蔽;<br>1'b1：中断不屏蔽; |

 版权所有 © 华为技术有限公司 

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | trans_err_int_mask | 1 | RW | 0x0 | 传输错误中断屏蔽位:<br>1'b0：中断屏蔽;<br>1'b1：中断不屏蔽; |
| | | | | | | trans_cmplt_int_mask | 0 | RW | 0x1 | 传输完成中断屏蔽位:<br>1'b0：中断屏蔽;<br>1'b1：中断不屏蔽; |
| TRANS_STATE | 传输状态寄存器 | 0x0050 | 32 | RO | 0x00000060 | reserved | 31:7 | - | 0x0000000 | reserved |
| | | | | | | scl_level | 6 | RO | 0x1 | SCL线电平状态 |
| | | | | | | sda_level | 5 | RO | 0x1 | SDA线电平状态 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | mtc_st_curr | 4:0 | RO | 0x00 | MTC状态机当前状态<br>5'd0：IDLE<br>5'd1：发送START<br>5'd2：发送Master code<br>5'd3：读取tx fifo<br>5'd4：发送general call或start byte<br>5'd5：读取数据<br>5'd6：发送STOP<br>5'd7：发送7bit设备地址<br>5'd8：发送10bit设备地址first addr<br>5'd9：发送10bit设备地址second addr<br>5'd10：发送Repeat START<br>5'd11：发送写数据<br>5'd12：10bit地址模式下发送Repeat START<br>5'd13：7bit地址模式下发送Repeat START<br>5'd14：HS模式下发送 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | Repeat START<br>5'd15：进入写数据hold状态<br>5'd16：进入读数据hold状态 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| TRANS_ERR | 传输错误寄存器 | 0x0054 | 32 | RO | 0x00000000 | trans_err_state | 31:0 | RO | 0x00000000 | 传输过程中错误状态<br>bit31~11：reserved<br>bit10：general call时接收到nack<br>bit9：general call时为读命令<br>bit8：start byte时接收到ack<br>bit7：start byte时未配置restart_en<br>bit6：hs模式下未配置restart_en<br>bit5：hs模式下发送完master code接收到ack<br>bit4：发送7bit地址是接收到nack<br>bit3：发送10bit first地址时接收到nack<br>bit2：发送10bit second地址时接收到nack<br>bit1：10bit地址模式下读未配置restart_en<br>bit0：写数据时接收到nack |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 属性 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|---|
| I2C_VERSION | 版本寄存器 | 0x0058 | 32 | RO | 0x00000030 | i2c_version | 31:0 | RO | 0x00000030 | I2C版本寄存器 |

## 7.4.3 调测参考

### 准备工具

i2c-tools是嵌入式常用的I$^2$C调试工具，本章节使用的i2c-tools工具为i2cdetect与i2ctransfer。

| 工具 | 功能说明 |
|---|---|
| i2cdetect | 列出I2C总线上的所有设备。 |
| i2ctransfer | 读写i2c设备某个寄存器值，支持16位/32位寄存器的读写。 |

其获取方式如下。

**步骤1** 访问如下链接，下载i2c-tools-4.3.tar.gz。

**https://mirrors.edge.kernel.org/pub/software/utils/i2c-tools/**

**步骤2** 登录Arm架构的Linux服务器。

**步骤3** 执行如下命令，切换至root用户。

**su - root**

**步骤4** 使用“WinSCP”，将软件包“ i2c-tools-4.3.tar.gz”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤5** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤6** 执行以下命令，解压i2c-tools-4.3.tar.gz。

**tar -zxvf i2c-tools-4.3.tar.gz**

**步骤7** 执行以下命令，进入“i2c-tools-4.3”目录。

**cd i2c-tools-4.3**

 版权所有 © 华为技术有限公司 

**步骤8** 执行以下命令进行编译。

**make USE_STATIC_LIB=1**

编译成功后，在当前目录的子目录“tools”下会生成对应的可执行文件“i2cdetect”与“i2ctransfer”。

**----结束**

## 调试操作

**步骤1** 登录待调试I²C的环境。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用“WinSCP”，将i2cdetect与i2ctransfer上传至root用户属组目录下，例如“/opt”。详细操作请参见14.3 使用WinSCP传输文件。

**步骤4** 执行以下命令，添加可执行权限。

**chmod +x i2ctransfer**

**chmod +x i2cdetect**

**步骤5** 执行以下命令，查看系统的I²C总线信息。

**./i2cdetect -l**

```
i2c-0   i2c             HiSilicon I2C Controller 300170000.i2c0 I2C adapter
i2c-1   i2c             HiSilicon I2C Controller 300180000.i2c1 I2C adapter
i2c-2   i2c             HiSilicon I2C Controller 300190000.i2c2 I2C adapter
i2c-3   i2c             HiSilicon I2C Controller 3001a0000.i2c3 I2C adapter
i2c-5   i2c             HiSilicon I2C Controller c4170000.i2c5  I2C adapter
i2c-6   i2c             HiSilicon I2C Controller c4180000.i2c6  I2C adapter
i2c-7   i2c             HiSilicon I2C Controller 82060000.i2c7  I2C adapter
i2c-8   i2c             HiSilicon I2C Controller 82070000.i2c8  I2C adapter
i2c-9   i2c             HiSilicon I2C Controller c4190000.i2c9  I2C adapter
i2c-13  i2c             HiSilicon I2C Controller 401040000.i2c13        I2C adapter
```

**步骤6** 执行以下命令，使用i2cdetect -y查询i2c上挂载的设备。

**./i2cdetect -y 5**

```
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- UU -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

**说明**

当前表明只有0x32挂载有设备。

**步骤7** 执行以下命令，写I²C。

**./i2ctransfer -y -f 8 w3@0x50 0x00 0x80 0x55**

 版权所有 © 华为技术有限公司 

**表 7-19** 参数说明

| 字段 | 说明 |
| --- | --- |
| 8 | I²C设备号。 |
| w3 | 表示写3个值，前两位为对应16位寄存器地址，之后则为要写的值。 |
| 0x50 | I²C从设备地址。 |
| 0x00 0x80 | 表示往0x0080地址写值。 |
| 0x55 | 表示给对应寄存器写0x55。 |

**步骤8** 执行以下命令，读I²C。

**./i2ctransfer -y -f 8 w2@0x50 0x00 0x80 r1**

**表 7-20** 参数说明

| 字段 | 说明 |
| --- | --- |
| 8 | I²C的总线号。 |
| w2 | 表示写2个值，即要读的对应16位寄存器地址。 |
| 0x50 | I²C从设备地址。 |
| 0x00 0x80 | 表示读取0x0080地址的值。 |
| r1 | 表示从寄存器开始读取1个字节。 |

**----结束**

## 7.4.4 Demo 参考

以下为Atlas 500 A2 智能小站新增RTC设备为例。

**步骤1** 获取dts源。

请参见**4.1 下载软件包**章节获取源码包“Ascend310B-source tar.gz”。

**步骤2** 登录Linux服务器。

**步骤3** 执行如下命令，切换至root用户。

**su - root**

**步骤4** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤5** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*


版权所有 © 华为技术有限公司

**步骤6** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤7** 执行如下命令，进入dts文件所在目录。

**cd Ascend310B-source/dtb/dts/hi1910b/hi1910BL**

**步骤8** 执行如下命令，编辑“hi1910B-asic-M150-B50.dts”文件。

**vim hi1910B-asic-M150-B50.dts**

修改dts配置支持rtc设备。

```
&i2c5_ao {
rtc@32 {
compatible = "epson,rx8900";
reg = <0x32>;
};
};
```

**表 7-21** 参数说明

| 字段 | 说明 |
| --- | --- |
| compatible | 表示节点支持某厂家的某设备，用于匹配驱动程序。 |
| 0x32 | rtc设备地址。 |

配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤9** 编译并生效内核dtb文件，详细操作请参见**8.3.1.3 编译并生效内核DTB文件**。

**----结束**

# 7.5 PWM

## 7.5.1 概述

PWM（Pulse Width Modulation）模块是一个用户可编程的PWM控制器，在Atlas 200I A2 加速模块中，可以利用GPIO管脚复用功能，配置GPIO端口复用为PWM模式，通过控制PWM的方波脉冲信号的占空比可以控制风扇的转速。

## 7.5.2 寄存器描述

Atlas 200I A2 加速模块提供了两个PWM管脚复用控制寄存器，描述如下。

 版权所有 © 华为技术有限公司 

| 管脚名称 | 信号名称定义 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- | --- |
| PWM2 | GPIO1_00/FAN_PWM0 | 0xC4000000 | 0xCC | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_pwm2；<br>001：pad_ddr_utxd；<br>011：pad_gpio1_00；<br>111：pad_prb_a[28]；<br>其它：保留 |
| PWM3 | GPIO1_01/FAN_PWM1 | 0xC4000000 | 0xD0 | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_pwm3；<br>001：pad_ddr_urxd；<br>011：pad_gpio1_01；<br>111：pad_prb_a[29]；<br>其它：保留 |

PWM寄存器基地址为0xC4080000，寄存器部分描述如下表。

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| PWM_PRD2 | 2通道脉冲周期寄存器 | 0x20 | 32 | 0x0FFFFFFF | reserved | 31:28 | - | 0x0 | 保留 |
| | | | | | pwm_prd2 | 27:0 | RW | 0xFFFFFFF | 通道2输出PWM信号的周期 |
| CH2_PWL | 2通道脉冲低电平起始寄存器 | 0x24 | 32 | 0x00000000 | reserved | 31:28 | - | 0x0 | 保留 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | ch2_pwl | 27:0 | RW | 0x0000000 | 第2通道PWM信号脉冲低电平开始时刻 |
| CH2_PWH | 2通道脉冲低电平起始寄存器 | 0x28 | 32 | 0x0000000 | reserved | 31:28 | - | 0x0 | 保留 |
| | | | | | ch2_pwh | 27:0 | RW | 0x0000000 | 第2通道PWM信号脉冲高电平开始时刻 |
| PWM_PRD3 | 3通道脉冲周期寄存器 | 0x2C | 32 | 0x0FFFFFFF | reserved | 31:28 | - | 0x0 | 保留 |
| | | | | | pwm_prd3 | 27:0 | RW | 0xFFFFFFF | 通道3输出PWM信号的周期 |
| CH3_PWL | 3通道脉冲低电平起始寄存器 | 0x30 | 32 | 0x0000000 | reserved | 31:28 | - | 0x0 | 保留 |
| | | | | | ch3_pwl | 27:0 | RW | 0x0000000 | 第3通道PWM信号脉冲低电平开始时刻 |
| CH3_PWH | 3通道脉冲低电平起始寄存器 | 0x34 | 32 | 0x0000000 | reserved | 31:28 | - | 0x0 | 保留 |
| | | | | | ch3_pwh | 27:0 | RW | 0x0000000 | 第3通道PWM信号脉冲高电平开始时刻 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|
| MEASURE_TIME | 风扇测量时间窗大小 | 0x0108 | 32 | 0x0000 | reserved | 31:14 | RW | 0x00 | 保留 |
| | | | | | set_cnt500ms | 13:0 | RW | 0x00 | 风扇测量时间窗大小，表示有多少个8KHz时钟 |

## 7.5.3 调测参考

### PWM 寄存器管脚复用配置

PWM管脚与GPIO管脚复用，使用PWM管脚时，需配置复用关系：配置寄存器IOMUX（基地址为0xC4000000，偏移地址为0xCC）为0，配置管脚功能为PWM。

### PWM 寄存器配置

通过配置PWM寄存器可实现管脚外接风扇的转速读写控制，以PWM第2通道为例具体配置步骤如下。

**步骤1** 配置寄存器MEASURE_TIME（偏移地址为0x108，Bit[13-0]），配置风扇测量时间窗大小。

**步骤2** 配置寄存器PWM_PRD2（偏移地址为0x20，Bit[27-0]），设置脉冲周期。

**步骤3** 配置寄存器CH2_PWL （偏移地址为0x24，Bit[27-0]），设置低电平开始时刻。

**步骤4** 配置寄存器CH2_PWH（偏移地址为0x28，Bit[27-0]），设置高电平开始时刻，即设置PWM占空比（风扇转速百分比）。

**步骤5** 查询寄存器CH2_PWH（偏移地址为0x28，Bit[27-0]），查询高电平开始时刻，可查询PWM占空比（风扇转速百分比）。

**----结束**

## 7.5.4 Demo 参考

**说明**

devmem为Linux开发中使用的一个开源调试工具，用户可基于开源代码编译此工具，从而进行寄存器配置。用户需自行下载devmem工具，参见**8.3.1.1 编译并生效内核Image文件（Euler）**配置内核编译选项，支持devmem工具进行寄存器配置（若有软实时OS需求，参见**8.3.1.2 编译并生效内核Image文件（openEuler-SP1）**配置内核编译选项）。


版权所有 © 华为技术有限公司

**步骤1** 设置PWM2管脚复用关系为pad_pwm2。

**devmem 0x00C40000CC 32 0**

**步骤2** 设置PWM风扇测量时间窗大小为250ms。

**devmem 0x00C4080108 32 2000**

**说明**

标准时间窗测量大小1S对应8000周期，250ms对应2000周期。

**步骤3** 设置PWM脉冲周期为10000。

**devmem 0x00C4080020 32 10000**

**步骤4** 设置PWM低电平开始时刻。

**devmem 0x00C4080024 32 0**

**步骤5** 设置PWM高电平开始时刻，即可设置PWM占空比（风扇转速百分比），此处以PWM占空比为45%举例。

**devmem 0x00C4080028 32** *4500*

**步骤6** 查询PWM占空比（风扇转速百分比）。

**devmem 0x00C4080028**

此处可查询到PWM占空比为45%。

**----结束**

# 7.6 CAN

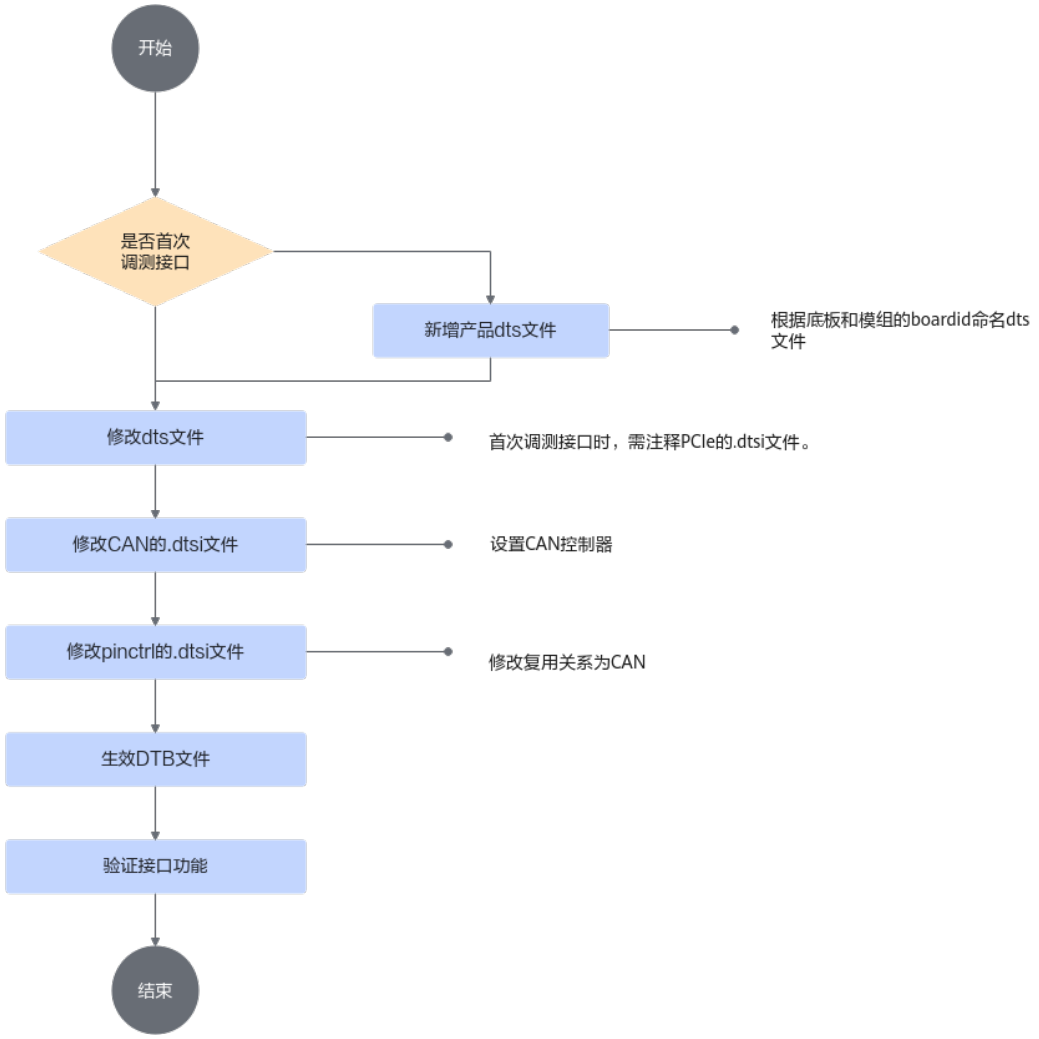
## 7.6.1 调测流程

CAN（Controller Area Network，控制器局域网）总线是广播类型的总线，在同一时刻网络上所有节点侦测的数据是一致的，它是一种基于消息广播模式的串行通信总线。

Atlas 200I A2 加速模块集成了4个独立的CAN设备，连接在APB总线上。可以与具有CAN接口的设备通讯，也可以多个芯片组成控制网络，实现信息的传输、交互和控制功能。本章节以调测Atlas 200I A2 加速模块整机CAN2接口为例。

 版权所有 © 华为技术有限公司 

**图 7-5** 调测流程




## 7.6.2 寄存器描述

**表 7-22** CAN 基地址

| CAN名称 | 基地址 |
| --- | --- |
| CAN0 | 0x00822A0000 |
| CAN1 | 0x00822B0000 |
| CAN2 | 0x00822C0000 |
| CAN3 | 0x00822D0000 |


版权所有 © 华为技术有限公司

**表 7-23** CAN 管脚复用控制寄存器

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| UTXD2 | 0x0082320000 | 0x040 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_utxd2；<br>001：pad_can_tx3；<br>011：pad_gpio2_17；<br>111：pad_prb_e[6]；<br>其它：保留。 |
| URXD2 | 0x0082320000 | 0x044 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_urxd2；<br>001：pad_can_rx3；<br>011：pad_gpio2_18；<br>111：pad_prb_e[7]；<br>其它：保留。 |
| URTS2 | 0x0082320000 | 0x048 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_urts2；<br>001：pad_can_rx2；<br>011：pad_gpio2_19；<br>111：pad_prb_e[8]；<br>其它：保留。 |
| UCTS2 | 0x0082320000 | 0x04C | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_ucts2；<br>001：pad_can_tx2；<br>011：pad_gpio2_20；<br>111：pad_prb_e[9]； |
| UTXD3 | 0x0082320000 | 0x050 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_utxd3；<br>001：pad_can_tx1；<br>011：pad_gpio2_21；<br>111：pad_prb_e[10]； |


版权所有 © 华为技术有限公司

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| URXD3 | 0x0082320000 | 0x054 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_urxd3；<br>001：pad_can_rx1；<br>011：pad_gpio2_22；<br>111：pad_prb_e[11]； |
| URTS3 | 0x0082320000 | 0x058 | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_urts3；<br>001：pad_can_rx0；<br>010：pad_urxd6；<br>011：pad_gpio2_23；<br>111：pad_prb_e[12]； |
| UCTS3 | 0x0082320000 | 0x05C | 0x00000003 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_ucts3；<br>001：pad_can_tx0；<br>010：pad_utxd6；<br>011：pad_gpio2_24；<br>111：pad_prb_e[13]； |

**说明**

- can_rx：接收。
- can_tx：发送。

## 7.6.3 调测参考

### 操作步骤

本操作以首次调测为例。

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。


版权所有 © 华为技术有限公司

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 新增整机dts文件。

例如用户整机的boardid为33150，故dts文件名称为“hi1910B-asic-M150-B33.dts”。具体请参见**DTB文件**。

1. 执行如下命令，进入配置文件所在目录。

   **cd Ascend310B-source/dtb/dts/hi1910b/hi1910BL**

2. 执行如下命令，新增适配boardid33150的dts文件。

   **cp hi1910B-asic-M150-B50.dts** *hi1910B-asic-M150-B33.dts*

   将“hi1910B-asic-M150-B50.dts”另存为boardid33150的dts文件“hi1910B-asic-M150-B33.dts”。

**步骤7** 修改整机dts文件。

1. 执行以下命令进入dts文件编辑模式。

   **vim** hi1910B-asic-M150-B33.dts

2. 在dts文件中，分别配置用户产品的实际整机boardid，注释PCIe的.dtsi文件和确认所使用的CAN和pinctrl的.dtsi文件及路径，具体内容如下加粗字体所示。

```
compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
hisi,boardid = <0x0 0x3 0x3 0x1 0x5 0x0>;   //修改为实际整机boardid
#address-cells = <0x2>;
#size-cells = <0x2>;
interrupt-parent = <0x1>;
model = "Hisilicon PhosphorHi1910B evb";

/include/ "base/hi1910B-asic-kernel-rc.dtsi"
/include/ "base/hi1910B-commmon-smmu.dtsi"
/include/ "base/hi1910B-mpam.dtsi"
/include/ "base/hi1910B-lp-pm.dtsi"
/include/ "base/hi1910B-lp-devmng.dtsi"
/include/ "base/hi1910B-mbigen-ao.dtsi"
/include/ "base/hi1910B-gpio.dtsi"
/include/ "base/hi1910B-ipcdrv.dtsi"
/include/ "base/hi1910B-ts-drv.dtsi"
/include/ "base/hi1910B-tsdrv.dtsi"
/include/ "base/hi1910B-network-M150-B50.dtsi"
/include/ "base/hi1910B-hidvpp.dtsi"
/include/ "base/hi1910B-hdmi.dtsi"
/include/ "product/hi1910B-audio.dtsi"
/include/ "base/hi1910B-mipitx.dtsi"
/include/ "base/hi1910B-vdp.dtsi"
/include/ "base/hi1910B-isp.dtsi"
/include/ "base/hi1910B-hilink.dtsi"
/include/ "base/hi1910B-mntn-milanr3.dtsi"
/include/ "base/hi1910B-gpu.dtsi"
/include/ "base/hi1910B-ctl.dtsi"
/include/ "base/hi1910B-itrustee.dtsi"
/include/ "base/hi1910B-sata.dtsi"
///include/ "product/hi1910B-pcie-rc-M150-B50.dtsi"  //注释PCIe的.dtsi文件
/include/ "product/hi1910B-mdio-M150-B50.dtsi"
/include/ "product/hi1910B-pinctrl-M150-B50.dtsi"   //确认所使用的pinctrl的.dtsi文件及路径
/include/ "product/hi1910B-can-M150-B50.dtsi"       //确认所使用的CAN的.dtsi文件及路径
/include/ "product/hi1910B-hiusbc-B50.dtsi"
/include/ "product/hi1910B-emmc-M150-B50.dtsi"
/include/ "product/hi1910B-i2c-M150-B50.dtsi"
/include/ "product/hi1910B-spi-M150-B50.dtsi"
```

 版权所有 © 华为技术有限公司 

**说明**

如果已调测SerDes接口，直接在已修改的产品dts文件中确认pinctrl的.dtsi文件及路径即可，无需修改boardid和注释PCIe的.dtsi文件。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤8** 修改CAN的.dtsi文件。

1. 执行如下命令进入“product”目录，进入CAN的.dtsi文件的编辑模式。

   **cd product**

   **vim** *hi1910B-can-M150-B50.dtsi*

2. 在CAN的.dtsi文件中确认已添加can2控制器，具体信息如下所示。

```
mttcan@2 {
    compatible = "hisi,mttcan";
    mttcan-id = <2>;
    cclk-freq-hz = <80000000>;
    hclk-freq-hz = <150000000>;
    interrupts = <0 180 0x04>,
                 <0 181 0x04>;
    interrupt-names = "int0", "int1";
    reg = <0x00 0x822C0000 0x00 0x200>,
          <0x00 0x822C1000 0x00 0x1000>,
          <0x00 0x80130A38 0x00 0x8>,
          <0x00 0x82320000 0x00 0x60>,
          <0x00 0xC014FFF8 0x00 0x8>;
    reg-names = "reg-base", "mram-base", "reset-base", "gpio-base", "plat-base";
    pinctrl-names = "default", "idle";
    pinctrl-0 = <&can2_rx_pmx_func &can2_tx_pmx_func>; //工作状态
    pinctrl-1 = <&can2_rx_pmx_idle &can2_tx_pmx_idle>; //休眠状态
};
```

**说明**

其他can控制器信息可以通过查看“/opt/Ascend310B-source/dtb/dts/hi1910b/hi1910BL/base”路径下的“hi1910B-can.dtsi”获取。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤9** 修改pinctrl的.dtsi文件。

1. 执行如下命令进入pinctrl的.dtsi文件的编辑模式。

   **vim** *hi1910B-pinctrl-M150-B50.dtsi*

2. 在pinctrl的.dtsi文件中配置utxd3、urxd3的复用关系，具体内容如下加粗字体所示。

```
    can2_rx_pmx_func: can2_rx_pmx_func {
        pinctrl-single,pins = <0x48    0x1>;//0x48为寄存器中管脚的Offset Address，0x1为二进制的管脚复用关系001（pad_can_rx2）发送
    };

    can2_rx_pmx_idle: can2_rx_pmx_idle {
        pinctrl-single,pins = <0x48    0x1>;//0x48为寄存器中管脚的Offset Address，0x1为二进制的管脚复用关系001（pad_can_rx2）发送
    };

    can2_tx_pmx_func: can2_tx_pmx_func {
        pinctrl-single,pins = <0x4C    0x1>;//0x4C为寄存器中管脚的Offset Address，0x1为二进制的管脚复用关系001（pad_can_tx2）接收
    };

    can2_tx_pmx_idle: can2_tx_pmx_idle {
        pinctrl-single,pins = <0x4C    0x1>;//0x4C为寄存器中管脚的Offset Address，0x1为二进制的管脚复用关系001（pad_can_tx2）接收
    };
```

 版权所有 © 华为技术有限公司 

**须知**

同一个管脚只能复用为一种功能，如uart3或can，不可同时复用为多个功能。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** 将新增的整机dts文件添加到编译路径中。

1. 执行如下命令，进入“dtbtool”目录。

   **cd** */opt*/**Ascend310B-source/dtb/dtbtool/**

2. 执行如下命令，修改CMakelists文件，将所增加的dts文件添加到编译路径中，具体修改如下加粗字体所示。

   **vim CMakeLists.txt**

```
set(DTS_BASE_DIR ${TOP_DIR}/tools/customize/dts)
if(PRODUCT STREQUAL "ascend310Besl" OR
   PRODUCT STREQUAL "ascend310B" OR
   PRODUCT STREQUAL "ascend310Bemu" OR
   PRODUCT STREQUAL "ascend310Brc" OR
   PRODUCT STREQUAL "ascend310Brcesl" OR
   PRODUCT STREQUAL "ascend310Brcemu")
    # 当前driver跟esl用相同的dts，后期需要更新到单独的目录
    set(DTS_SRC_DIR ${DTS_BASE_DIR}/hi1910b/hi1910BL)
    set(DTS_FILE_LIST hi1910B-default.dts unreleased/hi1911-esl/hi1911-esl-b600.dts unreleased/
hi1911-esl/hi1911-esl-rc-b600.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0.dts unreleased/
hi1911-fpga/hi1911-fpga-b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0_b2.dts
unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0.dts unreleased/hi1911-fpga/hi1911-fpga-rc-
b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0_b2.dts hi1910B-evb-900.dts hi1910B-
evb-901.dts hi1910B-evb-902.dts hi1910B-evb-903.dts hi1910B-evb-905.dts hi1910B-asic-M150-
B50.dts hi1910B-asic-M100-B51.dts hi1910B-asic-M150-B51.dts hi1910B-asic-M160-B51.dts hi1910B-
asic-M100-B00.dts hi1910B-asic-M101-B00.dts hi1910B-asic-M150-B00.dts hi1910B-asic-M151-B00.dts
hi1910B-asic-M150-B33.dts)
```

   **说明**

   新增加的dts只能在加粗字体段落末尾添加。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤11** 编译DTB文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译DTB文件。

   **bash build.sh dtb**

   出现如下回显，且生成“dt.img”文件表示编译内核userBaseConfig文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

   **说明**

   编译后的“dt.img”文件会自动存放于“Ascend310B-source/output”目录下。

**步骤12** 生效DTB文件，具体请参见**生效DTB文件**。

**----结束**

## 7.6.4 Demo 参考

具体请参见《**Atlas 500 A2 智能小站 用户指南**》的“CAN接口使用指导”章节。


版权所有 © 华为技术有限公司

# 7.7 MDIO

## 7.7.1 概述

MDIO是对外部以太网接口的PHY芯片进行配置和管理的接口控制模块，软件可以通过配置MDIO相应的寄存器实现对PHY芯片内部寄存器的读写，最终实现对PHY芯片的控制和管理。例如复位控制、自环控制、速度选择、双工选择、冲突测试、自协商使能、物理层状态获取等等。Atlas 200I A2 加速模块直接连PHY场景，不需要单独开发MDIO的驱动，在发布的版本中网口驱动已经包含MDIO的操作接口。在连接的芯片有独立的驱动，并且需要通过MDIO通道管理芯片的场景，需要参考此章节开发MDIO的驱动。

## 7.7.2 寄存器描述

Atlas 200I A2 加速模块提供了两个MDIO的控制器，寄存器基址分别为（**MDIO0**: 0xA0180000）和（**MDIO1**: 0xA0190000）；具体寄存器描述如下。

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|
| MDIO_COMMAND_REG | MDIO控制寄存器 | 0x0000 | 32 | 0x00008000 | rsv | 31:18 | RO | 0x0 | 保留 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | Mdio_clk_sel_exp | 17 | RW | 0x0 | MDIO的时钟模式选择扩展寄存器。与Mdio_clk_sel一起选择时钟模式，{Mdio_clk_sel_exp,Mdio_clk_sel}：<br>默认值为01<br>00：1MHz；<br>01：2.5MHz；<br>10：12.5MHz；<br>11：25MHz（需要保证宽带PHY器件的Tco（MDC的上升沿到MDIO稳定的时间）+双向电平转换器件的延时不大于25ns） |
| | | | | | Mdio_Auto_Scan | 16 | RW | 0x0 | 自动扫描PHY使能（只能扫描符合IEEE802.3-2005协议中第22章描述的寄存器）。配置该bit时，Mdio_Start必须为0 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | Mdio_clk_sel | 15 | RW | 0x1 | MDIO的时钟模式选择寄存器，具体描述见Mdio_clk_sel_exp |
| | | | | | Mdio_Start | 14 | RW/SC | 0x0 | MDIO启动命令控制字。<br>1：表示正在进行MDIO访问，不能进行操作，此位在MDIO访问结束后自动清零。（50us可以完成一次访问）；<br>0：表示MDIO访问已经结束，可以进行写1进行MDIO访问。<br>配置该bit时，Mdio_Auto_Scan必须为0 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | Mdio_St | 13:12 | RW | 0x0 | MDIO访问类型寄存器。<br>2’b00：表示访问的是符合IEEE802.3-2005协议中第45章描述的寄存器；<br>2’b01：表示访问的是符合IEEE802.3-2005协议中第22章描述的寄存器。<br>其余为无效值。 |
| | | | | | Mdio_Op | 11:10 | RW | 0x0 | MDIO操作寄存器。<br>当mdio_st为2’b00时，mdio_op：<br>2’b00代表写地址，<br>2’b01代表写数据，<br>2’b10代表连读操作，<br>2’b11代表非连读的读操作；<br>当st为2’b01时，mdio_op：<br>2’b01代表写，<br>2’b10代表读，<br>其余为无效值。 |

 版权所有 © 华为技术有限公司 

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | Mdio_Prtad | 9:5 | RW | 0x0 | MDIO访问的外部的端口地址。<br>当mdio_st为2’b00时，表示要访问的外部的端口地址；<br>当mdio_st为2’b01时，表示要访问的外部的PHY地址。 |
| | | | | | Mdio_Devad | 4:0 | RW | 0x0 | MDIO访问的外部的设备地址。<br>当mdio_st为2’b00时，表示要访问的外部的设备地址；<br>当mdio_st为2’b01时，表示要访问的外部PHY的寄存器地址。 |
| MDIO_ADDR_REG | MDIO间接地址寄存器 | 0x0004 | 32 | 0x00000000 | Rsv | 31:16 | RO | 0x0 | 保留 |
| | | | | | Mdio_Address | 15:0 | RW | 0x0 | 当mdio_st为2’b00时有效，代表要访问的外部某端口某设备的内部寄存器的地址，当mdio_st为其它值时无效。 |


版权所有 © 华为技术有限公司

| 寄存器名称 | 描述 | 偏移地址 | 宽度 | 默认值 | 字段名称 | 字段范围 | 字段属性 | 字段默认值 | 字段释义 |
|---|---|---|---|---|---|---|---|---|---|
| MDIO_WDATA_REG | MDIO写数据寄存器 | 0x0008 | 32 | 0x00000000 | Rsv | 31:16 | RO | 0x0 | 保留 |
| | | | | | Mdio_Wdata | 15:0 | RW | 0x0 | MDIO写访问的写数据。 |
| MDIO_RDATA_REG | MDIO读数据寄存器 | 0x000C | 32 | 0x00000000 | Rsv | 31:16 | RO | 0x0 | 保留 |
| | | | | | Mdio_Rdata | 15:0 | RO | 0x0 | MDIO读访问的读数据。 |
| MDIO_STA_REG | MDIO访问状态寄存器 | 0x0010 | 32 | 0x00000000 | Rsv | 31:1 | RO | 0x0 | 保留 |
| | | | | | Mdio_Sta | 0 | RO | 0x0 | MDIO读访问的状态。<br>1：表示读出错；<br>0：表示读正常。 |

## 7.7.3 调测参考

### 7.7.3.1 读操作

### MDIO 管脚复用配置

MDIO管脚与GPIO管脚复用，使用MDIO管脚时，需要配置复用关系：配置寄存器IOMUX（基地址为0xA0140000，偏移地址为0x68/0x6C、0x70/0x74）为0，配置管脚功能为MDC0/MDIO0，MDC1/MDIO1。

### Clause22 PHY 寄存器读操作

通过配置相关寄存器可实现对PHY外部寄存器的读访问，具体配置说明如下。

**步骤1** 配置寄存器MDIO_COMMAND_REG[Mdio_Prtad]（偏移地址为0x0000，Bit[9:5]），配置相应的外部PHY地址。

**步骤2** 配置寄存器MDIO_COMMAND_REG[Mdio_Devad]（偏移地址为0x0000，Bit[4:0]），配置相应的寄存器地址。

**步骤3** 配置寄存器MDIO_COMMAND_REG[Mdio_St]（偏移地址为0x0000，Bit[13:12]）为0x01，选择接口模式为GE模式。

**步骤4** 配置寄存器MDIO_COMMAND_REG[Mdio_Op]（偏移地址为0x0000，Bit[11:10]）为0x10，表示读操作。

 版权所有 © 华为技术有限公司 

**步骤5** 配置寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）为0x1，启动读操作。

**步骤6** 查询寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）是否为0，如果为0表示读操作正常，读完成。

**步骤7** 查询寄存器MDIO_RDATA_REG[Mdio_Rdata]（偏移地址为0x000C，Bit[15:0]），MDIO读访问的读数据。

**----结束**

**说明**

上述步骤可以缩减为3个步骤，步骤1到步骤5都是对同一个寄存器进行配置的，可以一次配置完成。

## Clause45 PHY 寄存器读数据操作

Clause45的MDIO读数据操作分为两个MDIO接口操作，一个是写地址操作，一个是读数据操作。通过配置相关寄存器可实现对PHY内部寄存器的读数据操作，具体配置说明如下。

**步骤1** 配置寄存器MDIO_ADDR_REG[Mdio_Address]（偏移地址为0x0004，Bit[15:0]），配置需要读取的PHY中DEV中寄存器的地址。

**步骤2** 配置寄存器MDIO_COMMAND_REG[Mdio_Prtad]（偏移地址为0x0000，Bit[9:5]），配置相应的外部PHY地址。

**步骤3** 配置寄存器MDIO_COMMAND_REG[Mdio_Devad]（偏移地址为0x0000，Bit[4:0]），配置相应的设备地址。

**步骤4** 配置寄存器MDIO_COMMAND_REG[Mdio_St]（偏移地址为0x0000，Bit[13:12]）为0x00，选择接口模式为XGE模式。

**步骤5** 配置寄存器MDIO_COMMAND_REG[Mdio_Op]（偏移地址为0x0000，Bit[11:10]）为0x00，表示写地址操作。

**步骤6** 配置寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）为0x1，启动写地址操作。

**步骤7** 查询寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）是否为0，如果为0表示写地址操作正常完成，进入下个步骤，不是0则在步骤7继续查询。

**步骤8** 配置寄存器MDIO_COMMAND_REG[Mdio_Prtad]（偏移地址为0x0000，Bit[9:5]），配置相应的外部PHY地址。

**步骤9** 配置寄存器MDIO_COMMAND_REG[Mdio_Devad]（偏移地址为0x0000，Bit[4:0]），配置相应的设备地址。

**步骤10** 配置寄存器MDIO_COMMAND_REG[Mdio_St]（偏移地址为0x0000，Bit[13:12]）为0x00，选择接口模式为XGE模式。

**步骤11** 配置寄存器MDIO_COMMAND_REG[Mdio_Op]（偏移地址为0x0000，Bit[11:10]）为0x11，表示读操作。

**步骤12** 配置寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）为0x1，启动读操作。

 版权所有 © 华为技术有限公司 

**步骤13** 查询寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）是否为0，如果为0表示读操作正常，读完成。

**步骤14** 查询寄存器MDIO_RDATA_REG[Mdio_Rdata]（偏移地址为0x000C，Bit[15:0]），MDIO读访问的读数据。

**----结束**

**说明**

上述步骤可以缩减为6个步骤，步骤2到步骤6都是对同一个寄存器进行配置的，可以一次配置完成。步骤8到步骤12都是对同一个寄存器进行配置的，可以一次配置完成。

### 7.7.3.2 写操作

#### MDIO 管脚复用配置

MDIO管脚与GPIO管脚复用，使用MDIO管脚时，需要配置复用关系：配置寄存器IOMUX（基地址为0xA0140000，偏移地址为0x68/0x6C、0x70/0x74）为0，配置管脚功能为MDC0/MDIO0，MDC1/MDIO1。

#### Clause22 PHY 寄存器写操作

通过配置相关寄存器可实现对PHY内部寄存器的写访问，具体配置说明如下。

**步骤1** 配置寄存器MDIO_WDATA_REG[Mdio_Wdata]（偏移地址为0x0008，Bit[15:0]），配置需要写入PHY寄存器的数值。

**步骤2** 配置寄存器MDIO_COMMAND_REG[Mdio_Prtad]（偏移地址为0x0000，Bit[9:5]），配置相应的外部PHY地址。

**步骤3** 配置寄存器MDIO_COMMAND_REG[Mdio_Devad]（偏移地址为0x0000，Bit[4:0]），配置相应的寄存器地址。

**步骤4** 配置寄存器MDIO_COMMAND_REG[Mdio_St]（偏移地址为0x0000，Bit[13:12]）为0x01，选择接口模式为GE模式。

**步骤5** 配置寄存器MDIO_COMMAND_REG[Mdio_Op]（偏移地址为0x0000，Bit[11:10]）为0x01，表示写操作。

**步骤6** 配置寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）为0x1，启动写操作。

**步骤7** 查询寄存器MDIO_STA_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）是否为0，如果为0表示写操作正常，读完成。

**----结束**

**说明**

上述步骤可以缩减为3个步骤，步骤2到步骤6都是对同一个寄存器进行配置的，可以一次配置完成。

#### Clause45 PHY 寄存器写数据操作

Clause45的MDIO写数据操作分为两个MDIO接口操作，一个是写地址操作，一个是写数据操作。通过配置相关寄存器可实现对PHY内部寄存器的写数据操作，具体配置说明如下。

 版权所有 © 华为技术有限公司 

**步骤1** 配置寄存器MDIO_ADDR_REG[Mdio_Address]（偏移地址为0x0004，Bit[15:0]），配置需要写的PHY中DEV中寄存器的地址。

**步骤2** 配置寄存器MDIO_COMMAND_REG[Mdio_Prtad]（偏移地址为0x0000，Bit[9:5]），配置相应的外部PHY地址。

**步骤3** 配置寄存器MDIO_COMMAND_REG[Mdio_Devad]（偏移地址为0x0000，Bit[4:0]），配置相应的设备地址。

**步骤4** 配置寄存器MDIO_COMMAND_REG[Mdio_St]（偏移地址为0x0000，Bit[13:12]）为0x00，选择接口模式为XGE模式。

**步骤5** 配置寄存器MDIO_COMMAND_REG[Mdio_Op]（偏移地址为0x0000，Bit[11:10]）为0x01，表示写地址操作。

**步骤6** 配置寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）为0x1，启动写地址操作。

**步骤7** 查询寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）是否为0，如果为0表示写地址操作正常完成，进入下个步骤，不是0则在步骤7继续查询。

**步骤8** 配置寄存器MDIO_WDATA_REG[Mdio_Wdata]（偏移地址为0x0008，Bit[15:0]），配置需要写入PHY寄存器的数值。

**步骤9** 配置寄存器MDIO_COMMAND_REG[Mdio_Prtad]（偏移地址为0x0000，Bit[9:5]），配置相应的外部PHY地址。

**步骤10** 配置寄存器MDIO_COMMAND_REG[Mdio_Devad]（偏移地址为0x0000，Bit[4:0]），配置相应的寄存器地址。

**步骤11** 配置寄存器MDIO_COMMAND_REG[Mdio_St]（偏移地址为0x0000，Bit[13:12]）为0x00，选择接口模式为XGE模式。

**步骤12** 配置寄存器MDIO_COMMAND_REG[Mdio_Op]（偏移地址为0x0000，Bit[11:10]）为0x01，表示写操作。

**步骤13** 配置寄存器MDIO_COMMAND_REG[Mdio_Start]（偏移地址为0x0000，Bit[14]）为0x1，启动写操作。

**步骤14** 查询寄存器MDIO_STA_REG[Mdio_Sta]（偏移地址为0x0010，Bit[0]）是否为0，如果为0表示写操作正常，读完成。

**----结束**

**说明**

上述步骤可以缩减为6个步骤，步骤2到步骤6都是对同一个寄存器进行配置的，可以一次配置完成。步骤9到步骤13都是对同一个寄存器进行配置的，可以一次配置完成。

## 7.7.4 Demo 参考

**须知**

该验证使用eth1网口作为验证对象，验证过程中会修改PHY的寄存器，不要使用此网口通信。


版权所有 © 华为技术有限公司

以下以GE类型的PHY读写为例。

**步骤1** 使用root用户登录Linux服务器，详细操作请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤2** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤3** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤4** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤5** 执行如下命令，进入“product”目录。

**cd Ascend310B-source/dtb/dts/hi1910b/hi1910BL/product**

**步骤6** 增加dts配置，以Atlas 200I DK A2 开发者套件设备上使用mdio1接口为例。

1. 执行以下命令，编辑“hi1910B-pinctrl-M100-B51.dtsi”文件。

   **vim hi1910B-pinctrl-M100-B51.dtsi**

   在“hi1910B-pinctrl-M100-B51.dtsi”中增加mdio1接口的复用配置。

```
pmx_io: pinmux@a0140000 {
        compatible = "pinctrl-single", "pinctrl-single1";
        reg = <0x0 0xa0140000 0x0 0x9c>;
        #gpio-range-cells = <3>;
        pinctrl-single,register-width = <32>;
        pinctrl-single,function-mask = <7>;

        pinctrl-names = "default", "idle";
        pinctrl-0 = <&mdc1_pmx_func &mdio1_pmx_func>;
        pinctrl-1 = <&mdc1_pmx_idle &mdc1_pmx_idle>;

        mdc1_pmx_func: mdc1_pmx_func {
                pinctrl-single,pins = <0x70     0x0>;
        };

        mdc1_pmx_idle: mdc1_pmx_idle {
                pinctrl-single,pins = <0x70     0x0>;
        };

        mdio1_pmx_func: mdio1_pmx_func {
                pinctrl-single,pins = <0x74     0x0>;
        };

        mdio1_pmx_idle: mdio1_pmx_idle {
                pinctrl-single,pins = <0x74     0x0>;
        };
};
```

2. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤7** 新增“hi1910B-customize-mdio.dtsi”文件。

1. 执行如下命令，新增“hi1910B-customize-mdio.dtsi”文件。

   **vim hi1910B-customize-mdio.dtsi**

2. 增加如下内容。

```
mdio@a0190000 {
        compatible = "hisilicon,hi1910b-mdio";
        reg = <0x0 0xa0190000 0x0 0x1000>,
              <0x0 0xa0120000 0x0 0x1000>;
```

 版权所有 © 华为技术有限公司 

```
            #address-cells = <0x1>;
            #size-cells = <0x0>;
        };
```

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤8** 板级的dts文件中增加“hi1910B-customize-mdio.dtsi”文件引用。

以Atlas 200I DK A2 开发者套件的配置为例。

1. 执行如下命令，进入dts文件所在目录。
   **cd** */opt*/**Ascend310B-source/dtb/dts/hi1910b/hi1910BL**
2. 执行如下命令，进入“hi1910B-asic-M100-B51.dts”编辑模式。
   **vim hi1910B-asic-M100-B51.dts**
3. 在“hi1910B-asic-M100-B51.dts”中增加**/include/ "product/hi1910B-customize-mdio.dtsi"**，如下图所示。

```
/include/ "base/hi1910B-itrustee.dtsi"
/include/ "base/hi1910B-sata.dtsi"
/include/ "product/hi1910B-pcie-rc-M100-B51.dtsi"
/include/ "product/hi1910B-pinctrl-M100-B51.dtsi"
/include/ "product/hi1910B-hiusbc-B51.dtsi"
/include/ "product/hi1910B-customize-mdio.dtsi"
/include/ "product/hi1910B-i2c-M100-B51.dtsi"
/include/ "product/hi1910B-spi-M100-B51.dtsi"
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤9** 执行如下命令，回到“Ascend310B-source”目录下。

**cd** */opt*/**Ascend310B-source**

**步骤10** 执行如下命令，编译DTB文件。

**bash build.sh dtb**

出现如下回显，且生成“dt.img”文件表示编译内核DTB文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

**说明**

编译后的dt.img文件会自动存放于“Ascend310B-source/output”目录下。

**步骤11** 执行如下命令，进入“Ascend310B-source”目录。

**cd** */opt*/**Ascend310B-source**

**步骤12** 执行如下命令，编译内核源码。

**bash build.sh kernelSource**

出现如下类似回显表示编译成功，“/linux-4.19”为生成内核源码路径。

```
/opt/Ascend310B-source/driver
kernel_directory is [/opt/Ascend310B-source/driver/kernel/linux-4.19]
generate kernel source success!
```

**步骤13** 增加mdio的驱动代码。

1. 执行如下命令，进入“Ascend310B-source/driver/drivers”目录。
   **cd** */opt*/**Ascend310B-source/driver/drivers**

 版权所有 © 华为技术有限公司 

2. 执行如下命令，创建并进入“usr”目录。

   **mkdir usr**

   **cd usr**

3. 执行如下命令，创建“hns_mdio.c”文件。

   **vim hns_mdio.c**

   添加如下代码，配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

   **说明**

   直接拷贝文档中代码到Linux系统中使用可能有格式不兼容问题，例如空格与Tab键使用错误，若出现类似报错，请先在Linux系统中调整代码格式正确。

```
#include <linux/errno.h>
#include <linux/etherdevice.h>
#include <linux/init.h>
#include <linux/of.h>
#include <linux/platform_device.h>
#include <linux/kernel.h>
#include <linux/cdev.h>
#include <linux/module.h>
#include <linux/netdevice.h>
#include <linux/of_platform.h>
#include <linux/phy.h>
#include <linux/regmap.h>
#include <linux/version.h>
#include <linux/of_address.h>
#include <linux/of_mdio.h>
#include <linux/securec.h>

#define MDIO_CDEV_MINORS 2
#define PHY_ADDR_FIXED 0
#define MDIO_CDEV_NAME "davinci_mdio_daemo"

#define MDIO_DRV_NAME "HiAscend_MDIO_Daemo"
#define MDIO_BUS_NAME "Hisilicon Ascend MII Bus Daemo"

#define MDIO_TIMEOUT 1000000

/* cfg phy bit map */
#define MDIO_CMD_ST_S 12
#define MDIO_CMD_START_B 14
#define MDIO_CMD_PRTAD_S 5
#define MDIO_CMD_OP_S 10
#define MDIO_CMD_DEVAD_M 0x1f
#define MDIO_CMD_DEVAD_S 0
#define MDIO_CMD_PRTAD_M 0x1f

/* mdio reg */
#define MDIO_RDATA_REG 0xc
#define MDIO_STA_REG 0x10
#define MDIO_COMMAND_REG 0x0
#define MDIO_ADDR_REG 0x4
#define MDIO_WDATA_REG 0x8
#define MDIO_WDATA_DATA_M 0xffff
#define MDIO_WDATA_DATA_S 0

#define MDIO_RDATA_DATA_M 0xffff
#define MDIO_RDATA_DATA_S 0

#define MDIO_ADDR_DATA_M 0xffff
#define MDIO_ADDR_DATA_S 0

#define MDIO_STATE_STA_B 0

enum mdio_st_clause {
```

 版权所有 © 华为技术有限公司 

```
    MDIO_ST_CLAUSE_45 = 0,
    MDIO_ST_CLAUSE_22
};

enum {
    MEM_MDIO_IOBASE,
    MEM_SYSCTRL_IOBASE,
    MEM_IOBASE_MAX,
};

enum mdio_c22_op_seq {
    MDIO_C22_WRITE = 1,
    MDIO_C22_READ = 2
};

struct peri_sc_mdio_reg {
    u16 mdio_clk_en;
    u16 mdio_clk_dis;
    u16 mdio_reset_req;
    u16 mdio_reset_dreq;
    u16 mdio_ctrl;
    u16 mdio_clk_st;
    u16 mdio_reset_st;
};

enum mdio_c45_op_seq {
    MDIO_C45_WRITE_ADDR = 0,
    MDIO_C45_WRITE_DATA,
    MDIO_C45_READ_INCREMENT,
    MDIO_C45_READ
};

struct hns_mdio_device {
    void *vbase; /* mdio reg base address */
    struct regmap *subctrl_vbase;
    const struct peri_sc_mdio_reg *sc_reg;

    struct platform_device *pdev;
    struct mii_bus *bus;

    struct cdev mdio_cdev;
    struct class *mdio_cdev_cls;
    dev_t mdio_devno;
};

static struct mii_bus *hns_mdiobus = NULL;

static void mdio_write_reg(void *base, u32 reg, u32 value)
{
    u8 __iomem *reg_addr = (u8 __iomem *)base;
    writel_relaxed(value, reg_addr + reg);
}

#define MDIO_WRITE_REG(a, reg, value) mdio_write_reg((a)->vbase, (reg), (value))

static u32 mdio_read_reg(void *base, u32 reg)
{
    u8 __iomem *reg_addr = (u8 __iomem *)base;
    return readl_relaxed(reg_addr + reg);
}

#define mdio_set_field(origin, mask, shift, val) do {                                   \
        (origin) &= (~((mask) << (shift)));         \
        (origin) |= (((val) & (mask)) << (shift)); \
    } while (0)

#define mdio_get_field(origin, mask, shift) (((origin) >> (shift)) & (mask))

static void mdio_set_reg_field(void *base, u32 reg, u32 mask, u32 shift, u32 val)
```

 版权所有 © 华为技术有限公司 

```
{
    u32 origin = mdio_read_reg(base, reg);

    mdio_set_field(origin, mask, shift, val);
    mdio_write_reg(base, reg, origin);
}

#define MDIO_SET_REG_FIELD(dev, reg, mask, shift, val) mdio_set_reg_field((dev)->vbase, (reg),
(mask), (shift), (val))

static u32 mdio_get_reg_field(void *base, u32 reg, u32 mask, u32 shift)
{
    u32 origin;

    origin = mdio_read_reg(base, reg);
    return mdio_get_field(origin, mask, shift);
}

#define MDIO_GET_REG_FIELD(dev, reg, mask, shift) mdio_get_reg_field((dev)->vbase, (reg), (mask),
(shift))

#define MDIO_GET_REG_BIT(dev, reg, bit) mdio_get_reg_field((dev)->vbase, (reg), 0x1ull, (bit))

static int hns_mdio_wait_ready(struct mii_bus *bus)
{
    struct hns_mdio_device *mdio_dev = bus->priv;
    u32 cmd_reg_value = 0;
    int i;

    /* waitting for MDIO_COMMAND_REG 's mdio_start==0 */
    /* after that can do read or write */
    for (i = 0; i < MDIO_TIMEOUT; i++) {
        cmd_reg_value = MDIO_GET_REG_BIT(mdio_dev, MDIO_COMMAND_REG, MDIO_CMD_START_B);
        if (!cmd_reg_value) {
            break;
        }
    }

    if ((i == MDIO_TIMEOUT) && cmd_reg_value) {
        printk(KERN_ERR "hns mdio wait timeout.\n");
        return -ETIMEDOUT;
    }

    return 0;
}

static void hns_mdio_cmd_write(struct hns_mdio_device *mdio_dev, u8 is_c45, u8 op, u8 phy_id, u16
cmd)
{
    u32 cmd_reg_value;
    u8 st = is_c45 ? MDIO_ST_CLAUSE_45 : MDIO_ST_CLAUSE_22;

    cmd_reg_value = st << MDIO_CMD_ST_S;
    cmd_reg_value |= op << MDIO_CMD_OP_S;
    cmd_reg_value |= (phy_id & MDIO_CMD_PRTAD_M) << MDIO_CMD_PRTAD_S;
    cmd_reg_value |= (cmd & MDIO_CMD_DEVAD_M) << MDIO_CMD_DEVAD_S;
    cmd_reg_value |= 1 << MDIO_CMD_START_B;

    MDIO_WRITE_REG(mdio_dev, MDIO_COMMAND_REG, cmd_reg_value);
}

static int hns_mdio_write(struct mii_bus *bus, int phy_id, int regnum, u16 data)
{
    int ret;
    struct hns_mdio_device *mdio_dev = (struct hns_mdio_device *)bus->priv;
    u8 devad = (((u32)regnum >> 16) & 0x1f);
    u8 is_c45 = !!((u32)regnum & MII_ADDR_C45);
    u16 reg = (u16)((u32)regnum & 0xffff);
    u8 op;
```

 版权所有 © 华为技术有限公司 

```
    u16 cmd_reg_cfg;

    /* wait for ready */
    ret = hns_mdio_wait_ready(bus);
    if (ret != 0) {
        printk(KERN_ERR "MDIO bus is busy\n");
        return ret;
    }

    if (!is_c45) {
        cmd_reg_cfg = reg;
        op = MDIO_C22_WRITE;
    } else {
        MDIO_SET_REG_FIELD(mdio_dev, MDIO_ADDR_REG, MDIO_ADDR_DATA_M,
MDIO_ADDR_DATA_S, reg);
        hns_mdio_cmd_write(mdio_dev, is_c45, MDIO_C45_WRITE_ADDR, phy_id, devad);

        /* check for read or write opt is finished */
        ret = hns_mdio_wait_ready(bus);
        if (ret != 0) {
            printk(KERN_ERR "MDIO bus is busy\n");
            return ret;
        }

        /* config the data needed writing */
        cmd_reg_cfg = devad;
        op = MDIO_C45_WRITE_ADDR;
    }

    MDIO_SET_REG_FIELD(mdio_dev, MDIO_WDATA_REG, MDIO_WDATA_DATA_M,
MDIO_WDATA_DATA_S, data);
    hns_mdio_cmd_write(mdio_dev, is_c45, op, phy_id, cmd_reg_cfg);

    return 0;
}

static int hns_mdio_read(struct mii_bus *bus, int phy_id, int regnum)
{
    int ret;
    u16 reg_val;
    u8 devad = (((u32)regnum >> 16) & 0x1f);
    u8 is_c45 = !!((u32)regnum & MII_ADDR_C45);
    u16 reg = (u16)((u32)regnum & 0xffff);
    struct hns_mdio_device *mdio_dev = (struct hns_mdio_device *)bus->priv;
    /* Step 1: wait for ready */
    ret = hns_mdio_wait_ready(bus);
    if (ret != 0) {
        printk(KERN_ERR "MDIO bus is busy\n");
        return ret;
    }

    if (!is_c45) {
        hns_mdio_cmd_write(mdio_dev, is_c45, MDIO_C22_READ, phy_id, reg);
    } else {
        MDIO_SET_REG_FIELD(mdio_dev, MDIO_ADDR_REG, MDIO_ADDR_DATA_M,
MDIO_ADDR_DATA_S, reg);

        /* Step 2; config the cmd-reg to write addr */
        hns_mdio_cmd_write(mdio_dev, is_c45, MDIO_C45_WRITE_ADDR, phy_id, devad);

        /* Step 3: check for read or write opt is finished */
        ret = hns_mdio_wait_ready(bus);
        if (ret != 0) {
            printk(KERN_ERR "MDIO bus is busy\n");
            return ret;
        }

        hns_mdio_cmd_write(mdio_dev, is_c45, MDIO_C45_WRITE_ADDR, phy_id, devad);
    }
```

 版权所有 © 华为技术有限公司 

```
    /* Step 5: waitting for MDIO_COMMAND_REG 's mdio_start==0, */
    /* check for read or write opt is finished */
    ret = hns_mdio_wait_ready(bus);
    if (ret != 0) {
        printk(KERN_ERR "MDIO bus is busy\n");
        return ret;
    }
    reg_val = MDIO_GET_REG_BIT(mdio_dev, MDIO_STA_REG, MDIO_STATE_STA_B);
    if (reg_val != 0) {
        printk(KERN_ERR "ERROR! MDIO Read failed!\n");
        return -EBUSY;
    }
    /* Step 6; get out data */
    reg_val = (u16)MDIO_GET_REG_FIELD(mdio_dev, MDIO_RDATA_REG, MDIO_RDATA_DATA_M,
MDIO_RDATA_DATA_S);
    return reg_val;
}

/* *
 * hns_mdio_bus_name - get mdio bus name
 * @name: mdio bus name
 * @np: mdio device node pointer
 */
static void hns_mdio_bus_name(char *name, struct device_node *np)
{
    const u32 *addr = NULL;
    u64 taddr = OF_BAD_ADDR;
    int ret;

    addr = of_get_address(np, 0, NULL, NULL);
    if (addr != NULL) {
        taddr = of_translate_address(np, addr);
    }

    ret = snprintf_s(name, MII_BUS_ID_SIZE, MII_BUS_ID_SIZE - 1, "%s@%llx", np->name, (unsigned
long long)taddr);
    if (ret > 0) {
        printk(KERN_INFO "get bus name suceess\n");
    }
}

static int mdio_open(struct inode *node, struct file *file)
{
    struct hns_mdio_device *mdio_dev;

    mdio_dev = container_of(node->i_cdev, struct hns_mdio_device, mdio_cdev);
    file->private_data = (void *)mdio_dev;

    return 0;
}

static int mdio_release(struct inode *node, struct file *file)
{
    file->private_data = NULL;
    return 0;
}

static long mdio_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    return 0;
}

static const struct file_operations mdio_ops = {
    .owner = THIS_MODULE,
    .open = mdio_open,
    .release = mdio_release,
    .unlocked_ioctl = mdio_ioctl,
};
```

 版权所有 © 华为技术有限公司 

```
static int hns_mdio_cdev_create(struct hns_mdio_device *mdio_dev)
{
    struct device *drv_dev = NULL;
    int ret;

    ret = alloc_chrdev_region(&mdio_dev->mdio_devno, 0, MDIO_CDEV_MINORS, MDIO_CDEV_NAME);
    if (ret != 0) {
        printk(KERN_ERR "alloc_chrdev_region failed, ret = %d\n", ret);
        goto _out;
    }

    mdio_dev->mdio_cdev_cls = class_create(THIS_MODULE, MDIO_CDEV_NAME);
    if (IS_ERR(mdio_dev->mdio_cdev_cls)) {
        printk(KERN_ERR "class_create faild, ret = %pK\n", mdio_dev->mdio_cdev_cls);
        goto _unregister_region;
    }

    drv_dev = device_create(mdio_dev->mdio_cdev_cls, NULL, mdio_dev->mdio_devno, NULL,
MDIO_CDEV_NAME);
    if (IS_ERR(drv_dev)) {
        printk(KERN_ERR "failed to create device, ret = %ld\n", PTR_ERR(drv_dev));
        goto _class_del;
    }

    cdev_init(&mdio_dev->mdio_cdev, &mdio_ops);
    mdio_dev->mdio_cdev.owner = THIS_MODULE;
    ret = cdev_add(&mdio_dev->mdio_cdev, mdio_dev->mdio_devno, MDIO_CDEV_MINORS);
    if (ret != 0) {
        printk(KERN_ERR "cdev_add failed, ret = %d\n", ret);
        goto _device_del;
    }

    printk(KERN_INFO "mdio cdev create finish, name is %s\n", MDIO_CDEV_NAME);
    return 0;

_device_del:
    device_destroy(mdio_dev->mdio_cdev_cls, mdio_dev->mdio_devno);
_class_del:
    class_destroy(mdio_dev->mdio_cdev_cls);
_unregister_region:
    unregister_chrdev_region(mdio_dev->mdio_devno, MDIO_CDEV_MINORS);
_out:
    return -EINVAL;
}

int hns_mdio_cdev_del(struct hns_mdio_device *mdio_dev)
{
    cdev_del(&mdio_dev->mdio_cdev);
    device_destroy(mdio_dev->mdio_cdev_cls, mdio_dev->mdio_devno);
    class_destroy(mdio_dev->mdio_cdev_cls);
    unregister_chrdev_region(mdio_dev->mdio_devno, MDIO_CDEV_MINORS);
    mdio_dev->mdio_cdev_cls = NULL;

    printk(KERN_INFO "mdio cdev del finish\n");
    return 0;
}

static const struct peri_sc_mdio_reg hi1911_peri_sc_mdio_reg = {
    .mdio_clk_en = 0x320,
    .mdio_clk_dis = 0x324,
    .mdio_reset_req = 0x418,
    .mdio_reset_dreq = 0x41C,
    .mdio_ctrl = 0,
    .mdio_clk_st = 0x5320,
    .mdio_reset_st = 0x5418,
};

static const struct of_device_id hns_mdio_match[] = {
    {
```

 版权所有 © 华为技术有限公司 

```
        .compatible = "hisilicon,hi1910b-mdio",
        .data = &hi1911_peri_sc_mdio_reg
    },
    { }
};

MODULE_DEVICE_TABLE(of, hns_mdio_match);

static int hns_mdiobus_register(struct device_node *np, struct mii_bus *new_bus, int *no_phy)
{
    int ret;

    ret = of_mdiobus_register(new_bus, np);
    if (ret != 0) {
        printk(KERN_ERR "Cannot register as MDIO bus!\n");
        return ret;
    }
    return ret;
}

static int hns_mdio_bus_info_init(struct platform_device *pdev,
    struct hns_mdio_device *mdio_dev, struct mii_bus *new_bus, struct device_node *np)
{
    int ret;
    int no_phy = 0;
    struct resource *res = NULL;

    new_bus->name = MDIO_BUS_NAME;
    new_bus->read = hns_mdio_read;
    new_bus->write = hns_mdio_write;
    new_bus->priv = mdio_dev;
    hns_mdio_bus_name(new_bus->id, np);

    res = platform_get_resource(pdev, IORESOURCE_MEM, MEM_MDIO_IOBASE);
    mdio_dev->vbase = devm_ioremap_resource(&pdev->dev, res);
    if (IS_ERR(mdio_dev->vbase)) {
        ret = PTR_ERR(mdio_dev->vbase);
        return ret;
    }

    res = platform_get_resource(pdev, IORESOURCE_MEM, MEM_SYSCTRL_IOBASE);
    mdio_dev->subctrl_vbase = devm_ioremap(&pdev->dev, res->start, resource_size(res));
    if (IS_ERR(mdio_dev->subctrl_vbase)) {
        ret = PTR_ERR(mdio_dev->subctrl_vbase);
        return ret;
    }

#if LINUX_VERSION_CODE >= KERNEL_VERSION(4, 5, 0)
#else
    new_bus->irq = devm_kcalloc(&pdev->dev, PHY_MAX_ADDR, sizeof(int), GFP_KERNEL);
    if (!new_bus->irq) {
        return -ENOMEM;
    }
#endif

    new_bus->parent = &pdev->dev;
    platform_set_drvdata(pdev, new_bus);
    hns_mdiobus = new_bus;

    ret = hns_mdiobus_register(np, new_bus, &no_phy);
    if (ret != 0) {
        platform_set_drvdata(pdev, NULL);
        return ret;
    }

    ret = hns_mdio_cdev_create(mdio_dev);
    if (ret != 0) {
        printk(KERN_ERR "hns_mdio_cdev_create failed\n");
        if (!no_phy) {
```

 版权所有 © 华为技术有限公司 

```
            mdiobus_unregister(new_bus);
        }
        platform_set_drvdata(pdev, NULL);
    }

    return 0;
}

static void hhns_mdio_test(struct mii_bus *bus)
{
    u16 phy_id_1;
    u16 phy_id_2;
    u16 value;
    int ret;

    printk(KERN_INFO "Now let's test the mdio function.");
    ret = hns_mdio_read(bus, 0x1, 0x2);
    if (ret < 0) {
        printk(KERN_ERR "Function hns_mdio_read phy addr 0x1, reg 0x2 failed.");
        return;
    }
    phy_id_1 = (u16)ret;

    ret = hns_mdio_read(bus, 0x1, 0x3);
    if (ret < 0) {
        printk(KERN_ERR "Function hns_mdio_read phy addr 0x1, reg 0x3 failed.");
        return;
    }
    phy_id_2 = (u16)ret;
    printk(KERN_INFO "Now get phy addr 0x1, reg 0x2 value [0x%x].", phy_id_1);
    printk(KERN_INFO "Now get phy addr 0x1, reg 0x3 value [0x%x].", phy_id_2);

    ret = hns_mdio_read(bus, 0x1, 0x0);
    if (ret < 0) {
        printk(KERN_ERR "Function hns_mdio_read phy addr 0x1, reg 0x0 failed.");
        return;
    }
    value = (u16)ret;
    printk(KERN_INFO "Now get phy addr 0x1, reg 0x0 value [0x%x].", value);

    value &= 0xFEFF;
    ret = hns_mdio_write(bus, 0x1, 0x0, value);
    if (ret != 0) {
        printk(KERN_ERR "Function hns_mdio_write phy addr 0x1, reg 0x0 failed.");
        return;
    }
    printk(KERN_INFO "Now set phy addr 0x1, reg 0x0 value [0x%x].", value);

    ret = hns_mdio_read(bus, 0x1, 0x0);
    if (ret < 0) {
        printk(KERN_ERR "Function hns_mdio_read phy addr 0x1, reg 0x0 failed.");
        return;
    }
    value = (u16)ret;
    printk(KERN_INFO "Now get phy addr 0x1, reg 0x0 value [0x%x].", value);
}

/* *
 * hns_mdio_probe - probe mdio device
 * @pdev: mdio platform device
 *
 * Return 0 on success, negative on failure
 */
static int hns_mdio_probe(struct platform_device *pdev)
{
    struct device_node *np = NULL;
    struct hns_mdio_device *mdio_dev = NULL;
    struct mii_bus *new_bus = NULL;
    const struct of_device_id *of_id = NULL;
```

 版权所有 © 华为技术有限公司 

```
    int ret;

    if (pdev == NULL) {
        printk(KERN_ERR  "pdev is NULL!\n");
        return -ENODEV;
    }

    np = pdev->dev.of_node;
    mdio_dev = (struct hns_mdio_device *)devm_kzalloc(&pdev->dev, sizeof(struct hns_mdio_device),
GFP_KERNEL);
    if (mdio_dev == NULL) {
        return -ENOMEM;
    }

    new_bus = devm_mdiobus_alloc(&pdev->dev);
    if (new_bus == NULL) {
        printk(KERN_ERR  "mdiobus_alloc fail!\n");
        return -ENOMEM;
    }

    of_id = of_match_device(hns_mdio_match, &pdev->dev);
    if (of_id == NULL) {
        return -ENODEV;
    }
    mdio_dev->sc_reg = of_id->data;

    ret = hns_mdio_bus_info_init(pdev, mdio_dev, new_bus, np);
    if (ret != 0) {
        printk(KERN_ERR "hns_mdio_bus_info_init failed.(ret=%d)", ret);
        return ret;
    }
    hhns_mdio_test(new_bus);
    return 0;
}

static int hns_mdio_remove(struct platform_device *pdev)
{
    struct hns_mdio_device *mdio_dev = NULL;
    struct mii_bus *bus = NULL;

    bus = platform_get_drvdata(pdev);
    if (bus == NULL) {
        printk(KERN_ERR "bus is NULL");
        return -EINVAL;
    }

    mdio_dev = (struct hns_mdio_device *)bus->priv;
    if (mdio_dev == NULL) {
        printk(KERN_ERR "mdio_dev is NULL");
        return -EINVAL;
    }

    mdiobus_unregister(bus);
    platform_set_drvdata(pdev, NULL);

    hns_mdio_cdev_del(mdio_dev);

    printk(KERN_INFO "mdio remove finish\n");
    return 0;
}

static struct platform_driver hns_mdio_driver = {
    .remove = hns_mdio_remove,
    .probe = hns_mdio_probe,
    .driver = {
        .of_match_table = hns_mdio_match,
        .name = MDIO_DRV_NAME,
    },
};
```

 版权所有 © 华为技术有限公司 

```
module_platform_driver(hns_mdio_driver);

MODULE_LICENSE("GPL");
MODULE_ALIAS("platform:" MDIO_DRV_NAME);
MODULE_DESCRIPTION("Hisilicon HNS MDIO Daemo driver");
```

4. 执行如下命令，创建Makefile文件。

   **vim Makefile**

   添加如下代码，配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

   **说明**

   直接拷贝文档中代码到Linux系统中使用可能有格式不兼容问题，例如空格与Tab键使用错误，若出现类似报错，请先在Linux系统中调整代码格式正确。

```
ccflags-y += -Wall -Werror -Wtrampolines $(WDATE_TIME) -Wfloat-equal -Wvla -Wundef -funsigned-char -Wformat=2 -Wstack-usage=2048 -Wcast-align
ccflags-y += -Wextra -Wno-unused-parameter -Wno-sign-compare -Wno-missing-field-initializers

obj-m += hns_mdio_daemo.o
hns_mdio_daemo-objs := hns_mdio.o

.PHONY : clean
clean :
	rm -fr Module.symvers ; rm -fr Module.markers ; rm -fr modules.order
	rm -fr *.mod.c *.mod *.o .*.cmd *.ko *~
	rm -fr .tmp_versions
```

5. 执行如下命令，创建module.mk文件。

   **vim module.mk**

   添加如下代码，配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

```
LOCAL_PATH := $(call my-dir)

include $(CLEAR_VARS)

LOCAL_MODULE := hns_mdio_daemo

LOCAL_KO_SRC_FOLDER := $(LOCAL_PATH)

LOCAL_INSTALLED_KO_FILES := hns_mdio_daemo.ko

include $(BUILD_DEVICE_KO)
```

**步骤14** 执行如下命令，编译驱动。

**make -C** */opt*/**Ascend310B-source/driver/kernel/linux-4.19 ARCH=arm64 M=`pwd` modules CROSS_COMPILE=aarch64-target-linux-gnu-**

类似回显信息如下。

```
make: Entering directory '/opt/Ascend310B-source/driver/kernel/linux-4.19'
  CC [M]  /opt/Ascend310B-source/driver/drivers/usr/hns_mdio.o
  LD [M]  /opt/Ascend310B-source/driver/drivers/usr/hns_mdio_daemo.o
WARNING: Symbol version dump "Mr'mrmodule.symvers" is missing.
         Modules may not have dependencies or modversions.
  MODPOST /opt/Ascend310B-source/driver/drivers/usr/Module.symvers
WARNING: modpost: Symbol info of vmlinux is missing. Unresolved symbol check will be entirely skipped.
  CC [M]  /opt/Ascend310B-source/driver/drivers/usr/hns_mdio_daemo.mod.o
  LD [M]  /opt/Ascend310B-source/driver/drivers/usr/hns_mdio_daemo.ko
make: Leaving directory '/opt/Ascend310B-source/driver/kernel/linux-4.19'
```

 版权所有 © 华为技术有限公司 

**说明**

- 编译后生成的“hns_mdio_daemo.ko”在当前目录下。
- 下次编译前建议执行**make clean**清除上次编译生成的可执行文件及配置文件。

**步骤15** 验证是否可以通过mdio访问PHY芯片。

1. 升级**步骤10**中dt.img，然后重启系统，具体请参见**生效DTB文件**。
2. 登录Atlas 200I DK A2 开发者套件系统环境。
3. 使用“WinSCP”，将编译后的hns_mdio_daemo.ko文件上传至任意目录下，例如“/run”。详细操作请参见**14.3 使用WinSCP传输文件**。
4. 执行如下命令，进入“run”目录，再生效“hns_mdio_daemo.ko”文件。

   **cd** */run*

   **insmod hns_mdio_daemo.ko**
5. 执行如下命令，查看驱动模块是否加载成功。

   **lsmod | grep hns_mdio**

   出现以下回显表示加载成功。

   ```
   hns_mdio_daemo        16384  0
   ```
6. 执行下dmesg，在结尾出现如下打印，表示可以通过mdio正常读写PHY的寄存器。

   ```
   [  465.402588] Now let's test the mdio function.
   [  465.402722] Now get phy addr 0x1, reg 0x2 value [0x0].
   [  465.402725] Now get phy addr 0x1, reg 0x3 value [0x11a].
   [  465.402793] Now get phy addr 0x1, reg 0x0 value [0x1140].
   [  465.402796] Now set phy addr 0x1, reg 0x0 value [0x1040].
   ```

**----结束**

# 7.8 音频

## 7.8.1 HI-MPI

### 7.8.1.1 Demo 参考 1

### 前提条件

- 编译环境
  - 已具备一台带有网络端口且操作系统为Ubuntu 22.04的x86架构的Linux服务器。
  - Linux服务器已安装交叉编译工具链，安装交叉编译工具链具体请参见**8.1 安装工具链**。
  - Linux服务器已安装x86_64对应的toolkit开发套件包，toolkit开发套件包获取方式请参见**开发套件包**，安装开发套件包操作具体请参见**安装开发套件包**。
- Atlas 200I A2 加速模块环境
  - Atlas 200I A2 加速模块环境已升级驱动和固件，具体请参见**9 驱动与固件升级**。升级CANN包具体请参见**CANN软件安装**。


版权所有 © 华为技术有限公司

- Atlas 200I A2 加速模块环境已接入3.5MM音频耳机口。

## 操作步骤

**步骤1** 搭建audio_sample工具编译工程并构建工具。

1. 登录Linux服务器。
2. 执行如下命令，切换至root用户。

   **su - root**
3. 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的audio验证工具包“**audio_sample.tar.gz**”上传至Linux系统root用户属组目录下，如“/opt”，详细操作请参见**14.3 使用WinSCP传输文件**。
4. 执行如下命令，进入“opt”目录。

   **cd** */opt*
5. 执行如下命令，将audio验证工具包“**audio_sample.tar.gz**”解压至“opt”目录下，生成子目录“audio_sample”。

   **tar -zxvf audio_sample.tar.gz -C ./**
6. 执行如下命令，进入“audio_sample”路径并创建“lib”文件夹。

   **cd audio_sample && mkdir lib**
7. 执行如下命令，将编译依赖库文件从toolkit包安装路径拷贝至“lib”目录下。例如，当前安装路径为“/usr/local/Ascend/ascend-toolkit/7.0.RC1”。

   **cp** */usr/local/Ascend/ascend-toolkit/7.0.RC1***/runtime/lib64/stub/aarch64/libacl_audio_mpi.so ./lib && cp** */usr/local/Ascend/ascend-toolkit/7.0.RC1***/runtime/lib64/stub/aarch64/libacl_dvpp_mpi.so ./lib && cp** */usr/local/Ascend/ascend-toolkit/7.0.RC1***/compiler/lib64/stub/linux/aarch64/libc_sec.so ./lib**
8. 执行如下命令，修改Makefile文件。

   **vim Makefile**

   修改“PREFIX_DIR_SELF_DEFINED”为audio_sample验证工具包存放路径，“CANN_TOOKIT_PATH”为toolkit开发套件包安装路径，“CC”为toolkit开发着套件包GCC路径。

   修改“PREFIX_DIR_SELF_DEFINED”为“/opt”，修改“CANN_TOOKIT_PATH”为“/usr/local/Ascend/ascend-toolkit/7.0.RC1”，修改“CC”为“$(CANN_TOOKIT_PATH)/toolkit/toolchain/hcc/bin/aarch64-target-linux-gnu-gcc”。如下图所示。

 版权所有 © 华为技术有限公司 

```
PREFIX_DIR_SELF_DEFINED :=/opt
PRJ_SELF_DEFINED :=audio_sample

#目标文件名，输入任意你想要的执行文件名
TARGET  :=audio_sample

#编译环境CANN-toolkit套件包安装路径
CANN_TOOKIT_PATH :=/usr/local/Ascend/ascend-toolkit/7.0.RC1

#compile and lib parameter
INC_DIR :=  -I$(CANN_TOOKIT_PATH)/runtime/include \
                -I$(CANN_TOOKIT_PATH)/runtime/include/acl/media \
                        -I$(PREFIX_DIR_SELF_DEFINED)/$(PRJ_SELF_DEFINED)/include

LIB_DIR := -L$(PREFIX_DIR_SELF_DEFINED)/$(PRJ_SELF_DEFINED)/lib

CFLAGS := -Wall -Wfloat-equal -fPIE -fstack-protector-strong
LIBS += -lacl_dvpp_mpi -lacl_audio_mpi -lc_sec -ldl
LDFLAGS  := -Wl,-z,noexecstack,-z,relro,-z,now -pie -s

#编译环境CANN-toolkit套件包aarch64-target-linux-gnu-gcc路径
export CC=$(CANN_TOOKIT_PATH)/toolkit/toolchain/hcc/bin/aarch64-target-linux-gnu-gcc
```

按“Esc”键，输入**:wq!**，按“Enter”保存并退出编辑。

9. 执行如下命令，编译audio验证工具，则会在当前目录下生成audio_sample工具。

   **make clean && make all**

**步骤2** 运行audio_sample工具。

1. 登录Atlas 200I A2 加速模块的环境。
2. 将生成的工具上传至Atlas 200I A2 加速模块环境，例如“/opt”目录下。
3. 执行如下命令，进入“/opt”目录，并增加工具的可执行权限。

   **cd** */opt* **&& chmod +x audio_sample**
4. （可选）若当前系统为Ubuntu，执行如下命令配置环境变量。

   **export LD_LIBRARY_PATH=/lib64:$LD_LIBRARY_PATH**
5. 根据具体的命令格式，执行如下命令。

   **./audio_sample** *mode filename*

   **说明**

   - mode：取值为0或1。
     - 0：播放音频。
     - 1：录制音频。
   - filename：用户自定义命名，仅支持pcm格式。

**----结束**

### 7.8.1.2 Demo 参考 2

请参见Gitee上Ascend/samples开源代码仓中的**audio gitee**。

### 7.8.1.3 调测参考

详细请参见《应用开发接口》的**“AI音频输入/AO音频输出”**章节。

 版权所有 © 华为技术有限公司 

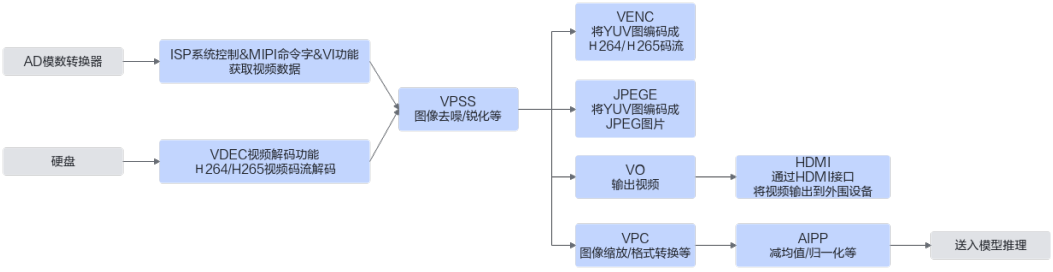
# 7.9 视频

## 7.9.1 概述

Atlas 200I A2 加速模块的视频接口包含HDMI、MIPI-CSI与MIPI-DSI。

MIPI-CSI接口对应图7-6中的ISP系统控制，MIPI-DSI与HDMI对应图7-6中的VO。

**图 7-6** 图像/视频数据处理




## 7.9.2 HDMI

### 7.9.2.1 Demo 参考 1

#### 注意事项

7.9.2.4 图形化桌面会修改DTB中的“hi1910B-vdp.dtsi”和对应的dts文件，所以使能图形化桌面后，会导致本Demo和7.9.2.2 Demo参考2的功能不可用。

#### 前提条件

- 已具备一台带有网络端口且操作系统为Ubuntu 22.04的x86架构的Linux服务器。
- Atlas 200I A2 加速模块环境已升级驱动和固件，具体请参见9 驱动与固件升级。升级CANN包具体请参见CANN软件安装。
- 当前环境已通过hdmi连接线连接屏幕。
- 已安装交叉编译工具链并准备需要输出的图片（图片当前仅支持yuv格式），安装交叉编译工具链具体请参见8.1 安装工具链。

#### 操作步骤

**步骤1** 搭建hdmi_sample工具编译工程并构建工具。

1. 登录Linux服务器。
2. 执行如下命令，切换至root用户。

   **su - root**

3. 使用WinSCP工具或其他工具，将4.1 下载软件包获取的驱动包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run”（若有软实时OS需求，使用驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-


版权所有 © 华为技术有限公司

aarch64.run）与hdmi验证工具包“hdmi_sample.tar.gz”上传至Linux系统root用户属组目录下，如“/opt”，详细操作请参见**14.3 使用WinSCP传输文件**。

4. 请从**昇腾社区**下载“Ascend-cann-nnrt_*<version>*_linux-aarch64.run”软件包并上传至“opt”目录下。
5. 执行如下命令，进入“opt”目录。

   **cd** */opt*
6. 执行如下命令，将驱动包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run”（若有软实时OS需求，使用驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run）解压至“opt”的子目录“driver”下。

   **bash Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-aarch64.run --noexec --extract=./driver**

   若有软实时OS需求，该命令为**bash Ascend-hdk-310b-npu-driver-soc_***<version>***_linux-rt-aarch64.run --noexec --extract=./driver**。
7. 执行如下命令，将“Ascend-cann-nnrt_*<version>*_linux-aarch64.run”软件包解压至“opt”的子目录“Ascend”下。并将解压后软件包“CANN-runtime-*<version>*-linux.aarch64.run”解压至“opt”的子目录“cann”下。

   **bash Ascend-cann-nnrt_***<version>***_linux-aarch64.run --noexec --extract=./Ascend**

   **bash Ascend/run_package/CANN-runtime-***<version>***-linux.aarch64.run --noexec --extract=./cann**
8. 执行如下命令，将hdmi_sample验证工具包“hdmi_sample.tar.gz”解压至“opt”目录下，生成子目录“hdmi_sample”。

   **tar -zxvf hdmi_sample.tar.gz -C ./**
9. 执行如下命令，进入“hdmi_sample”路径并创建lib文件夹。

   **cd hdmi_sample && mkdir lib**
10. 执行如下命令，将编译依赖CANN包中的库文件拷贝至“lib”目录下。

    **cp ../cann/runtime/lib64/libacl_vo_mpi.so ./lib && cp ../cann/runtime/lib64/libacl_hdmi_mpi.so ./lib && cp ../cann/runtime/lib64/libascendalog.so ./lib && cp ../cann/runtime/lib64/libmmpa.so ./lib**
11. 执行如下命令，将编译依赖驱动包中的库文件拷贝至“lib”目录下。

    **cp ../driver/libmid*.so ./lib/ && cp ../driver/libmpi*.so ./lib && cp ../driver/libslog.so ./lib && cp ../driver/libascend_hal.so ./lib && cp ../driver/libdevmmap.so ./lib && cp ../driver/libc_sec.so ./lib**
12. 执行如下命令，将编译依赖的头文件拷贝至对应目录下。

    **mkdir acl && cp -R ../cann/runtime/include/acl/* ./acl && cp ../cann/runtime/include/securec* ./acl**
13. （**可选**）“pictures”文件夹提供了默认图片可用于展示。若需展示自定义图片，执行以下命令，更改图片名称。

    **vim include/hdmi_sample_comm.h**

    修改“HDMI_1080P_YUNFILE_NAME”为1080P图片名称，修改“HDMI_4K_YUNFILE_NAME”为4K图片名称。修改完成后按“Esc”键，输入**:wq!**，按“Enter”保存并退出编辑。默认图片名称如下所示。

    ```
    #define HDMI_1080P_YUNFILE_NAME "ut_1920x1080_nv12.yuv"
    #define HDMI_4K_YUNFILE_NAME "ut_3840x2160_nv12.yuv"
    ```
14. 执行如下命令，修改Makefile文件。

 版权所有 © 华为技术有限公司 

**vim Makefile**

修改“PREFIX_DIR_SELF_DEFINED”为hdmi_sample验证工具包存放路径，CC为8.1 安装工具链中配置的交叉工具链路径。

例如：修改“PREFIX_DIR_SELF_DEFINED”为/opt，修改“CC”为/opt/compiler/toolchain/bin/aarch64-target-linux-gnu-gcc。如下所示。

```
PREFIX_DIR_SELF_DEFINED := /opt
PRJ_SELF_DEFINED := hdmi_sample

#目标文件名，输入任意你想要的执行文件名
TARGET  := hdmi_sample

#compile and lib parameter
INC_DIR :=  -I$(PREFIX_DIR_SELF_DEFINED)/$(PRJ_SELF_DEFINED)/acl \
            -I$(PREFIX_DIR_SELF_DEFINED)/$(PRJ_SELF_DEFINED)/acl/dvpp
            -I$(PREFIX_DIR_SELF_DEFINED)/$(PRJ_SELF_DEFINED)/acl/media
                        -I$(PREFIX_DIR_SELF_DEFINED)/$(PRJ_SELF_DEFINE
                        -I$(PREFIX_DIR_SELF_DEFINED)/$(PRJ_SELF_DEFINE

LIB_DIR := -L$(PREFIX_DIR_SELF_DEFINED)/$(PRJ_SELF_DEFINED)/lib

MACRO := -Wall
LIBS += -lmid_vo_mpi -lascendalog -lacl_vo_mpi -lacl_hdmi_mpi -lmid_ho
CFLAGS  := -g -Wall -O2

#aarch64-target-linux-gnu-gcc路径
export CC=/opt/compiler/toolchain/bin/aarch64-target-linux-gnu-gcc
```

按“Esc”键，输入**:wq!**，按“Enter”保存并退出编辑。

15. 执行如下命令，编译hdmi验证工具，则会在当前目录下生成hdmi_sample工具。

    **make clean && make all**

**步骤2** 运行hdmi_sample工具。

1. 登录Atlas 200I A2 加速模块的环境。
2. 将生成的工具与展示图片（默认可使用hdmi_sample验证工具包“pictures”文件夹中所附带的图片）上传至Atlas 200I A2 加速模块环境，例如“/opt”目录下。
3. 执行如下命令，进入“/opt”目录，并增加工具的可执行权限。

   **cd** */opt* **&& chmod +x hdmi_sample**
4. （可选）若当前系统为Ubuntu，通过如下命令配置环境变量。

   **export LD_LIBRARY_PATH=/lib64:$LD_LIBRARY_PATH**
5. 根据具体的命令格式，执行如下命令可成功可将图片投送至屏幕中。

   **./hdmi_sample** *port resolution*

   **说明**

   Atlas 200I A2 加速模块提供了2路HDMI接口，分别对应HDMI0与HDMI1。

   依赖源码生成的工具支持1080P和4K图片投递，帮助信息格式如下。

   ```
   Usage: hdmi_sample port resolution
   port                : Port only can be set 0 or 1, (0: hdmi0, 1: hdmi1).
   resolution          : Resolution only can be set 0 or 1, (0: 1080P, 1: 4K).
   ```

**----结束**

 版权所有 © 华为技术有限公司 

### 7.9.2.2 Demo 参考 2

#### 注意事项

**7.9.2.4 图形化桌面**会修改DTB中的“hi1910B-vdp.dtsi”和对应的dts文件，所以使能图形化桌面后，会导致**7.9.2.1 Demo参考1**和本Demo的功能不可用。

请参见Gitee上Ascend/samples开源代码仓中的**hdmi_gitee**。

### 7.9.2.3 调测参考

详细请参见《 应用开发接口 》的**“NVR场景视频解码、处理和显示”**章节。

### 7.9.2.4 图形化桌面

#### 注意事项

图形化桌面会修改DTB中的“hi1910B-vdp.dtsi”和对应的dts文件，所以使能图形化桌面后，会导致**7.9.2.1 Demo参考1**和**7.9.2.2 Demo参考2**的功能不可用。

#### 前提条件

- 使用HDMI线缆，将显示屏与整机连接，连接至HDMI 0。
- 键盘、鼠标插到整机的USB接口上。

#### 操作步骤

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source tar.gz”上传至root用户属组目录下，例如“/opt”。工具使用方法请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 修改dts文件。按照整机环境配置对应的dts文件，参见**DTB文件**中的表格。以下以“hi1910B-asic-M100-B51.dts”文件为例进行修改。

1. 执行如下命令，进入配置文件所在目录。

   **cd Ascend310B-source/dtb/dts/hi1910b/hi1910BL**

2. 执行如下命令，打开“hi1910B-asic-M100-B51.dts”文件。

   **vim hi1910B-asic-M100-B51.dts**

3. 将/include/"base/**hi1910B-vdp.dtsi**"修改为/include/"base/**hi1910B-drm.dtsi**"。


版权所有 © 华为技术有限公司

4. 修改“hi1910B-asic-M100-B51.dts”的“bootargs”。

   修改“console”、“cma”和“rootfstype”参数的值，如下代码加粗字体所示（在原有基础上修改，不可全部复制替换）。如没有对应参数，请按照如下加粗字体新增即可。

   ```
   bootargs = "console=ttyAMA0,115200 console=tty1 log_redirect=0x1fc000@0x22741000 root=/dev/mmcblk0p2 rw rdinit=/sbin/init rodata=off syslog earlycon=pl011,mmio32,0xC4010000 initrd=0x2BA00000,300M rootwait mpam=of enable_ascend_share_pool ascend_enable_all enable_sp_multi_group_mode init_all_gicr enable_fpga no_console_suspend fbcon=map:6 cma=256M rootfstype=ext4 selinux=0";
   ```

5. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。
6. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

**步骤7** 执行如下命令，编译DTB文件。

**bash build.sh dtb**

出现如下回显，且生成“dt.img”文件表示编译内核DTB文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

**说明**

编译后的dt.img文件会自动存放于“Ascend310B-source/output”目录下。

**步骤8** 生效DTB文件，参见**生效DTB文件**。

**步骤9** 登录整机（整机已有NPU驱动），找到驱动安装目录。

- Ubuntu的驱动安装目录：/usr/lib/aarch64-linux-gnu/
- openEuler的驱动安装目录：/usr/lib64/

在驱动安装目录下查找libdrm.so.2.4.0，将其放在整机的“/usr/lib64”目录下后更名为“libdrm.so.2”。

**步骤10** 登录整机环境，执行如下命令，切换至root用户。

**su - root**

**步骤11** 插入图形化桌面驱动（每次重启后需要重新插入驱动）。

1. 执行如下命令，插入驱动。

   **insmod /var/davinci/driver/ascend_vdp_drm.ko**
2. 每次重新插入后，执行如下命令，查看dri下是否存在by-path和card0文件。若不存在需重新插入驱动。

   **ls /dev/dri/**

**步骤12** 安装桌面管理环境。

**说明**

用户可选择其他安装方式，以下以开源Gnome桌面插件为例，进行安装。

- Ubuntu 22.04
  a. 执行如下命令，进行安装。

     **apt-get install -y gdm3 gnome-shell gnome-session gnome-terminal**
  b. 执行如下命令，查询进程是否为“activce”。

 版权所有 © 华为技术有限公司 

**systemctl status gdm3**

**systemctl status display-manager.service**

若回显如下，需执行3。

```
Active: inactive (dead)
```

c. 执行如下命令，将进程使能为“active”。

**systemctl start gdm3**

**systemctl start display-manager.service**

- openEuler 22.03

a. 执行如下命令，进行安装。

**dnf install gdm gnome-shell gnome-session gnome-terminal**

b. 执行如下命令，重启进程。

**systemctl restart gdm**

**systemctl restart display-manager.service**

**步骤13** 配置图形化桌面加载开关。

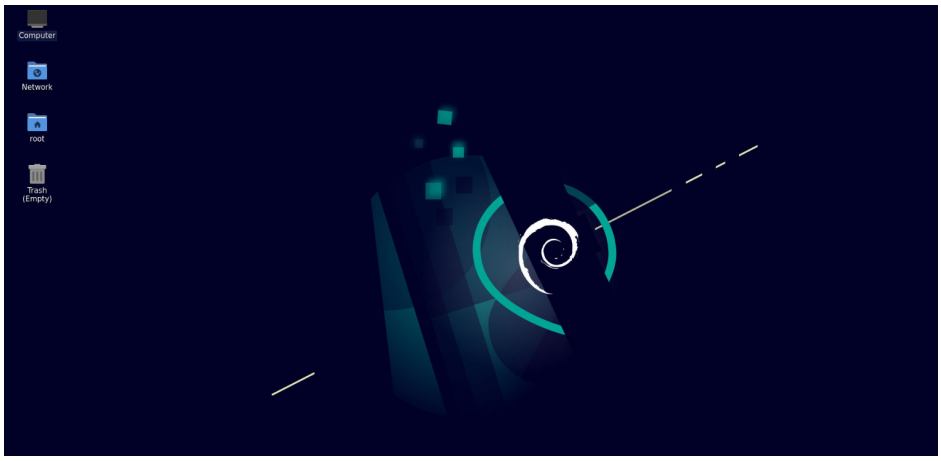
**说明**

图形化桌面加载开关由用户自定义实现，实现思路可参考以下步骤。

1. 将桌面驱动拷贝到系统驱动默认搜索路径“/lib/modules/5.10.0+”。
2. 在配置文件中添加并定义“开”和“关”的标志符。
3. 实现一个脚本，读取开关标志符。如果“开”的标志为1，则加载桌面驱动，否则不执行操作。

**步骤14** 使用HwHiAiUser用户名和密码登录整机，进入主界面如图1 图形化桌面所示。

**图 7-7** 图形化桌面




**----结束**

## 7.9.3 MIPI-CSI

### 7.9.3.1 Demo 参考 1

请参见Gitee上Ascend/samples开源代码仓中的**camera_sample**。


版权所有 © 华为技术有限公司

### 7.9.3.2 Demo 参考 2

Atlas 200I A2 加速模块支持HDMI转CSI功能。此Demo只支持3840*2160 yuv422图片传输。

## 前提条件

Atlas 200I A2 加速模块环境已升级驱动和固件至最新版本，具体请参见**9 驱动与固件升级**。已升级CANN包至最新版本，具体请参见**CANN软件安装**。

## 操作步骤

**步骤1** 下载Gitee上Ascend/samples开源代码仓中的**camera_sample**源码包。

**步骤2** 登录Linux服务器。

**步骤3** 执行如下命令，切换至root用户。

**su - root**

**步骤4** 使用WinSCP工具或其他工具，将**步骤1**下载的samples-master.zip源码包上传至Linux系统root用户属组目录下，如“/opt”，详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤5** 执行如下命令，进入“/opt”目录。

**cd** */opt*

**步骤6** 执行如下命令，解压zip包文件。

**unzip samples-master.zip**

**步骤7** 执行如下命令，进入“samples-master”目录。

**cd samples-master/cplusplus/level1_single_api/6_media/1_camera/camera_sample/sensor**

**步骤8** 执行如下命令，创建“sensor_isp”目录。

**mkdir sensor_isp**

**步骤9** 执行如下命令，进入“sensor_isp”目录。

**cd sensor_isp**

**步骤10** 执行如下命令，创建“isp_cmos.c”文件。

**vim isp_cmos.c**

添加如下代码，配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

```
#include <stdio.h>
#include <string.h>
#include <assert.h>
#include "hi_mpi_isp.h"
#include "hi_mpi_ae.h"
#include "hi_mpi_awb.h"
#include "hi_common_isp.h"
#include "hi_sns_ctrl.h"

#ifdef __cplusplus
```

 版权所有 © 华为技术有限公司 

```
#if __cplusplus
extern "C" {
#endif
#endif /* End of #ifdef __cplusplus */
#define ISP_SENSOR_ID 0

static hi_s32 cmos_get_ae_default(hi_vi_pipe vi_pipe, hi_isp_ae_sensor_default *pstAeSnsDft)
{
    return HI_SUCCESS;
}

/* the function of sensor set fps */
static hi_void cmos_fps_set(hi_vi_pipe ViPipe, hi_float f32Fps, hi_isp_ae_sensor_default *pstAeSnsDft)
{
    return;
}

static hi_void cmos_slow_framerate_set(hi_vi_pipe vi_pipe, hi_u32 u32FullLines, hi_isp_ae_sensor_default
*pstAeSnsDft)
{
    return;
}


/* while isp notify ae to update sensor regs, ae call these funcs. */
static hi_void cmos_inttime_update(hi_vi_pipe vi_pipe, hi_u32 u32IntTime)
{
    return;
}

static hi_void cmos_again_calc_table(hi_vi_pipe ViPipe, hi_u32 *pu32AgainLin, hi_u32 *pu32AgainDb)
{
    return;
}

static hi_void cmos_dgain_calc_table(hi_vi_pipe vi_pipe, hi_u32 *pu32DgainLin, hi_u32 *pu32DgainDb)
{
    return;
}

static hi_void cmos_gains_update(hi_vi_pipe vi_pipe, hi_u32 u32Again, hi_u32 u32Dgain)
{
    return;
}

static hi_void cmos_get_inttime_max(hi_vi_pipe vi_pipe, hi_u16 u16ManRatioEnable,
    hi_u32 *au32Ratio, hi_u32 *au32IntTimeMax,
    hi_u32 *au32IntTimeMin, hi_u32 *pu32LFMaxIntTime)
{
    return;
}

/* Only used in LINE_WDR mode */
static hi_void cmos_ae_fswdr_attr_set(hi_vi_pipe vi_pipe, hi_isp_ae_fswdr_attr *pstAeFSWDRAttr)
{
    return;
}
static hi_s32 cmos_init_ae_exp_function(hi_isp_ae_sensor_exp_func *pstExpFuncs)
{
    int ret;

    CMOS_CHECK_POINTER(pstExpFuncs);

    ret = memset(pstExpFuncs, 0, sizeof(hi_isp_ae_sensor_exp_func));

    pstExpFuncs->pfn_cmos_get_ae_default    = cmos_get_ae_default;
    pstExpFuncs->pfn_cmos_fps_set           = cmos_fps_set;
    pstExpFuncs->pfn_cmos_slow_framerate_set = cmos_slow_framerate_set;
    pstExpFuncs->pfn_cmos_inttime_update    = cmos_inttime_update;
```

 版权所有 © 华为技术有限公司 

```
    pstExpFuncs->pfn_cmos_gains_update      = cmos_gains_update;
    pstExpFuncs->pfn_cmos_again_calc_table  = cmos_again_calc_table;
    pstExpFuncs->pfn_cmos_dgain_calc_table  = cmos_dgain_calc_table;
    pstExpFuncs->pfn_cmos_get_inttime_max   = cmos_get_inttime_max;
    pstExpFuncs->pfn_cmos_ae_fswdr_attr_set = cmos_ae_fswdr_attr_set;

    return HI_SUCCESS;
}

static hi_s32 cmos_get_awb_default(hi_vi_pipe vi_pipe, hi_isp_awb_sensor_default *pstAwbSnsDft)
{
    return HI_SUCCESS;
}

static hi_s32 cmos_init_awb_exp_function(hi_isp_awb_sensor_exp_func *pstExpFuncs)
{
    int ret;
    CMOS_CHECK_POINTER(pstExpFuncs);

    ret = memset(pstExpFuncs, 0, sizeof(hi_isp_awb_sensor_exp_func));

    pstExpFuncs->pfn_cmos_get_awb_default = cmos_get_awb_default;

    return HI_SUCCESS;
}

static hi_s32 cmos_get_isp_default(hi_vi_pipe ViPipe, hi_isp_cmos_default *pstDef)
{
    return HI_SUCCESS;
}


static hi_s32 cmos_get_isp_black_level(hi_vi_pipe ViPipe, hi_isp_cmos_black_level *pstBlackLevel)
{
    return HI_SUCCESS;
}

static hi_void cmos_set_pixel_detect(hi_vi_pipe vi_pipe, hi_bool bEnable)
{
    return;
}


static hi_s32 cmos_set_wdr_mode(hi_vi_pipe ViPipe, hi_u8 u8Mode)
{
    return HI_SUCCESS;
}

static hi_s32 cmos_get_sns_regs_info(hi_vi_pipe ViPipe, hi_isp_sns_regs_info *pstSnsRegsInfo)
{
    return HI_SUCCESS;
}

static hi_s32 cmos_set_image_mode(hi_vi_pipe ViPipe, hi_isp_cmos_sensor_image_mode
*pstSensorImageMode)
{
    return HI_SUCCESS;
}

static hi_void sensor_global_init(hi_vi_pipe ViPipe)
{
    return;
}


void sensor_init(hi_vi_pipe vi_pipe)
{
    return;
}
```

 版权所有 © 华为技术有限公司 

```
void sensor_exit(hi_vi_pipe vi_pipe)
{
    return;
}
static hi_s32 cmos_init_sensor_exp_function(hi_isp_sensor_exp_func *pstSensorExpFunc)
{
    int ret;

    CMOS_CHECK_POINTER(pstSensorExpFunc);

    ret = memset(pstSensorExpFunc, 0, sizeof(hi_isp_sensor_exp_func));

    pstSensorExpFunc->pfn_cmos_sensor_init          = sensor_init;
    pstSensorExpFunc->pfn_cmos_sensor_exit          = sensor_exit;
    pstSensorExpFunc->pfn_cmos_sensor_global_init   = sensor_global_init;
    pstSensorExpFunc->pfn_cmos_set_image_mode       = cmos_set_image_mode;
    pstSensorExpFunc->pfn_cmos_set_wdr_mode         = cmos_set_wdr_mode;
    pstSensorExpFunc->pfn_cmos_get_isp_default      = cmos_get_isp_default;
    pstSensorExpFunc->pfn_cmos_get_isp_black_level  = cmos_get_isp_black_level;
    pstSensorExpFunc->pfn_cmos_set_pixel_detect     = cmos_set_pixel_detect;
    pstSensorExpFunc->pfn_cmos_get_sns_reg_info     = cmos_get_sns_regs_info;

    return HI_SUCCESS;
}
static hi_s32 sensor_set_bus_info(hi_vi_pipe vi_pipe, hi_isp_sns_commbus sns_bus_info)
{
    return HI_SUCCESS;
}

static hi_s32 sensor_ctx_init(hi_vi_pipe ViPipe)
{

    return HI_SUCCESS;
}

static hi_void sensor_ctx_exit(hi_vi_pipe ViPipe)
{
    return;
}

static hi_s32 sensor_register_callback(hi_vi_pipe ViPipe, hi_isp_3a_alg_lib *pstAeLib, hi_isp_3a_alg_lib
*pstAwbLib, hi_isp_3a_alg_lib *pstAfLib)
{
    hi_s32 s32Ret;
    hi_isp_sensor_register stIspRegister;
    hi_isp_ae_sensor_register  stAeRegister;
    hi_isp_awb_sensor_register stAwbRegister;
    hi_isp_sns_attr_info    stSnsAttrInfo;

    if (((ViPipe) < 0) || ((ViPipe) >= HI_ISP_MAX_PIPE_NUM)) {    \
        printf("Err viPipe %d!\n", ViPipe); \
        return HI_ERR_ISP_ILLEGAL_PARAM;
    }

    CMOS_CHECK_POINTER(pstAeLib);
    CMOS_CHECK_POINTER(pstAwbLib);
    CMOS_CHECK_POINTER(pstAfLib);

    s32Ret = sensor_ctx_init(ViPipe);

    if (HI_SUCCESS != s32Ret)
    {
        return HI_FAILURE;
    }

    stSnsAttrInfo.sensor_id = ISP_SENSOR_ID;

    s32Ret  = cmos_init_sensor_exp_function(&stIspRegister.sns_exp);
```

 版权所有 © 华为技术有限公司 

```
    if (HI_SUCCESS != s32Ret)
    {
        printf("sensor none cmos_init_sensor_exp_function err\n");
        return s32Ret;
    }
    s32Ret = hi_mpi_isp_sensor_reg_callback(ViPipe, &stSnsAttrInfo, &stIspRegister);
    if (HI_SUCCESS != s32Ret)
    {
        printf("sensor none register callback function failed!\n");
        return s32Ret;
    }

    s32Ret  = cmos_init_ae_exp_function(&stAeRegister.sns_exp);
    if (HI_SUCCESS != s32Ret)
    {
        printf("sensor none cmos_init_ae_exp_function err\n");
        return s32Ret;
    }
    s32Ret = hi_mpi_ae_sensor_reg_callback(ViPipe, pstAeLib, &stSnsAttrInfo, &stAeRegister);
    if (HI_SUCCESS != s32Ret)
    {
        printf("sensor none sensor register callback function to ae lib failed!\n");
        return s32Ret;
    }

    s32Ret  = cmos_init_awb_exp_function(&stAwbRegister.sns_exp);
    if (HI_SUCCESS != s32Ret)
    {
        printf("sensor none cmos_init_awb_exp_function err\n");
        return s32Ret;
    }
    s32Ret = hi_mpi_awb_sensor_reg_callback(ViPipe, pstAwbLib, &stSnsAttrInfo, &stAwbRegister);
    if (HI_SUCCESS != s32Ret)
    {
        printf("sensor none sensor register callback function to awb lib failed!\n");
        return s32Ret;
    }

    return HI_SUCCESS;
}


static hi_s32 sensor_unregister_callback(hi_vi_pipe ViPipe, hi_isp_3a_alg_lib *pstAeLib, hi_isp_3a_alg_lib
*pstAwbLib, hi_isp_3a_alg_lib *pstAfLib)
{
    hi_s32 s32Ret;

    CMOS_CHECK_POINTER(pstAeLib);
    CMOS_CHECK_POINTER(pstAwbLib);
    CMOS_CHECK_POINTER(pstAfLib);

    s32Ret = hi_mpi_isp_sensor_unreg_callback(ViPipe, ISP_SENSOR_ID);
    if (HI_SUCCESS != s32Ret)
    {
        printf("sensor unregister callback function failed!\n");
        return s32Ret;
    }

    s32Ret = hi_mpi_ae_sensor_unreg_callback(ViPipe, pstAeLib, ISP_SENSOR_ID);
    if (HI_SUCCESS != s32Ret)
    {
        printf("sensor unregister callback function to ae lib failed!\n");
        return s32Ret;
    }

    s32Ret = hi_mpi_awb_sensor_unreg_callback(ViPipe, pstAwbLib, ISP_SENSOR_ID);
    if (HI_SUCCESS != s32Ret)
    {
        printf("isp sensor unregister callback function to awb lib failed!\n");
```

 版权所有 © 华为技术有限公司 

```
        return s32Ret;
    }

    sensor_ctx_exit(ViPipe);

    return HI_SUCCESS;
}

static void sensor_standby(hi_vi_pipe vi_pipe)
{
    return;
}
static void sensor_restart(hi_vi_pipe vi_pipe)
{
    return;
}

static hi_s32 sensor_write_register(hi_vi_pipe vi_pipe, hi_u32 addr, hi_u32 data)
{
    return HI_SUCCESS;
}
static hi_s32 sensor_read_register(hi_vi_pipe vi_pipe, hi_u32 addr)
{
    return HI_SUCCESS;
}

static hi_s32 sensor_set_init(hi_vi_pipe ViPipe, hi_isp_init_attr *pstInitAttr)
{
    return HI_SUCCESS;
}
hi_isp_sns_obj g_sns_isp_obj =
{
    .pfn_register_callback    = sensor_register_callback,
    .pfn_un_register_callback  = sensor_unregister_callback,
    .pfn_standby              = sensor_standby,
    .pfn_restart              = sensor_restart,
    .pfn_write_reg            = sensor_write_register,
    .pfn_read_reg             = sensor_read_register,
    .pfn_set_bus_info         = sensor_set_bus_info,
    .pfn_set_init             = sensor_set_init
};

#ifdef __cplusplus
#if __cplusplus
}
#endif
#endif /* End of #ifdef __cplusplus */
```

**步骤11** 执行如下命令，创建“CMakeLists.txt”文件。

**vim CMakeLists.txt**

添加如下代码，配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

```
# Copyright (c) Huawei Technologies Co., Ltd. 2023. All rights reserved.

# CMake lowest version requirement
cmake_minimum_required(VERSION 3.5.1)

### Sensor SONY IMX219 ###
# project information
project(sns_isp)

set(LIB_PATH $ENV{NPU_HOST_LIB})
# Dynamic libraries in the stub directory can only be used for compilation
if (NOT DEFINED ENV{NPU_HOST_LIB})
    set(LIB_PATH "/usr/local/Ascend/ascend-toolkit/latest/CANN-6.4/runtime/lib64/stub/aarch64")
    message(STATUS "set default LIB_PATH: ${LIB_PATH}")
```

 版权所有 © 华为技术有限公司 

```
else ()
    message(STATUS "env LIB_PATH: ${LIB_PATH}")
endif()
set(INC_PATH $ENV{NPU_HOST_INC})
# Dynamic libraries in the stub directory can only be used for compilation
if (NOT DEFINED ENV{NPU_HOST_INC})
    set(INC_PATH "/usr/local/Ascend/ascend-toolkit/latest/CANN-6.4/runtime/include")
    message(STATUS "set default INC_PATH: ${INC_PATH}")
else ()
    message(STATUS "env INC_PATH: ${INC_PATH}")
endif()

set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${LIB_PATH})

link_directories(
    ${LIB_PATH}
    ${LIB_PATH}/stub
)

add_library(sns_isp SHARED
        isp_cmos.c
)

include_directories(sns_isp
        ${INC_PATH}/acl/media
        ../include
)

target_link_libraries(sns_isp
        acl_isp_mpi
)
```

**说明**

其中“/usr/local/Ascend/ascend-toolkit/latest/CANN-6.4/runtime/lib64/stub/aarch64”和“/usr/local/Ascend/ascend-toolkit/latest/CANN-6.4/runtime/include”为CANN包安装路径，请根据实际情况替换。

**步骤12** 新增“isp_sensor_config.h”文件。

1. 执行如下命令，进入“sensor_sample”目录。

   cd */opt*/**samples-master/cplusplus/level1_single_api/6_media/1_camera/camera_sample/sensor_sample**

2. 执行如下命令，新增“isp_sensor_config.h”文件。

   **vim isp_sensor_config.h**

   添加如下代码。

```
#include "hi_mipi_rx.h"
#include "hi_mpi_isp.h"
#include "hi_mpi_vi.h"

#include "sensor_management.h"

combo_dev_attr_t MIPI_4lane_SENSOR_NONE_12BIT_1920x1080_ATTR =
{
    .devno = 2,
    .input_mode = INPUT_MODE_MIPI,
    .data_rate = MIPI_DATA_RATE_X1,
    .img_rect = {0, 0, 3840, 2160},
    {
        .mipi_attr =
        {
            DATA_TYPE_YUV422_8BIT,
            HI_MIPI_WDR_MODE_NONE,
            {4, 5, 6, 7, -1, -1, -1, -1}
        }
    }
```

 版权所有 © 华为技术有限公司 

```
};

hi_isp_pub_attr ISP_PUB_ATTR_SENSOR_NONE_1080P_30FPS =
{
    {0, 0, 3840, 2160},
    {3840, 2160},
    30,
    HI_ISP_BAYER_RGGB,
    HI_WDR_MODE_NONE,
    1,
};

// hi_vi_dev_attr 1920X1080
hi_vi_dev_attr DEV_ATTR_SENSOR_NONE_1080P_BASE =
{
    HI_VI_MODE_MIPI_YUV422,
    HI_VI_SCAN_PROGRESSIVE,
    HI_VI_DATA_SEQ_VUVU,
    HI_VI_DATA_TYPE_YUV,
    {3840, 2160},
    {
        HI_WDR_MODE_NONE,
        2160
    },
    HI_DATA_RATE_X1
};

// hi_vi_pipe_attr 1920x1080 RAW8
hi_vi_pipe_attr PIPE_ATTR_SENSOR_NONE_1920x1080_RAW8_420_3DNR_RFR =
{
    HI_VI_PIPE_BYPASS_NONE,
    HI_TRUE,
    {3840, 2160},
    HI_PIXEL_FORMAT_YVU_SEMIPLANAR_422,
    HI_COMPRESS_MODE_NONE,
    HI_DATA_BIT_WIDTH_8,
    { -1, -1}
};


hi_vi_chn_attr CHN_ATTR_SENSOR_NONE_1920x1080_420_SDR8_LINEAR =
{
    {3840, 2160},
    HI_PIXEL_FORMAT_YVU_SEMIPLANAR_422,
    HI_DYNAMIC_RANGE_SDR8,
    HI_VIDEO_FORMAT_LINEAR,
    HI_COMPRESS_MODE_NONE,
    0,
    { -1, -1}
};
```

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤13** 修改“vi_with_sensor.c”文件。

1. 执行如下命令，进入“vi_with_sensor.c”所在目录。

   **cd** */opt*/**samples-master/cplusplus/level1_single_api/6_media/1_camera/camera_sample**

2. 执行如下命令，进入“vi_with_sensor.c”文件编译模式。

   **vim vi_with_sensor.c**

3. 将代码中的“sony_imx477”修改为“sensor_isp”信息，同时修改sampleViInfo的值，此样例mipi dev2是4条lane。具体如下代码加粗字体内容。

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <errno.h>
```

 版权所有 © 华为技术有限公司 

```
#include "vi_with_sensor.h"
#include "sample_comm.h"
#include "sample_comm_vi.h"
#include "sample_comm_isp.h"
#include "image_dump_util.h"
#include "acl/acl_rt.h"
#include "acl/acl.h"

#define HI_MAX_PIPE_NUM  4
#define HI_MAX_NAME_SIZE 128
SAMPLE_VI_CONFIG_S g_stViConfig;
hi_s32 pipe_mult_sensor[HI_MAX_PIPE_NUM] = {0};

static hi_s32 yuv_cnt = 0;     //新增支持CSI接收YUV422图片

hi_u32 hi_pixel_format_bit_width(hi_pixel_format enPixelFormat)
{
    switch (enPixelFormat) {
        case HI_PIXEL_FORMAT_RGB_BAYER_8BPP:
            return 8;
        case HI_PIXEL_FORMAT_RGB_BAYER_10BPP:
            return 10;
        case HI_PIXEL_FORMAT_RGB_BAYER_12BPP:
            return 12;
        case HI_PIXEL_FORMAT_RGB_BAYER_14BPP:
            return 14;
        case HI_PIXEL_FORMAT_RGB_BAYER_16BPP:
            return 16;
        default:
            return 0;
    }
}

static hi_s32 sample_sys_init()
{
    hi_s32 ret = HI_FAILURE;
    hi_mpi_sys_exit();

    SAMPLE_PRT("begin hi_mpi_sys_init !\n");
    ret = hi_mpi_sys_init();
    if (HI_SUCCESS != ret) {
        SAMPLE_PRT("hi_mpi_sys_init failed!\n");
        return HI_FAILURE;
    }
    SAMPLE_PRT("begin aclInit !\n");
    ret = aclInit(NULL);
    if (ret != HI_SUCCESS) {
        SAMPLE_PRT("aclInit failed!\n");
        return HI_FAILURE;
    }
    return HI_SUCCESS;
}

hi_void close_camera(void)
{
    hi_s32 s32Ret;
    (void)Sample_Comm_Vi_StopVi(&g_stViConfig);
    s32Ret = hi_mpi_sys_exit();
    if (s32Ret != HI_SUCCESS) {
        SAMPLE_PRT("hi_mpi_sys_exit failed!\n");
    } else {
        SAMPLE_PRT("hi_mpi_sys_exit succ \n");
    }
    s32Ret = aclFinalize();
    if (s32Ret != ACL_SUCCESS) {
        SAMPLE_PRT("aclFinalize failed!\n");
    } else {
        SAMPLE_PRT("aclFinalize succ \n");
    }
```

 版权所有 © 华为技术有限公司 

```
    return;
}

hi_s32 dump_raw(hi_vi_pipe ViPipe)
{
    hi_s32 s32Ret       = 0;
    hi_u32 u32Nbit      = 12;
    hi_u32 u32FrmCnt    = 1;
    hi_u32 u32RawDepth  = 2;
    hi_u32 u32ByteAlign = 1;
    hi_u32 u32RatioShow = 0;
    hi_compress_mode enCompressMode = HI_COMPRESS_MODE_NONE;
    hi_vi_frame_dump_attr astBackUpDumpAttr;
    hi_vi_frame_dump_attr stDumpAttr;
    hi_vi_pipe_attr stPipeAttr;

    if (1 > u32FrmCnt || MAX_FRM_CNT < u32FrmCnt) {
        printf("invalid FrmCnt %d, FrmCnt range from 1 to %d\n", u32FrmCnt, MAX_FRM_CNT);
        exit(HI_FAILURE);
    }

    s32Ret = hi_mpi_vi_get_pipe_attr(ViPipe, &stPipeAttr);
    if (HI_SUCCESS != s32Ret) {
        printf("Get Pipe %d attr failed!\n", ViPipe);
        return s32Ret;
    }
    u32Nbit = hi_pixel_format_bit_width(stPipeAttr.pixel_format);
    if (u32Nbit == 0) {
        printf("the enPixFmt is %d, cannot dump raw!\n", stPipeAttr.pixel_format);
        return -1;
    }
    s32Ret = hi_mpi_vi_get_pipe_frame_dump_attr(ViPipe, &astBackUpDumpAttr);
    if (HI_SUCCESS != s32Ret) {
        printf("Get Pipe %d dump attr failed!\n", ViPipe);
        return s32Ret;
    }
    memcpy(&stDumpAttr, &astBackUpDumpAttr, sizeof(hi_vi_frame_dump_attr));
    stDumpAttr.enable = HI_TRUE;
    stDumpAttr.depth = u32RawDepth;
    s32Ret = hi_mpi_vi_set_pipe_frame_dump_attr(ViPipe, &stDumpAttr);
    if (HI_SUCCESS != s32Ret) {
        printf("Set Pipe %d dump attr failed!\n", ViPipe);
        return s32Ret;
    }
    // dump raw
    DumpLinearBayer(ViPipe, u32Nbit, enCompressMode, u32FrmCnt, u32ByteAlign, u32RatioShow);
    stDumpAttr.enable = HI_FALSE;
    s32Ret = hi_mpi_vi_set_pipe_frame_dump_attr(ViPipe, &stDumpAttr);
    if (HI_SUCCESS != s32Ret) {
        printf("Set Pipe %d dump attr failed!\n", ViPipe);
        return s32Ret;
    }
    return s32Ret;
}

hi_s32 dump(hi_s32 Pipe)
{
    hi_char szYuvName[HI_MAX_NAME_SIZE];
    hi_u32 depth = 3;
    hi_s32 s32MilliSec = 120000;
    hi_video_frame_info stFrame;
    hi_s32 s32Ret;
    hi_vi_chn_attr stChnAttr;
    hi_s32 Chn = 0;
    FILE* pfd = HI_NULL;

    s32Ret = hi_mpi_vi_get_chn_attr(Pipe, Chn, &stChnAttr);
    if (s32Ret != HI_SUCCESS)  {
        SAMPLE_PRT("get chn attr error!!!\n");
```

 版权所有 © 华为技术有限公司 

```
        return HI_FAILURE;
    }

    stChnAttr.depth = depth;
    s32Ret = hi_mpi_vi_set_chn_attr(Pipe, Chn, &stChnAttr);
    if (s32Ret != HI_SUCCESS) {
        SAMPLE_PRT("set chn attr error!!!\n");
        return HI_FAILURE;
    }

    memset(&stFrame, 0, sizeof(stFrame));

    for (int i = 0; i < HI_MAX_PIPE_NUM; i++) {
        s32Ret = hi_mpi_vi_get_chn_frame(Pipe, Chn, &stFrame, s32MilliSec);
        if (s32Ret != HI_SUCCESS) {
            SAMPLE_PRT("Get frame fails s32Ret = 0x%x!!\n", s32Ret);
            continue;
        }
        SAMPLE_PRT("hi_mpi_vi_get_chn_frame success\n!");
        break;
    }
    if (s32Ret != HI_SUCCESS) {
        SAMPLE_PRT("get frame error for 4 times, now exit !!!\n");
        return HI_FAILURE;
    }
    /* get frame */
    snprintf(szYuvName, HI_MAX_NAME_SIZE, "./vi_pipe%d_chn%d_w%d_h%d.yuv",
        Pipe, Chn, stFrame.v_frame.width, stFrame.v_frame.height);
    /* open file */
    pfd = fopen(szYuvName, "wb");
    if (HI_NULL == pfd) {
        SAMPLE_PRT("open file failed:%s!\n", strerror(errno));
    }
    hi_u32 u32Size = (stFrame.v_frame.width) * (stFrame.v_frame.height) * 3 / 2;
    fwrite(stFrame.v_frame.phys_addr[0], u32Size, 1, pfd);
    fflush(pfd);
    fclose(pfd);
    s32Ret = hi_mpi_vi_release_chn_frame(Pipe, Chn, &stFrame);
    if (HI_SUCCESS != s32Ret) {
        SAMPLE_PRT("Release frame error ,now exit s32Ret is %#x!!!\n", s32Ret);
    }
    return s32Ret;
}

hi_s32 start_camera(hi_u32 index, case_info sensor_info)
{
    hi_s32 ret;
    hi_size stSize;
    hi_s32 sensorId;
    hi_s32 num;
    lane_divide_mode_t divide_mode;
    SAMPLE_SNS_TYPE_E sensor_type;
    SAMPLE_VI_INFO_S sampleViInfo[HI_MAX_PIPE_NUM];

    ret = sample_sys_init();
    if (ret != HI_SUCCESS) {
        goto sys_init_failed;
    }
    // Obtaining Sensor and MIPI Information
    if (sensor_info.sensor_num == 1) {
        divide_mode = 1;
        g_stViConfig.s32WorkingViNum = 1; // simple pipe
    } else {
        divide_mode = 3;
        g_stViConfig.s32WorkingViNum = HI_MAX_PIPE_NUM;
    }
    // 1: imx219
    if (sensor_info.sensor_type == 1) {
        sensor_type = SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL;
```

 版权所有 © 华为技术有限公司 

```
    } else if (sensor_info.sensor_type == 2){
        sensor_type = SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL;  //修改支持CSI接收YUV422图片
    } else {
        goto sys_init_failed;
    }
    SAMPLE_PRT("The input sensor type is %#x\n", sensor_type);

    g_stViConfig.mipi_lane_divide_mode = divide_mode;
    for (num = 0; num < g_stViConfig.s32WorkingViNum; num++) {
        ret = get_sample_vi_info_by_sensorType(sensor_type, &sampleViInfo[num + 2], num + 2);//根据环境修改sampleViInfo的值，此样例mipi dev2是4条lane
        if (ret != HI_SUCCESS) {
            return -1;
        }
        hi_vi_dev_attr stViDevAttr;
        Sample_Comm_Vi_GetDevAttrBySns(sensor_type, &stViDevAttr);
        g_stViConfig.as32WorkingViId[num] = num + 2;//根据环境修改sampleViInfo的值，此样例mipi dev2是4条lane
        memcpy(&(g_stViConfig.astViInfo[num + 2]), &sampleViInfo[num + 2], sizeof(SAMPLE_VI_INFO_S));//根据环境修改sampleViInfo的值，此样例mipi dev2是4条lane
    }

    ret = Sample_Comm_Vi_StartVi(&g_stViConfig);
    if (ret != HI_SUCCESS) {
        goto start_vi_failed;
    }

    return 0;
start_vi_failed:
    Sample_Comm_Vi_StopVi(&g_stViConfig);
sys_init_failed:
    return -1;
}

hi_s32 get_sample_vi_info_by_sensorType(const SAMPLE_SNS_TYPE_E sensor_type,
    SAMPLE_VI_INFO_S *sample_vi_info, hi_s32 dev_id)
{
    static SAMPLE_VI_INFO_S sample_vi_info_linear = {
        .stSnsInfo = {
            .s32SnsId = 4,
            .s32BusId = 2,
            .MipiDev = 0,
        },
        .stDevInfo = {
            .enWDRMode = HI_WDR_MODE_NONE,
        },
        .stPipeInfo = {
            .aPipe = {2, -1, -1, -1},//根据环境修改sampleViInfo的值，此样例mipi dev2是4条lane
        },
        .stChnInfo = {
            .ViChn = 0,
            .enPixFormat = HI_PIXEL_FORMAT_YVU_SEMIPLANAR_420,
            .enDynamicRange = HI_DYNAMIC_RANGE_SDR8,
            .enVideoFormat = HI_VIDEO_FORMAT_LINEAR,
            .enCompressMode = HI_COMPRESS_MODE_NONE
        }
    };
    switch (sensor_type) {
        case SONY_IMX219_MIPI_3K_10BIT_NORMAL:
        case SONY_IMX219_MIPI_1920x1080_10BIT_NORMAL:
        case SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL:
            memcpy(sample_vi_info, &sample_vi_info_linear, sizeof(SAMPLE_VI_INFO_S));
            sample_vi_info->stSnsInfo.enSnsType = sensor_type;
            sample_vi_info->stSnsInfo.s32BusId = dev_id;
            sample_vi_info->stSnsInfo.s32SnsId = 0;
            sample_vi_info->stSnsInfo.MipiDev = dev_id;
            sample_vi_info->stPipeInfo.aPipe[0] = dev_id;
            sample_vi_info->stDevInfo.ViDev = dev_id;
```

 版权所有 © 华为技术有限公司 

```
            pipe_mult_sensor[dev_id] = dev_id;
            break;
        case SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL:
            memcpy(sample_vi_info, &sample_vi_info_linear, sizeof(SAMPLE_VI_INFO_S));
            sample_vi_info->stSnsInfo.enSnsType = sensor_type;
            sample_vi_info->stSnsInfo.s32BusId = 0 + yuv_cnt*2;
            sample_vi_info->stSnsInfo.s32SnsId = 1;
            sample_vi_info->stSnsInfo.MipiDev = 2 + yuv_cnt*2;
            sample_vi_info->stPipeInfo.aPipe[0] = 2 + yuv_cnt;
            sample_vi_info->stDevInfo.ViDev = 2 + yuv_cnt*2;
            pipe_mult_sensor[dev_id] = 2 + yuv_cnt;
            yuv_cnt++;                  //修改支持CSI接收YUV422图片
            break;
        default:
            SAMPLE_PRT("Function %s() Error. Undefined sensor type: %#x\n", __FUNCTION__,
sensor_type);
            return HI_FAILURE;
    }

    return HI_SUCCESS;
}

hi_s32 vi_with_sensor(hi_u32 index, case_info info) {  //修改支持CSI接收YUV422图片
    hi_s32 ret;
    ret = start_camera(index, info);
    if (ret == 0) {
        sleep(1);
        // ?camera ?Pipe0, ?camera ?Pipe0,2
        dump(2);
        dump(3);
    }
    close_camera();
    return 0;
}
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤14** 修改“sensor_management.c”文件。

1. 执行如下命令，进入“sensor_management.c”文件目录。

   cd */opt*/**samples-master/cplusplus/level1_single_api/6_media/1_camera/camera_sample/sensor_sample**

2. 执行如下命令，进入“sensor_management.c”文件编译模式。

   **vim sensor_management.c**

3. 修改代码。

   a. 将“hi_s32 Sample_Set_Mipi_2lane”和“hi_s32 Sample_Comm_Vi_GetComboAttrBySns”函数修改为如下代码的“hi_s32 Sample_Comm_Vi_GetComboAttrBySns”函数。

   b. 将“sony_imx477”修改为“sensor_isp”信息，具体如下代码中加粗字体内容。

```
#include "sensor_management.h"
#include "sony_imx219_config.h"
#include "isp_sensor_config.h"

hi_s32 Sample_Comm_Vi_GetComboAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, combo_dev_t
MipiDev, combo_dev_attr_t* pstComboAttr)
{
    switch (enSnsType) {
        case SONY_IMX219_MIPI_3K_10BIT_NORMAL:
            memcpy(pstComboAttr,
&MIPI_2lane_CHN0_SENSOR_IMX219_10BIT_24M_NOWDR_ATTR, sizeof(combo_dev_attr_t));
            pstComboAttr->devno = MipiDev;
            switch(MipiDev){
                case 0:
```

 版权所有 © 华为技术有限公司 

```
                pstComboAttr->mipi_attr.lane_id[0]= 0;
                pstComboAttr->mipi_attr.lane_id[1]= 2;
                break;
            case 1:
                pstComboAttr->mipi_attr.lane_id[0]= 1;
                pstComboAttr->mipi_attr.lane_id[1]= 3;
                break;
            case 2:
                pstComboAttr->mipi_attr.lane_id[0]= 4;
                pstComboAttr->mipi_attr.lane_id[1]= 6;
                break;
            case 3:
                pstComboAttr->mipi_attr.lane_id[0]= 5;
                pstComboAttr->mipi_attr.lane_id[1]= 7;
                break;
            default:
                printf("Function %s() Error. MipiDev: %d\n", __FUNCTION__, MipiDev);
                return HI_FAILURE;
        }
        break;
    case SONY_IMX219_MIPI_1920x1080_10BIT_NORMAL:
    case SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL:
        memcpy(pstComboAttr,
&MIPI_2lane_CHN0_SENSOR_IMX219_10BIT_1920x1080_NOWDR_ATTR,
sizeof(combo_dev_attr_t));
        pstComboAttr->devno = MipiDev;
        switch(MipiDev){
            case 0:
                pstComboAttr->mipi_attr.lane_id[0]= 0;
                pstComboAttr->mipi_attr.lane_id[1]= 2;
                break;
            case 1:
                pstComboAttr->mipi_attr.lane_id[0]= 1;
                pstComboAttr->mipi_attr.lane_id[1]= 3;
                break;
            case 2:
                pstComboAttr->mipi_attr.lane_id[0]= 4;
                pstComboAttr->mipi_attr.lane_id[1]= 6;
                break;
            case 3:
                pstComboAttr->mipi_attr.lane_id[0]= 5;
                pstComboAttr->mipi_attr.lane_id[1]= 7;
                break;
            default:
                printf("Function %s() Error. MipiDev: %d\n", __FUNCTION__, MipiDev);
                return HI_FAILURE;
        }
        break;
    case SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL:
        memcpy(pstComboAttr, &MIPI_4lane_SENSOR_NONE_12BIT_1920x1080_ATTR,
sizeof(combo_dev_attr_t));
    break;
    default:
        return HI_FAILURE;
  }
  return HI_SUCCESS;
}

hi_s32 Sample_Comm_Vi_GetSnsRstSourceBySns(SAMPLE_SNS_TYPE_E enSnsType, combo_dev_t
MipiDev, sns_rst_source_t* sns_reset_port)
{
  switch (enSnsType) {
    case SONY_IMX219_MIPI_3K_10BIT_NORMAL:
    case SONY_IMX219_MIPI_1920x1080_10BIT_NORMAL:
    case SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL:
    case SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL:
        *sns_reset_port = 0;
        break;
    default:
```

 版权所有 © 华为技术有限公司 

```
            printf("Function %s() Error. Undefined sensor type: %#x\n", __FUNCTION__, enSnsType);
            return HI_FAILURE;
    }
    return HI_SUCCESS;
}

hi_s32 Sample_Comm_Vi_GetSnsClkCfgBySns(SAMPLE_SNS_TYPE_E enSnsType, combo_dev_t MipiDev,
sns_clk_cfg_t* sns_clk_cfg)
{
    switch (enSnsType) {
        case SONY_IMX219_MIPI_3K_10BIT_NORMAL:
        case SONY_IMX219_MIPI_1920x1080_10BIT_NORMAL:
        case SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL:
        case SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL:
            memcpy(sns_clk_cfg, &g_mipi_sns_clk_cfg_attr, sizeof(sns_clk_cfg_t));
            break;
        default:
            printf("Function %s() Error. Undefined sensor type: %#x\n", __FUNCTION__, enSnsType);
            return HI_FAILURE;
    }
    return HI_SUCCESS;
}

hi_s32 Sample_Comm_Isp_GetIspAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, hi_isp_pub_attr*
pstPubAttr)
{
    switch (enSnsType)
    {
        case SONY_IMX219_MIPI_3K_10BIT_NORMAL:
            memcpy(pstPubAttr, &ISP_PUB_ATTR_IMX219_24M_20FPS_raw10, sizeof(hi_isp_pub_attr));
            break;
        case SONY_IMX219_MIPI_1920x1080_10BIT_NORMAL:
            memcpy(pstPubAttr, &ISP_PUB_ATTR_IMX219_1920x1080_20FPS_raw10,
sizeof(hi_isp_pub_attr));
            break;
        case SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL:
            memcpy(pstPubAttr, &ISP_PUB_ATTR_IMX219_1920x1080_30FPS_raw10,
sizeof(hi_isp_pub_attr));
            break;
        case SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL:
            memcpy(pstPubAttr, &ISP_PUB_ATTR_SENSOR_NONE_1080P_30FPS,
sizeof(hi_isp_pub_attr));
            break;
        default:
            return HI_FAILURE;
    }

    return HI_SUCCESS;
}

hi_s32 Sample_Comm_Vi_GetDevAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, hi_vi_dev_attr*
pstViDevAttr)
{
    switch (enSnsType)
    {
        case SONY_IMX219_MIPI_3K_10BIT_NORMAL:
            memcpy(pstViDevAttr, &DEV_ATTR_IMX219_24M, sizeof(hi_vi_dev_attr));
            break;
        case SONY_IMX219_MIPI_1920x1080_10BIT_NORMAL:
        case SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL:
            memcpy(pstViDevAttr, &DEV_ATTR_IMX219_1920x1080, sizeof(hi_vi_dev_attr));
            break;
        case SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL:
            memcpy(pstViDevAttr, &DEV_ATTR_SENSOR_NONE_1080P_BASE, sizeof(hi_vi_dev_attr));
            break;
        default:
            return HI_FAILURE;
    }
```

 版权所有 © 华为技术有限公司 

```
    return HI_SUCCESS;
}

hi_s32 Sample_Comm_Vi_GetPipeAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, hi_vi_pipe_attr*
pstPipeAttr)
{
    switch (enSnsType)
    {
        case SONY_IMX219_MIPI_3K_10BIT_NORMAL:
            memcpy(pstPipeAttr, &IMX219_PIPE_ATTR_3264x2448_RAW10_420_3DNR_RFR,
sizeof(hi_vi_pipe_attr));
            break;
        case SONY_IMX219_MIPI_1920x1080_10BIT_NORMAL:
        case SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL:
            memcpy(pstPipeAttr, &IMX219_PIPE_ATTR_1920x1080_RAW10_420_3DNR_RFR,
sizeof(hi_vi_pipe_attr));
            break;
        case SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL:
            memcpy(pstPipeAttr, &PIPE_ATTR_SENSOR_NONE_1920x1080_RAW8_420_3DNR_RFR,
sizeof(hi_vi_pipe_attr));
            break;
        default:
        return HI_FAILURE;
    }

    return HI_SUCCESS;
}

hi_s32 Sample_Comm_Vi_GetChnAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, hi_vi_chn_attr*
pstChnAttr)
{
    switch (enSnsType)
    {
        case SONY_IMX219_MIPI_3K_10BIT_NORMAL:
            memcpy(pstChnAttr, &IMX219_CHN_ATTR_3264x2448_420_SDR8_LINEAR,
sizeof(hi_vi_chn_attr));
            break;
        case SONY_IMX219_MIPI_1920x1080_10BIT_NORMAL:
        case SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL:
            memcpy(pstChnAttr, &IMX219_CHN_ATTR_1920x1080_420_SDR8_LINEAR,
sizeof(hi_vi_chn_attr));
            break;
        case SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL:
            memcpy(pstChnAttr, &CHN_ATTR_SENSOR_NONE_1920x1080_420_SDR8_LINEAR,
sizeof(hi_vi_chn_attr));
            break;
        default:
            return HI_FAILURE;
    }
    return HI_SUCCESS;
}

const hi_isp_sns_obj* g_enSnsObj[MAX_SENSOR_NUM] = {
    &g_sns_imx219_obj,
    &g_sns_isp_obj,
};

const hi_isp_sns_obj* Sample_Comm_Isp_GetSnsObj(hi_u32 u32SnsId)
{
    return g_enSnsObj[u32SnsId];
}
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤15** 修改“sensor_management.h”文件。

1. 执行如下命令，进入“sensor_management.h”文件编译模式。

   **vim sensor_management.h**

 版权所有 © 华为技术有限公司 

2. 将代码中的“sony_imx477”修改为“sensor_isp”信息，具体如下加粗字体内容。

```
#include "hi_mipi_rx.h"
#include "hi_mpi_isp.h"
#include "hi_common_sns.h"
#include "hi_common_3a.h"
#include "hi_mpi_vi.h"
#include "hi_sns_ctrl.h"
#define MAX_SENSOR_NUM 20

#ifndef SENSOR_MANAGEMENT_H
#define SENSOR_MANAGEMENT_H

typedef unsigned int combo_dev_t;
typedef unsigned int sns_rst_source_t;

typedef enum HISAMPLE_SNS_TYPE_E
{
    SONY_IMX219_MIPI_3K_10BIT_NORMAL = 0x60,
    SONY_IMX219_MIPI_1920x1080_10BIT_NORMAL = 0x62,
    SONY_IMX219_MIPI_1920x1080_10BIT_30FPS_NORMAL = 0x64,
    SENSOR_NONE_MIPI_1920x1080_8BIT_YUV_30FPS_NORMAL = 0x71,

    SAMPLE_SNS_TYPE_BUTT,
} SAMPLE_SNS_TYPE_E;

const hi_isp_sns_obj* Sample_Comm_Isp_GetSnsObj(hi_u32 u32SnsId);
hi_s32 Sample_Comm_Vi_GetComboAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, combo_dev_t
MipiDev, combo_dev_attr_t* pstComboAttr);
hi_s32 Sample_Comm_Isp_GetIspAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, hi_isp_pub_attr*
pstPubAttr);
hi_s32 Sample_Comm_Vi_GetDevAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, hi_vi_dev_attr*
pstViDevAttr);
hi_s32 Sample_Comm_Vi_GetChnAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, hi_vi_chn_attr*
pstChnAttr);
hi_s32 Sample_Comm_Vi_GetPipeAttrBySns(SAMPLE_SNS_TYPE_E enSnsType, hi_vi_pipe_attr*
pstPipeAttr);
hi_s32 Sample_Comm_Vi_GetSnsRstSourceBySns(SAMPLE_SNS_TYPE_E enSnsType, combo_dev_t
MipiDev, sns_rst_source_t* sns_reset_port);
hi_s32 Sample_Comm_Vi_GetSnsClkCfgBySns(SAMPLE_SNS_TYPE_E enSnsType, combo_dev_t MipiDev,
sns_clk_cfg_t* sns_clk_cfg);
extern const hi_isp_sns_obj* g_enSnsObj[];

// Sensor Object
extern hi_isp_sns_obj g_sns_imx219_obj;
extern hi_isp_sns_obj g_sns_isp_obj;
#endif
```

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤16** 修改“camera_sample”目录下的“CMakeLists.txt”文件。

1. 执行如下命令，进入“camera_sample”文件目录。

   cd */opt*/**samples-master/cplusplus/level1_single_api/6_media/1_camera/camera_sample**

2. 执行如下命令，进入“CMakeLists.txt”文件编译模式。

   **vim CMakeLists.txt**

3. 将代码中的“sony_imx477”修改为“sensor_isp”信息，具体如下加粗字体内容。

```
# Copyright (c) Huawei Technologies Co., Ltd. 2023. All rights reserved.

# CMake lowest version requirement
cmake_minimum_required(VERSION 3.5.1)

# project information
project(vi_l1_sample)
```

 版权所有 © 华为技术有限公司 

```
set(LIB_PATH $ENV{NPU_HOST_LIB})
# Dynamic libraries in the stub directory can only be used for compilation
if (NOT DEFINED ENV{NPU_HOST_LIB})
    set(LIB_PATH "/usr/local/Ascend/ascend-toolkit/latest/CANN-6.4/runtime/lib64/stub/aarch64")
    message(STATUS "set default LIB_PATH: ${LIB_PATH}")
else ()
    message(STATUS "env LIB_PATH: ${LIB_PATH}")
endif()

set(INC_PATH $ENV{NPU_HOST_INC})
# Dynamic libraries in the stub directory can only be used for compilation
if (NOT DEFINED ENV{NPU_HOST_INC})
    set(INC_PATH "/usr/local/Ascend/ascend-toolkit/latest/CANN-6.4/runtime/include")
    message(STATUS "set default INC_PATH: ${INC_PATH}")
else ()
    message(STATUS "env INC_PATH: ${INC_PATH}")
endif()

link_directories(
    ${LIB_PATH}
    ${LIB_PATH}/stub
)

add_executable(vi_l1_sample
  main.c
  vi_with_sensor.c
  ./common/sample_comm_isp.c
  ./common/sample_comm_vi.c
  ./sensor_sample/sensor_management.c
  ./common/image_dump_util.c
)

include_directories(vi_l1_sample
    ${INC_PATH}
    ${INC_PATH}/acl/media
    ./sensor_sample
    ./common
    ./sensor/include
    ./sensor/sony_imx219
    ./sensor/sensor_isp
)

add_compile_options(vi_l1_sample
    -O2
    -Wall
    -fpic
)

target_link_libraries(vi_l1_sample
    acl_dvpp_mpi
    acl_isp_mpi
    sns_imx219
    sns_isp
    acl_vi_mpi
    acl_vpss_mpi
    acl_isp_ae_mpi
    acl_isp_awb_mpi
    ascendcl
    pthread
)
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤17** 修改“sensor”目录下的“CMakeLists.txt”文件。

1. 执行如下命令，进入“sensor”文件目录。

   cd */opt*/**samples-master/cplusplus/level1_single_api/6_media/1_camera/camera_sample/sensor**

 版权所有 © 华为技术有限公司 

2. 执行如下命令，进入“CMakeLists.txt”文件编译模式。

   **vim CMakeLists.txt**

3. 将代码中的“sony_imx477”修改为“sensor_isp”信息，具体修改如下加粗字体内容。

   ```
   add_subdirectory(sony_imx219)
   add_subdirectory(sensor_isp)
   ```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤18** 参考**camera_sample**编译生成“libsns_imx219.so”、“libsns_isp.so”文件和CSI工具。

**步骤19** 登录Atlas 200I A2 加速模块整机，具体请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤20** 使用WinSCP工具或其他工具，将**步骤18**编译成的“libsns_isp.so”和“vi_l1_sample”上传至Atlas 200I A2 加速模块整机目录，例如“/mnt”。

“libsns_isp.so”还需要放到CANN包安装路径下，例如：“/usr/local/Ascend/nnrt/7.0.0/runtime/lib64/”路径下。

**步骤21** 执行如下命令，进入“/mnt”目录。

**cd** */mnt*

**步骤22** 执行如下命令，配置环境变量。

**export LD_LIBRARY_PATH=/lib64/:$LD_LIBRARY_PATH**

**步骤23** 执行如下命令，运行CSI工具。

**./vi_l1_sample** 1 1 2

- 参数1：测试程序模式，1为sensor出yuv图，2为出raw图 。Atlas 200I A2 加速模块暂不支持raw图。
- 参数2：camera num，可选1或2。
- 参数3：sensor_type，1为“imx219”，2为“isp”。

**说明**

CSI工具若存在有变化（例如：升级驱动、系统重启、退出CSI工具等），则需要hdmi信号输入退出后，再重启CSI工具。

**步骤24** 用户自行配置HDMI信号输入。

**步骤25** 执行如下命令，查看是否收到数据。

**cat /proc/umap/mipi_rx**

如出现以下类似回显信息，则表示收到数据。

```
----mipi phy data info-----------------------------------------------------
 phy_id       lane_id          phy_data              mipi_data             lvds_data             freq
     0      0, 1, 2, 3    0x00,0x00,0x00,0x00    0x00,0x00,0x00,0x00    0x00,0x00,0x00,0x00      n/a
     1      4, 5, 6, 7    0xc4,0x48,0xbf,0x22    0x1e,0x00,0x1e,0x3a    0x00,0x00,0x00,0x00    156MHZ
```

**步骤26** 执行如下命令，在“vi_l1_sample”工具目录下查看生成的图片。

**cd** */mnt*

**ls**

 版权所有 © 华为技术有限公司 

生成图片为“vi_pipe2_chn0_w3840_h2160.yuv”。

**----结束**

#### 7.9.3.3 调测参考

详细请参见《应用开发接口》的**“ISP系统控制及3A算法注册”**、**“MIPI Rx ioctl命令字”**与**“VI视频输入功能”**章节。

### 7.9.4 MIPI-DSI

#### 7.9.4.1 Demo 参考

请参见Gitee上Ascend/samples开源代码仓中的**raspberry_sample**。

#### 7.9.4.2 调测参考

当前硬件支持，无配套软件功能，DSI接口视频输出二次开发能力规划中。

## 7.10 SPI

### 7.10.1 概述

SPI是串行外围设备接口，是一种高速的，全双工，同步的通信总线，支持一主多从。

### 7.10.2 寄存器描述

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| SPI0_SCLK | 0x00823200 00 | 0x060 | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_spi0_sclk；<br>010：pad_scl12；<br>011：pad_gpio2_25；<br>100：pad_urts3；<br>111：pad_prb_e[14]；<br>其它：保留。 |


版权所有 © 华为技术有限公司

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| SPI0_CSN | 0x0082320000 | 0x064 | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_spi0_csn；<br>010：pad_sda11；<br>011：pad_gpio2_26；<br>100：pad_utxd3；<br>111：pad_prb_e[15]；<br>其它：保留。 |
| SPI0_SDO | 0x0082320000 | 0x068 | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_spi0_sdo；<br>010：pad_sda12；<br>011：pad_gpio2_27；<br>100：pad_ucts3；<br>111：pad_prb_e[16]；<br>其它：保留。 |
| SPI0_SDI | 0x0082320000 | 0x06C | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_spi0_sdi；<br>010：pad_scl11；<br>011：pad_gpio2_28；<br>100：pad_urxd3；<br>111：pad_prb_e[17]；<br>其它：保留。 |
| SPI1_SCLK | 0x0082320000 | 0x070 | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_spi1_sclk；<br>011：pad_gpio2_29；<br>111：pad_prb_e[18]；<br>其它：保留。 |
| SPI1_CSN | 0x0082320000 | 0x074 | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_spi1_csn；<br>011：pad_gpio2_30；<br>111：pad_prb_e[19]；<br>其它：保留。 |


版权所有 © 华为技术有限公司

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| SPI1_SDO | 0x0082320000 | 0x078 | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_spi1_sdo；<br>011：pad_gpio2_31；<br>111：pad_prb_e[20]；<br>其它：保留。 |
| SPI1_SDI | 0x0082320000 | 0x07C | 0x00000000 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_spi1_sdi；<br>011：pad_gpio3_00；<br>111：pad_prb_e[21]；<br>其它：保留。 |
| SPI2_SCLK | 0x00C4000000 | 0x0D4 | 0x00000000 | 000：pad_spi2_sclk；<br>001：pad_spi5_sclk；<br>011：pad_gpio0_20；<br>110：pad_mdc_dc_md0；<br>111：pad_prb_a[30]；<br>其它：保留。 |
| SPI2_CSN | 0x00C4000000 | 0x0D8 | 0x00000000 | 000：pad_spi2_csn；<br>001：pad_spi5_csn；<br>010：pad_sda10；<br>011：pad_gpio0_21；<br>111：pad_prb_a[31]；<br>其它：保留。 |
| SPI2_SDO | 0x00C4000000 | 0x0DC | 0x00000000 | 000：pad_spi2_sdo；<br>001：pad_spi5_sdo；<br>011：pad_gpio0_22；<br>110：pad_mdc_dc_md1；<br>111：pad_prb_a[32]；<br>其它：保留。 |

 版权所有 © 华为技术有限公司 

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| SPI2_SDI | 0x00C4000000 | 0x0E0 | 0x00000000 | 000：pad_spi2_sdi;<br>001：pad_spi5_sdi;<br>010：pad_scl10;<br>011：pad_gpio0_23;<br>111：pad_prb_a[33];<br>其它：保留。 |
| SCL0 | 0x3001D000 | 0x000 | 0x00000000 | 000：pad_scl0;<br>001：pad_spi3_sclk;<br>010：pad_sensor_hs1;<br>011：pad_gpio5_00;<br>111：pad_prb_c[0];<br>其它：保留。 |
| SDA0 | 0x3001D000 | 0x004 | 0x00000000 | 000：pad_sda0;<br>001：pad_spi3_sdo;<br>010：pad_sensor_hs3;<br>011：pad_gpio5_01;<br>111：pad_prb_c[1];<br>其它：保留。 |
| SCL1 | 0x3001D000 | 0x008 | 0x00000000 | 000：pad_scl1;<br>001：pad_spi3_sdi;<br>011：pad_gpio5_02;<br>111：pad_prb_c[2];<br>其它：保留。 |
| SDA1 | 0x3001D000 | 0x00C | 0x00000000 | 000：pad_sda1;<br>001：pad_spi3_csn;<br>011：pad_gpio5_03;<br>111：pad_prb_c[3];<br>其它：保留。 |
| SCL2 | 0x3001D000 | 0x010 | 0x00000000 | 000：pad_scl2;<br>001：pad_spi4_sclk;<br>011：pad_gpio5_04;<br>111：pad_prb_c[4];<br>其它：保留。 |


版权所有 © 华为技术有限公司

| 管脚名称 | MUX Base Address | Offset Address | Reset Value | 复用关系 |
| --- | --- | --- | --- | --- |
| SDA2 | 0x3001D0000 | 0x014 | 0x00000000 | 000：pad_sda2；<br>001：pad_spi4_sdo；<br>011：pad_gpio5_05；<br>111：pad_prb_c[5]；<br>其它：保留。 |
| SCL3 | 0x3001D0000 | 0x018 | 0x00000000 | 000：pad_scl3；<br>001：pad_spi4_sdi；<br>011：pad_gpio5_06；<br>111：pad_prb_c[6]；<br>其它：保留。 |
| SDA3 | 0x3001D0000 | 0x01C | 0x00000000 | 000：pad_sda3；<br>001：pad_spi4_csn；<br>011：pad_gpio5_07；<br>111：pad_prb_c[7]；<br>其它：保留。 |

## 7.10.3 调测参考

以首次调测为例，操作步骤如下。

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 新增整机dts文件。

例如用户整机的boardid为33150，故dts文件名称为“hi1910B-asic-M150-B33.dts”。

 版权所有 © 华为技术有限公司 

1. 执行如下命令，进入配置文件所在目录。
   **cd dtb/dts/hi1910b/hi1910BL**
2. 执行如下命令，新增适配boardid33150的dts文件。
   **cp hi1910B-asic-M150-B50.dts hi1910B-asic-M150-B33.dts**
   将“hi1910B-asic-M150-B50.dts”另存为boardid33150的dts文件“hi1910B-asic-M150-B33.dts”。

**步骤8** 参考product目录下的任意一个spi.dtsi文件配置新增文件的SPI内容，例如hi1910B-spi-M100-B51.dtsi。

1. 执行如下命令进入“product”目录下。
   **cd product**
2. 打开“hi1910B-spi-M100-B51.dtsi”文件参考其中spi配置。
   **vim hi1910B-spi-M100-B51.dtsi**
   文件内容如下。

```
spi0@82020000 {
		compatible = "hisilicon,udrv_spi";
		#address-cells = <1>;
		#size-cells = <0>;
		num-cs = <1>; /* 支持cs片选个数量 */
		bus-num = <0>; /* spi控制器id */
		clk-freq = <150000000>; /* 时钟频率 */
		reg = <0x0 0x82020000 0 0x10000>;
		interrupts = <0x0 161 0x4>;
		cs@0 {
			compatible = "rohm,dh2228fv";
			reg = <0>; /* cs片选id */
			spi-max-frequency = <25000000>;
		};
	};

	spi1@82030000 {
		compatible = "hisilicon,udrv_spi";
		#address-cells = <1>;
		#size-cells = <0>;
		num-cs = <1>; /* 支持cs片选个数量 */
		bus-num = <1>; /* spi控制器id */
		clk-freq = <150000000>; /* 时钟频率 */
		reg = <0x0 0x82030000 0 0x10000>;
		interrupts = <0x0 162 0x4>;
		cs@0 {
			compatible = "rohm,dh2228fv";
			reg = <0>; /* cs片选id */
			spi-max-frequency = <25000000>;
		};
	};

	spi3@3001B0000 {
		compatible = "hisilicon,udrv_spi";
		#address-cells = <1>;
		#size-cells = <0>;
		num-cs = <1>; /* 支持cs片选个数量 */
		bus-num = <3>; /* spi控制器id */
		clk-freq = <150000000>; /* 时钟频率 */
		reg = <0x3 0x1B0000 0 0x10000>;
		interrupts = <0x0 251 0x4>;
		// To enable the SPI function, uncomment the following two lines and comment the pinctrl configuration of the I2C 0 1
		// pinctrl-names = "default";
		// pinctrl-0 = <&iomg000_pmx_func &iomg001_pmx_func &iomg002_pmx_func &iomg003_pmx_func>;
		cs@0 {
			compatible = "rohm,dh2228fv";
```

 版权所有 © 华为技术有限公司 

```
                reg = <0>; /* cs片选id */
                spi-max-frequency = <25000000>;
        };
};
```

3. 创建“hi1910B-spi-Mxxx-Bxx.dtsi”文件，例如hi1910B-spi-M150-B33.dtsi。将需要使用的对应SPI控制拷贝进去，如使用spi0，内容如下。

```
spi0@82020000 {
        compatible = "hisilicon,udrv_spi";
        #address-cells = <1>;
        #size-cells = <0>;
        num-cs = <1>; /* 支持cs片选个数量 */
        bus-num = <0>; /* spi控制器id */
        clk-freq = <150000000>; /* 时钟频率 */
        reg = <0x0 0x82020000 0 0x10000>;
        interrupts = <0x0 161 0x4>;
        cs@0 {
                compatible = "rohm,dh2228fv";
                reg = <0>; /* cs片选id */
                spi-max-frequency = <25000000>;
        };
};
```

4. 执行如下命令，返回“hi1910B-asic-Mxxx-Bxx.dts”文件所在目录，例如hi1910B-asic-M150-B33.dts。

   **cd** */opt*/**Ascend310B-source/dtb/dts/hi1910b/hi1910BL**

5. 在“hi1910B-asic-Mxxx-Bxx.dts”文件中增加引用“hi1910B-spi-Mxxx-Bxx.dtsi”。例如在“hi1910B-asic-M150-B33.dts”文件中增加引用“hi1910B-spi-M150-B33.dtsi”。

   ```
   /include/ "product/hi1910B-spi-Mxxx-Bxx.dtsi"
   ```

   示例：

   ```
   /include/ "product/hi1910B-spi-M150-B33.dtsi"
   ```

6. 编译并生效dtb，请参见**8.3.1.3 编译并生效内核DTB文件**。

**须知**

1. SPI2用作可信启动，不对外开放使用，无法外接SPI设备使用。
2. SPI3，SPI4在配置时要配置pinctrl复用，对应的管脚参考**7.10.2 寄存器描述**章节，pinctrl配置方式参考**7.3 UART**章节。

**----结束**

## 7.10.4 demo 参考

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*


版权所有 © 华为技术有限公司

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 执行如下命令，编译内核源码。

**bash build.sh kernelSource**

**步骤8** 进入到spi内核源码目录“Ascend310B-source/driver/kernel/linux-4.19/tools/spi”。

**cd** */opt*/**Ascend310B-source/driver/kernel/linux-4.19/tools/spi**

**步骤9** 使用交叉编译工具编译。

**make CROSS_COMPILE=aarch64-target-linux-gnu-**

```
mkdir -p include/linux/spi 2>&1 || true
ln -sf /opt/Ascend310B-sdk/Ascend310B-source/driver/kernel/linux-4.19/tools/spi/../../include/uapi/linux/spi/
spidev.h include/linux/spi/spidev.h
make -f /opt/Ascend310B-sdk/Ascend310B-source/driver/kernel/linux-4.19/tools/build/Makefile.build dir=.
obj=spidev_test
make[1]: Entering directory '/opt/Ascend310B-sdk/Ascend310B-source/driver/kernel/linux-4.19/tools/spi'
  CC       spidev_test.o
  LD       spidev_test-in.o
make[1]: Leaving directory '/opt/Ascend310B-sdk/Ascend310B-source/driver/kernel/linux-4.19/tools/spi'
  LINK     spidev_test
make -f /opt/Ascend310B-sdk/Ascend310B-source/driver/kernel/linux-4.19/tools/build/Makefile.build dir=.
obj=spidev_fdx
make[1]: Entering directory '/opt/Ascend310B-sdk/Ascend310B-source/driver/kernel/linux-4.19/tools/spi'
  CC       spidev_fdx.o
  LD       spidev_fdx-in.o
make[1]: Leaving directory '/opt/Ascend310B-sdk/Ascend310B-source/driver/kernel/linux-4.19/tools/spi'
  LINK     spidev_fdx
```

**步骤10** 编译成功会在当前目录下生成spidev_test，spidev_fdx二进制工具。可执行**ll**命令查看，回显信息如下所示。

```
total 300
drwxr-xr-x  3 root root  4096 Sep 16 14:54 ./
drwxr-xr-x 36 root root  4096 Sep 16 14:50 ../
-rw-r--r--  1 root root    60 Sep 11 10:16 Build
drwxr-xr-x  3 root root  4096 Sep 16 14:54 include/
-rw-r--r--  1 root root  1745 Sep 11 10:16 Makefile
-rwxr-xr-x  1 root root 22568 Sep 16 14:54 spidev_fdx*
-rw-r--r--  1 root root  2798 Sep 11 10:16 spidev_fdx.c
-rw-r--r--  1 root root 22336 Sep 16 14:54 spidev_fdx-in.o
-rw-r--r--  1 root root    88 Sep 16 14:54 .spidev_fdx-in.o.cmd
-rw-r--r--  1 root root 22328 Sep 16 14:54 spidev_fdx.o
-rw-r--r--  1 root root  6816 Sep 16 14:54 .spidev_fdx.o.cmd
-rw-r--r--  1 root root  6479 Sep 16 14:54 .spidev_fdx.o.d
-rwxr-xr-x  1 root root 37120 Sep 16 14:54 spidev_test*
-rw-r--r--  1 root root 10529 Sep 11 10:16 spidev_test.c
-rw-r--r--  1 root root 54312 Sep 16 14:54 spidev_test-in.o
-rw-r--r--  1 root root    91 Sep 16 14:54 .spidev_test-in.o.cmd
-rw-r--r--  1 root root 54312 Sep 16 14:54 spidev_test.o
-rw-r--r--  1 root root  7975 Sep 16 14:54 .spidev_test.o.cmd
-rw-r--r--  1 root root  7633 Sep 16 14:54 .spidev_test.o.d
```

**步骤11** 将Atlas 200I A2 加速模块SPI的MOSI管脚和MISO管脚环回测试。

 版权所有 © 华为技术有限公司 

**说明**

- MOSI（Master Output Slave Input）：主设备数据输出，从设备数据输入。
- MISO（Master Input Slave Output）：主设备数据输入，从设备数据输出。

  具体管脚定义请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“原理图设计指南 > 信号接口设计 > 其他接口 > SPI接口”章节。

1. 登录Atlas 200I A2 加速模块的环境。
2. 将spidev_test二进制工具上传到Atlas 200I A2 加速模块环境任意目录，执行如下命令增加可执行权限。

   **chmod +x spidev_test**

   spidev_test支持的参数可执行**./spidev_test --help**命令查看，如下所示。

```
-D --device   device to use (default /dev/spidev1.1)
-s --speed    max speed (Hz)
-d --delay    delay (usec)
-b --bpw      bits per word
-i --input    input data from a file (e.g. "test.bin")
-o --output   output data to a file (e.g. "results.bin")
-l --loop     loopback
-H --cpha     clock phase
-O --cpol     clock polarity
-L --lsb      least significant bit first
-C --cs-high  chip select active high
-3 --3wire    SI/SO signals shared
-v --verbose  Verbose (show tx buffer)
-p            Send data (e.g. "1234\xde\xad")
-N --no-cs    no chip select
-R --ready    slave pulls low to pause
-2 --dual     dual transfer
-4 --quad     quad transfer
-8 --octal    octal transfer
-S --size     transfer size
-I --iter     iterations
```

**步骤12** 执行如下命令，发送数据测试。

**./spidev_test -D** */dev/spidev0.0* **-p "**|*x04*|*x05*|*x20*|*x01***" -v**

回显信息如下，表示RX和TX数据和所发数据一致，收发功能正常。

```
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 kHz)
TX | 04 05 20 01 __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __  |.. .|
RX | 04 05 20 01 __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __  |.. .
```

**----结束**

# 7.11 I2S

## 7.11.1 概述

I2S = Inter-IC Sound = Integrated Interchip Sound, 是一种数字音频传输标准，用于数字音频数据在系统内器件之间传输，例如编解码器CODEC、DSP、数字输入/输出接口、ADCQ、DAC和数字滤波器等。


版权所有 © 华为技术有限公司

## 7.11.2 寄存器描述

| 管脚名称 | MUX Base Address | Offset Address | 复用关系 |
| --- | --- | --- | --- |
| I2S0_MCLK | 0x04_0014_0000 | 0x030 | Bits[31:3]：预留<br>Bits[2:0]：<br>000：pad_i2s0_mclk；<br>010：pad_gpclk0；<br>011：pad_gpio7_02；<br>100：pad_utxd7；<br>101：pad_spi7_csn；<br>111：pad_prb_d[2]；<br>其它：保留。 |
| I2S0_BCLK_TX | 0x04_0014_0000 | 0x034 | 000：pad_i2s0_bclk_tx；<br>010：pad_spi8_csn；<br>011：pad_gpio7_03；<br>111：pad_prb_d[3]；<br>其它：保留。 |
| I2S0_WS_TX | 0x04_0014_0000 | 0x038 | 000：pad_i2s0_ws_tx；<br>010：pad_spi8_sdi；<br>011：pad_gpio7_04；<br>111：pad_prb_d[4]；<br>其它：保留。 |
| I2S0_SD_TX | 0x04_0014_0000 | 0x03C | 000：pad_i2s0_sd_tx；<br>010：pad_spi8_sclk；<br>011：pad_gpio7_05；<br>100：pad_gpclk1；<br>111：pad_prb_d[5]；<br>其它：保留。 |
| I2S0_SD_RX | 0x04_0014_0000 | 0x040 | 000：pad_i2s0_sd_rx；<br>010：pad_spi8_sdo；<br>011：pad_gpio7_06；<br>100：pad_gpclk0；<br>111：pad_prb_d[6]；<br>其它：保留。 |


版权所有 © 华为技术有限公司

| 管脚名称 | MUX Base Address | Offset Address | 复用关系 |
| --- | --- | --- | --- |
| I2S1_MCLK | 0x04_0014_0000 | 0x044 | 000：pad_i2s1_mclk;<br>001：pad_i2s_mclk_dbg;<br>010：pad_gpclk1;<br>011：pad_gpio7_07;<br>100：pad_urxd7;<br>101：pad_spi7_sdi;<br>111：pad_prb_d[7];<br>其它：保留。 |
| I2S1_BCLK_TX | 0x04_0014_0000 | 0x048 | 000：pad_i2s1_bclk_tx;<br>001：pad_i2s_bclk_tx_dbg;<br>010：pad_gpclk2;<br>011：pad_gpio7_08;<br>100：pad_utxd8;<br>101：pad_spi7_sdo;<br>111：pad_prb_d[8];<br>其它：保留。 |
| I2S1_WS_TX | 0x04_0014_0000 | 0x04C | 000：pad_i2s1_ws_tx;<br>001：pad_i2s_ws_tx_dbg;<br>011：pad_gpio7_09;<br>100：pad_urxd8;<br>101：pad_spi7_sclk;<br>111：pad_prb_d[9];<br>其它：保留。 |
| I2S1_SD_TX | 0x04_0014_0000 | 0x050 | 000：pad_i2s1_sd_tx;<br>001：pad_i2s_sd_tx_dbg;<br>011：pad_gpio7_10;<br>111：pad_prb_d[10];<br>其它：保留。 |
| I2S1_SD_RX | 0x04_0014_0000 | 0x054 | 000：pad_i2s1_sd_rx;<br>001：pad_i2s_sd_rx_dbg;<br>011：pad_gpio7_11;<br>111：pad_prb_d[11];<br>其它：保留。 |


版权所有 © 华为技术有限公司

## 7.11.3 调测参考

**步骤1** 登录**4.2 准备环境**所搭建Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 新增整机dts文件。

例如用户整机的boardid为33150，故dts文件名称为“hi1910B-asic-M150-B33.dts”。

1. 执行如下命令，进入dts文件目录。

   **cd dtb/dts/hi1910b/hi1910BL**

2. 执行如下命令，新增适配boardid33150的dts文件。

   **cp hi1910B-default.dts hi1910B-asic-M150-B33.dts**

   将“hi1910B-default.dts”另存为boardid33150的dts文件“hi1910B-asic-M150-B33.dts”。

3. 执行如下命令，修改dts文件。board_id为33150。

   **vi hi1910B-asic-M150-B33.dts**

   将boardid修改为33150，同时确认dts所使用的audio.dtsi和pinctrl.dtsi文件，具体如下代码加粗字体。

```
compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
    hisi,boardid =<0x0 0x3 0x3 0x1 0x5 0x0>; //修改boardid
    #address-cells = <0x2>;
    #size-cells = <0x2>;
    interrupt-parent = <0x1>;
    model = "Hisilicon PhosphorHi1910B evb";

    /include/ "base/hi1910B-asic-kernel-rc.dtsi"
    /include/ "base/hi1910B-commmon-smmu.dtsi"
    /include/ "base/hi1910B-mpam.dtsi"
    /include/ "base/hi1910B-lp-pm.dtsi"
    /include/ "base/hi1910B-lp-devmng.dtsi"
    /include/ "base/hi1910B-mbigen-ao.dtsi"
    /include/ "base/hi1910B-gpio.dtsi"
    // /include/ "base/hi1910B-bindcore.dtsi"
    // /include/ "base/hi1910B-can.dtsi"
    /include/ "base/hi1910B-emmc.dtsi"
    /include/ "base/hi1910B-ipcdrv.dtsi"
    /include/ "base/hi1910B-ts-drv.dtsi"
    /include/ "base/hi1910B-tsdrv.dtsi"
    /include/ "base/hi1910B-network-M100-B51.dtsi"
```


版权所有 © 华为技术有限公司

```
/include/ "base/hi1910B-hidvpp.dtsi"
/include/ "base/hi1910B-hdmi.dtsi"
/include/ "product/hi1910B-audio.dtsi"   //确认所使用的audio.dtsi文件
/include/ "base/hi1910B-mipitx.dtsi"
/include/ "base/hi1910B-vdp.dtsi"
/include/ "base/hi1910B-isp.dtsi"
/include/ "base/hi1910B-hilink.dtsi"
/include/ "base/hi1910B-mntn-milanr3.dtsi"
/include/ "base/hi1910B-gpu.dtsi"
/include/ "base/hi1910B-ctl.dtsi"
    /include/ "base/hi1910B-itrustee.dtsi"
    /include/ "base/hi1910B-sata.dtsi"
    // /include/ "product/hi1910B-pcie-rc-M100-B51.dtsi"
    /include/ "product/hi1910B-pinctrl-M100-B51.dtsi"  //确认所使用的pinctrl.dtsi文件
/include/ "product/hi1910B-hiusbc-B51.dtsi"
    /include/ "product/hi1910B-i2c-M100-B51.dtsi"
    /include/ "product/hi1910B-spi-M100-B51.dtsi"
/include/ "base/hi1910B-abl-qos.dtsi"
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤8** 修改audio.dtsi文件。

1. 执行如下命令，进入“product”目录。

   **cd product**

2. 执行如下命令，修改audio.dtsi文件。

   **vim hi1910B-audio.dtsi**

3. 新增如下加粗字体内容，增加I2S引用。

```
audio: audio {
        compatible = "vendor,aiao";
        reg = <0x4 0x00160000 0x0 0x7000>, /* asp_wrap_reg */
              <0x4 0x00130000 0x0 0x10000>, /* adss_sub_subctrl_reg */
              <0x4 0x0314e000 0x0 0x400>, /* asp_cfg_reg */
              <0x4 0x00120000 0x0 0x4000>, /* adss_disp_reg */
              <0x4 0x0314b000 0x0 0x1000>, /* dma */
              <0x4 0x03152000 0x0 0x290>; /* i2s */
        reg-names = "asp-wrap","adss-sub","asp-cfg","adss-disp","dma","i2s";
        // 中断
        dmac = <0>;
        irq_name = "asp_dmac_irq";
        interrupt-names = "asp_dmac_irq";
        interrupts = <590 0x4>;
        interrupt-parent = <&mbigen_audio>;
        status = "ok";
        pinctrl-names = "default";
        pinctrl-0 = <&i2s0_mclk_pmx_func &i2s0_bclk_tx_pmx_func
                     &i2s0_ws_tx_pmx_func &i2s0_sd_tx_pmx_func
                     &i2s0_sd_rx_pmx_func
                     &i2s1_mclk_pmx_func &i2s1_bclk_tx_pmx_func
                     &i2s1_ws_tx_pmx_func &i2s1_sd_tx_pmx_func
                     &i2s1_sd_rx_pmx_func>;
};

audio-sys {
        compatible = "hisilicon,hisi-audio-sys";
};
```

4. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤9** 修改pinctrl.dtsi文件，增加pin复用关系。

1. 执行如下命令，修改pinctrl.dtsi文件。

   **vi hi1910B-pinctrl-M100-B51.dtsi**

2. 将“pmx_media: pinmux@400140000”修改为如下代码内容。

```
pmx_media: pinmux@400140000 {
            compatible = "pinctrl-single", "pinctrl-single1";
```

 版权所有 © 华为技术有限公司 

```
reg = <0x4 0x140000 0x0 0x60>;
#gpio-range-cells = <3>;
pinctrl-single,register-width = <32>;
pinctrl-single,function-mask = <7>;

pinctrl-names = "default", "default", "default";

i2s0_mclk_pmx_func: i2s0_mclk_pmx_func {
        pinctrl-single,pins = <0x30     0x0>;
};

i2s0_mclk_pmx_idle: i2s0_mclk_pmx_idle {
        pinctrl-single,pins = <0x30     0x0>;
};


i2s0_bclk_tx_pmx_func: i2s0_bclk_tx_pmx_func {
        pinctrl-single,pins = <0x34     0x0>;
};

i2s0_bclk_tx_pmx_idle: i2s0_bclk_tx_pmx_idle {
        pinctrl-single,pins = <0x34     0x0>;
};

i2s0_ws_tx_pmx_func: i2s0_ws_tx_pmx_func {
        pinctrl-single,pins = <0x38     0x0>;
};

i2s0_ws_tx_pmx_idle: i2s0_ws_tx_pmx_idle {
        pinctrl-single,pins = <0x38     0x0>;
};

i2s0_sd_tx_pmx_func: i2s0_sd_tx_pmx_func {
        pinctrl-single,pins = <0x3c     0x0>;
};

i2s0_sd_tx_pmx_idle: i2s0_sd_tx_pmx_idle {
        pinctrl-single,pins = <0x3c     0x0>;
};

i2s0_sd_rx_pmx_func: i2s0_sd_rx_pmx_func {
        pinctrl-single,pins = <0x40     0x0>;
};

i2s0_sd_rx_pmx_idle: i2s0_sd_rx_pmx_idle {
        pinctrl-single,pins = <0x40     0x0>;
};

i2s1_mclk_pmx_func: i2s1_mclk_pmx_func {
        pinctrl-single,pins = <0x44     0x0>;
};

i2s1_mclk_pmx_idle: i2s1_mclk_pmx_idle {
        pinctrl-single,pins = <0x44     0x0>;
};


i2s1_bclk_tx_pmx_func: i2s1_bclk_tx_pmx_func {
        pinctrl-single,pins = <0x48     0x0>;
};

i2s1_bclk_tx_pmx_idle: i2s1_bclk_tx_pmx_idle {
        pinctrl-single,pins = <0x48     0x0>;
};

i2s1_ws_tx_pmx_func: i2s1_ws_tx_pmx_func {
        pinctrl-single,pins = <0x4c     0x0>;
};
```


版权所有 © 华为技术有限公司

```
            i2s1_ws_tx_pmx_idle: i2s1_ws_tx_pmx_idle {
                pinctrl-single,pins = <0x4c    0x0>;
            };

            i2s1_sd_tx_pmx_func: i2s1_sd_tx_pmx_func {
                pinctrl-single,pins = <0x50    0x0>;
            };

            i2s1_sd_tx_pmx_idle: i2s1_sd_tx_pmx_idle {
                pinctrl-single,pins = <0x50    0x0>;
            };

            i2s1_sd_rx_pmx_func: i2s1_sd_rx_pmx_func {
                pinctrl-single,pins = <0x54    0x0>;
            };

            i2s1_sd_rx_pmx_idle: i2s1_sd_rx_pmx_idle {
                pinctrl-single,pins = <0x54    0x0>;
            };
    };
```

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** 将新增的整机dts文件添加到编译路径中。

1. 执行如下命令，进入“dtbtool”目录。

   **cd** */opt*/**Ascend310B-source/dtb/dtbtool/**

2. 执行如下命令，修改CMakelists文件，将所增加的dts文件添加到编译路径中，具体修改如下加粗字体所示。

   **vim CMakeLists.txt**

```
set(DTS_BASE_DIR ${TOP_DIR}/tools/customize/dts)
if(PRODUCT STREQUAL "ascend310Besl" OR
   PRODUCT STREQUAL "ascend310B" OR
   PRODUCT STREQUAL "ascend310Bemu" OR
   PRODUCT STREQUAL "ascend310Brc" OR
   PRODUCT STREQUAL "ascend310Brcesl" OR
   PRODUCT STREQUAL "ascend310Brcemu")
    # 当前driver跟esl用相同的dts，后期需要更新到单独的目录
    set(DTS_SRC_DIR ${DTS_BASE_DIR}/hi1910b/hi1910BL)
    set(DTS_FILE_LIST hi1910B-default.dts unreleased/hi1911-esl/hi1911-esl-b600.dts unreleased/
hi1911-esl/hi1911-esl-rc-b600.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0.dts unreleased/
hi1911-fpga/hi1911-fpga-b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-b600_b0_b2.dts
unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0.dts unreleased/hi1911-fpga/hi1911-fpga-rc-
b600_b0_b1.dts unreleased/hi1911-fpga/hi1911-fpga-rc-b600_b0_b2.dts hi1910B-evb-900.dts hi1910B-
evb-901.dts hi1910B-evb-902.dts hi1910B-evb-903.dts hi1910B-evb-905.dts hi1910B-asic-M150-
B50.dts hi1910B-asic-M100-B51.dts hi1910B-asic-M150-B51.dts hi1910B-asic-M160-B51.dts hi1910B-
asic-M100-B00.dts hi1910B-asic-M101-B00.dts hi1910B-asic-M150-B00.dts hi1910B-asic-M151-
B00.dts  hi1910B-asic-M150-B33.dts)
```

   **说明**

   新增加的dts只能在加粗字体段落末尾添加。

3. 配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤11** 编译DTB文件。

1. 执行如下命令，返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

2. 执行如下命令，编译DTB文件。

   **bash build.sh dtb**

   出现如下回显，且生成“dt.img”文件表示编译内核userBaseConfig文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

 版权所有 © 华为技术有限公司 

**说明**

编译后的“dt.img”文件会自动存放于“Ascend310B-source/output”目录下。

**步骤12** 升级“dt.img”文件。具体请参见**生效DTB文件**。

**----结束**

 版权所有 © 华为技术有限公司 

# 8 二次开发场景

8.1 安装工具链
8.2 启动流程介绍
8.3 源码编译
8.4 Hboot2源码编译
8.5 驱动自动加载配置
8.6 重构驱动包
8.7 编辑用户自定义脚本
8.8 节能配置

## 8.1 安装工具链

本章节介绍用户如何安装相应的交叉编译工具链到编译程序所在的Linux系统。

**环境准备**：一台带网络端口且操作系统为x86 Ubuntu 22.04的Linux服务器。

### 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 执行如下命令，创建目录，例如“/opt/compiler”。

**mkdir** */opt/compiler*

**步骤4** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的交叉编译工具链包“toolchain.tar.gz”上传至“/opt/compiler”目录。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤5** 执行如下命令，进入到“/opt/compiler”目录。

**cd** */opt/compiler*

 版权所有 © 华为技术有限公司 

**步骤6** 执行如下命令，解压交叉编译工具。

**tar -xvf toolchain.tar.gz -C ./**

**步骤7** 执行如下命令，在“/etc/profile”配置文件中增加交叉编译工具链路径。

**echo "export PATH=*/opt/compiler/*toolchain/bin:\$PATH: " >> /etc/profile**

**步骤8** 执行如下命令，使环境变量生效。

**source /etc/profile**

**步骤9** 执行如下命令，查看交叉编译工具链版本。

**aarch64-target-linux-gnu-gcc -v**

显示有版本信息，则表明安装工具链成功。具体版本号请以实际情况为准。

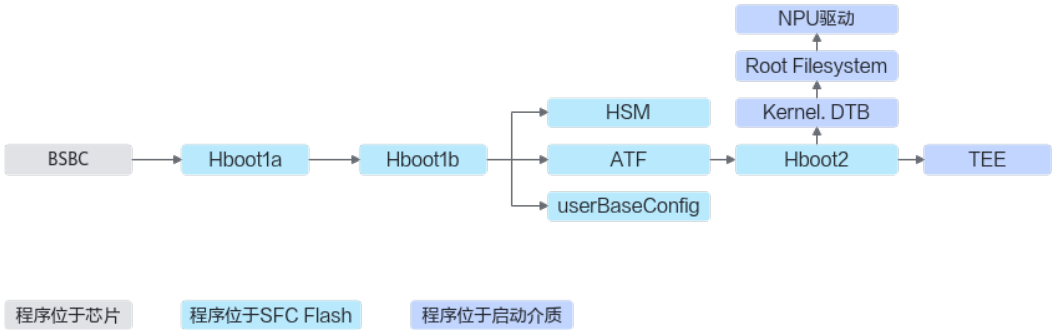
```
gcc version 7.3.0 (Do-Compiler V100R003C01B001)
```

**----结束**

# 8.2 启动流程介绍

## 8.2.1 概述

Atlas 200I A2 加速模块的启动流程如下。




## 8.2.2 Hboot2

Hboot2程序出厂预置在Atlas 200I A2 加速模块的SFC Flash上，主要功能为根据BOOTSEL真值表选择启动介质以及从启动介质中加载Kernel、DTB等程序。BOOTSEL真值表的详细信息请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“启动接口设计”章节。

## 8.2.3 用户弹性配置介绍

userBaseConfig为用户弹性配置，用户可通过userBaseConfig灵活配置SerDes。详细操作请参见**8.3.2 编译并生效userBaseConfig文件**。

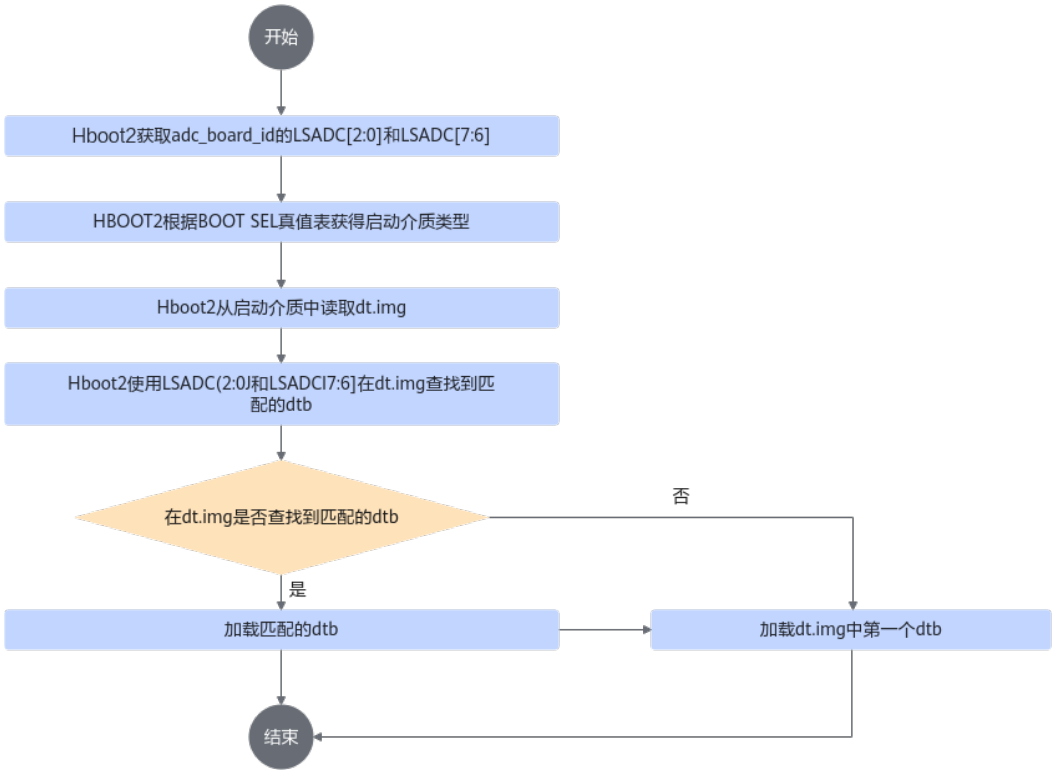
## 8.2.4 内核介绍

Atlas 200I A2 加速模块使用Linux 5.10内核。

 版权所有 © 华为技术有限公司 

## 8.2.5 DTB 介绍

Atlas 200I A2 加速模块提供dt.img镜像集成不同的DTB，Hboot2根据Atlas 200I A2 加速模块的adc_board_id和底板的adc_board_id在dt.img中选择匹配的DTB加载，当无匹配的DTB时会加载dt.img中第一个DTB。




用户可基于dts自定义不同的DTB使能整机丰富的硬件接口，dts源码提供了可供整机参考的dts，详细介绍如下。

| dts Sample | dts域段作用介绍 | 整机示例 |
| --- | --- | --- |
| hi1910B-asic-M150-B50.dts | M150：表示算力为20T的Atlas 200I A2 加速模块。<br>B50：表示adc_board_id的LSADC[7:6]为50，LSADC[7:6]用于区分不同的底板，详细请参考《**Atlas 200I A2 加速模块 硬件开发指南**》。<br>dts名字和dts内容对应的域段：hisi,boardid = <0x0 0x5 0x0 0x1 0x5 0x0>。 | Atlas 500 A2智能小站 |


版权所有 © 华为技术有限公司

| dts Sample | dts域段作用介绍 | 整机示例 |
| --- | --- | --- |
| hi1910B-asic-M100-B51.dts | M100：表示算力为8T的Atlas 200I A2 加速模块。<br>B51：表示adc_board_id的LSADC[7:6]为51，LSADC[7:6]用于区分不同的底板，详细请参考《**Atlas 200I A2 加速模块 硬件开发指南**》。<br>dts名字和dts内容对应的域段：hisi，boardid = <0x0 0x5 0x1 0x1 0x0 0x0>。 | Atlas 200I DK A2 开发者套件 |
| hi1910B-asic-M150-B51.dts | M150：表示算力为20T的Atlas 200I A2 加速模块。<br>B51：表示adc_board_id的LSADC[7:6]为51，LSADC[7:6]用于区分不同的底板，详细请参考《**Atlas 200I A2 加速模块 硬件开发指南**》。<br>dts名字和dts内容对应的域段：hisi，boardid = <0x0 0x5 0x1 0x1 0x0 0x0>。 | 20T的Atlas 200I A2 加速模块 +Atlas 200I DK A2 开发者套件底板 |
| hi1910B-asic-default.dts | Hboot2根据adc_board_id的LSADC[2:0]和LSADC[7:6]在dt.img中无法找到匹配的DTB时，会默认加载DTB中第一位置的hi1910B-asic-default.dtb进行启动，默认DTB中包含CPU、保留内存、uart0、eth0、eth1配置。 | NA |

**说明**

hi1910B-asic-M150-B50.dts、hi1910B-asic-M100-B51.dts、hi1910B-asic-M150-B51.dts和hi1910B-asic-default.dts在源码包Ascend310B-source tar.gz中，详细获取方式请参见**4.1 下载软件包**。

用户有以下两种方式自定义dts。

- 基于dts Sample示例定义整机的dts，比如整机底板的LSADC[7:6]为30，搭配20T算力的Atlas 200I A2 加速模块时，dts可以为hi1910B-asic-M150-B30.dts，同时修改dts中boardid字段为如下。

```
{
compatible = "hisilicon,hi1910B-evb", "hisilicon,ascend610";
hisi,boardid = <0x0 0x3 0x0 0x1 0x5 0x0>;
#address-cells = <0x2>;
#size-cells = <0x2>;
interrupt-parent = <0x1>;
model = "Hisilicon PhosphorHi1910B evb";
```

- 基于hi1910B-asic-default.dts定义整机的dts，适配整机的硬件接口。


版权所有 © 华为技术有限公司

## 8.2.6 文件系统介绍

烧录工具提供单Root Filesystem和冗余Root Filesystem的支持，同时提供了boot_tool工具查询启动的相关信息，例如boot_tool get image_info查询Root Filesystem所在的分区信息。烧录工具的详细介绍请参见13.2 烧写工具介绍。

**图 8-1** 单 Root Filesystem

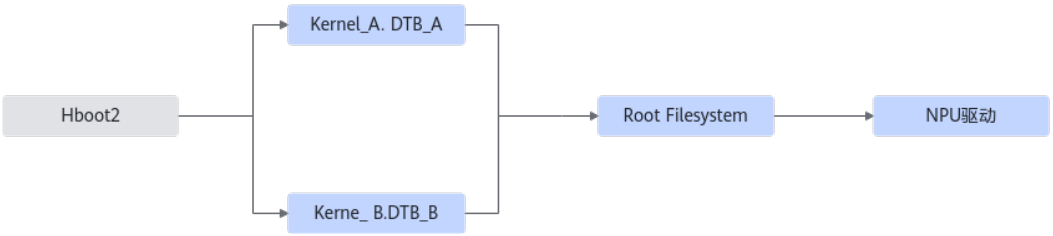
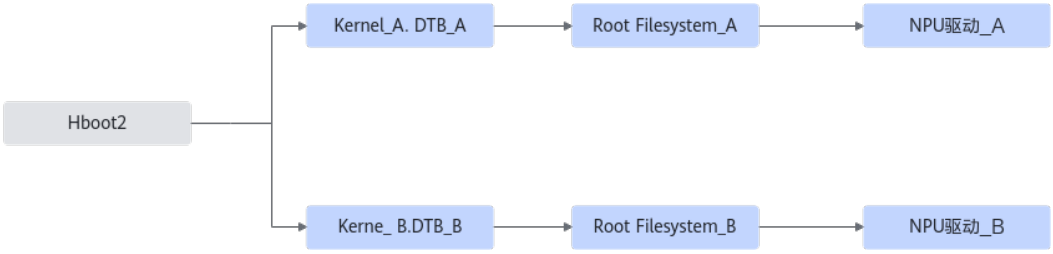



**图 8-2** 冗余 Root Filesystem




# 8.3 源码编译

## 8.3.1 编译内核

### 8.3.1.1 编译并生效内核 Image 文件（Euler）

本章节针对Atlas 200I A2 加速模块如何编译内核Image进行介绍。

Image固件在进行安装升级及安全启动时会进行文件头的校验，若使用未添加文件头的固件则会导致升级失败。用户可通过如下编译过程，自动生成带有文件头的Image文件。

#### 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake、rpm2cpio与交叉编译工具链。


版权所有 © 华为技术有限公司

**说明**

- 所有的依赖必须用root用户进行安装。
- 用户可以通过如下命令进行安装上述依赖。

  **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake rpm2cpio**
- 交叉编译工具链安装请参见**8.1 安装工具链**。

## 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** （可选）执行如下命令，提高系统内存访问权限，可通过devmem工具实现物理地址的读写。

**sed -i '/CONFIG_STRICT_DEVMEM=y/d; /CONFIG_IO_STRICT_DEVMEM=y/d' kernel/kernel/kernel/kbuild/defconfigs/arch/arm64/configs/ascend310B_defconfig**

**说明**

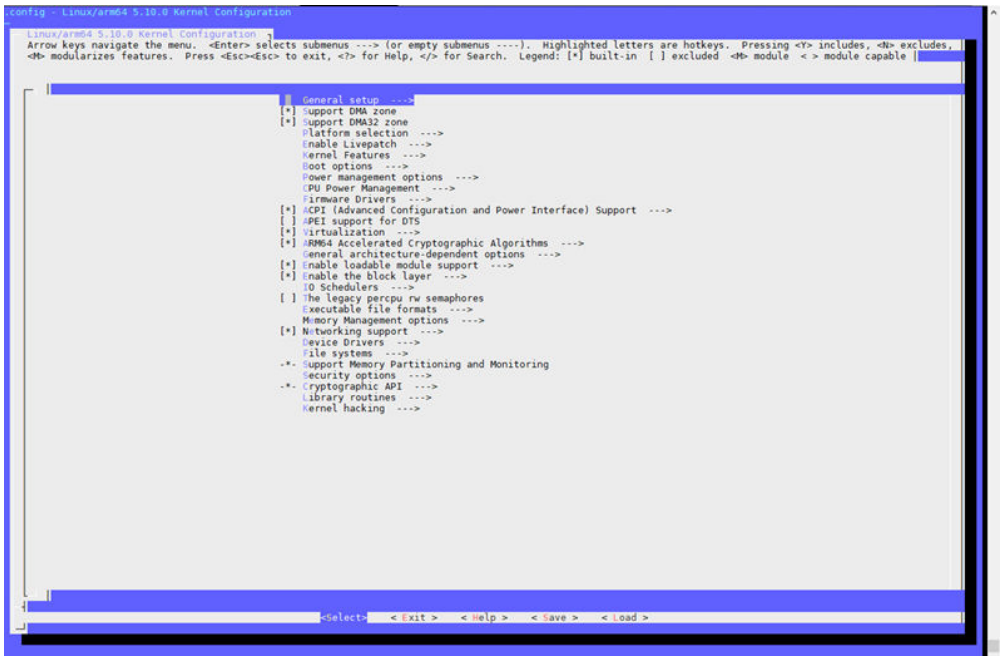
该操作会提高系统内存访问权限，若无特殊调测需要建议关闭。

**步骤8** 执行如下命令，编译OS内核。

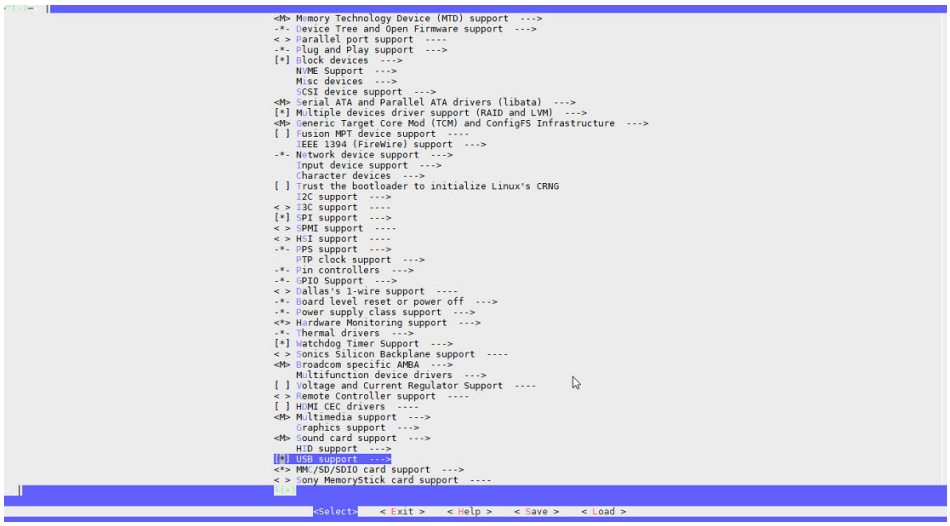
**bash build.sh kernel**

执行过程会弹出内核配置选项图形界面。

 版权所有 © 华为技术有限公司 




- 如需修改，参考如下步骤操作。
  a. 选择“Device Drivers”，并按“Enter”键。
  b. 用户可根据实际需求进行配置。
     如需使能USB功能，键盘空格键控制选中“USB support”，星号表示已选中。




  c. 选择Save，并单击OK，一直选择Exit直至退出图形界面后开始编译内核。
- 如不修改直接选择Exit退出。

出现如下回显，表示编译内核Image文件成功。

```
generate /opt/Ascend310B-source/output/Image success!
sign /opt/Ascend310B-source/output/Image success!
```

**说明**

编译后的Image文件会自动存放于“Ascend310B-source/output”目录下。

**----结束**

 版权所有 © 华为技术有限公司 

## 生效 Image 文件

**步骤1** 登录Atlas 200I A2 加速模块系统环境。

**步骤2** 使用“WinSCP”，将编译后的“Image”文件上传至任意目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤3** 执行如下命令，进入“opt”目录。

**cd** */opt*

**步骤4** 执行如下命令，升级Image文件。

**dd if=Image of=/dev/mmcblk1 count=61440 seek=32768 bs=512**

```
-sh-5.1# dd if=Image of=/dev/mmcblk1 count=61440 seek=32768 bs=512
56849+1 records in
56849+1 records out
29106944 bytes (29 MB, 28 MiB) copied, 2.36973 s, 12.3 MB/s
-sh-5.1#
```

**步骤5** 执行如下命令，重启生效。

**reboot**

**步骤6** 用户可根据实际配置的功能进行验证。如使能USB功能后，验证USB功能是否正常。

**----结束**

## 8.3.1.2 编译并生效内核 Image 文件（openEuler-SP1）

本章节针对Atlas 200I A2 加速模块如何编译内核Image进行介绍。

Image固件在进行安装升级及安全启动时会进行文件头的校验，若使用未添加文件头的固件则会导致升级失败。用户可通过如下编译过程，自动生成带有文件头的Image文件。

Atlas 200I A2 加速模块支持Euler内核和openEuler内核，发布的源码包**Ascend310B-source tar.gz**中自带Euler内核源码，不包含openEuler内核源码。需基于openEuler开源社区源码编译openEuler 22.03 LTS SP1内核并进行驱动包的重构，具体操作可参见**8.6 重构驱动包**。

## 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake、rpm2cpio与交叉编译工具链。

**说明**

- 所有的依赖必须用root用户进行安装。
- 用户可以通过如下命令进行安装上述依赖。

  **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake rpm2cpio**
- 交叉编译工具链安装请参见**8.1 安装工具链**。

 版权所有 © 华为技术有限公司 

## 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 基于开源社区下载openEuler 22.03 LTS SP1内核源码，并上传至Linux服务器。

1. 访问**链接**下载“openEuler-22.03-LTS-SP1-everything-aarch64-dvd.iso”镜像文件。
2. 访问镜像文件，进入“Packages”目录，获取rpm源码包“kernel-rt-source-5.10.0-136.12.0.rt62.59.oe2203sp1.aarch64.rpm”。
3. 使用WinSCP工具或其他工具将rpm源码包“kernel-rt-source-5.10.0-136.12.0.rt62.59.oe2203sp1.aarch64.rpm”上传到“Ascend310B-source.tar.gz”源码包解压子目录下，例如“/opt/Ascend310B-source”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤8** （可选）执行如下两条命令，提高系统内存访问权限，可通过devmem工具实现物理地址的读写。

**sed -i 's/6981/6979/g' kernel/kernel/kernel/rtkernel_patch/0042-ascend310B_defconfig.patch**

**sed -i '/+CONFIG_STRICT_DEVMEM=y/d; /+CONFIG_IO_STRICT_DEVMEM=y/d' kernel/kernel/kernel/rtkernel_patch/0042-ascend310B_defconfig.patch**

**说明**

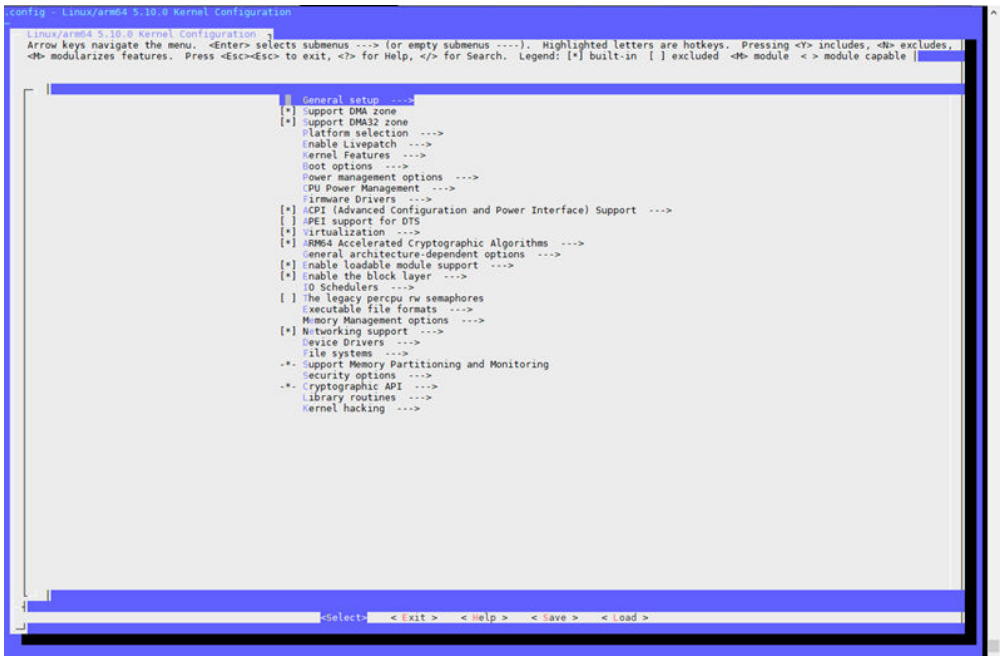
该操作会提高系统内存访问权限，若无特殊调测需要建议关闭。

**步骤9** 执行如下命令，编译OS内核。

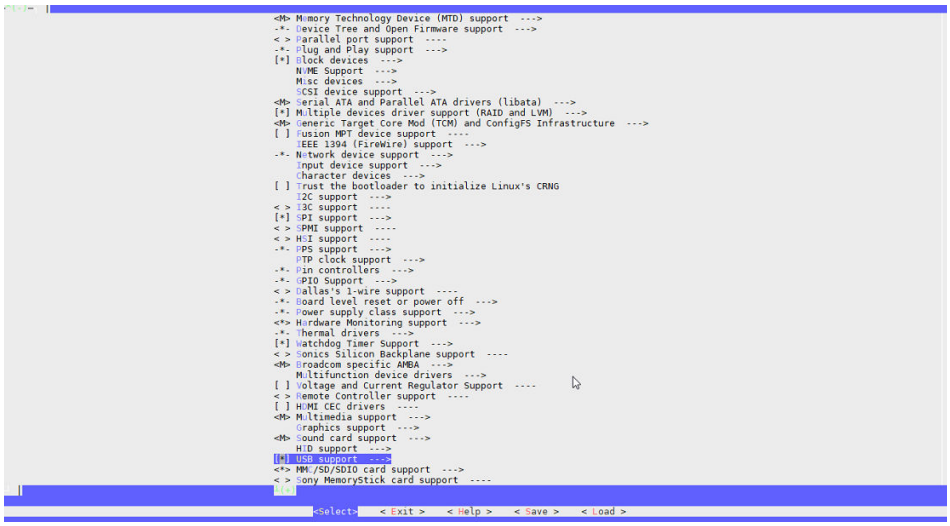
**bash build.sh rtKernel**

执行过程会弹出内核配置选项图形界面。

 版权所有 © 华为技术有限公司 




- 如需修改，参考如下步骤操作。
  a. 选择“Device Drivers”，并按“Enter”键。
  b. 用户可根据实际需求进行配置。
     如需使能USB功能，键盘空格键控制选中“USB support”，星号表示已选中。




  c. 选择Save，并单击OK，一直选择Exit直至退出图形界面后开始编译内核。
- 如不修改直接选择Exit退出。

出现如下回显，表示编译内核Image文件成功。

```
generate /opt/Ascend310B-source/output/Image success!
sign /opt/Ascend310B-source/output/Image success!
```

**说明**

编译后的Image文件会自动存放于“Ascend310B-source/output”目录下。

**----结束**

 版权所有 © 华为技术有限公司 

## 生效 Image 文件

**步骤1** 登录Atlas 200I A2 加速模块系统环境。

**步骤2** 使用“WinSCP”，将编译后的“Image”文件上传至任意目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤3** 执行如下命令，进入“opt”目录。

**cd** */opt*

**步骤4** 执行如下命令，升级Image文件。

**dd if=Image of=/dev/mmcblk1 count=61440 seek=32768 bs=512**

```
-sh-5.1# dd if=Image of=/dev/mmcblk1 count=61440 seek=32768 bs=512
56849+1 records in
56849+1 records out
29106944 bytes (29 MB, 28 MiB) copied, 2.36973 s, 12.3 MB/s
-sh-5.1#
```

**步骤5** 执行如下命令，重启生效。

**reboot**

**步骤6** 用户可根据实际配置的功能进行验证。如使能USB功能后，验证USB功能是否正常。

**----结束**

## 8.3.1.3 编译并生效内核 DTB 文件

## 操作场景

当用户需要定制自定义设备树文件时，如开启UART0串口功能，可以通过修改源码包中对应形态的dts文件以重新编译内核DTB文件，并重构驱动包进行安装。

**须知**

若Atlas 200I A2 加速模块环境已存在NPU驱动，只需升级dt.img，获取制作dt.img使用的Ascend-hdk-310b-sdk-soc_*<version>*.zip时，软件包的版本需与当前Atlas 200I A2 加速模块的驱动版本号保持一致。否则，版本号不一致，升级dt.img后Atlas 200I A2 加速模块环境无法登录，只能通过重新烧写系统镜像进行恢复。

## 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake、rpm2cpio与交叉编译工具链。


版权所有 © 华为技术有限公司

**说明**

- 所有的依赖必须用root用户进行安装。
- 用户可以通过如下命令进行安装上述依赖。

  **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake rpm2cpio**
- 交叉编译工具链安装请参见**8.1 安装工具链**。

## 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 修改对应设备的dts文件。

- 如果需要使能DTB相关功能，可参考此步骤，下文以Atlas 200I DK A2 开发者套件新增调试串口功能为例进行说明。
  a. 执行如下命令，进入“dtb”目录。

     **cd dtb**

  b. 执行如下命令，打开Atlas 200I DK A2 开发者套件boardid为51100的dts文件。

     **vim** *dts/hi1910b/hi1910BL/hi1910B-asic-M100-B51.dts*

  c. 可通过修改bootargs字段中的加粗字体，使能配置。

     此处以使能UART0串口作为控制台设备为例进行配置（Atlas 200I DK A2 开发者套件默认已使能UART0串口配置）。

     ```
     chosen {
                 bootargs = "console=ttyAMA0,115200 log_redirect=0x1fc000@0x22741000
     root=/dev/mmcblk1p1 rw rdinit=/sbin/init rodata=off syslog
     earlycon=pl011,mmio32,0xC4010000 loglevel=5 initrd=0x2BA00000,300M rootwait mpam=of
     enable_ascend_share_pool ascend_enable_all enable_sp_multi_group_mode init_all_gicr
     enable_fpga no_console_suspend use_kallsyms_lookup_symbol selinux=0
     systemd.unified_cgroup_hierarchy=0 fbcon=map:6";
         };
     ```

  d. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

     **:wq!**

  e. 返回“Ascend310B-source”目录。

 版权所有 © 华为技术有限公司 

**cd** */opt*/**Ascend310B-source**

**步骤8** 执行如下命令，编译DTB文件。

**bash build.sh dtb**

出现如下回显，且生成“dt.img”文件表示编译内核DTB文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

**说明**

编译后的dt.img文件会自动存放于“Ascend310B-source/output”目录下。

**----结束**

## 生效 DTB 文件

**步骤1** 登录Atlas 200I A2 加速模块系统环境，具体操作请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤2** 使用“WinSCP”，将编译后的“dt.img”文件上传至任意目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤3** 执行如下命令，进入“opt”目录。

**cd** */opt*

**步骤4** 执行如下命令，查询启动方式。

boot_tool get boot_sel

- 若回显如下，表示启动方式从eMMC启动。

```
boot from eMMC
```

- 若回显如下，表示启动方式从SD卡启动。

```
boot from SD
```

**步骤5** 升级dt.img文件。

- eMMC启动。
  - 升级主区DTB，执行如下命令。

    **dd if=dt.img of=/dev/mmcblk0 count=4096 seek=114688 bs=512**

    ```
    852+1 records in
    852+1 records out
    436480 bytes (436 kB, 426 KiB) copied, 0.113982 s, 3.8 MB/s
    ```

  - 升级备区DTB，执行如下命令。

    **dd if=dt.img of=/dev/mmcblk0 count=4096 seek=376832 bs=512**

    ```
    852+1 records in
    852+1 records out
    436480 bytes (436 kB, 426 KiB) copied, 0.112121 s, 3.9 MB/s
    ```

- M.2启动。
  - 升级主区DTB，执行如下命令。

    **dd if=dt.img of=/dev/sda count=4096 seek=114688 bs=512**

    ```
    1504+1 records in
    1504+1 records out
    770304 bytes (770 kB, 752 KiB) copied, 0.0117018 s, 65.8 MB/s
    ```

 版权所有 © 华为技术有限公司 

- 升级备区DTB，执行如下命令。

  **dd if=dt.img of=/dev/sda count=4096 seek=376832 bs=512**

  ```
  1504+1 records in
  1504+1 records out
  770304 bytes (770 kB, 752 KiB) copied, 0.0247949 s, 31.1 MB/s
  ```

- SD卡启动。
  - 升级主区DTB，执行如下命令。

    **dd if=dt.img of=/dev/mmcblk1 count=4096 seek=114688 bs=512**

    ```
    852+1 records in
    852+1 records out
    436480 bytes (436 kB, 426 KiB) copied, 0.113982 s, 3.8 MB/s
    ```

  - 升级备区DTB，执行如下命令。

    **dd if=dt.img of=/dev/mmcblk1 count=4096 seek=376832 bs=512**

    ```
    852+1 records in
    852+1 records out
    436480 bytes (436 kB, 426 KiB) copied, 0.112121 s, 3.9 MB/s
    ```

**说明**

建议用户在调测阶段仅升级主区与备区中的其中一个dt.img文件。

**步骤6** 执行如下命令，重启生效。

**reboot**

建议查看串口日志确认升级重启进展。

**步骤7** 用户可根据实际配置的功能进行验证。如使能UART0串口配置后，验证UART0串口功能是否正常。

**----结束**

# 8.3.2 编译并生效 userBaseConfig 文件

## 操作场景

- 当用户需要配置serdes信息时，比如serdes_type，可以通过修改user_base_config.xml从而修改对应的serdes配置。
- 当用户需要配置过温过流保护功能时，可以通过修改user_base_config.xml从而修改对应的过温过流功能配置。

**须知**

如果关闭过流过温功能，可能会导致设备损坏，请谨慎配置。

## 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake、rpm2cpio与交叉编译工具链。

 版权所有 © 华为技术有限公司 

**说明**

- 所有的依赖必须用root用户进行安装。
- 用户可以通过如下命令进行安装上述依赖。

  **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake rpm2cpio**
- 交叉编译工具链安装请参见**8.1 安装工具链**。

## 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 修改userBaseConfig配置文件。

**说明**

如果需要使能userBaseConfig相关功能，请执行此步骤。

1. 执行如下命令，进入配置文件所在目录。

   **cd config/adaptive/adaptive_config/**
2. 打开user_base_config.xml文件。

   **vim user_base_config.xml**
3. 配置serdes信息，修改board_id字段为对应的board_id，chip0_strategy为使用的策略索引。

```
<item name="version"><!--版本号-->
    <param version="20000"/>
</item>

<item name="product_strategy_info"><!--产品板策略ID-->
    <!--30150 对应十六进制75c6-->
    <param board_id ="75c6" phy_num = "1" chip0_strategy="1" max_power="0"  equal="0"/>
</item>

<item name="evb_strategy_info">
</item>
```

   详细的配置说明，请参见**A.2 自定义配置user_base_config.xml文件参数说明**。
4. 设置serdes复用关系。详细的配置说明，请参见**A.2 自定义配置user_base_config.xml文件参数说明**。

 版权所有 © 华为技术有限公司 

**须知**

弹性配置中的serdes复用关系参数必须与实际硬件单板保持一致，Serdes错误配置可能导致启动挂死，功能不可用。

user_base_config.xml文件示例如下回显加粗字体。

```
    <item name="strategy_info_array"><!--启动策略矩阵-->
        <subitem subname="sub_strategy0">
            <module class="serdes">
                <submodule subclass="serdes_info">
                    <param lan_index="0" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx
= "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0" port_index = "0"/>
                    <param lan_index="1" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx
= "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0" port_index = "0"/>
                    <param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx
= "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0" port_index = "0"/>
                    <param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx =
"0" lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
                    <param lan_index="4" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx =
"0" lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN3" port_index = "0"/>
                    <param lan_index="5" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx
= "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
                    <param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx
= "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
                    <param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx
= "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
                </submodule>
            </module>
            <module class="recovery">
                <submodule subclass="recovery_info">
                    <param recovery_enable="0" gpio_index="1"/>
                </submodule>
            </module>
        </subitem>
        <subitem subname="sub_strategy1">
            <module class="serdes">
                <submodule subclass="serdes_info">
                    <param lan_index="0" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0"
polarity_rx = "0" lan_order = "0" bandwidth="4"  align_mode="0" frequency="PCIE_GEN3"
port_index = "0"/>
                    <param lan_index="1" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0"
polarity_rx = "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN3"
port_index = "0"/>
                    <param lan_index="2" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0"
polarity_rx = "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN3"
port_index = "0"/>
                    <param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0"
polarity_rx = "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="PCIE_GEN3"
port_index = "0"/>
                    <param lan_index="4" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0"
polarity_rx = "0" lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN3"
port_index = "0"/>
                    <param lan_index="5" serdes_type="USB"  ssc_enable="0" polarity_tx = "0"
polarity_rx = "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0"
port_index = "0"/>
                    <param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0"
polarity_rx = "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0"
port_index = "0"/>
                    <param lan_index="7" serdes_type="USB"  ssc_enable="0" polarity_tx = "0"
polarity_rx = "0" lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0"
port_index = "0"/>
                </submodule>
            </module>
            <module class="recovery">
                <submodule subclass="recovery_info">
                    <param recovery_enable="0" gpio_index="4"/>
                </submodule>
```

 版权所有 © 华为技术有限公司 

```
        </module>
```

5. （可选）配置过温过流保护功能。若需查看详细的过温过流功能配置说明，请参见**A.2 自定义配置user_base_config.xml文件参数说明**。

**须知**

如果关闭过流过温功能，可能会导致设备损坏，请谨慎配置。

配置过温过流功能示例如下。

```
<!--LP低功耗配置，无特殊说明数字为10进制, feature/edp/tdp各节点均支持缺省，配置定义与sysbaeconfig相同-->
        <module class="lp">
            <!--特性开关-->
            <submodule subclass="feature">
                <!--功能开关: 16进制掩码定义, 每比特位0为关闭1为使能-->
                <!--  各项开关均为可选配置，默认不关闭，默认全1配置"-->
                <!--过流保护功能开关, bit0-过流硬件降频，bit1-过流告警, bit2-过流软件降频,以下为功能全使能 -->
                <param edp="0x7"/>
                <!--过温保护功能开关, bit0-超高温下电, bit1-高温告警, bit2-过温降频,以下为功能全使能 -->
                <param tdp="0x7"/>
            </submodule>
            <!--edp过流保护-->
            <submodule subclass="edp">
                <!--acg降频等级0(10%), 1(18%) 2(25%),其它不支持-->
                <param acg_drop="2"/>
                <!--mask(适用档位掩码, 16进制）-->
                <!--hardware(单位0.1A， ina226硬件过流阈值)-->
                <!--alarm  (单位0.1A， 软件监测过流告警阈值）-->
                <!--ave   (单位0.1A， 软件监测过流降频阈值)-->
                <!--示例：mask(适用0/1/2/3档) ina226阈值5A，软件告警3A, 降频5A-->
                <param mask="0xF" hardware="50" alarm="30" ave="50"/>
            </submodule>
            <!--tdp过温保护-->
            <submodule subclass="tdp">
                <!--ctrl_temp  (单位0.1℃， 软件监测过温降频阈值)-->
                <!--ultra_high  (单位0.1℃， tsensor超高温下电阈值，需大于60度（600），建议不超过105度)-->
                <!--alarm_high  (单位0.1℃， 软件监测高温告警阈值)-->
                <!--alarm_low  (单位0.1℃， 软件监测低温告警阈值（保留）)-->
                <!--示例：过温降频95, 告警92度， 超温下电105度, 低温告警-20度(保留）-->
                <param ctrl_temp="950" ultra_high="1050" alarm_high="920" alarm_low="-200"/>
            </submodule>
        </module>
```

6. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

7. 返回“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

**步骤8** 执行如下命令，编译userBaseConfig文件。

**bash build.sh usrBaseConfig**

出现如下回显，表示编译内核userBaseConfig文件成功。

```
generate /opt/Ascend310B-source/output/userBaseConfig.bin success!
sign /opt/Ascend310B-source/output/userBaseConfig.bin success!
```

 版权所有 © 华为技术有限公司 

**说明**

编译后的userBaseConfig.bin文件会自动存放于“Ascend310B-source/output”目录下。

**----结束**

## 生效 userBaseConfig 文件

**步骤1** 登录Atlas 200I A2 加速模块系统环境。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用“WinSCP”，将编译后的userBaseConfig.bin文件上传至任意目录下，例如“/run”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 升级userBaseConfig.bin文件。

**cd** */run*

**/var/davinci/driver/upgrade-tool --device_index -1 --component Usr_Base_Config --path userBaseConfig.bin**

出现如下回显，表示升级userBaseConfig.bin文件成功。

```
{"device": 0, "succeed"}
```

**步骤5** 若修改userBaseConfig文件时，涉及修改启动策略矩阵，则需配置dts文件，同步升级dt.img，详细操作请参见**7.1 SerDes（PCIe/SATA/USB/ETH）**。

**步骤6** 升级完成后重启生效。

**reboot**

**----结束**

# 8.3.3 编译驱动

### 8.3.3.1 编译默认驱动

## 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake、rpm2cpio与交叉编译工具链。

**说明**

- 所有的依赖必须用root用户进行安装。
- 用户可以通过如下命令进行安装上述依赖。

  **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake rpm2cpio**
- 交叉编译工具链安装请参见**8.1 安装工具链**。

 版权所有 © 华为技术有限公司 

## 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 执行如下命令，编译驱动。

**bash build.sh driver**

出现如下回显，表示编译驱动成功。

```
### build driver_device modules successfully ###
generate /opt/Ascend310B-sdk/Ascend310B-source/output/driver_modules success!
make driver success!
```

**说明**

- 编译后的驱动文件会自动存放于“Ascend310B-source/output/driver_modules”目录下。
- 用户请将编译后的驱动文件上传至自有的环境进行加载。

**----结束**

## 8.3.3.2 编译并生效新增用户驱动

## 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake、rpm2cpio与交叉编译工具链。

**说明**

- 所有的依赖必须用root用户进行安装。
- 用户可以通过如下命令进行安装上述依赖。

  **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake rpm2cpio**
- 交叉编译工具链安装请参见**8.1 安装工具链**。

 版权所有 © 华为技术有限公司 

## 编译驱动文件

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“**Ascend310B-source.tar.gz**”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“drivers”目录。

**cd Ascend310B-source/driver/drivers**

**步骤7** 执行如下命令，创建并进入“usr”目录。

**mkdir usr**

**cd usr**

**步骤8** 执行如下命令，新增hello.c。

**vim hello.c**

hello.c文件的配置参考如下所示。

```
#include <linux/module.h>
#include <linux/kernel.h>

int __init myhello_init(void)
{
        printk("==========user hello mod init=========\n");
        return 0;
}

void __exit myhello_exit(void)
{

        printk("===========user hello mod exit=========\n");
}

MODULE_LICENSE("GPL");
module_init(myhello_init);
module_exit(myhello_exit);
```

配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤9** 执行如下命令，新增Makefile。

**vim Makefile**

Makefile文件的配置参考如下所示。

```
ccflags-y += -Wall -Werror -Wtrampolines $(WDATE_TIME) -Wfloat-equal -Wvla -Wundef -funsigned-char -Wformat=2 -Wstack-usage=2048 -Wcast-align
ccflags-y += -Wextra -Wno-unused-parameter -Wno-sign-compare -Wno-missing-field-initializers
```

 版权所有 © 华为技术有限公司 

```
obj-m += myhello.o
myhello-objs := hello.o
```

配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤10** 执行如下命令，新增module.mk。

**vim module.mk**

module.mk文件的配置参考如下所示。

```
LOCAL_PATH := $(call my-dir)
include $(CLEAR_VARS)
LOCAL_MODULE := myhello
LOCAL_KO_SRC_FOLDER := $(LOCAL_PATH)
LOCAL_INSTALLED_KO_FILES := myhello.ko
include $(BUILD_DEVICE_KO)
```

配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤11** 执行如下命令，进入“product_modules”目录。

**cd /**opt**/Ascend310B-source/driver/build/product_modules**

**步骤12** 执行如下命令，修改device-ctrlcpu.mk文件。

**vim device-ctrlcpu.mk**

新增DRIVER_DEVICE_MODULES += myhello.ko

```
################################################################################
####                    official    release modules                    ########
################################################################################

#define driver released modules in DEVICE
# DRIVER_DEVICE_MODULES += drv_prof.ko drv_platform.ko drv_devdrv.ko drv_devm
# DRIVER_DEVICE_MODULES += tee_drv.ko
# DRIVER_DEVICE_MODULES += hnae3.ko hns3.ko hclgeplf.ko
# DRIVER_DEVICE_MODULES += trs_tsmng.ko ascend_queue.ko
DRIVER_DEVICE_MODULES += drv_prof.ko drv_platform.ko drv_soc_resmng.ko trs_nv
DRIVER_DEVICE_MODULES += ipc_drv.ko drv_icm.ko ascend_virtmng_dev.ko drv_mttc
DRIVER_DEVICE_MODULES += drv_i2c_uniform.ko drv_sys_ctrl_uniform.ko drv_usb_u
DRIVER_DEVICE_MODULES += drv_log.ko ascend_xsmem.ko drv_dp_proc_mng.ko mntn_b
DRIVER_DEVICE_MODULES += drv_ascend_ctl.ko drv_user_cfg.ko drv_dfm.ko drv_pki
DRIVER_DEVICE_MODULES += myhello.ko
#add ALL module variables
CUSTOM_DEVICE_MODULES_VARIABLES := \
  DRIVER_DEVICE_MODULES

~
```

配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤13** 执行如下命令，返回“Ascend310B-source”目录。

**cd** */opt*/**Ascend310B-source**

**步骤14** 执行如下命令，编译驱动模块。

**bash build.sh driver**

出现如下回显表示编译成功。

```
### build driver_device modules successfully ###
generate /opt/Ascend310B-sdk/Ascend310B-source/output/driver_modules success!
make driver success!
```

 版权所有 © 华为技术有限公司 

**说明**

编译后myhello.ko在“Ascend310B-source/output/driver_modules/”目录下。

**----结束**

### 生效驱动文件

**步骤1** 登录Atlas 200I A2 加速模块系统环境。

**步骤2** 使用“WinSCP”，将编译后的myhello.ko文件上传至任意目录下，例如“/run”。详细操作请参见14.3 使用WinSCP传输文件。

**步骤3** 执行如下命令，进入“run”目录，再生效myhello.ko文件。

**cd** */run*

**insmod myhello.ko**

**步骤4** 执行如下命令，查看驱动模块是否加载成功。

**lsmod |grep myhello**

出现以下回显表示加载成功。

```
myhello              16384  0
```

**----结束**

## 8.4 Hboot2 源码编译

## 8.4.1 编译 Hboot2

### 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake与交叉编译工具链。

**说明**

- 所有的依赖必须用root用户进行安装。
- 用户可以通过如下命令进行安装上述依赖。

  **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake**
- 交叉编译工具链安装请参见8.1 安装工具链。

### 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

 版权所有 © 华为技术有限公司 

**步骤3** 执行如下命令，安装编译所需依赖库。

**apt-get install uuid-dev**

**步骤4** 使用WinSCP工具或其他工具，将4.1 下载软件包获取的源码包“Ascend310B-hboot2-source.tar.gz”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见10.2-使用WinSCP传输文件。

**步骤5** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤6** 执行如下命令，解压源码包“Ascend310B-hboot2-source.tar.gz”。

**tar -xzvf Ascend310B-hboot2-source.tar.gz**

**步骤7** 执行如下命令，进入“Ascend310B-hboot2-source”目录。

**cd Ascend310B-hboot2-source**

**步骤8** 根据用户需求修改对应的源码文件，具体操作请参见8.4.3 Demo参考。

**步骤9** 执行如下命令，进入“Ascend310B-hboot2-source”目录。

**cd** */opt*/**Ascend310B-hboot2-source**

**步骤10** 执行如下命令，编译“AS310B_HBOOT2_UEFI.fd”文件。

**bash build.sh hboot2**

出现如下回显，且生成“AS310B_HBOOT2_UEFI.fd”文件表示编译“AS310B_HBOOT2_UEFI.fd”文件成功。

```
generate /opt/Ascend310B-hboot2-source/output/AS310B_HBOOT2_UEFI.fd success!
sign /opt/Ascend310B-hboot2-source/output/AS310B_HBOOT2_UEFI.fd success!
```

**说明**

编译后的“AS310B_HBOOT2_UEFI.fd”文件会自动存放于“Ascend310B-hboot2-source/output”目录下。

**----结束**

# 8.4.2 生效 Hboot2

## 前提条件

已参见13.3.4 efuse-tool工具介绍章节，取消Hboot2的安全启动校验。

**说明**

hboot2固件支持用户二次开发，用户升级自己编译的hboot2固件前，需要使用efuse-tool工具对hboot2域段进行烧写，取消hboot2的安全启动校验，否则会导致生效Hboot2失败。

## 操作步骤

**步骤1** 登录Atlas 200I A2 加速模块系统环境。

**步骤2** 使用“WinSCP”，将编译后的“AS310B_HBOOT2_UEFI.fd”文件上传至任意目录下，例如“/opt”。详细操作请参见14.3 使用WinSCP传输文件。

 版权所有 © 华为技术有限公司 

**步骤3** 执行如下命令，进入“opt”目录。

**cd** */opt*

**步骤4** 执行如下命令，升级“AS310B_HBOOT2_UEFI.fd”文件。

**/var/davinci/driver/upgrade-tool --device_index 0 --component hboot2 --path ./AS310B_HBOOT2_UEFI.fd**

出现如下回显，表示升级“AS310B_HBOOT2_UEFI.fd”文件成功。

```
{"device": 0, "succeed"}
```

**步骤5** 执行如下命令，重启生效。

**reboot**

**----结束**

## 8.4.3 Demo 参考

### Demo 功能

串口处系统启动日志中增加一行打印，打印内容如下。

```
[TestDxeEntry][13L] Test!!
```

### 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake与交叉编译工具链。

**说明**

- 所有的依赖必须用root用户进行安装。
- 用户可以通过如下命令进行安装上述依赖。

  **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake**
- 交叉编译工具链安装请参见**8.1 安装工具链**。

### 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 执行如下命令，安装编译所需依赖库。

**apt-get install uuid-dev**

**步骤4** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“Ascend310B-hboot2-source.tar.gz”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见**10.2-使用WinSCP传输文件**。

 版权所有 © 华为技术有限公司 

**步骤5** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤6** 执行如下命令，解压源码包“Ascend310B-hboot2-source.tar.gz”。

**tar -xzvf Ascend310B-hboot2-source.tar.gz**

**步骤7** 执行如下命令，进入“Ascend310B-hboot2-source/drivers/firmware/bios/HwPkg/UEFI/Common/Drivers”目录。

**cd** */opt*/**Ascend310B-hboot2-source/drivers/firmware/bios/HwPkg/UEFI/Common/Drivers**

**步骤8** 执行如下命令，创建“TestDxe”文件夹。

**mkdir TestDxe**

**步骤9** 执行如下命令，进入“TestDxe”目录。

**cd TestDxe**

**步骤10** 执行如下命令，新增“TestDxe.c”文件。

**vim TestDxe.c**

添加如下代码，配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

```
#include <Uefi.h>
#include <types.h>
#include <Library/DebugLib.h>
#include <Library/IoLib.h>
#include <Library/UefiBootServicesTableLib.h>
#include <Resource.h>


EFI_STATUS EFIAPI TestDxeEntry(EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE *SystemTable)
{
EFI_STATUS Status = EFI_SUCCESS;

PRINT_WARNING("[%a][%dL] Test!!\n", __FUNCTION__, __LINE__);

return Status;
}
```

**步骤11** 执行如下命令，新增“TestDxe.inf”文件。

**vim TestDxe.inf**

添加如下代码，配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

```
[Defines]
INF_VERSION                = 0x00010005
BASE_NAME                  = TestDxe
FILE_GUID                  = 4D035FE8-A0BB-42D4-B661-21771E6906A5
MODULE_TYPE                  = DXE_DRIVER
VERSION_STRING               = 1.0
ENTRY_POINT                = TestDxeEntry

[Sources.common]
TestDxe.c

[Packages]
ArmPlatformPkg/ArmPlatformPkg.dec
MdePkg/MdePkg.dec
```

 版权所有 © 华为技术有限公司 

```
ArmPkg/ArmPkg.dec
HwPkg/HwPkg.dec

[LibraryClasses]
UefiDriverEntryPoint
UefiBootServicesTableLib
DebugLib
IoLib
BlackBoxLib

[Guids]

[Depex]
TRUE
```

**步骤12** 执行如下命令，进入“/opt/Ascend310B-hboot2-source/drivers/firmware/bios/HwPkg/UEFI/Products/as310b/as310b”目录。

**cd *[/opt](/opt)*/Ascend310B-hboot2-source/drivers/firmware/bios/HwPkg/UEFI/Products/as310b/as310b**

**步骤13** 执行如下命令，修改“as310b.dsc”文件。

**vim as310b.dsc**

文件新增内容如下图红框，配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

```
610   # Fat filesystem and GPT and MBR
611   MdeModulePkg/Universal/Disk/DiskIoDxe/DiskIoDxe.inf
612   MdeModulePkg/Universal/Disk/PartitionDxe/PartitionDxe.inf
613   FatPkg/EnhancedFatDxe/Fat.inf
614   MdeModulePkg/Universal/Disk/UnicodeCollation/EnglishDxe/EnglishDxe.inf
615   HwPkg/UEFI/Common/Drivers/TestDxe/TestDxe.inf
616
617 !ifdef $(TEST)
618   TestPkg/Common/Ebl/EblAS310B.inf
619 !endif
620
621   # BDS stage
622   MdeModulePkg/Universal/DevicePathDxe/DevicePathDxe.inf
623   HwPkg/UEFI/Common/Bds/Bds.inf
624
```

**步骤14** 执行如下命令，修改“as310b.fdf”文件。

**vim as310b.fdf**

文件新增内容如下图红框，配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

```
181   # PCIE RC
182   INF HwPkg/UEFI/Products/as310b/as310b/Drivers/PcieRcInitDxe/PcieRcInitDxe.inf
183
184   # only Asic config, some FPGA combination may not support pcie
185   !ifdef $(ASIC)
186   #INF ArmPkg/Drivers/ArmPciCpuIo2Dxe/ArmPciCpuIo2Dxe.inf
187   #INF MdeModulePkg/Bus/Pci/PciHostBridgeDxe/PciHostBridgeDxe.inf
188   #INF MdeModulePkg/Bus/Pci/PciBusDxe/PciBusDxe.inf
189   #INF MdeModulePkg/Bus/Pci/NvmExpressDxe/NvmExpressDxe.inf
190   !endif
191   INF HwPkg/UEFI/Common/Drivers/TestDxe/TestDxe.inf
192
193   # network
194   !ifdef $(ASIC)
195   !ifdef $(TEST)
196   INF TestPkg/as310b/Snpas310bDxe/SnpPV600DxeMac0.inf
197   !endif
198   !endif
```

 版权所有 © 华为技术有限公司 

**步骤15** 执行如下命令，进入“opt/Ascend310B-hboot2-source”目录

**cd** */opt*/**Ascend310B-hboot2-source**

**步骤16** 执行如下命令，编译“AS310B_HBOOT2_UEFI.fd”文件。

**bash build.sh hboot2**

出现如下回显，且生成“AS310B_HBOOT2_UEFI.fd”文件表示编译“AS310B_HBOOT2_UEFI.fd”文件成功。

```
generate /opt/Ascend310B-hboot2-source/output/AS310B_HBOOT2_UEFI.fd success!
sign /opt/Ascend310B-hboot2-source/output/AS310B_HBOOT2_UEFI.fd success!
```

**说明**

编译后的“AS310B_HBOOT2_UEFI.fd”文件会自动存放于“Ascend310B-hboot2-source/output”目录下。

**步骤17** 准备串口线，连接PC和Atlas 200I A2 加速模块底板串口，具体请参见**14.2 使用PuTTY登录设备（串口方式）**，观测串口处系统启动日志。

**步骤18** 生效“AS310B_HBOOT2_UEFI.fd”文件，具体操作请参考**8.4.2 生效Hboot2**。

**步骤19** 查看串口的系统启动日志，出现以下回显表示功能修改已生效。

```
Pirnt userSerdesInfo:
Serdes info:
serdes: 0 0 0 0 0 0 4 0 3 0
serdes: 1 0 0 0 0 0 255 0 3 0
serdes: 2 0 0 0 0 0 255 0 3 0
serdes: 3 0 0 0 0 0 255 0 3 0
serdes: 4 0 0 0 0 0 1 0 3 0
serdes: 5 2 0 0 0 0 255 0 5 0
serdes: 6 2 0 0 0 0 255 0 5 0
serdes: 7 2 0 0 0 0 255 0 5 0
[TestDxeEntry][13L] Test!!
[0.820] [BOOT_TRACE] auto boot.
This is sd boot entry
emmcsd init
count(1)
end
```

**----结束**

# 8.5 驱动自动加载配置

## 使用场景

当用户需要配置内核通用驱动的自动加载或第三方驱动的自动加载时，可参见此章节修改相关配置文件。

 版权所有 © 华为技术有限公司 

## 前提条件

一台带网络端口且操作系统为Ubuntu 22.04的x86架构的Linux服务器。

## 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的驱动包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run”（若有软实时OS需求，请使用带-rt参数的驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run）上传至Linux系统root用户属组目录下，例如“/opt”，详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入“opt”目录。

**cd** */opt*

**步骤5** 将驱动包解压至“opt”的子目录“repack”下。

**bash Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run --noexec --extract=./repack**

若有软实时OS需求，该命令为**bash Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run --noexec --extract=./repack**。

**步骤6** 执行如下命令进入解压目录。

**cd repack**

**步骤7** 执行如下命令，解压modules.tar.gz。

**tar -zxvf modules.tar.gz**

**步骤8** 查看驱动包中是否包含通用驱动。

示例：执行以下命令，查看驱动包中是否包含通用驱动rfkill.ko。

**find ./ -name** *rfkill.ko*

若有以下回显，则表示有通用驱动rfkill.ko。

```
./5.10.0+/kernel/net/rfkill/rfkill.ko
```

若驱动包中包含该通用驱动，请参见**步骤9**进行配置。

若驱动包中不包含该通用驱动，请参见**步骤10**进行配置。

**步骤9** 若驱动包中包含通用驱动，新增自动加载配置，步骤如下。

1. 执行如下命令，进入“scripts”目录。
   **cd scripts**
2. 执行如下命令，打开11-atlas.conf配置文件。
   **vim 11-atlas.conf**
3. 新增需要自动加载的驱动名。
   示例：新增自动加载驱动rfkill.ko。

 版权所有 © 华为技术有限公司 

```
rfkill
```

4. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

5. 将修改后文件11-atlas.conf拷贝至**8.6 重构驱动包**中的重打包目录“scripts”下，再执行重打包流程。详细操作请参见**8.6 重构驱动包**。

**步骤10** 若驱动包中不包含通用驱动，用户需新增驱动，步骤如下。

1. 执行如下命令，进入“scripts”目录。

   **cd scripts**

2. 修改filelist.csv文件，使驱动升级有效。

   a. 执行以下命令，打开filelist.csv配置文件。

      **vim filelist.csv**

   b. 修改filelist.csv配置文件，新增驱动文件配置。

      示例：添加如下内容，增加myhello.ko的配置。

      ko,copy,**myhello.ko**,driver/**myhello.ko**,TRUE,**440**,root:root,all,NA,all,NA,FALSE,NA,driver,NA

      **说明**

      - 加粗字体**440**为新增驱动文件的权限，用户可根据自身需要进行修改。
      - 驱动文件myhello.ko的生成方式请参见**8.3.3.2 编译并生效新增用户驱动**。

   c. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

      **:wq!**

3. 修改minirc_sys_init_ext.sh脚本，使驱动自动加载。

   a. 执行以下命令，打开minirc_sys_init_ext.sh配置文件。

      **vim minirc_sys_init_ext.sh**

   b. 在load_device_modules()函数中新增需要自动加载的驱动配置信息。

      示例：新增如下加粗字体内容，配置myhello.ko驱动文件的自动加载。

      ```
      load_device_modules()
      {
          echo "load device ext modules"
          ko_path="${targetdir}"/driver
          ko_check ${ko_path}/myhello.ko
          echo "finish load device ext modules"
      }
      ```

   c. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

      **:wq!**

4. 将修改后的filelist.csv与minirc_sys_init_ext.sh拷贝至**8.6 重构驱动包**中重打包目录“scripts”下。
5. 将**步骤10.2.b**中生成的myhello.ko拷贝至**8.6 重构驱动包**中重打包目录“drivers”下。
6. 再执行重打包流程，详细操作请参见**8.6 重构驱动包**。

**----结束**

## 生效驱动文件

**步骤1** 升级驱动，具体请参见**9.3.1.1 升级当前区**。

 版权所有 © 华为技术有限公司 

**步骤2** 查看驱动模块是否加载成功。

- 若驱动包中包含通用驱动，执行如下命令。

  **lsmod |grep rfkill**

  出现以下类似回显表示加载成功。

  ```
  rfkill                 45056  1
  ```

- 若驱动包中不包含通用驱动，执行如下命令。

  **lsmod |grep myhello**

  出现以下类似回显表示加载成功。

  ```
  myhello                16384  0
  ```

**----结束**

# 8.6 重构驱动包

## 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake、rpm2cpio与交叉编译工具链。

  **说明**

  - 所有的依赖必须用root用户进行安装。
  - 用户可以通过如下命令进行安装上述依赖。

    **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake rpm2cpio**
  - 交叉编译工具链安装请参见**8.1 安装工具链**。

## 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至Linux系统root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

 版权所有 © 华为技术有限公司 

**步骤7** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的驱动包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run”（若有软实时OS需求，使用驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run）上传至Linux系统root用户属组目录下，例如**步骤6**中的“Ascend310B-source”目录。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤8** 执行以下命令，为驱动包添加可执行权限。

**chmod u+x Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run**

若有软实时OS需求，该命令为**chmod u+x Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run**。

**步骤9** （可选）新增或替换文件。

在二次开发场景中，若用户需要新增或替换驱动包中的Image、dt.img、驱动文件（ko文件）或驱动文件的加载脚本等，可根据需要，选择执行以下操作。

1. 执行以下命令，在“Ascend310B-source”目录下新建“repack”目录并进入。若已创建，则无需重复创建。

   **mkdir repack**

   **cd repack**

2. 执行以下命令，在“repack”目录下新建“firmware”目录，用于替换Image或dt.img文件。新建“drivers”目录，用于替换驱动文件（ko文件）。新建“scripts”目录，用于替换驱动文件的加载脚本。

   **mkdir firmware**

   **mkdir drivers**

   **mkdir scripts**

3. 执行以下命令，进入“firmware”目录。

   **cd firmware**

   上传Image或dt.img文件，文件生成方式请参见**8.3.1 编译内核**，文件传输方式请参见**14.3 使用WinSCP传输文件**。

   执行以下命令，查看上传的Image或dt.img文件。

   **ls**

4. 执行以下命令回到“repack”目录，再进入“drivers”目录。

   **cd** */opt*/**Ascend310B-source/repack**

   **cd drivers**

   上传新的驱动文件（ko文件），文件生成方式请参见**8.3.3 编译驱动**，文件传输方式请参见**14.3 使用WinSCP传输文件**。

   执行以下命令，查看上传的驱动文件（ko文件）。

   **ls**

5. 执行以下命令回到“repack”目录，再进入“scripts”目录。

   **cd** */opt*/**Ascend310B-source/repack**

   **cd scripts**

   上传含通用驱动的自动加载配置文件11-atlas.conf或上传不含通用驱动的自动加载配置文件filelist.csv和minirc_sys_init_ext.sh，文件生成方式请参见**8.5 驱动自动加载配置**，文件传输方式请参见**14.3 使用WinSCP传输文件**。

   执行以下命令，查看上传的驱动文件的加载脚本。

 版权所有 © 华为技术有限公司 

**ls**

6. 执行如下命令，回到“Ascend310B-source”目录。

   **cd** */opt*/**Ascend310B-source**

**步骤10** 执行如下命令，编译驱动。

**bash build.sh repack ./Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run**

若有软实时OS需求，该命令为**bash build.sh repack ./Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run**。

出现如下回显，表示驱动重打包成功。

```
repack success!
```

**说明**

- 编译后的Ascend-hdk-310b-npu-driver-soc_<*version*>_linux-aarch64-repack.run（若有软实时OS需求，则驱动包为Ascend-hdk-310b-npu-driver-soc_<*version*>_linux-rt-aarch64-repack.run）驱动文件会自动存放于“Ascend310B-source/”目录下。
- *xxx*表示软件包版本号。
- 用户请将编译后的驱动包上传至自有的环境进行安装，安装操作请参见**9 驱动与固件升级**。

**----结束**

# 8.7 编辑用户自定义脚本

本章节介绍用户如何在制作OS阶段进行用户自定义软件包的预置。

您可以参考制卡工具包中携带的install_hook_demo.sh脚本，在生成的自定义脚本minirc_install_hook.sh中进行软件包预置及系统启动服务调用。

**表 8-1** install_hook_demo.sh 脚本参数

| 参数 | 说明 |
| --- | --- |
| ROOT_PATH=$1 | 制作过程中根文件的系统路径，不可修改，用户可以此作为系统安装后的根目录。 |
| BOOT_SH="${ROOT_PATH}/var/minirc_hook.sh" | 制作过程中系统启动预留钩子脚本名，不可修改。 |
| function hook_head<br>function hook_end | 系统启动预留钩子脚本起始及结尾代码，不可修改。 |

**说明**

用户可对以下脚本中的**粗体**部分进行修改。

```
#!/bin/bash

#rootfs root path
ROOT_PATH=$1
```

 版权所有 © 华为技术有限公司 

```
#boot scripts, no need to modify
BOOT_SH="${ROOT_PATH}/var/minirc_hook.sh"
######minirc_hook.sh header and ending, no need to modify######
function hook_head()
{
    echo "
#!/bin/bash
" >${BOOT_SH}
}
function hook_end()
{
    echo "
exit 0
" >>${BOOT_SH}
}
#####################################################################
#########################user function##############################
function func_sample()
{
echo "start install sample lib"
#do copy
installAclLib
installAicpuKernels
}
#####################################################################
# start
hook_head
func_sample
hook_end
```

针对添加用户自定义安装包场景，用户在脚本“**func_sample()**”函数中，需要做如下操作。

**步骤1** 确认安装包格式。

- 如果安装包是压缩包格式，需要先解压安装包到预安装系统的指定目录下。
- 如果待安装文件是.run或.deb格式等不需要解压的文件，可直接拷贝文件到预安装系统的指定目录中。

**说明**

预安装系统的指定目录：制卡时创建的目录，路径为${ROOT_PATH}。

**步骤2** 添加安装命令到预安装系统的启动脚本中（${BOOT_SH}，启动脚本在Atlas 200I A2加速模块系统启动后会自动调用执行）。添加的命令可以是安装命令，也可以是启动用户服务的命令。

通用安装流程为：进入到解压目录，然后更改安装脚本权限，并执行安装脚本，最后删除解压目录。

**----结束**

## 使用样例

下面以添加安装TestCase.tar.gz包为例，介绍下如何修改install_hook_demo.sh脚本。新增内容如脚本中的**粗体**部分。

```
#!/bin/bash
#rootfs root path
ROOT_PATH=$1
#boot scripts, no need to modify
BOOT_SH="${ROOT_PATH}/var/minirc_hook.sh"
######minirc_hook.sh header and ending, no need to modify######
function hook_head()
{
```

 版权所有 © 华为技术有限公司 

```
    echo "
#!/bin/bash
" >${BOOT_SH}
}
function hook_end()
{    echo "
exit 0
" >>${BOOT_SH}  }

#user package name, you must change name
AICPU_KERNELS_PACKAGE=$(ls TestCase.tar.gz)

# using this as generateAclLibInstallShell()
function genAicpuKernInstShell()
{
   echo "
cd /root/aicpu_kernels_device/
chmod 750 *.sh
chmod 750 scripts/*.sh
scripts/install.sh --run

rm -rf /root/aicpu_kernels_device
" >>${BOOT_SH}
}

# using this as installAclLib()
function installAicpuKernels()
{
   tar zxf ${CUR_PATH}/${AICPU_KERNELS_PACKAGE} -C ${ROOT_PATH}/root/
   genAicpuKernInstShell
}
######################################################################
#########################user function#############################
function main()
{    echo "start install sample lib"
     installAicpuKernels
}
######################################################################
# start
hook_head
main
hook_end
```

# 8.8 节能配置

## 操作场景

当Atlas 200I A2 加速模块功耗比较高时，用户可以选择关闭ISP、VPSS模块。

## 前提条件

- 一台带网络端口且操作系统为Ubuntu 22.04的x86 Linux服务器。
- Linux服务器已安装python、make、gcc、unzip、pigz、bison、flex、libncurses-dev、squashfs-tools、bc、device-tree-compiler、libssl-dev、cmake、rpm2cpio与交叉编译工具链。

 版权所有 © 华为技术有限公司 

**说明**

- 所有的依赖必须用root用户进行安装。
- 用户可以通过如下命令进行安装上述依赖。

  **apt-get install -y python3 make gcc unzip pigz bison flex libncurses-dev squashfs-tools bc device-tree-compiler libssl-dev cmake rpm2cpio**
- 交叉编译工具链安装请参见**8.1 安装工具链**。

## 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的源码包“Ascend310B-source.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤4** 执行如下命令，进入源码包所在目录，例如“/opt”。

**cd** */opt*

**步骤5** 执行如下命令，解压源码包“Ascend310B-source.tar.gz”。

**tar -xzvf Ascend310B-source.tar.gz**

**步骤6** 执行如下命令，进入“Ascend310B-source”目录。

**cd Ascend310B-source**

**步骤7** 修改对应设备的dts文件。

1. 执行如下命令，进入“dtb”目录。

   **cd dtb**
2. 执行如下命令，打开Atlas 200I A2 加速模块dts文件。

   **vim dts/hi1910b/hi1910BL/base/hi1910B-isp.dtsi**
3. 将文件中的**isp_enable** 、**vpss_enable**字段的默认值**<0x1>**修改为**<0x0>**。

```
media {
    #address-cells = <2>;
    #size-cells = <2>;
    compatible = "simple-bus";
    isp_enable = <0x0>;
    vpss_enable = <0x0>;
    ranges;
    ...
};
```

 版权所有 © 华为技术有限公司 

**说明**

- 如果ISP和NVR场景都不需要，配置为：
  isp_enable = <0x0>;
  vpss_enable = <0x0>;
- 如果ISP场景不需要，但NVR场景需要VPSS，配置为：
  isp_enable = <0x0>;
  vpss_enable = <0x1>;
- 场景名词解释。
  ISP：图像信号处理器
  NVR：网络硬盘录像机
  VPSS：视频处理子系统

4. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。
   **:wq!**
5. 返回“Ascend310B-source”目录。
   **cd** */opt*/**Ascend310B-source**

**步骤8** 执行如下命令，编译DTB文件。

**bash build.sh dtb**

出现如下回显，且生成“dt.img”文件表示编译内核DTB文件成功。

```
generate /opt/Ascend310B-source/output/dt.img success!
sign /opt/Ascend310B-source/output/dt.img success!
```

**说明**

编译后的dt.img文件会自动存放于“Ascend310B-source/output”目录下。

**步骤9** 生效DTB文件。具体步骤请参见**生效DTB文件**。

**步骤10** 登录Atlas 200I A2 加速模块，验证配置是否成功。

- 如果dts文件中配置为：
  isp_enable = <0x0>;
  vpss_enable = <0x0>;
  a. 执行如下命令，查看设备节点。
     ls /dev/vi*
     ls /dev/isp*
     ls /dev/vpss*
  b. 若显示如下内容，表示配置成功。
     ```
     ls: cannot access '/dev/vi*': No such file or directory
     ls: cannot access '/dev/isp*': No such file or directory
     ls: cannot access '/dev/vpss*': No such file or directory
     ```
- 如果dts文件中配置为：
  isp_enable = <0x0>;
  vpss_enable = <0x1>;
  a. 执行如下命令，查看设备节点。
     ls /dev/vi*
     ls /dev/isp*

 版权所有 © 华为技术有限公司 

ls /dev/vpss*

b. 若显示如下内容，表示配置成功。

```
ls: cannot access '/dev/vi*': No such file or directory
ls: cannot access '/dev/isp*': No such file or directory
/dev/vpss
```

**----结束**

 版权所有 © 华为技术有限公司 

# 9 驱动与固件升级

9.1 升级前必读
9.2 升级准备
9.3 升级驱动
9.4 升级固件
9.5 版本回退

## 9.1 升级前必读

Atlas 200I A2 加速模块可以配置单文件系统或双文件系统（主备分区）升级驱动操作如下。

- 对于单文件系统，升级驱动的详细操作请参见**9.3.1.1 升级当前区**。
- 对于双文件系统，升级当前区（主区）驱动的详细操作请参见**9.3.1.1 升级当前区**，升级备区驱动的详细操作请参见**9.3.1.2 升级备分区**。

Atlas 200I A2 加速模块升级固件请参见**9.4 升级固件**。

### 升级影响

无。

### 注意事项

- 升级过程禁止进行其他维护操作动作。
- 升级过程中Atlas 200I A2 加速模块需保持供电稳定，如发生异常掉电，再次上电后系统无法启动时，请备份数据后参考**6.1.2.4 SD卡制作系统镜像**章节制作镜像包。
- 升级驱动和固件过程中系统会修改如下目录（“/home/HwHiAiUser”、“/var/davinci/”、“/var/log/npu”和“/usr/local/Ascend”）中的文件属组，为防止提权风险，请用户勿将自定义文件与上述目录存放在一起，否则可能会被修改为相同属组。
- 请使用同一版本的驱动与固件。

 版权所有 © 华为技术有限公司 

- 若有软实时OS需求，请勿直接使用**4.1 下载软件包**获取的驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run进行升级，请务必自行编译生成内核Image文件后重构驱动run包进行升级。
- Ascend HDK 23.0.3及后续版本不支持回退Ascend HDK 23.0.RC1.1以及Ascend HDK 23.0.RC1.5版本，请勿进行回退操作。

# 9.2 升级准备

Atlas 200I A2 加速模块升级需要准备升级所需的驱动包和固件包，软件包获取方式请参见**4.1 下载软件包**。

1. 在升级前检查Atlas 200I A2 加速模块的固件版本和驱动版本是否配套，配套关系具体请参见《**Ascend HDK 23.0.3 版本配套表**》，若不配套请联系华为技术工程师解决。
   - 获取Atlas 200I A2 加速模块固件版本。
     i. 登录Atlas 200I A2 加速模块OS，并切换到root用户。
     ii. 执行如下命令，获取固件版本信息。
     **/var/davinci/driver/upgrade-tool --device_index -1 --component -1 --version**
   - 获取Atlas 200I A2 加速模块驱动版本。
     i. 登录Atlas 200I A2 加速模块OS，并切换到root用户。
     ii. 执行如下命令，获取sys_version.conf文件内容。
     **cat /etc/sys_version.conf**
2. 已安装基础固件。
   a. 登录Atlas 200I A2 加速模块OS，并切换到root用户。
   b. 执行如下命令，获取基础固件版本信息。
   **/var/davinci/driver/upgrade-tool --device_index -1 --component -1 --all --version**

# 9.3 升级驱动

## 9.3.1 通过 run 包升级

### 9.3.1.1 升级当前区

#### 前提条件

Atlas 200I A2 加速模块已安装操作系统。

#### 操作步骤

**步骤1** 使用PuTTY登录Atlas 200I A2 加速模块OS的命令行。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

 版权所有 © 华为技术有限公司 

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的驱动升级包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run”（若有软实时OS需求，使用重构后的驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run）上传至Atlas 200I A2 加速模块所在OS的任一目录下，例如“/home/HwHiAiUser/software”。

**步骤4** 执行如下命令，进入升级包所在目录，例如“/home/HwHiAiUser/software”。

**cd** */home/HwHiAiUser/software*

**步骤5** 执行如下命令，增加软件包的可执行权限。

**chmod +x** Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run

若有软实时OS需求，请执行如下命令。

**chmod +x** Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run。

**步骤6** 执行如下命令，升级驱动包，此处以**--full**参数为例。

**./**Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run **--full**

若有软实时OS需求，请执行如下命令。

**./**Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run **--full**

若系统出现如下关键回显信息，表示升级成功。

```
Driver package installed successfully!
```

**说明**

执行完此命令后，driver相关文件默认存储在“/var/davinci/driver/”目录下。

**步骤7** 执行重启命令，进行Atlas 200I A2 加速模块的重启，从而完成Driver包的升级。

**reboot**

**说明**

启动过程中请勿将Atlas 200I A2 加速模块断电。

**步骤8** 待Atlas 200I A2 加速模块启动完成后，执行如下命令，查看升级后的驱动版本号。

**npu-smi info**

若回显信息中“Version”版本号与驱动包版本信息一致，则表明升级成功。

```
| npu-smi 23.0.0                   Version: 23.0.0                   |
```

**----结束**

## 9.3.1.2 升级备分区

## 前提条件

- Atlas 200I A2 加速模块已配置双文件系统。
- Atlas 200I A2 加速模块已安装操作系统。

 版权所有 © 华为技术有限公司 

## 操作步骤

**步骤1** 使用PuTTY登录Atlas 200I A2 加速模块OS的命令行。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的驱动升级包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run”（若有软实时OS需求，使用重构后的驱动包Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run）上传至Atlas 200I A2 加速模块所在OS的任一目录下，例如“/home/HwHiAiUser/software”。

**步骤4** 执行如下命令，进入升级包所在目录，例如“/home/HwHiAiUser/software”。

**cd** */home/HwHiAiUser/software*

**步骤5** 执行如下命令，增加软件包的可执行权限。

**chmod +x** Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run

若有软实时OS需求，请执行如下命令。

**chmod +x** Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run。

**步骤6** 检查“/mnt/p2”或者“/mnt/p3”是否存在。

**ls /mnt/p2**

**ls /mnt/p3**

若不存在，则需先执行如下命令创建对应目录。

**mkdir -p /mnt/p2**

**mkdir -p /mnt/p3**

**步骤7** 执行如下命令查看当前OS启动的分区，并做记录。

**cat /proc/cmdline**

- 对于Atlas 200I A2 加速模块使用SD卡启动方式，显示如下：

```
root@davinci-mini:~# cat /proc/cmdline
console=ttyAMA0,115200 log_redirect=0x1fc000@0x22741000 root=/dev/mmcblk1p2 rw rdinit=/sbin/init rodata=off
```

  - 若回显信息中存在“root=/dev/mmcblk1p2”，则执行**mount /dev/mmcblk1p3 /mnt/p3**。
  - 若回显信息中存在“root=/dev/mmcblk1p3”，则执行**mount /dev/mmcblk1p2 /mnt/p2**。

- 对于Atlas 500 A2 智能小站使用M.2启动方式，显示如下：

```
root@davinci-mini:~# cat /proc/cmdline
console=ttyAMA0,115200 log_redirect=0x1fc000@0x22741000 root=PARTUUID=360c8c0e-5ed1-4d0b-8cd6-1db37d9bfc1e rw rdinit=/sbin/init rodata=off syslog earlycon=pl011,mmio32,0xC4010000 loglevel=5 initrd=0x2BA00000,300M rootwait mpam=of enable_ascend_share_pool ascend_enable_all enable_sp_multi_group_mode init_all_gicr enable_fpga no_console_suspend selinux=0 systemd.unified_cgroup_hierarchy=0 audit=1 audit_backlog_limit=8192 fbcon=map:6
```

  则执行**readlink -f /dev/disk/by-partuuid/***{PARTUUID}*命令查询OS启动分区，例如**readlink -f /dev/disk/by-partuuid/***360c8c0e-5ed1-4d0b-8cd6-1db37d9bfc1e*。

  显示如下：

```
root@davinci-mini:~# readlink -f /dev/disk/by-partuuid/360c8c0e-5ed1-4d0b-8cd6-1db37d9bfc1e
/dev/sda3
```

 版权所有 © 华为技术有限公司 

  - 若回显信息为“/dev/sda2”，则执行**mount /dev/sda3 /mnt/p3**。
  - 若回显信息为“/dev/sda3”，则执行**mount /dev/sda2 /mnt/p2**。

**步骤8** 根据上一步查询结果执行如下命令，升级驱动包到备区。

- 对于Atlas 200I A2 加速模块使用SD卡启动方式。

  使用普通NPU驱动包。

  - 若“root=/dev/mmcblk1p2”，则执行**./**Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run **--mnt-install-path=**/mnt/p3。
  - 若“root=/dev/mmcblk1p3”，则执行**./**Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run **--mnt-install-path=**/mnt/p2。

  若有软实时OS需求，使用重构后的NPU驱动包。

  - 若“root=/dev/mmcblk1p2”，则执行**./**Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run **--mnt-install-path=**/mnt/p3。
  - 若“root=/dev/mmcblk1p3”，则执行**./**Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run **--mnt-install-path=**/mnt/p2。

- 对于Atlas 500 A2 智能小站使用M.2启动方式。
  - 若查询结果为“/dev/sda2”，则执行**./**Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run **--mnt-install-path=**/mnt/p3。
  - 若查询结果为“/dev/sda3”，则执行**./**Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-aarch64.run **--mnt-install-path=**/mnt/p2。

  若系统出现如下关键回显信息，表示升级成功。

```
Driver package installed successfully!
```

**说明**

执行完此命令后，driver相关文件默认存储在“/var/davinci/driver/”目录下。

**步骤9** 执行重启命令，进行Atlas 200I A2 加速模块的重启，从而完成Driver包的升级。

**reboot**

**说明**

启动过程中请勿将Atlas 200I A2 加速模块断电。

**步骤10** 待Atlas 200I A2 加速模块启动完成后，执行如下命令，查看升级后的驱动版本号。

**npu-smi info**

若回显信息中“Version”版本号与驱动包版本信息一致，则表明升级成功。

```
| npu-smi 23.0.0                   Version: 23.0.0                   |
```

**步骤11** 查看当前OS启动的分区是否切换。

- 对于Atlas 200I A2 加速模块使用SD卡启动方式，执行以下命令查看OS启动的分区。

  **cat /proc/cmdline**

  与**步骤7**记录的分区信息对比，此处回显信息中OS启动分区应切到备区。例如，原先为“root=/dev/mmcblk1p2”，则此处查询结果应为“root=/dev/mmcblk1p3”。

- 对于Atlas 500 A2 智能小站使用M.2启动方式，执行以下命令查看OS启动的分区。

 版权所有 © 华为技术有限公司 

**cat /proc/cmdline**

根据显示信息，再执行**readlink -f /dev/disk/by-partuuid/***{PARTUUID}*命令查看OS启动分区。

与**步骤7**记录的分区信息对比，此处回显信息中OS启动分区应切到备区。例如，原先为“/dev/sda2”，则此处查询结果应为“/dev/sda3”。

**----结束**

## 9.3.2 通过 tar.gz 包升级

通过**6.1.3 SD卡制作和启动系统镜像（方法二）**或**6.3 eMMC制作和启动系统镜像**启动的Atlas 200I A2 加速模块，可通过tar.gz包的升级方式实现主备区同时升级，保证主备区的版本一致。

### 前提条件

Atlas 200I A2 加速模块已通过**6.1.3 SD卡制作和启动系统镜像（方法二）**或**6.3 eMMC制作和启动系统镜像**启动。

### 操作步骤

**步骤1** 登录Atlas 200I A2 加速模块OS的命令行。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具或其他工具，根据实际需求将对应软件包上传至Atlas 200I A2 加速模块所在OS的任一目录下，例如“/home/HwHiAiUser/software”。软件包请参见**4.1 下载软件包**获取，工具使用方法请参见**14.3 使用WinSCP传输文件**。

- 非软实时OS需求，将“Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz”、签名校验文件“Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz.cms”和“Ascend-hdk-310b-npu-soc_*<version>*_linux-aarch64.tar.gz.crl”包上传至Atlas 200I A2 加速模块所在OS的任一目录下，例如“/home/HwHiAiUser/software”。
- 软实时OS需求，将重构后的驱动包“Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run”以及“Ascend-hdk-310b-npu-soc_<version>_linux-rt-aarch64.tar.gz”包上传至Atlas 200I A2 加速模块所在OS的任一目录下，例如“/home/HwHiAiUser/software”，并执行如下命令进行软件包重构。

  **mkdir rt && tar -zxvf Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz -C ./rt && rm -rf rt/Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64.run && cp Ascend-hdk-310b-npu-driver-soc_*<version>*_linux-rt-aarch64-repack.run rt/ && cd rt && tar zcvf Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz ./* && cp Ascend-hdk-310b-npu-soc_*<version>*_linux-rt-aarch64.tar.gz ../ && cd .. && rm -rf rt**

  **说明**

  *<version>*表示NPU版本号，具体请根据实际情况进行替换。

**步骤4** 执行如下命令，升级驱动包。

 版权所有 © 华为技术有限公司 

**/usr/local/scripts/upgrade_drv.sh -u** */home/HwHiAiUser/software*/**Ascend-hdk-310b-npu-soc**_*<version>* **_linux-aarch64.tar.gz** */home/HwHiAiUser/software*/**Ascend-hdk-310b-npu-soc**_*<version>* **_linux-aarch64.tar.gz.cms** */home/HwHiAiUser/software*/**Ascend-hdk-310b-npu-soc**_*<version>* **_linux-aarch64.tar.gz.crl**

若有软实时OS需求，请执行如下命令。

**/usr/local/scripts/upgrade_drv.sh -u** */home/HwHiAiUser/software*/**Ascend-hdk-310b-npu-soc**_*<version>* **_linux-rt-aarch64.tar.gz** */home/HwHiAiUser/software*/**Ascend-hdk-310b-npu-soc**_*<version>* **_linux-rt-aarch64.tar.gz.cms** */home/HwHiAiUser/software*/**Ascend-hdk-310b-npu-soc**_*<version>* **_linux-rt-aarch64.tar.gz.crl**

若系统出现如下关键回显信息，表示升级成功。

```
Driver package installed successfully!
......
user upgrade success
end upgrade_drv.sh
```

**说明**

执行完此命令后，driver相关文件默认存储在“/var/davinci/driver/”目录下。

**步骤5** 执行重启命令，进行Atlas 200I A2 加速模块的重启，从而完成Driver包的升级。

**reboot**

**说明**

启动过程中请勿将Atlas 200I A2 加速模块断电。

**步骤6** 待Atlas 200I A2 加速模块启动完成后，执行如下命令，查看升级后的驱动版本号。

**npu-smi info**

若回显信息中“Version”版本号与驱动包版本信息一致，则表明升级成功。

```
| npu-smi 23.0.0                   Version: 23.0.0                   |
```

**说明**

- 升级完成启动后若需进行切区，需等待1分钟以上保证对区版本升级完成，否则会导致对区版本升级失败。
- 升级完成启动后会再次进行自动重启来完成固件的同步升级，此为正常现象，待自动重启后可正常使用。

**----结束**

# 9.4 升级固件

## 前提条件

Atlas 200I A2 加速模块已安装操作系统。

## 操作步骤

**步骤1** 使用PuTTY登录Atlas 200I A2 加速模块OS的命令行。

 版权所有 © 华为技术有限公司 

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 使用WinSCP工具，将4.1 下载软件包获取的固件升级包“Ascend-hdk-310b-npu-firmware-soc_*x.x.x.x.X*.run”上传至Atlas 200I A2 加速模块所在系统的任一目录下，例如“/home/HwHiAiUser/software”。

**步骤4** 执行如下命令，进入升级包所在目录，例如“/home/HwHiAiUser/software”。

**cd** */home/HwHiAiUser/software*

**步骤5** 执行如下命令，增加软件包的可执行权限。

**chmod +x** Ascend-hdk-310b-npu-firmware-soc_*x.x.x.x.X*.run

**步骤6** 执行如下命令，升级固件包。

**./**Ascend-hdk-310b-npu-firmware-soc_*x.x.x.x.X*.run **--full**

**说明**

执行完此命令后，固件相关文件存储在“/usr/local/Ascend/firmware”目录下。

若系统出现如下关键回显信息，表示升级成功。

```
Firmware package installed successfully!
```

**步骤7** 执行重启命令，进行Atlas 200I A2 加速模块的重启，从而完成固件包的升级。

**reboot**

**步骤8** 待Atlas 200I A2 加速模块启动完成后，执行如下命令，查看升级后的固件版本号。

**/var/davinci/driver/upgrade-tool --device_index -1 --component -1 --all --version**

获取版本信息，与固件包版本信息一致即表示升级成功。

**----结束**

# 9.5 版本回退

回退操作方式和升级操作相同，回退驱动具体操作请参见9.3.1 通过run包升级，回退固件具体操作请参见9.4 升级固件。


版权所有 © 华为技术有限公司

# 10 关键特性介绍

10.1 主备分区介绍
10.2 安全启动介绍
10.3 安全关机介绍
10.4 休眠唤醒介绍

## 10.1 主备分区介绍

### 功能介绍

用户制作启动镜像包前可以通过修改配置文件使能主备分区功能，详细操作请参见**步骤13**。通过**6.1.3 SD卡制作和启动系统镜像（方法二）**、**6.2 M.2制作和启动系统镜像**和**6.3 eMMC制作和启动系统镜像**制作的启动镜像默认使能主备分区功能。

在主区修改的文件会同步备份到/home/data下，当主分区意外损坏时可以切换到备区，切换后/home/data下关键数据能够同步到备区，用户可以正常登录使用。主备分区切换操作请参见**示例9：主备文件系统场景下切换启动分区**。

- 通过**6.1.2 SD卡制作和启动系统镜像（方法一）**制作的SD启动盘，主备分区关系表如**表10-1**和**表10-2**所示。

**表 10-1** SD 卡使能主备分区映射表

| SD卡分区 | 说明 | 挂载目录 | 默认分区大小 |
| --- | --- | --- | --- |
| /dev/mmcblk1ªp1 | 占位分区，保留p1盘符。 | 无 | 100M |
| /dev/mmcblk1ªp2 | 系统主分区。 | / | 6G |
| /dev/mmcblk1ªp3 | 系统备分区。 | / | 6G |


版权所有 © 华为技术有限公司

| SD卡分区 | 说明 | 挂载目录 | 默认分区大小 |
| --- | --- | --- | --- |
| /dev/mmcblk1[a]p4 | 关键数据同步备份分区。 | /home/data | 1G |
| /dev/mmcblk1[a]p5 | 存储日志分区。 | /home/log | 1G |
| a：SD卡分区：mmcblk1。 | | | |

**表 10-2** SD 卡不使能主备分区映射表

| SD卡分区 | 说明 | 挂载目录 | 默认分区大小 |
| --- | --- | --- | --- |
| /dev/mmcblk1[a]p1 | 占位分区，保留p1盘符。 | 无 | 100M |
| /dev/mmcblk1[a]p2 | 系统主分区。 | / | 6G |
| /dev/mmcblk1[a]p3 | 占位分区，保留p3盘符。 | 无 | 100M |
| /dev/mmcblk1[a]p4 | 用户数据区[b]。 | /home/data | 1G |
| /dev/mmcblk1[a]p5 | 存储日志分区。 | /home/log | 1G |
| a：SD卡分区：mmcblk1。<br>b：不使能主备分区时，该分区无数据同步备份功能。 | | | |

- 通过**6.1.3 SD卡制作和启动系统镜像（方法二）**、**6.2 M.2制作和启动系统镜像**和**6.3 eMMC制作和启动系统镜像**制作的启动镜像，主备分区关系如**表10-3**所示。

**表 10-3** 使能主备分区映射表

| M.2分区 | eMMC分区 | SD卡分区 | 说明 | 挂在目录 | 默认区分大小 |
| --- | --- | --- | --- | --- | --- |
| /dev/sd*x*[a]1 | /dev/mmcblk0p1 | /dev/mmcblk1p1 | 系统启动镜像分区。 | 无 | 1.5G |
| /dev/sd*x*[a]2 | /dev/mmcblk0p2 | /dev/mmcblk1p2 | 系统主分区。 | / | 6G |
| /dev/sd*x*[a]3 | /dev/mmcblk0p3 | /dev/mmcblk1p3 | 系统备分区。 | / | 6G |
| /dev/sd*x*[a]4 | /dev/mmcblk0p4 | /dev/mmcblk1p4 | 关键数据同步备份分区。 | /home/data | 1G |


版权所有 © 华为技术有限公司

| M.2分区 | eMMC分区 | SD卡分区 | 说明 | 挂在目录 | 默认区分大小 |
| --- | --- | --- | --- | --- | --- |
| /dev/sd*x*[a]5 | /dev/mmcblk0p5 | /dev/mmcblk1p5 | 存储日志分区。 | /home/log | 1G |
| /dev/sd*x*[a]6 | /dev/mmcblk0p6 | /dev/mmcblk1p6 | 中间件&OM分区。 | /usr/local/mindx | 2G |
| /dev/sd*x*[a]7 | /dev/mmcblk0p7 | /dev/mmcblk1p7 | 软件包。 | /home/package | 3G |
| /dev/sd*x*[a]8 | /dev/mmcblk0p8 | /dev/mmcblk1p8 | 容器镜像分区。 | 无 | 7G |
| a：sd*x*，可通过**fdisk -l**查找M.2 SATA SSD硬盘所在的M.2设备名称，例如“/dev/sda”。 | | | | | |

## 关键文件同步

用户制作启动恢复出厂镜像包时使能了主备分区功能，当前提供白名单（backup_file_list.txt、backup_dir_list.txt）控制主备文件系统同步文件或文件夹。

在主区修改的文件会同步备份到“/home/data”目录下，当主分区意外损坏时可以切换到备区，切换后“/home/data”目录下关键数据能够同步到备区，用户可以正常登录使用。主备分区切换操作请参见**示例9：主备文件系统场景下切换启动分区**。

**说明**

- 仅支持在“/etc”目录下新增需要同步的目录或文件。
- 需保证主备区均存在要同步的目录或文件，否则会导致主备同步功能异常。

该机制会实时同步影响用户登录的关键配置文件，同步**表10-4**和**表10-5**所示的文件。

**表 10-4** backup_dir_list.txt 关键配置目录

| 文件 | 说明 |
| --- | --- |
| /etc/profile.d | linux系统配置文件目录 |
| /etc/sysctl.d | linux系统内核参数配置文件目录 |
| /etc/ld.so.conf.d | 动态链接库配置文件目录 |
| /etc/modules-load.d | 内核模块配置文件目录 |
| /etc/sysconfig/network-scripts | 网络配置目录 |
| /etc/netplan | |
| /etc/ssh | |
| /etc/modprobe.d | 模块配置目录 |
| /var/spool/cron | 定时任务目录 |

 版权所有 © 华为技术有限公司 

| 文件 | 说明 |
| --- | --- |
| /var/spool/cron/crontabs | |
| /etc/udev/rules.d/ | USB规则文件目录 |
| /usr/share/cracklib/ | 弱密码字典目录 |

**表 10-5** backup_file_list.txt 关键配置文件

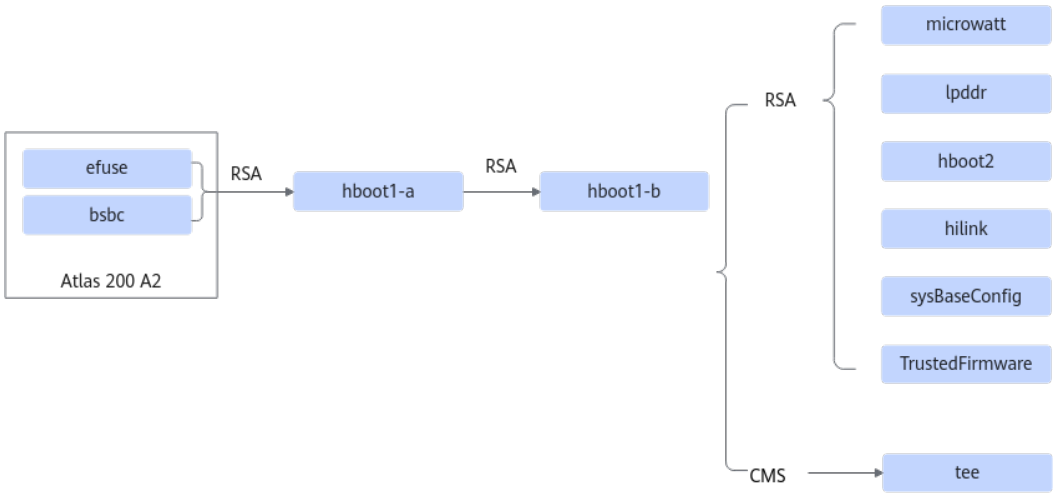
| 文件 | 说明 |
| --- | --- |
| /etc/profile | 系统环境变量配置文件 |
| /etc/sysctl.conf | 内核参数配置文件 |
| /etc/sudoers | sudo配置文件 |
| /etc/security/limits.conf | 系统资源限制配置文件 |
| /etc/sysconfig/network | 网络配置文件 |
| /etc/resolv.conf | |
| /etc/ntp.conf | |
| /etc/sysconfig/ntpd | |
| /etc/hosts | 主机名配置文件 |
| /etc/hostname | |
| /etc/sysconfig/docker | Docker配置文件 |
| /etc/docker/daemon.json | |
| /etc/localtime | 系统时钟配置文件 |
| /etc/adjtime | |
| /etc/rc.d/rc.local | 系统启动脚本 |
| /etc/rsyslog.conf | 系统日志配置文件 |
| /etc/hwsipcrl/ascendsip.crl | 证书文件 |
| /etc/ascend_driver.conf | 驱动配置文件 |

# 10.2 安全启动介绍

Atlas 200I A2 加速模块安全启动是基于BSBC（BootROM Secure Boot Code）作为信任根，结合eFuse的物理特性实现硬件级的安全启动校验。相对软件实现的安全启动特性，采用BSBC结合eFuse的芯片级安全启动特性更加可靠。当前，Atlas 200I A2 加速模块对其关键固件做了安全校验，其主要实现框图如图10-1所示。

 版权所有 © 华为技术有限公司 

**图 10-1** Atlas 200I A2 加速模块安全启动框图




Atlas 200I A2 加速模块涉及安全启动的驱动包含9个二进制文件。利用BSBC和eFuse中根公钥Hash值，对hboot1-a进行RSA校验。校验通过后，由hboot1-a引导对hboot1-b进行RSA校验。之后通过hboot1-b对剩余7个固件分别进行RSA/CMS（Cryptographic Message Syntax）校验，从而构建整个系统的安全可信链。

# 10.3 安全关机介绍

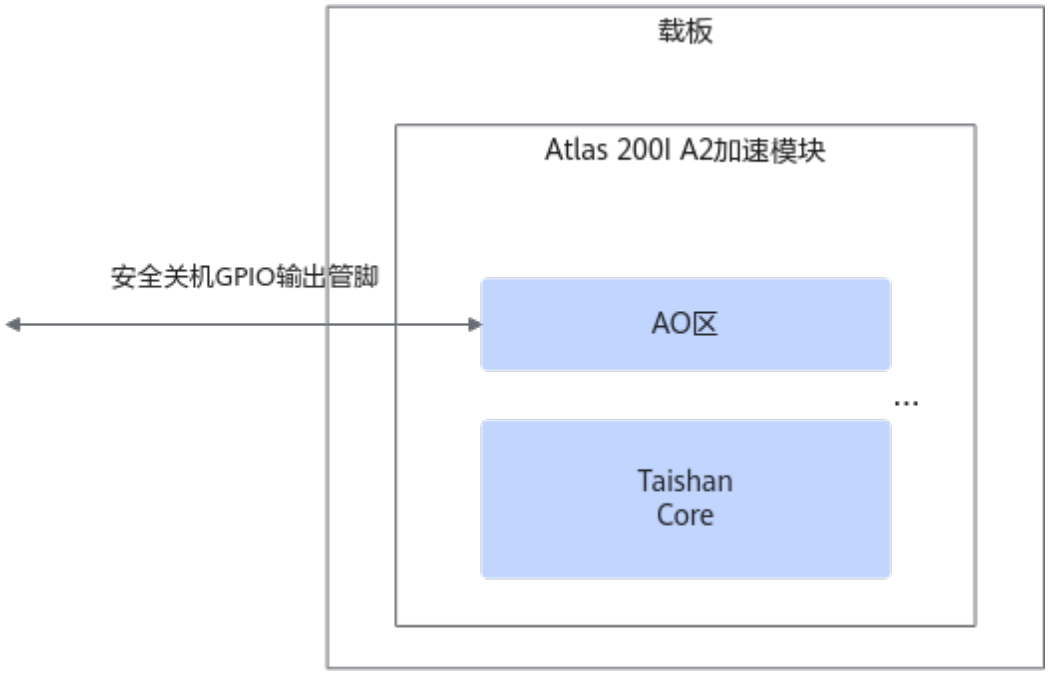
## 实现原理

Atlas 200I A2 加速模块提供安全关机特性，让用户对Atlas 200I A2 加速模块进行安全关机操作，避免强制关机从而导致数据丢失。

如图10-2所示，AO区为整个Atlas 200I A2 加速模块的管理模块，载板不下电AO区一直在运行，而用户程序、进程以及服务等都运行在Taishan Core中，用户可通过执行**poweroff**命令实现安全关机功能。命令执行后，AO区控制Taishan Core安全下电保证数据安全性，并将安全关机GPIO输出管脚拉高，表明Atlas 200I A2 加速模块已关机。


版权所有 © 华为技术有限公司

**图 10-2** 安全关机系统框图




### 操作步骤

**步骤1** 使用root用户登录设备，详细操作请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤2** 执行如下命令，设备安全关机。

**poweroff**

网络无法连通表示Atlas 200I A2 加速模块已安全关机。

**----结束**

## 10.4 休眠唤醒介绍

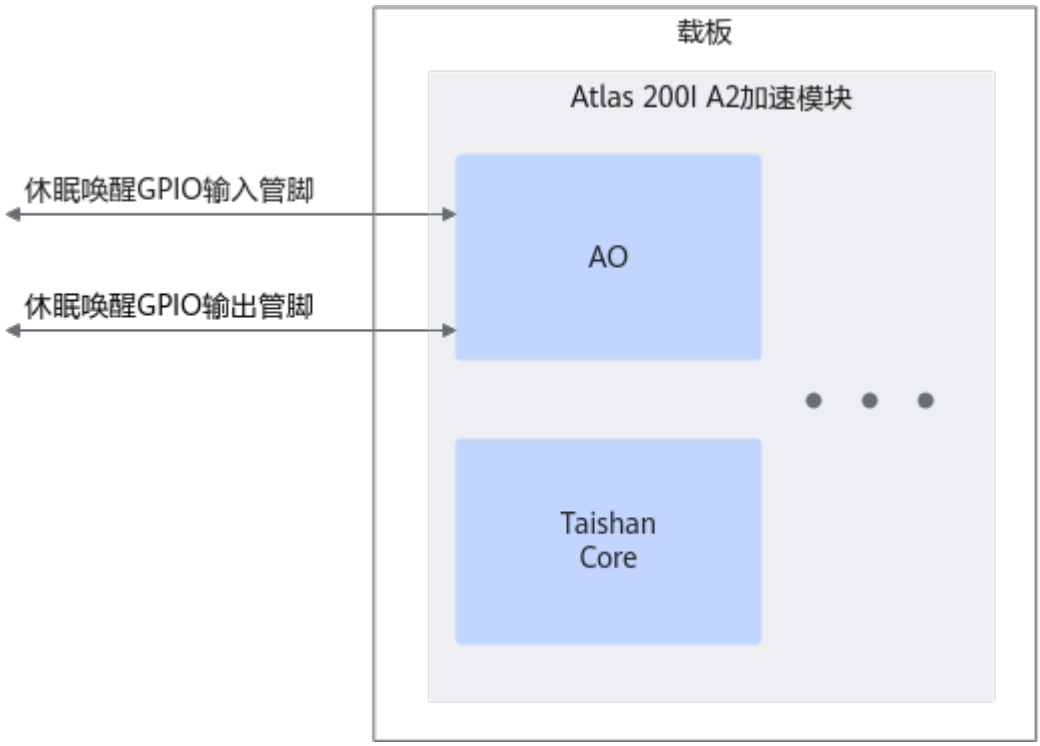
### 实现原理

Atlas 200I A2 加速模块提供休眠唤醒特性，让用户对Atlas 200I A2 加速模块进行休眠唤醒操作，以满足低功耗需求。

如**图10-3**所示，AO区为整个Atlas 200I A2 加速模块的管理模块，载板不下电AO区一直在运行，而用户程序、进程以及服务等都运行在Taishan Core中。用户可通过执行**npu-smi**命令和操作管脚实现休眠唤醒功能，此时AO区控制Taishan Core休眠或者唤醒，并改变相关GPIO管脚状态。管脚状态参见**管脚状态说明**。


版权所有 © 华为技术有限公司

**图 10-3** 休眠唤醒系统框图




## 管脚状态说明

GPIO8_0管脚为休眠唤醒输入管脚，用于控制Atlas 200I A2 加速模块是否处于休眠状态。

- 当GPIO8_0（对应Atlas 200I A2 加速模块管脚为SYS_SLEEP_IN）输入为高电平时，表示当前为休眠状态。
- 当GPIO8_0（对应Atlas 200I A2 加速模块管脚为SYS_SLEEP_IN）输入为低电平时，表示当前为唤醒状态。

GPIO8_1管脚为休眠唤醒输出管脚，用于指示Atlas 200I A2 加速模块是否处于休眠状态。

- 当GPIO8_1（对应Atlas 200I A2 加速模块管脚为SYS_SLEEP_OUT）输出为高电平时，表示当前为休眠状态。
- 当GPIO8_1（对应Atlas 200I A2 加速模块管脚为SYS_SLEEP_OUT）输出为低电平时，表示当前为唤醒状态。

## 前提条件

- Atlas 200I A2 加速模块已安装操作系统。
- 能正常执行npu-smi命令。

## 操作步骤

- 方法一：使用npu-smi命令进行休眠状态。

**步骤1** 登录Atlas 200I A2 加速模块系统环境。详细操作请参见**14.1 使用PuTTY登录设备（网口方式）**。

 版权所有 © 华为技术有限公司 

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 执行如下命令，进入休眠状态，200ms后自动唤醒。

**npu-smi set -t power-state -i 0 -c 0 -d 200**

**说明**

**-d**参数设置休眠时间，单位为毫秒(ms)，取值范围为[200，604800000]。

**----结束**

- 方法二：根据管脚状态说明，操作GPIO管脚实现休眠唤醒，具体操作请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“休眠唤醒控制”章节。

**须知**

使用npu-smi命令进行休眠后，休眠唤醒输入管脚为低电平；若提前唤醒，需将此管脚变为高电平后再变为低电平，即可实现唤醒。

 版权所有 © 华为技术有限公司 

# 11 产品化方案参考

11.1 （可选）MAC地址烧录

## 11.1 （可选）MAC 地址烧录

Atlas 200I A2 加速模块出厂时默认已烧录MAC地址，若用户需要设置自己定制的MAC地址则需重新烧录MAC地址。

### 前提条件

- 准备需要烧录的对应数量的合法MAC地址。
- 烧录MAC地址的Atlas 200I A2 加速模块环境系统正常启动，npu-smi工具可正常使用。
- Atlas 200I A2 加速模块支持查询与烧录eth0、eth1、eth2与eth3网口对应的MAC地址。

### 操作步骤

**步骤1** 使用root用户登录设备。

**步骤2** 执行如下命令查询当前设备信息，记录设备id和芯片id。

**npu-smi info**

**说明**

Atlas 200I A2 加速模块只有一个NPU芯片。

参数具体含义可参考《**npu-smi 命令参考**》。

**步骤3** 根据步骤2查询的设备id和芯片id，通过**npu-smi info -t mac-addr -i** *id* **-c** *chip_id*查询指定芯片MAC地址。此处以设备id为0、芯片id为0举例，使用示例如下。

```
npu-smi info -t mac-addr -i 0 -c 0
      mac-addr 0                    : 2e:66:xx:xx:11:ca
      mac-addr 1                    : 30:fd:xx:xx:94:eb
      mac-addr 2                    : 30:fd:xx:xx:94:ec
      mac-addr 3                    : 30:fd:xx:xx:94:ed
```

参数具体含义可参考《**npu-smi 命令参考**》。

 版权所有 © 华为技术有限公司 

**步骤4** 将定制的MAC地址烧录到指定网口，通过**npu-smi set -t mac-addr -i** *id* **-c** *chip_id* **-d** *mac_id* **-s** *mac_string*设置指定芯片网口的MAC地址。此处以设置设备id为0、芯片id为0、编号为0网口（eth0）的MAC地址举例，使用示例如下：

```
npu-smi set -t mac-addr -i 0 -c 0 -d 0 -s 00:05:xx:xx:54:16
      Status                  : OK
      Message                 : Set chip mac-addr successfully. Reset system for the configuration to take
effect.
```

参数具体含义可参考《**npu-smi 命令参考**》。

**步骤5** 设置完成后，通过**npu-smi info -t mac-addr -i** *id* **-c** *chip_id*再次查询指定芯片MAC地址，查看是否与步骤4中设置的MAC地址一致。此处查看设备id为0、芯片id为0、编号为0网口（eth0）的MAC地址，使用示例如下：

```
npu-smi info -t mac-addr -i 0 -c 0
      mac-addr 0                : 00:25:xx:xx:54:16
      mac-addr 1                : 30:fd:xx:xx:94:eb
      mac-addr 2                : 30:fd:xx:xx:94:ec
      mac-addr 3                : 30:fd:xx:xx:94:ed
```

**步骤6** 若需对指定芯片其他网口进行定制化MAC地址烧录，重复执行步骤4~5。

**步骤7** 烧录完成后，设备重启系统烧录的MAC才能生效。

**步骤8** 重启完成后，执行如下命令查询设备系统中对应网口的MAC地址与步骤4中设置的MAC是否一致。

**ifconfig**

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
inet xx.xx.xx.xx  netmask 255.255.xx.0  broadcast xx.xx.xx.xx
inet6 xx::xx:xx:xx:xx  prefixlen 64  scopeid 0x20<link>
ether 00:25:xx:xx:54:16  txqueuelen 1000  (Ethernet)
RX packets 67871  bytes 5199644 (5.1 MB)
RX errors 0  dropped 3756  overruns 0  frame 0
TX packets 10079  bytes 774203 (774.2 KB)
TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

如果不一致，说明烧录失败，请联系华为技术支持解决。

**----结束**

 版权所有 © 华为技术有限公司 

# 12 设备检查

12.1 查询基本信息
12.2 查询详细信息
12.3 查询产品类型
12.4 查询所有芯片健康状态

## 12.1 查询基本信息

### 前提条件

登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

### 操作步骤

查询所有设备的基本信息。

```
npu-smi info
+------------------------------------------------------------------------------------------------+
| npu-smi 23.0.0                                   Version: 23.0.0                               |
+---------------------------+---------------+----------------------------------------------------+
| NPU     Name              | Health        | Power(W)      Temp(C)           Hugepages-Usage(page) |
| Chip    Device            | Bus-Id        | AICore(%)     Memory-Usage(MB)                        |
+===========================+===============+====================================================+
| 0       310B              | OK            | 7.6           42                15    / 1304          |
| 0       0                 | NA            | 0             1003 / 10810                              |
+===========================+===============+====================================================+
```

**表 12-1** 打印信息说明

| 字段 | 说明 |
| --- | --- |
| npu-smi | npu-smi工具版本 |
| Version | 驱动版本 |


版权所有 © 华为技术有限公司

| 字段 | 说明 |
| --- | --- |
| NPU | 设备id |
| Name | 芯片名称 |
| Health | 芯片的健康状态<br>有如下五种状态：OK、Warning、Alarm、Critical或UNKNOWN<br>• OK：正常<br>• Warning：一般告警<br>• Alarm：重要告警<br>• Critical：紧急告警<br>• UNKNOWN：该设备不存在或未启动 |
| Power(W) | 芯片功率 |
| Temp(C) | 芯片温度（单位℃） |
| Hugepages-Usage(page) | 大页占比（单位page），每一个page的大小是2048KB |
| Chip | 芯片id |
| Device | 芯片编号 |
| Bus-Id | BUS id |
| AICore(%) | AICore占用率 |
| Memory-Usage(MB) | 内存占比 |

# 12.2 查询详细信息

## 前提条件

登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

## 操作步骤

查询设备0的具体信息。

```
npu-smi info -t board -i 0
        NPU ID                  : 0
        Product Name            : NA
        Model                   : NA
        Manufacturer            : NA
        Serial Number           : NA
        Software Version        : 23.0.0
        Firmware Version        : 7.1.0.3.220
        Board ID                : 0x45
        PCB ID                  : B
        BOM ID                  : 1
```


版权所有 © 华为技术有限公司

```
    Chip Count                  : 1
    Faulty Chip Count           : 0
```

**表 12-2** 打印信息说明

| 字段 | 说明 |
| --- | --- |
| NPU ID | 设备id |
| Product Name | 产品名称 |
| Model | 产品型号 |
| Manufacturer | 生产厂家 |
| Serial Number | 产品序列号 |
| Software Version | 驱动版本 |
| Firmware Version | 固件版本 |
| Board ID | 底板类型<br>**说明**<br>当使用非root用户执行该命令时，不支持查询Board ID，显示为NA。 |
| PCB ID | PCB版本编号 |
| BOM ID | BOM版本编号 |
| Chip Count | 芯片个数 |
| Faulty Chip Count | 故障芯片个数 |

**说明**

若需要升级固件与驱动，请参见**9 驱动与固件升级**。

# 12.3 查询产品类型

## 前提条件

登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

## 操作步骤

查询设备0的产品类型。具体产品型号以实际查询的产品为准。

```
npu-smi info -t product -i 0
    NPU ID                      : 0
    Chip Count                  : 1

    Chip ID                     : 0
    Product Type                : Atlas 200I A2
```


版权所有 © 华为技术有限公司

**表 12-3** 打印信息说明

| 字段 | 说明 |
| --- | --- |
| NPU ID | 设备id |
| Chip Count | 芯片个数 |
| Chip ID | 芯片id |
| Product Type | 产品类型 |

# 12.4 查询所有芯片健康状态

## 前提条件

登录Atlas 200I A2 加速模块OS。详细信息请参见**14.2 使用PuTTY登录设备（串口方式）**。

## 操作步骤

查询设备0所有芯片的健康状态。

```
npu-smi info -t health -i 0
        NPU ID                  : 0
        Chip Count              : 1

        Chip ID                 : 0
        Health                  : OK
```

**表 12-4** 打印信息说明

| 字段 | 说明 |
| --- | --- |
| NPU ID | 设备id |
| Chip Count | 芯片个数 |
| Chip ID | 芯片id |
| Health | 芯片的健康状态<br>有如下五种状态：OK、Warning、Alarm、Critical或UNKNOWN<br>• OK：正常<br>• Warning：一般告警<br>• Alarm：重要告警<br>• Critical：紧急告警<br>• UNKNOWN：该设备不存在或未启动 |


版权所有 © 华为技术有限公司

# 13 工具介绍与使用

13.1 系统管理工具介绍
13.2 烧写工具介绍
13.3 其他工具介绍

## 13.1 系统管理工具介绍

### 13.1.1 npu-smi 工具

- npu-smi是昇腾AI处理器的系统管理工具，可以用于收集设备信息，查看设备健康状态，对设备进行配置以及执行固件升级、清除设备信息等功能。
- 驱动安装后会默认安装npu-smi工具。
- 安装完成后，npu-smi放置在“/usr/local/sbin/”路径下。
- 详细npu-smi命令介绍请参考《**npu-smi 命令参考**》。

### 13.1.2 DCMI 工具

- DCMI（DaVinci Card Management Interface）接口包含设备管理接口、配置管理接口、MCU升级控制接口、芯片复位接口以及用户自定义信息配置接口。
- 用户可使用这些接口进行设备管理、配置管理、芯片复位启动等操作，可调用接口完成二次开发。
- 详细DCMI接口介绍请参见《**DCMI API参考**》。

## 13.2 烧写工具介绍

sdtool.tar.gz制卡脚本工具包用于SD卡制卡的烧写工具，可以对SD卡进行固件烧写、文件系统配置等操作。

制卡前可以通过修改配置文件来自定义分区大小和数量、使能主备分区功能等。

详细使用方法介绍请参见**6.1.2.4 SD卡制作系统镜像**的介绍说明。

 版权所有 © 华为技术有限公司 

# 13.3 其他工具介绍

## 13.3.1 GPIO 工具介绍

### 13.3.1.1 概述

gpio_operate工具用于设置GPIO管脚的输入与输出方向，也可将每个GPIO管脚独立的设为0或1。

### 13.3.1.2 使用指南

- gpio_operate命令必须使用root用户执行。详细请参见**14.1 使用PuTTY登录设备（网口方式）**。
- Atlas 200I A2 加速模块当前不支持查询和设置**第6组**和**第8组**的GPIO管脚。

### 13.3.1.3 使用示例

#### 13.3.1.3.1 获取 GPIO 帮助信息

#### 命令功能

**gpio_operate -h**用于获取gpio_operate命令帮助信息。

#### 参数说明

无

#### 使用实例

获取gpio_operate命令帮助信息。

```
gpio_operate -h
Usage: gpio_operate <Command|-h> [Options...]
gpio_operate Command:
        -h                      : This command's help information.
        set_value               : Set gpio pin value.
        get_value               : Get gpio pin value.
        set_direction           : Set gpio pin direction value.
        get_direction           : Get gpio pin direction value.
```

#### 13.3.1.3.2 查询 GPIO 管脚值

#### 命令功能

**gpio_operate get_value** *gpio_group gpio_pin*用于查询GPIO管脚值。


版权所有 © 华为技术有限公司

## 参数说明

| 类型 | 描述 |
| --- | --- |
| *gpio_group* | GPIO组号，取值为[0，8] |
| *gpio_pin* | GPIO管脚号，取值为[0，31] |

## 使用实例

查询第0组第2个管脚的值。

```
gpio_operate get_value 0 2
Get gpio pin value successed, value is 1.
```

### 13.3.1.3.3 设置 GPIO 管脚值

## 命令功能

**gpio_operate set_value** *gpio_group gpio_pin value*用于设置GPIO管脚值。

## 参数说明

| 类型 | 描述 |
| --- | --- |
| *gpio_group* | GPIO组号，取值为[0，8] |
| *gpio_pin* | GPIO管脚号，取值为[0，31] |
| *value* | GPIO管脚值，取值为[0，1]<br>当GPIO管脚方向为输入方向时，不允许设置GPIO管脚值。 |

## 使用实例

设置第0组第2个管脚的值为0。

```
gpio_operate set_value 0 2 0
Set gpio pin value successed.
```

### 13.3.1.3.4 查询 GPIO 管脚方向

## 命令功能

**gpio_operate get_direction** *gpio_group gpio_pin*用于查询GPIO管脚方向。

## 参数说明

| 类型 | 描述 |
| --- | --- |
| *gpio_group* | GPIO组号，取值为[0，8] |


版权所有 © 华为技术有限公司

| 类型 | 描述 |
| --- | --- |
| *gpio_pin* | GPIO管脚号，取值为[0，31] |

## 使用实例

查询第0组第2个管脚的GPIO方向。

```
gpio_operate get_direction 0 2
Get gpio pin direction value successed, value is 0.
```

**表 13-1** 打印信息说明

| 字段 | 说明 |
| --- | --- |
| direction | GPIO管脚方向，取值为[0，1]<br>● 0：输入方向<br>● 1：输出方向 |

### 13.3.1.3.5 设置 GPIO 管脚方向

## 命令功能

**gpio_operate set_direction** *gpio_group gpio_pin direction*用于设置GPIO管脚方向。

## 参数说明

| 类型 | 描述 |
| --- | --- |
| *gpio_group* | GPIO组号，取值为[0，8] |
| *gpio_pin* | GPIO管脚号，取值为[0，31] |
| *direction* | GPIO管脚方向，取值为[0，1]<br>● 0：输入方向<br>● 1：输出方向 |

## 使用实例

设置第0组第2个管脚的GPIO方向为输出方向。

```
gpio_operate set_direction 0 2 1
Set gpio pin direction value successed.
```

 版权所有 © 华为技术有限公司 

# 13.3.2 vrd-tool 工具介绍

## 命令功能

vrd-tool工具用于升级vrd器件，包括vrd信息查询，vrd固件升级和生效。

**⚠ 注意**

vrd器件控制芯片电源，升级过程中请勿将设备断电，断电可能会导致产品无法启动。

## 命令格式

**vrd-tool -h**用于显示查询命令帮助信息。

**vrd-tool --help**用于显示查询命令帮助信息。

**vrd-tool -v**用于显示vrd-tool工具版本信息。

**vrd-tool info**用于查询vrd固件相关信息。

**vrd-tool upgrade -h**用于显示查询vrd升级命令帮助信息。

**vrd-tool upgrade -f** *file*用于升级vrd固件文件。

**vrd-tool upgrade -a**用于生效已升级的vrd固件文件。

**说明**

**vrd-tool upgrade -a**此命令会导致设备重启，请谨慎操作。

## 参数说明

| 类型 | 描述 |
| --- | --- |
| -h/--help | 帮助信息 |
| -v | vrd-tool工具版本信息 |
| info | vrd固件信息 |
| upgrade -h | 升级命令帮助信息 |
| upgrade -f *file* | 升级vrd固件文件 |
| upgrade -a | 生效vrd固件文件 |

## 注意事项

vrd-tool工具不支持在非root用户和容器下运行。

vrd-tool工具需要在主区使用，具体切换操作请参见**13.3.3 boot_tool工具介绍**。


版权所有 © 华为技术有限公司

## 使用示例

此处以Atlas 200I DK A2 开发者套件为例。

**步骤1** 登录Atlas 200I DK A2 开发者套件系统环境。

**步骤2** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的工具包“vrdtool.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤3** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

**步骤4** 执行如下命令，解压源码包“vrdtool.tar.gz”。

**tar -xzvf vrdtool.tar.gz**

**步骤5** 执行如下命令，进入工具包目录。

**cd vrdtool**

**步骤6** 执行如下命令，增加操作权限。

**chmod +x vrd-tool**

**步骤7** 执行如下命令，查看设备的vrd器件类型。

**./vrd-tool info**

显示如下回显，vrd器件类型为MPS。

```
supplier                : MPS
```

**步骤8** 执行如下命令，升级MPS类型的单个vrd固件文件。

**./vrd-tool upgrade -f** */opt/vrdtool/msp_firmware/V01_IT22MMDA_MPS_MPS_U12_0X60_AI-DVFS_0V9_0V8-CORE_0V825.txt*

出现如下回显，表示升级成功。

```
vrd firmware starts to upgrade
The vrd firmware has been upgraded,need to active vrd
```

出现如下回显，表示升级文件被篡改，请重新获取工具包“**vrdtool.tar.gz**”。

```
vrd firmware signature verification failed. Error:-9014
```

**说明**

vrd固件文件有两个，只能对一个vrd固件文件升级和生效后再进行另一个vrd固件文件升级和生效。不能同时升级两个vrd固件文件。

**步骤9** 执行如下命令，生效vrd文件。

**./vrd-tool upgrade -a**

出现如下回显，表示生效成功。

```
The vrd firmware is taking effect
```

**说明**

生效vrd文件会导致产品重启，请谨慎使用。

 版权所有 © 华为技术有限公司 

**步骤10** 待Atlas 200I DK A2 开发者套件启动完成后，执行如下命令，进入工具包目录。

**cd** */opt*/**vrdtool**

**步骤11** 执行如下命令，升级MPS类型的另一个vrd固件文件。

**./vrd-tool upgrade -f** */opt/vrdtool/mps_firmware/V01_IT22MMDA_MPS_MPS_U15_0X6C_CPU-DVFS_0V9.txt*

出现如下回显，表示升级成功。

```
vrd firmware starts to upgrade
The vrd firmware has been upgraded,need to active vrd
```

**说明**

vrd固件文件有两个，只能对一个vrd固件文件升级和生效后再进行另一个vrd固件文件升级和生效。不能同时升级两个vrd固件文件。

**步骤12** 执行如下命令，生效vrd文件。

**./vrd-tool upgrade -a**

出现如下回显，表示生效成功。

```
The vrd firmware is taking effect
```

**说明**

生效vrd文件会导致产品重启，请谨慎使用。

**步骤13** 待Atlas 200I DK A2 开发者套件启动完成后，执行如下命令，查看升级后的版本号。

**./vrd-tool info**

- MPS回显示例如下：

  ```
  supplier                : MPS
  ai_vrd_version          : 1
  ai_vrd_avail_times      : 255
  cpu_vrd_version         : 1
  cpu_vrd_avail_times     : 255
  ```

  MPS中的“ai_vrd_avail_times”和“cpu_vrd_avail_times”数值无实际意义，MPS升级次数不做限制。

- IFX回显示例如下：

  ```
  supplier                : IFX
  ai_vrd_version          : 1
  ai_vrd_avail_times      : 16
  cpu_vrd_version         : 1
  cpu_vrd_avail_times     : 22
  ```

  IFX中的“ai_vrd_avail_times”和“cpu_vrd_avail_times”数值表示剩余的可升级次数。

**----结束**

# 13.3.3 boot_tool 工具介绍

## 命令功能

boot_tool命令用于查询和设置启动相关信息。

 版权所有 © 华为技术有限公司 

## 命令格式

**boot_tool get -h**用于显示查询命令帮助信息。

**boot_tool set -h**用于显示设置命令帮助信息。

**boot_tool -h**用于显示boot_tool命令帮助信息。

**boot_tool get boot_sel**用于查询启动介质信息，例如SD卡。

**boot_tool get partition_map**用于查询启动介质中启动镜像分区信息。

**boot_tool get upgrade_control_info**用于查询启动介质中升级控制信息。

**boot_tool get image_info**用于查询启动介质中启动镜像信息。

**boot_tool set clear_force_recover**用于清除强制恢复出厂标志。

**boot_tool set swap_region**用于主备文件系统场景下切换启动分区。

**boot_tool set clear_reset_count**用于清除CPU启动计数信息。

**boot_tool set force_recover_flag**用于设置强制恢复出厂标志。

## 参数说明

| 类型 | 描述 |
| --- | --- |
| -h | 帮助信息 |
| boot_sel | 启动介质信息 |
| partition_map | 启动介质中启动镜像分区信息 |
| upgrade_control_info | 启动介质中升级控制信息 |
| image_info | 启动介质中启动镜像信息 |
| clear_force_recover | 清除强制恢复出厂标志 |
| swap_region | 主备文件系统场景下切换启动分区 |
| clear_reset_count | 清除CPU启动计数信息 |
| force_recover_flag | 设置强制恢复出厂标志 |

## 使用指南

boot_tool命令必须使用root用户执行。切换root用户请参考**14.1 使用PuTTY登录设备（网口方式）**。

## 使用示例

- 示例1：显示查询命令帮助信息。

  **boot_tool get -h**

  回显示例如下：

  ```
  Usage: boot_tool get Options...
        -h                          : This command's help information.
  ```

 版权所有 © 华为技术有限公司 

```
    boot_sel                    : Query boot media information, such as eMMC, SD, and M.2 SSD.
    partition_map               : Query the boot partition table information.
    upgrade_control_info        : Query upgrade control information.
    image_info                  : Query boot image information.
```

- 示例2：显示设置命令帮助信息。

  **boot_tool set -h**

  回显示例如下：

```
Usage: boot_tool set Options...
    -h                      : This command's help information.
    clear_force_recover         : Clear the forcible factory restoration flag.
    swap_region                 : Switch the boot partition in the active/standby file system scenario.
    clear_reset_count           : Clear reset count.
    force_recover_flag          : Set the forcible factory restoration flag.
```

- 示例3：显示boot_tool命令帮助信息。

  **boot_tool -h**

  回显示例如下：

```
Usage: boot_tool <Command|-h> [Options...]
boot_tool Command:
    -h                      : This command's help information.
    get                     : Query boot information.
    set                     : Set boot information.
```

- 示例4：查询启动介质信息。

  **boot_tool get boot_sel**

  回显示例如下：

```
boot from SD
```

- 示例5：查询启动介质中启动镜像分区信息。

  **boot_tool get partition_map**

  回显示例如下：

```
partition map info  :
partition count     : 7
partition map       : 0xdf
```

- 示例6：查询启动介质中升级控制信息。

  **boot_tool get upgrade_control_info**

  回显示例如下：

```
boot upgrade control info     :
force_recover_flag          : 0x0
upgrade_part_count          : 4
################################################################
RAW_DATA     control info     :
upgrade_type                : 0
upgrade_status              : 0x0
upgrade_part_flag           : 0x0
################################################################
RECOVER      control info     :
upgrade_type                : 0
upgrade_status              : 0x0
upgrade_part_flag           : 0x0
################################################################
RECOVER_DATA control info     :
upgrade_type                : 0
upgrade_status              : 0x0
upgrade_part_flag           : 0x0
################################################################
ROOTFS       control info     :
upgrade_type                : 0
upgrade_status              : 0x0
```

 版权所有 © 华为技术有限公司 

```
upgrade_part_flag            : 0x0
#################################################################
```

- 示例7：查询启动介质中启动镜像信息。

  **boot_tool get image_info**

  回显示例如下：

```
#################################################################
boot_a   info      :
partition_name     : /dev/mmcblk1p1
component_count    : 3
component_type     : 0
component_name     : kernel
data_size          : 29098496
component_type     : 1
component_name     : dtb
data_size          : 528384
component_type     : 2
component_name     : tee
data_size          : 1831100
#################################################################
boot_b   info      :
partition_name     : /dev/mmcblk1p1
component_count    : 3
component_type     : 0
component_name     : kernel
data_size          : 29098496
component_type     : 1
component_name     : dtb
data_size          : 528384
component_type     : 2
component_name     : tee
data_size          : 1831100
#################################################################
```

- 示例8：清除强制恢复出厂标志。

  **boot_tool set clear_force_recover**

  回显示例如下：

```
Clear force recover flag success.
```

- 示例9：主备文件系统场景下切换启动分区。

  **boot_tool set swap_region**

  回显示例如下：

```
Set switch_region ok.
```

- 示例10：清除CPU启动计数信息。

  **boot_tool set clear_reset_count**

  回显示例如下：

```
Set clear_reset_count ok.
```

- 示例11：设置强制恢复出厂标志。

  **boot_tool set force_recover_flag**

  回显示例如下：

```
The device does not support this operation.
```

  **说明**

  当前不支持设置强制恢复出厂标志。

 版权所有 © 华为技术有限公司 

# 13.3.4 efuse-tool 工具介绍

## 命令功能

efuse-tool工具用于烧写熔丝器件，包括检查对应域段熔丝是否烧写，烧写是否正常以及对开放域段熔丝进行烧写。

**⚠ 注意**

熔丝器件一旦烧写无法回退，使用efuse-tool工具前请确认是否需要烧录对应功能域段，请谨慎操作，避免导致产品无法恢复到烧写前的功能，影响正常使用。

## 命令格式

**efuse-tool -h**用于查询命令帮助信息。

**efuse-tool 0 check -hboot2** 用于查询hboot2域段是否被烧写。

**efuse-tool 0 burn -hboot2** 用于烧写hboot2域段。

## 参数说明

| 类型 | 参数 | 描述 |
| --- | --- | --- |
| device_id | 0 | 设备ID |
| option | -hboot2 | 开放域段参数 |
| - | -h | 帮助信息 |
| - | check | 检查参数 |
| - | burn | 烧写参数 |

## 注意事项

efuse-tool工具不支持在非root用户、虚拟机和容器下运行。

## 使用示例

此处以Atlas 200I DK A2 开发者套件为例。

**步骤1** 登录Atlas 200I DK A2 开发者套件系统环境。

**步骤2** 使用WinSCP工具或其他工具，将**4.1 下载软件包**获取的工具包“efuse-tool.tar.gz”上传至root用户属组目录下，例如“/opt”。详细操作请参见**14.3 使用WinSCP传输文件**。

**步骤3** 执行如下命令，进入源码包所在目录“/opt”。

**cd** */opt*

 版权所有 © 华为技术有限公司 

**步骤4** 执行如下命令，解压源码包“efuse-tool.tar.gz”。

**tar -xzvf efuse-tool.tar.gz**

**步骤5** 执行如下命令，进入工具包目录。

**cd efusetool**

**步骤6** 执行如下命令，升级固件包。

**./Ascend-hdk-310b-npu-firmware-soc_*x.x.x.x.X*.run --upgrade**

**步骤7** 执行如下命令，升级驱动包。

**./Ascend-hdk-310b-npu-driver-soc_<*version*>_linux-aarch64.run --upgrade**

若有软实时OS需求，该命令为**./Ascend-hdk-310b-npu-driver-soc_<*version*>_linux-rt-aarch64.run --upgrade**。

**步骤8** 执行如下命令，重启Atlas 200I DK A2 开发者套件。

**reboot**

**步骤9** 待Atlas 200I DK A2 开发者套件启动完成后，执行如下命令，进入工具包目录。

**cd** */opt/***efusetool**

**步骤10** 执行如下命令，查看Atlas 200I DK A2 开发者套件的“hboot2”域段是否被烧写。

**./efuse-tool 0 check -hboot2**

出现如下回显，表示域段正常且未烧写。

```
The domain segment has not been burned
```

**步骤11** Atlas 200I DK A2 开发者套件的“hboot2”域段未被烧写时，执行如下命令进行烧写。

**./efuse-tool 0 burn -hboot2**

出现如下回显，表示域段烧写成功。

```
The domain segment has been burned
```

**步骤12** 参见**9.1 升级前必读~9.4 升级固件**，将驱动与固件版本升级到客户使用的版本，保证业务正常使用。

**----结束**

 版权所有 © 华为技术有限公司 

# 14 常用操作

14.1 使用PuTTY登录设备（网口方式）
14.2 使用PuTTY登录设备（串口方式）
14.3 使用WinSCP传输文件

## 14.1 使用 PuTTY 登录设备（网口方式）

### 操作场景

使用PuTTY工具，可以通过局域网远程访问设备，对设备实施配置、维护操作。

### 必备事项

**前提条件**

已通过网线连接PC与设备的管理网口。

**数据**

需准备如下数据。

- 待连接设备的IP地址
- 登录待连接设备的用户名和密码

**软件**

PuTTY.exe：此工具为第三方软件。

### 操作步骤

**步骤1** 设置PC机的IP地址、子网掩码或者路由，使PC机能和设备网络互通。

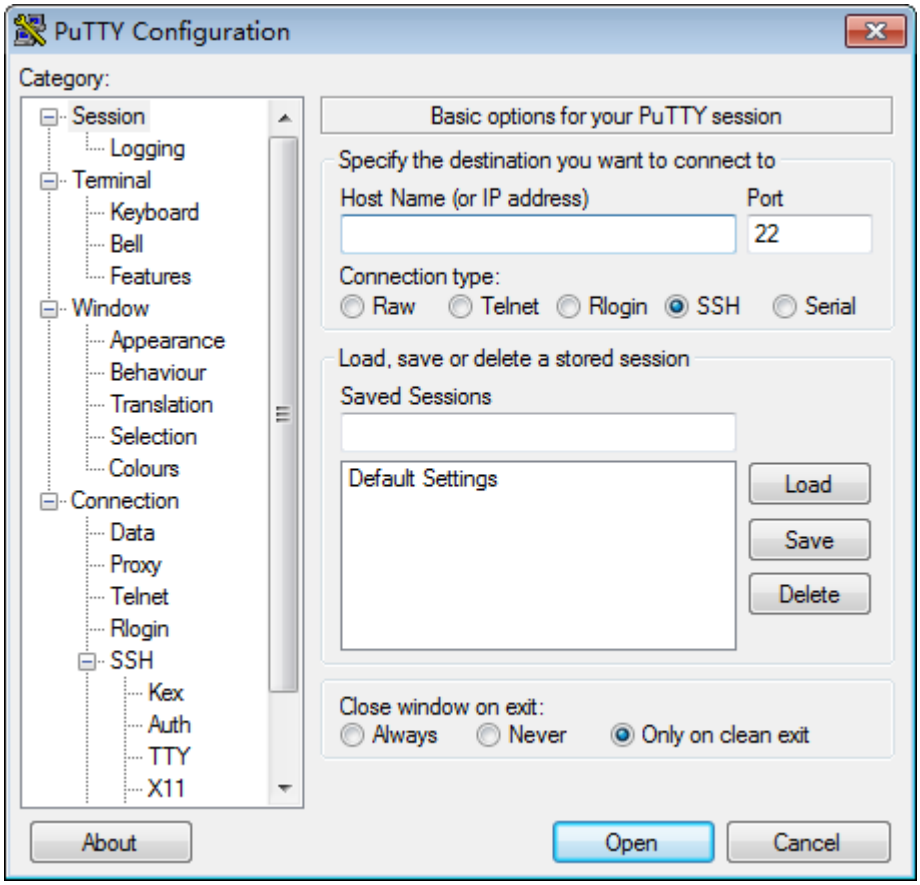
可在PC机的cmd命令窗口，通过**Ping *设备IP地址***命令，检查网络是否互通。

**步骤2** 双击“PuTTY.exe”。

弹出“PuTTY Configuration”窗口，如**图14-1**所示。

 版权所有 © 华为技术有限公司 

**图 14-1** PuTTY Configuration




**步骤3** 填写登录参数。

参数说明如下。

- Host Name（or IP address）：输入要登录设备的IP地址，Atlas 200I A2 加速模块的默认IP地址为**192.168.0.2**。
- Port：默认设置为“22”。
- Connection type：默认选择“SSH”。
- Close window on exit：默认选择“Only on clean exit”。

**说明**

配置“Host Name”后，再配置“Saved Sessions”并单击“Save”保存，则后续使用时直接双击“Saved Sessions”下保存的记录即可登录设备。

**步骤4** 单击“Open”。

进入“PuTTY”运行界面，提示“login as:”，等待用户输入用户名。

**说明**

- 如果首次登录该目标设备，则会弹出“PuTTY Security Alert”窗口。单击“是”表示信任此站点，进入“PuTTY”运行界面。
- 登录设备时，如果帐号输入错误，必须重新连接PuTTY。

**步骤5** 按提示分别输入用户名和密码。默认用户名和密码请参见《**Atlas 系列硬件产品 账户清单**》。

 版权所有 © 华为技术有限公司 

- 首次登录时，先使用HwHiAiUser登录，再执行**su - root**切换为root用户，并执行**passwd root**修改初始密码。
- 非首次登录，可使用HwHiAiUser或root用户登录。

登录完成后，命令提示符左侧显示出当前登录服务器的主机名。

**说明**

为保证系统安全性，请您在首次操作时修改初始密码，并定期更新。

**----结束**

# 14.2 使用 PuTTY 登录设备（串口方式）

## 必备事项

**前提条件**

- 已通过串口线缆连接PC与设备。
- 已经安装最新版本的PuTTY。

**数据**

登录待连接设备的用户名和密码。

**软件**

PuTTY.exe：此工具为第三方软件。要求PuTTY能支持串口登录。

## 操作步骤

**步骤1** 查看串口编号。

1. 打开PC机的“设备管理器”窗口。
2. 单击“端口”查看连接Atlas 200I A2 加速模块的串口编号。
   其中COM1、COM2和COM3为系统自带串口。

**步骤2** 双击“PuTTY.exe”。

弹出“PuTTY Configuration”窗口。

**步骤3** 在左侧导航树中选择“Connection > Serial”。

**步骤4** 设置如下登录参数，其他参数保持默认即可。

- Serial Line to connect to：COM*n*（**步骤1**中获取的串口编号）
- Speed（baud）：115200

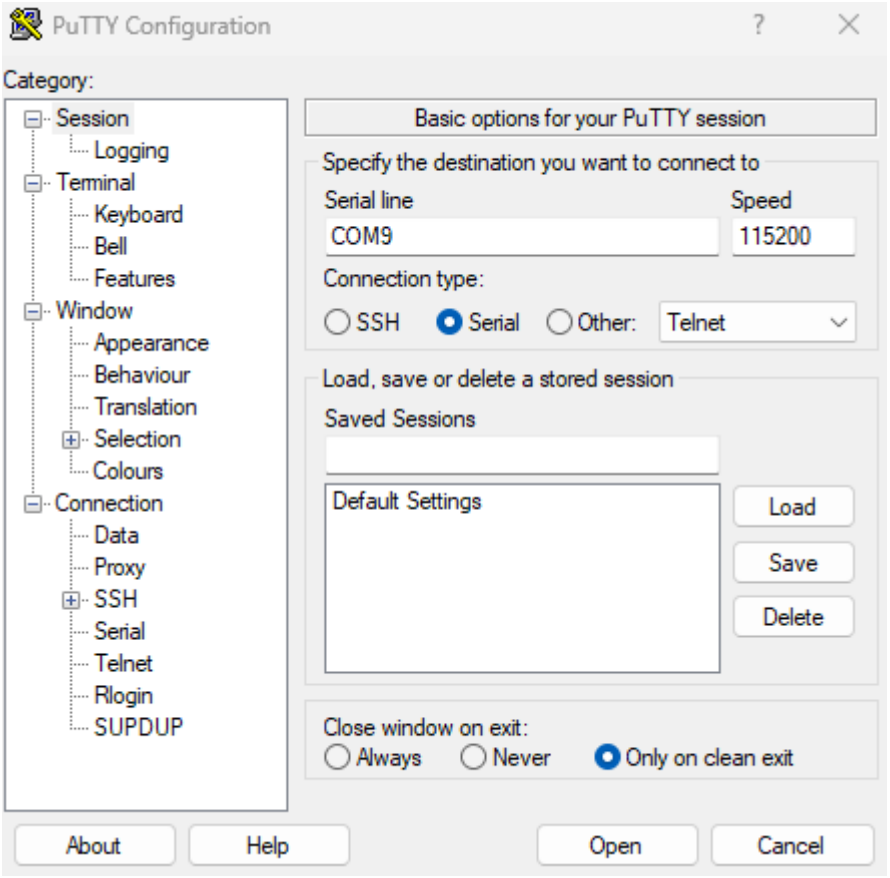
*n*表示不同串口的编号，取值为整数。

**步骤5** 在左侧导航树中选择“Session”。

**步骤6** 选择“Connection type”为“Serial”，如**图14-2**所示。

 版权所有 © 华为技术有限公司 

**图 14-2** PuTTY Configuration




**步骤7** 单击“Open”。

进入“PuTTY”运行界面，提示“login as:”，等待用户输入用户名。

**步骤8** 按提示分别输入用户名和密码。默认用户名和密码请参见《**Atlas 系列硬件产品 账户清单**》。

- 首次登录时，先使用HwHiAiUser登录，再执行**su - root**切换为root用户，并执行**passwd root**修改初始密码。
- 非首次登录，可使用HwHiAiUser或root用户登录。

登录完成后，命令提示符左侧显示出当前登录服务器的主机名。

**说明**

为保证系统安全性，请您在首次操作时修改初始密码，并定期更新。

**----结束**

# 14.3 使用 WinSCP 传输文件

## 操作场景

在PC机上使用WinSCP工具进行文件传输。

 版权所有 © 华为技术有限公司 

## 必备事项

**前提条件**

目的设备已开启SFTP服务。

**数据**

需准备如下数据。

- 待连接服务器的IP地址
- 登录待连接服务器的用户名和密码

**软件**

WinSCP.exe：此工具为第三方软件，请自行准备。

## 操作步骤

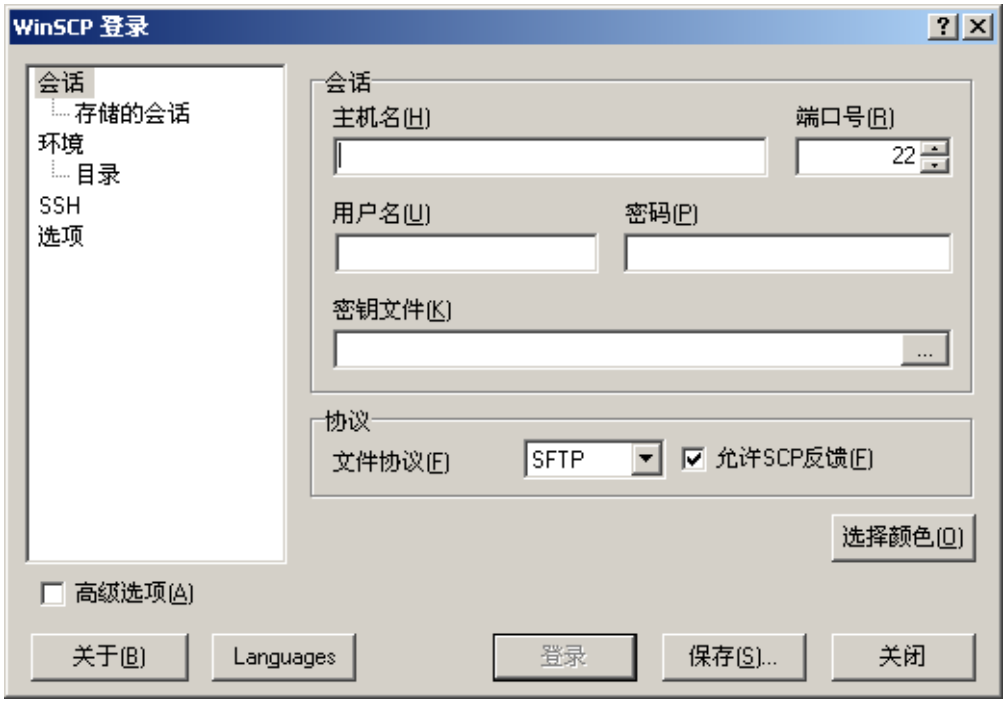
**步骤1** 打开“WinSCP”文件夹，双击“WinSCP.exe”。

弹出“WinSCP登录”对话框，如图14-3所示。

**说明**

若系统非中文操作系统，可以单击“Languages”进行界面语言的选择。

**图 14-3** WinSCP 登录




**步骤2** 单击“会话”。

**步骤3** 设置登录参数。

参数说明如下。

 版权所有 © 华为技术有限公司 

- 主机名（H）：待连接设备的IP地址，例如“192.168.2.10”。
- 端口号（R）：默认为“22”。
- 用户名（U）：待连接设备的操作系统用户名，例如“admin123”。
- 密码（P）：待连接设备的操作系统用户的密码，例如“admin123”。
- 密钥文件（K）：默认为空，保留默认值。
- 协议：选择默认文件协议“SFTP”，并勾选“允许SCP反馈（F）”。

**步骤4** 单击“登录”。

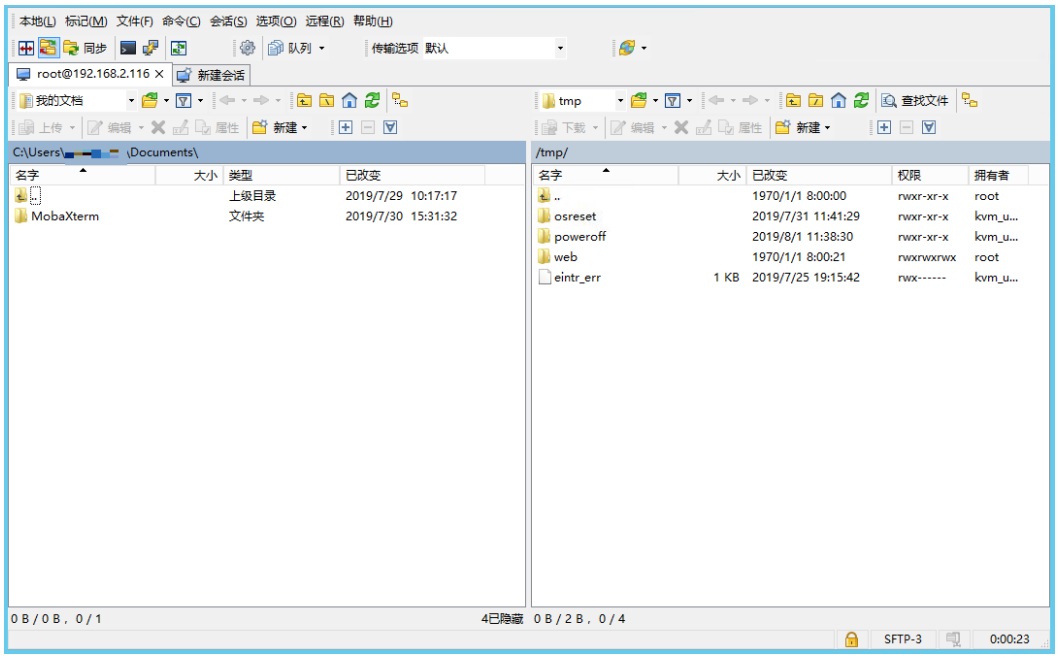
进入“WinSCP”文件传输界面。

**说明**

如果首次登录时没有选择密钥文件，此时会弹出一个警告提示框，询问“是否连接并添加密钥到缓存？”，单击“是（Y）”，进入“WinSCP”文件传输界面。

**步骤5** 根据实际需求，在界面左右区的指定目录中进行文件夹的创建、删除和复制等操作。

**图 14-4** WinSCP 界面




**说明**

界面左侧区域为本地PC的目录，右侧区域为已连接设备的目录。

**----结束**

 版权所有 © 华为技术有限公司 

# 15 FAQ

15.1 使用root用户SSH登录服务器出现Access Denied错误
15.2 驱动升级过程中报错显示无法删除文件夹
15.3 emmc-head工具不可用
15.4 更新网络源时签名不可用
15.5 制卡过程中出现“[ERROR] Can not get disk, please use fdisk -l to check available disk name!”的报错
15.6 运行程序超内存导致设备异常挂死，设备能ping通，但SSH无法远程连接
15.7 系统启动时在加载PCIe设备处卡死，无法正常启动
15.8 SD卡作为系统启动介质时，启动过程中显示挂载分区失败

## 15.1 使用 root 用户 SSH 登录服务器出现 Access Denied 错误

### 问题描述

使用root用户SSH登录服务器出现“Access Denied”错误。

### 原因分析

sshd_config配置文件中PermitRootLogin字段的值为no，不允许远程登录。

### 解决方法

**步骤1** 使用HwHiAiUser帐号登录OS后台。

**步骤2** 执行以下命令，切换root账户。

**su root**

输入默认密码，默认密码请参见《**Atlas 系列硬件产品 账户清单**》。

 版权所有 © 华为技术有限公司 

**步骤3** 执行以下命令，修改sshd_config文件。

**vim /etc/ssh/sshd_config**

在该行后添加一行PermitRootLogin yes。

```
#PermitRootLogin prohibit-password
PermitRootLogin yes
```

配置完成后，按“Esc”键，再执行**:wq!**保存修改，并按“Enter”键退出。

**步骤4** 执行如下命令，重启ssh。

```
/etc/init.d/ssh restart
```

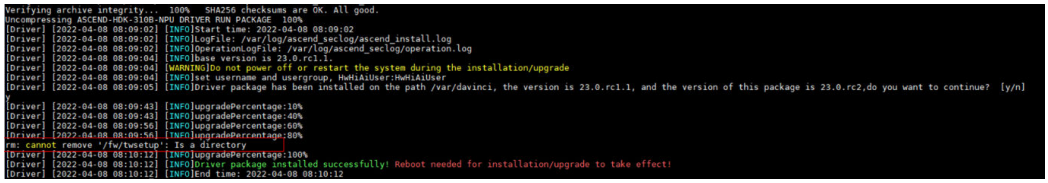
**步骤5** 重新使用ssh远程连接。

**----结束**

# 15.2 驱动升级过程中报错显示无法删除文件夹

## 问题描述

驱动升级过程中报错显示无法删除文件夹。




## 原因分析

驱动升级过程中会使用**rm -f**命令删除“/fw”目录下的所有文件，并进行重装，故该目录下若创建文件夹，则会出现无法删除文件夹报错，类似报错为同一原因。

## 解决方法

该报错不影响驱动升级，可忽略该报错，或者不在该目录下创建文件夹即可。

# 15.3 emmc-head 工具不可用

## 问题一

检查emmc-head工具时提示-bash: ./emmc-head: No such file or directory。

**原因分析**

emmc-head工具需要使用“/lib64/”目录下的ld-linux-aarch64.so.1文件，当前环境的“/lib64/”目录下缺少此文件。

**解决方案**

- Arm环境
  执行如下命令，复制ld-linux-aarch64.so.1到“/lib64/”目录下。

 版权所有 © 华为技术有限公司 

**cp /lib/ld-linux-aarch64.so.1 /lib64/**

- X86环境

  **cp /usr/aarch64-linux-gnu/lib/ld-linux-aarch64.so.1 /lib64/**

**说明**

- 若当前环境没有“/lib64/”目录，可以执行mkdir -p /lib64命令创建。
- 若“/lib/”目录下无ld-linux-aarch64.so.1文件，可以使用find / -name ld-linux-aarch64.so.1命令查找。

## 问题二

检查emmc-head工具时提示./emmc-head: error while loading shared libraries: libc.so.6: cannot open shared object file: No such file or directory。

**原因分析**

若为emmc-head工具需要使用“/lib”目录下的libc.so.6文件，当前环境的“/lib/”目录下缺少此文件。

**解决方案**

- Arm环境

  执行如下命令，复制libc.so.6到“/lib/”目录下。

  **cp /usr/lib/aarch64-linux-gnu/libc.so.6 /lib/**

- X86环境

  **cp /usr/aarch64-linux-gnu/lib/libc.so.6 /lib/**

**说明**

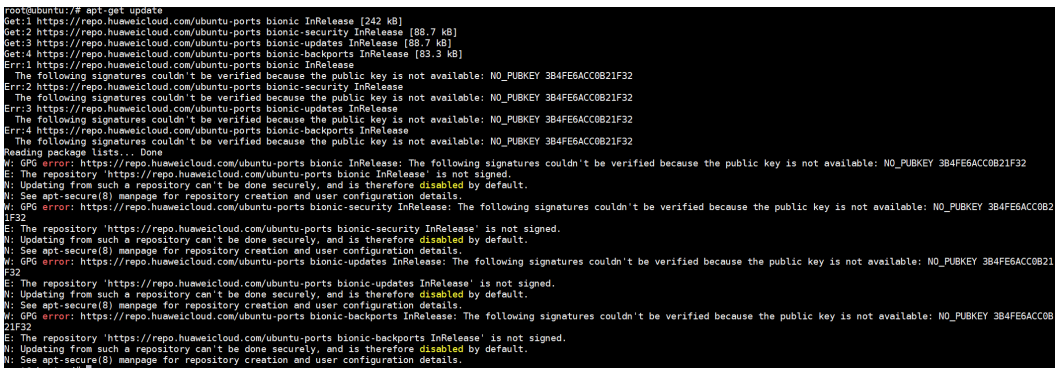
- 若当前环境没有“/lib/”目录，可以执行mkdir -p /lib命令创建。
- 若“/lib/”目录下无libc.so.6文件，可以使用find / -name libc.so.6命令查找。

# 15.4 更新网络源时签名不可用

## 问题现象

更新网络源时报错“The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 3B4FE6ACC0B21F32”，如图15-1所示。

**图 15-1** 网络源更新报错




 版权所有 © 华为技术有限公司 

## 原因分析

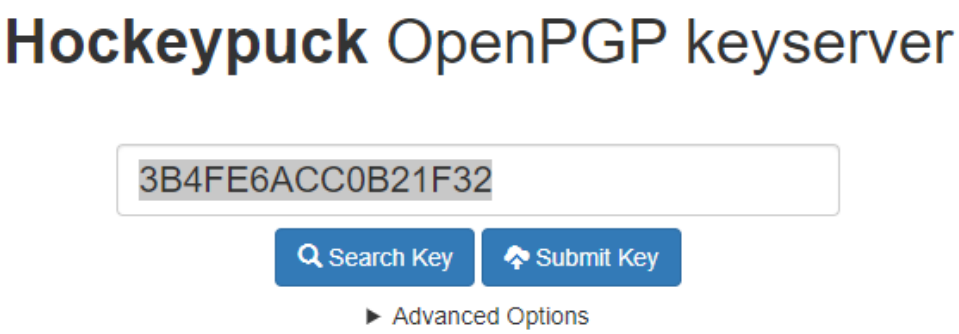
公钥不可用导致签名无法被验证，所以无法安全地从此网络源进行更新，根据安全保护原则，默认情况下禁用更新。

## 解决措施

**步骤1** 进入网站**http://keyserver.ubuntu.com/**。

**步骤2** 输入“3B4FE6ACC0B21F32”，单击“Search key”，如**图15-2**所示。

**图 15-2** 秘钥服务器界面




**步骤3** 单击如下红框，查看秘钥内容。




**步骤4** 回到部署环境，执行**exit**命令退出当前根系统，在Ubuntu 22.04镜像目录下执行命令创建“key3B4FE6ACC0B21F32”文档。

**vim key3B4FE6ACC0B21F32**

将**步骤3**的秘钥内容复制到文档中，保存并退出。

**步骤5** 执行以下命令，将“key3B4FE6ACC0B21F32”文件复制到“/etc/apt/”目录下。

**cp key3B4FE6ACC0B21F32 squashfs-root/etc/apt/**

**步骤6** 执行以下命令，切换根系统。

**chroot squashfs-root**

**步骤7** 执行以下命令，添加秘钥。

**apt-key add /etc/apt/key3B4FE6ACC0B21F32**

提示“OK”表示成功。

**步骤8** 重新执行apt-get update命令更新网络源。

**----结束**

 版权所有 © 华为技术有限公司 

# 15.5 制卡过程中出现“[ERROR] Can not get disk, please use fdisk -l to check available disk name!”的报错

## 问题描述

制卡过程中出现“[ERROR] Can not get disk, please use fdisk -l to check available disk name!”的报错，相关打印信息如图15-3所示。

**图 15-3** 制卡打印

```
root@ubuntu-59:~/Hi1911# python3 make_sd_card.py local /dev/sdd
Begin to make SD Card...
[ERROR] Can not get disk, please use fdisk -l to check available disk name!
root@ubuntu-59:~/Hi1911#
```

## 可能原因

1. 在非root用户下进行制卡操作。
2. 用户输入的盘符不存在。
3. 用户使用的linux版本安装语言或终端输出为中文，该场景下制作脚本无法匹配中文相关字段。相关打印信息如图15-4所示。

**图 15-4** 打印信息

```
[root@localhost ~]# fdisk -l
WARNING: fdisk GPT support is currently new, and therefore in an experimental phase. Use at your own discretion.

磁盘 /dev/sda：268.4 GB, 268435456000 字节，524288000 个扇区
Units = 扇区 of 1 * 512 = 512 bytes
扇区大小(逻辑/物理)：512 字节 / 512 字节
I/O 大小(最小/最佳)：512 字节 / 512 字节
磁盘标签类型：gpt
Disk identifier: 1FC965F3-8CE0-4960-88FE-33906D6F816B


#         Start          End    Size  Type            Name
 1         2048       411647    200M  EFI System      EFI System Partition
 2       411648      2508799      1G  Microsoft basic
 3      2508800    524285951  248.8G  Linux LVM

磁盘 /dev/mapper/nlas-root：53.7 GB, 53687091200 字节，104857600 个扇区
Units = 扇区 of 1 * 512 = 512 bytes
扇区大小(逻辑/物理)：512 字节 / 512 字节
I/O 大小(最小/最佳)：512 字节 / 512 字节
```

## 解决方案

对于可能原因1，切换到root用户进行制卡。

对于可能原因2，执行**fdisk -l**确认正确盘符后（如/dev/sdb），重新制卡。

对于可能原因3，可按如下操作解决。

**步骤1** 执行如下命令，确认当前环境语言。

**echo $LANG**

显示如下，表示当前环境语言为中文。

```
[root@localhost ~]# echo $LANG
zh_CN.UTF-8
```


版权所有 © 华为技术有限公司

**步骤2** 执行如下命令，将当前终端的环境语言临时修改为英文。

**LANG=en_US**

**步骤3** 执行如下命令，查看当前环境语言是否修改成功。

**fdisk -l**

显示如下，表示当前环境语言修改为英文。

```
[root@localhost ~]# echo $LANG
zh_CN.UTF-8
[root@localhost ~]# LANG=en_US
[root@localhost ~]# fdisk -l
WARNING: fdisk GPT support is currently new, and therefore in an experimental phase. Use at your own discretion.

Disk /dev/sda: 268.4 GB, 268435456000 bytes, 524288000 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: gpt
Disk identifier: 1FC965F3-8CE0-4960-88FE-33906D6F816B


#         Start          End    Size  Type            Name
 1         2048       411647    200M  EFI System      EFI System Partition
 2       411648      2508799      1G  Microsoft basic
 3      2508800    524285951  248.8G  Linux LVM
```

**----结束**

# 15.6 运行程序超内存导致设备异常挂死，设备能 ping 通，但 SSH 无法远程连接

## 问题描述

运行程序超内存导致设备异常挂死，设备能ping通，但SSH无法远程连接。

## 原因分析

通过串口连接后，实时运行大内存业务程序，设备异常挂死前日志循环打印“OOM type”，分析定位为内存不足，运行业务超系统设置的内存限额。

## 解决方法

**步骤1** 参考**6.1.2.4 SD卡制作系统镜像**制卡。

**步骤2** 设置OOM内存管理机制。

**说明**

配置方法重启后失效，用户可以根据业务诉求进行持久化配置。

1. 执行如下命令，启用enable_oom_killer。
   1表示启用，0表示禁用。
   **echo *1* > /proc/sys/vm/enable_oom_killer**
2. 执行如下命令，启用memory cgroup的OOM机制。
   0表示启用，1表示禁用。
   **echo *0* > /sys/fs/cgroup/memory/usermemory/memory.oom_control**
3. 执行如下命令，查看OOM内存设置状态。
   **cat /proc/sys/vm/enable_oom_killer**

 版权所有 © 华为技术有限公司 

```
1
```

**cat /sys/fs/cgroup/memory/usermemory/memory.oom_control**

```
oom_kill_disable 0
under_oom 0
oom_kill 0
oom_kill_local 0
```

**----结束**

# 15.7 系统启动时在加载 PCIe 设备处卡死，无法正常启动

## 问题描述

系统启动时在加载PCIe设备处卡死，无法正常启动。

```
NOTICE:  [domain on]:mpidr 100
NOTICE:  [domain on]:mpidr 200
NOTICE:  [domain on]:mpidr 300
[    0.010240] KASLR disabled due to lack of seed
[    0.097474] Initramfs unpacking failed: invalid magic at start of compressed archive
[    0.187344] pinctrl-single 82320000.pinmux: please update dts to use #pinctrl-cells = <1>
[    0.195802] pinctrl-single c4000000.pinmux: please update dts to use #pinctrl-cells = <1>
[    0.204253] pinctrl-single 400140000.pinmux: please update dts to use #pinctrl-cells = <1>
[    0.212736] pinctrl-single 3001d0000.pinmux: please update dts to use #pinctrl-cells = <1>
[    0.222091] pci_bus 0000:00: Unknown NUMA node; performance will be reduced
[    0.230911] pci_bus 0001:08: Unknown NUMA node; performance will be reduced
[   60.241236] rcu: INFO: rcu_sched detected stalls on CPUs/tasks:
[   60.247148] rcu:     3-...0: (0 ticks this GP) idle=76e/1/0x4000000000000000 softirq=14/14 fqs=5656
[   60.256006] rcu:     (detected by 0, t=15002 jiffies, g=-1171, q=11)
[  240.261236] rcu: INFO: rcu_sched detected stalls on CPUs/tasks:
[  240.267145] rcu:     3-...0: (0 ticks this GP) idle=76e/1/0x4000000000000000 softirq=14/14 fqs=21117
[  240.276089] rcu:     (detected by 0, t=60007 jiffies, g=-1171, q=11)
[  420.281237] rcu: INFO: rcu_sched detected stalls on CPUs/tasks:
[  420.287147] rcu:     3-...0: (0 ticks this GP) idle=76e/1/0x4000000000000000 softirq=14/14 fqs=36328
[  420.296092] rcu:     (detected by 2, t=105015 jiffies, g=-1171, q=13)
```

## 原因分析

系统UserBaseConfig与DTB文件中PCIe控制器未配置正确。

## 解决方法

参考**7.1.2 PCIe接口调测参考**正确配置PCIe。

# 15.8 SD 卡作为系统启动介质时，启动过程中显示挂载分区失败

## 问题描述

SD卡作为系统启动介质时，启动过程中显示挂载分区失败。


版权所有 © 华为技术有限公司

```
[    2.577737] JBD2: journal transaction 52285 on mmcblk1p2-8 is corrupt.
[    2.587382] EXT4-fs (mmcblk1p2): error loading journal
[    2.594463] VFS: Cannot open root device "mmcblk1p2" or unknown-block(179,2): error -117
[    2.602551] Please append a correct "root=" boot option; here are the available partitions:
[    2.610895] b300        31166976 mmcblk1
[    2.610896]  driver: mmcblk
[    2.617678]   b301          102400 mmcblk1p1 c0c5ab94-9d39-4941-9371-6710befec583
[    2.617679]
[    2.626628]   b302         6291456 mmcblk1p2 40ccfb74-b8f1-4265-8a74-22dbc70d2ad7
[    2.626629]
[    2.635579]   b303          102400 mmcblk1p3 f9881427-1003-4593-92ca-6465f0bb9a8f
[    2.635580]
[    2.644529]   b304         1048576 mmcblk1p4 cff831cb-ef1d-4773-80f0-84ed39b3f54f
[    2.644530]
[    2.653485]   b305         1048576 mmcblk1p5 8f735693-9615-453d-b84b-911f800c7415
[    2.653486]
[    2.662438] Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(179,2)
```

## 原因分析

SD卡损坏，无法正确的挂载分区。

## 解决方法

更换SD卡，参考**5.2 烧写镜像**重新制卡。


版权所有 © 华为技术有限公司

# A 附录

## A.1 用户信息

Atlas 200I A2 加速模块烧录镜像默认用户信息如表A-1所示。

**表 A-1** 用户信息

| 用户组 | 用户 | 密码 |
| --- | --- | --- |
| root | root | Mind@123 |
| HwHiAiUser | HwHiAiUser | Mind@123 |
| HwSysUser | HwSysUser | Mind@123 |
| HwDmUser | HwDmUser | Mind@123 |
| HwBaseUser | HwBaseUser | Mind@123 |

HwDmUser用户用于运行设备管理相关的进程，HwSysUser用户用于运行不涉及外部数据处理的进程，其它进程通过系统默认生成的HwHiAiUser用户运行。HwDmUser用户和HwHiAiUser用户存在一些公共组件，这些组件的文件属主会设置为HwBaseUser用户。

如果需要增加新的业务用户和用户组，需要将该业务用户添加到自定义用户组外，还需要添加到HwHiAiUser、HwBaseUser、HwDmUser三个用户组。如果只是增加新的用户，只需要将新用户添加到HwHiAiUser、HwBaseUser、HwDmUser三个用户组。

```
useradd -g <用户自定义组> <guest用户名>
usermod -aG HwHiAiUser <guest用户名>
usermod -aG HwBaseUser <guest用户名>
usermod -aG HwDmUser <guest用户名>
```


版权所有 © 华为技术有限公司

# A.2 自定义配置 user_base_config.xml 文件参数说明

## 配置 serdes 参数

```
<item name="product_strategy_info"><!--产品板策略ID-->
<!--33150 对应十六进制817e-->
<param board_id ="817e" phy_num = "1" chip0_strategy="0" max_power="0"  equal="0"/>
</item>
```

**表 A-2** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| board_id | 如33150对应十六进制817e。 | 十六进制的boardid。<br>boardid为用户实际使用底板的adc_board_id与Atlas 200I A2 加速模块的adc_board_id的组合值。 |
| phy_num | 1。 | 多p系统才会使用，固定为1。 |
| chip0_strategy | 0~n。 | 策略索引号。对应代码中的sub_strategy0和sub_strategy1。 |
| max_power | • 0表示20T。<br>• 2表示8T。 | 最大算力档位。 |
| euqal | 0-n。 | 匹配DTB时等效board_id使用，保持默认配置0即可。 |

如下所示为xml文件中弹性配置的SerDes信息，SerDes主要内容为serdes_info，具体参数含义请参见**表A-3**。

```
<subitem subname="sub_strategy0">
    <module class="serdes">
    <submodule subclass="serdes_info">
        <param lan_index="0" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0" port_index = "0"/>
        <param lan_index="1" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0" port_index = "0"/>
        <param lan_index="2" serdes_type="SATA" ssc_enable="1" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="SATA3.0" port_index = "0"/>
        <param lan_index="3" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN2" port_index = "0"/>
        <param lan_index="4" serdes_type="PCIE" ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="1"  align_mode="0" frequency="PCIE_GEN3" port_index = "0"/>
        <param lan_index="5" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
        <param lan_index="6" serdes_type="USB"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "0" bandwidth="ff" align_mode="0" frequency="USB3.0" port_index = "0"/>
        <param lan_index="7" serdes_type="ETH"  ssc_enable="0" polarity_tx = "0" polarity_rx = "0"
lan_order = "1" bandwidth="ff" align_mode="0" frequency="1GE" port_index = "1"/>
    </submodule>
</module>
```


版权所有 © 华为技术有限公司

**须知**

**弹性配置中的serdes复用关系参数必须与实际硬件单板保持一致，Serdes错误配置可能导致启动挂死，功能不可用。**

SerDes参数配置规格如下。

- PCIe控制器0（lane0~lane1）支持RC/EP模式，其他PCIe控制器只支持RC模式。
- Macro0（lane0~lane3）中若存在PCIe和其他协议（SATA）共存，则PCIe只能支持到PCIE_GEN2；若只存在PCIe协议，则PCIe支持到PCIE_GEN3。
- PCIe支持降lane应用，如PCIe x4降lane到PCIe x2/x1。
- Macro0（lane0~lane3）支持PCIE和SATA协议，Macro1（lane4~lane7）支持PCIe、USB、GE协议，每个Macro同时最多只能配置两种协议。
- SATA只能在Macro0（lane0~lane3）使用。
- USB控制器0（lane4）只支持HOST，USB控制器1~USB控制器3（lane5~lane7）支持HOST+DEVICE。
- USB和GE只能在Macro1（lane4~lane7）使用。
- 若为系统直出的USB 2.0，则无需配置SerDes。

**表 A-3** 参数说明

| param | 配置值 | 说明 |
| --- | --- | --- |
| lan_index | 0~7。 | lane编号，Atlas 200I A2 加速模块一共有8条serdes lane。 |
| serdes_type | Atlas 200I A2 加速模块支持PCIE、SATA、USB、ETH。 | 该条lane使用的协议。 |
| ssc_enable | • 0：否。<br>• 1：是。 | 是否开展频[a]，只有用作PCIe和SATA时才能打开。 |
| polarity_tx | • 0：否。<br>• 1：是。 | tx极性是否翻转。 |
| polarity_rx | • 0：否。<br>• 1：是。 | rx极性是否翻转。 |
| lan_order | • 0：否。<br>• 1：是。 | lane序是否反转。只有用作ETH时需要修改，具体请参见《**Atlas 200I A2 加速模块 硬件开发指南**》的“以太网接口”章节。 |


版权所有 © 华为技术有限公司

| param | 配置值 | 说明 |
| --- | --- | --- |
| bandwidth | • 1表示x1。<br>• 2表示x2。<br>• 4表示x4。<br>• ff表示不配置。 | 用作PCIe时，表示位宽x4/x2/x1，其他协议不使用。<br>• x4：使用四条lane。<br>• x2：使用两条lane。<br>• x1：使用一条lane。 |
| align_mode | 0。 | 该lane是master还是slave，Atlas 200I A2 加速模块不使用该参数。 |
| frequency | • PCIE_GEN2。<br>• PCIE_GEN3。<br>• 1GE。<br>• 2.5GE。<br>• USB3.0。<br>• SATA3.0。 | 使用的频点[b]，主要用来区分PCIE GEN2/3，和ETH的1GE/2.5GE。 |
| port_index | 0~n。 | 用作ETH时，表示lane的网口端口号，其他协议不使用。不使用时设置为0。 |
| a：时钟展频通过频率调制的手段将集中在窄频带范围内的能量分散到设定的宽频带范围，通过降低时钟在基频和奇次谐波频率的幅度（能量），达到降低系统电磁辐射峰值的目的。<br>b：PCIe的频点是指数据在总线上传输的速率，也称为数据传输速度。 | | |

```
        <module class="recovery">
            <submodule subclass="recovery_info">
                <param recovery_enable="0" gpio_index="1"/>
            </submodule>
        </module>
```

“recovery”参数为保留字段，暂未使用。

## 配置过流过温保护参数

**须知**

如果关闭过流过温功能，可能会导致设备损坏，请谨慎配置。

```
<!--LP低功耗配置，无特殊说明数字为10进制, feature/edp/tdp各节点均支持缺省，配置定义与sysbaeconfig相同-->
        <module class="lp">
            <!--特性开关-->
            <submodule subclass="feature">
                <!--功能开关: 16进制掩码定义, 每比特位0为关闭1为使能-->
                <!--   各项开关均为可选配置，默认不关闭，默认全1配置"-->
                <!--过流保护功能开关, bit0-过流硬件降频，bit1-过流告警, bit2-过流软件降频,以下为功能全使能
-->
```


版权所有 © 华为技术有限公司

```
            <param edp="0x7"/>
            <!--过温保护功能开关, bit0-超高温下电, bit1-高温告警, bit2-过温降频,以下为功能全使能 -->
            <param tdp="0x7"/>
          </submodule>
          <!--edp过流保护-->
          <submodule subclass="edp">
            <!--acg降频等级0(10%), 1(18%) 2(25%),其它不支持-->
            <param acg_drop="2"/>
            <!--mask(适用档位掩码, 16进制 ）-->
            <!--hardware(单位0.1A， ina226硬件过流阈值)-->
            <!--alarm （单位0.1A， 软件监测过流告警阈值 ）-->
            <!--ave （单位0.1A， 软件监测过流降频阈值)-->
            <!--示例：mask(适用0/1/2/3档) ina226阈值5A，软件告警3A, 降频5A-->
            <param mask="0xF" hardware="50" alarm="30" ave="50"/>
          </submodule>
          <!--tdp过温保护-->
          <submodule subclass="tdp">
            <!--ctrl_temp （单位0.1℃， 软件监测过温降频阈值)-->
            <!--ultra_high （单位0.1℃， tsensor超高温下电阈值，需大于60度（ 600 ），建议不超过105度)--
>
            <!--alarm_high （单位0.1℃， 软件监测高温告警阈值)-->
            <!--alarm_low （单位0.1℃， 软件监测低温告警阈值（ 保留 ））-->
            <!--示例：过温降频95, 告警92度， 超温下电105度, 低温告警-20度(保留 ）-->
            <param ctrl_temp="950" ultra_high="1050" alarm_high="920" alarm_low="-200"/>
          </submodule>
        </module>
```

**表 A-4 参数说明**

| param | 配置值 | 说明 |
| --- | --- | --- |
| edp | • 0x0：（ 000 ）关闭过流硬件降频、过流告警和过流软件降频功能。<br>• 0x1：（ 001 ）关闭过流告警和过流软件降频功能。<br>• 0x2：（ 010 ）关闭过流硬件降频和过流软件降频功能。<br>• 0x3：（ 011 ）关闭过流软件降频功能。<br>• 0x4：（ 100 ）关闭过流硬件降频和过流告警功能。<br>• 0x5：（ 101 ）关闭过流告警功能。<br>• 0x6：（ 110 ）关闭过流硬件降频功能。<br>• 0x7：（ 111 ）开启过流硬件降频、过流告警和过流软件降频功能。 | • 16进制掩码定义，每比特位0为关闭1为使能。<br>• 各项开关均为可选配置，默认不关闭，默认全1配置。<br>• 过流保护功能开关，bit0-过流硬件降频，bit1-过流告警，bit2-过流软件降频。 |


版权所有 © 华为技术有限公司

| param | 配置值 | 说明 |
| --- | --- | --- |
| tdp | • 0x0：（000）关闭超高温下电、高温告警和过温降频功能。<br>• 0x1：（001）关闭高温告警和过温降频功能。<br>• 0x2：（010）关闭超高温下电和过温降频功能。<br>• 0x3：（011）关闭过温降频功能。<br>• 0x4：（100）关闭超高温下电和高温告警功能。<br>• 0x5：（101）关闭高温告警功能。<br>• 0x6：（110）关闭超高温下电功能。<br>• 0x7：（111）开启超高温下电、高温告警和过温降频功能。 | • 16进制掩码定义，每比特位0为关闭1为使能。<br>• 各项开关均为可选配置，默认不关闭，默认全1配置。<br>• 过温保护功能开关，bit0-超高温下电，bit1-高温告警，bit2-过温降频。 |
| acg_drop | • 0：降频10%。<br>• 1：降频18%。<br>• 2：降频25%。 | 降频等级。 |
| mask | 0xF。 | 适用档位掩码，16进制。<br>当前不生效，保持0xF不变就行。 |
| hardware | 单位0.1A。 | ina226硬件过流阈值。 |
| alarm | 单位0.1A。 | 软件监测过流告警阈值。 |
| ave | 单位0.1A。 | 软件监测过流降频阈值。 |
| ctrl_temp | 单位0.1℃。 | 软件监测过温降频阈值。 |
| ultra_high | 600~1050，单位0.1℃。 | tsensor超高温下电阈值，需大于60度（600），建议不超过105度。 |
| alarm_high | 单位0.1℃。 | 软件监测高温告警阈值。 |
| alarm_low | 单位0.1℃。 | 软件监测低温告警阈值。 |


版权所有 © 华为技术有限公司

# A.3 参数说明/常用命令

## 参数说明

软件包支持根据命令行完成一键式安装，各个命令之间可以配合使用，用户根据安装需要选择对应参数完成安装，所有参数都是可选参数。

安装命令格式： **./***xx*.run [options]

详细参数请参见**表A-5**。

**须知**

软件包中的**xx**请根据实际情况进行替换。

如果通过**./***xx*.run **--help**命令查询出的参数未解释在如下表格，则说明该参数预留或适用于其他芯片版本，用户无需关注。

**表 A-5** 安装包支持的参数说明

| 参数 | 说明 |
| --- | --- |
| --run | 运行态类型：只安装运行场景需安装的文件。 |
| --full | 完整安装类型：安装所有文件。 |
| --uninstall | 卸载。仅适用于Firmware软件包。 |
| --nox11 | 安装过程中不弹出图形终端窗口。<br>使用示例：**./***xx*.run **--full --nox11**<br>**说明**<br>该参数预留，暂不使用。 |
| --noexec | 不执行安装脚本。配套--extract=*path*使用。格式为**--noexec --extract=***path* |
| --extract=*<path>* | 解压缩安装包中文件到指定目录。使用该参数时，用户无需手动建立指定的目录，解压过程中会自动创建，例如：<br>--noexec --extract=./package，则命令执行后会自动将解压后的内容放在package目录。 |
| --upgrade | 升级。<br>• 首次安装场景，系统默认安装到“/usr/local/Ascend”目录。<br>• 非首次安装场景，仅支持从软件包所在路径升级。 |
| --help或者-h | 查看帮助。 |
| --check | 校验软件包的完整性、检查与已经安装的其他软件包之间的配套关系是否正确。 |


版权所有 © 华为技术有限公司

| 参数 | 说明 |
| --- | --- |
| --version | 查询版本号。 |
| --tar arg1 [arg2 …] | 对安装包执行tar命令，使用tar后面的参数作为命令的参数。例如执行**--tar xvf**命令，解压软件包的内容到当前目录。 |
| --list | 导出安装包文件列表。 |
| --info | 显示数据包详细信息。 |
| --quiet | 静默安装，跳过交互式信息。<br>使用该参数进行安装时，若安装目录权限大于755，则会提示安全风险，用户自行选择终止安装或选择其他权限为755的安装目录重新安装。<br>使用示例：**./***xx*.run **--full --quiet**<br>**说明**<br>Firmware软件包在使用该参数时，不能与--uninstall配合使用。 |
| --pre-install-path=<path> | SD卡预安装<br>● 仅支持手动输入绝对路径，<path>参数必须包含“ascend_pkg_install”。<br>● 该命令可以将run包内文件解压复制到输入的路径下，在制卡脚本“make_os_sd.sh”中调用，用于制卡时预装驱动包内文件，不用于安装和升级。<br>● 用户可以使用该命令将驱动包文件拷贝到输入的临时目录下，方便进行文件的复制删除等操作。 |
| --mnt-install-path=<path> | 安装挂载驱动包到其他分区（备区）。 |

# A.4 相关工具

**须知**

upgrade-tool工具不支持容器场景，部署容器时，禁止将upgrade-tool工具映射到容器内。

**步骤1** 登录Host侧的服务器。

**步骤2** 切换到软件包所在安装路径，根据以下表格中的命令调用工具，如下以Driver采用默认安装路径为例进行说明。


版权所有 © 华为技术有限公司

**表 A-6** 相关工具

| 功能描述 | 执行命令 | 参数 |
| --- | --- | --- |
| upgrade-tool工具可以查看固件版本、升级单个或多个固件等，只允许root用户执行。<br>请切换到“/var/davinci/driver”路径执行命令。 | 查看帮助信息：<br>**./upgrade-tool --help** | --help：查看帮助信息。 |
| | 列举所有Device：<br>**./upgrade-tool --mini_devices** | --mini_devices：所有设备列表。 |
| | 启动固件主备同步：<br>**./upgrade-tool --sync** | --sync：启动固件主备同步。 |
| | 获取指定设备的版本：<br>**./upgrade-tool --device_index <dev_id> --system_version** | • --device_index：设备编号。取值只能是0和-1，0表示对应编号的设备。-1表示所有设备。<br>• --system_version：系统版本。 |
| | 获取指定设备的组件信息：<br>**./upgrade-tool --device_index <dev_id> --components** | --components：列举所有有效的组件。 |
| | 查询设备状态：<br>**./upgrade-tool --device_index <dev_id> --status** | --status：设备状态。支持的设备状态具体包含：<br>• idle：空闲。<br>• upgrading：正在升级。<br>• not support：不支持。<br>• failed：失败。<br>• waitting_restart：等待重启。<br>• waitting_sync：等待固件同步。<br>• synchronizing：正在同步。<br>• wrong status：错误状态。 |
| | 查询某个设备中某一个组件对应的版本：<br>**./upgrade-tool --device_index <dev_id> --component <type> --version** | • --component：指定具体的组件。查询和升级单个组件时，需要指定组件名称（hboot1a、hboot1b、hboot2、ddr、hsm、hlink、Sys_Base_Config、atf、Usr_Base_Config）。查询和升级所有组件时，需要输入-1。<br>• --version：组件的版本。 |


版权所有 © 华为技术有限公司

| 功能描述 | 执行命令 | 参数 |
| --- | --- | --- |
| | 异步升级指定设备的固件包（暂不支持）：<br>**./upgrade-tool --device_index <dev_id> --component <type> --async --path <firmware_path>**<br>**--async**必须紧跟在**component <type>**后面 | --async：支持异步升级，即device侧收到host侧发送的升级请求后，返回请求成功响应到host侧（具体固件是否升级成功，需要通过“--status”参数去查询）。暂不支持。 |
| | 查看所有分区的固件版本：<br>**./upgrade-tool --device_index <dev_id> --component <type> --all --version** | --all --version：查看所有分区（Flash分区、主备分区）的固件版本。 |
| | 查询算力功率等级（暂不支持）：<br>**./upgrade-tool --device_index <dev_id> --component <type> --level** | --level：查询算力功率等级，暂不支持。 |
| | • 指定组件的flash区域进行升级：**./upgrade-tool --device_index <dev_id> --component <type> --path <firmware_path>**<br>• 指定组件的主备区域进行升级：**./upgrade-tool --device_index <dev_id> --component <type> --media --mmc --<main/backup> --path <firmware_path>** | • --path：指定组件对应的路径。若升级单个组件，例如AS310B_HBOOT1_a.bin，以Ascend-hdk-310b-npu-firmware-soc_*x.x.x.x.X*.run包解压到“/opt/tmp”目录下为例：则为**--path** */opt/tmp/***firmware/image/AS310B_HBOOT1_a.bin**。<br>• --media --mmc：指定组件的主备区域进行升级。flash和主备区域组件介绍详细请参见**A.5 组件分区介绍**。<br>需确保所有组件升级后版本一致。 |
| | 指定组件的recover区主备区域进行升级：<br>**./upgrade-tool --device_index <dev_id> --component <type> --media --mmc --recover --<main/backup> --path <firmware_path>** | --recover：指定组件的recover区主备区域进行升级，用于系统恢复。 |


版权所有 © 华为技术有限公司

| 功能描述 | 执行命令 | 参数 |
| --- | --- | --- |
| | 查询设备是否是物理机：<br>**./upgrade-tool --device_index <dev_id> --phymachflag** | --phymachflag：查询设备是否是物理机。如果不是物理机，不允许升级设备的固件包。 |

**----结束**

# A.5 组件分区介绍

Atlas 200I A2 加速模块启动时涉及hboot1a、hboot1b、hboot2、ddr、hsm、hlink、Sys_Base_Config、atf、Usr_Base_Config、kernel、DTB、tee组件，使用**/var/davinci/driver/upgrade-tool --device_index 0 --component -1 --all --version**命令查询所有组件，回显信息如下：

```
flash components:
Get component version(x.x.x.x.x) succeed for deviceId(0), componentType(11).
        {"device_id":0, "component":hboot1a, "version":x.x.x.x.x}
Get component version(x.x.x.x.x) succeed for deviceId(0), componentType(12).
        {"device_id":0, "component":hboot1b, "version":x.x.x.x.x}
Get component version(x.x.x.x.x) succeed for deviceId(0), componentType(13).
        {"device_id":0, "component":hboot2, "version":x.x.x.x.x}
Get component version(x.x.x.x.x) succeed for deviceId(0), componentType(14).
        {"device_id":0, "component":ddr, "version":x.x.x.x.x}
Get component version(x.x.x.x.x) succeed for deviceId(0), componentType(16).
        {"device_id":0, "component":hsm, "version":x.x.x.x.x}
Get component version(x.x.x.x.x) succeed for deviceId(0), componentType(18).
        {"device_id":0, "component":hlink, "version":x.x.x.x.x}
Get component version(x.x.x.x.x) succeed for deviceId(0), componentType(25).
        {"device_id":0, "component":Sys_Base_Config, "version":x.x.x.x.x}
Get component version(x.x.x.x.x) succeed for deviceId(0), componentType(30).
        {"device_id":0, "component":atf, "version":x.x.x.x.x}
Get component version(x.x.x.x.x) succeed for deviceId(0), componentType(31).
        {"device_id":0, "component":Usr_Base_Config, "version":x.x.x.x.x}

(current boot partition: main)
main partition:
        {"device_id":0, "component":kernel, "version":x.x.x.x.x}
        {"device_id":0, "component":dtb, "version":x.x.x.x.x}
        {"device_id":0, "component":tee, "version":x.x.x.x.x}
backup partition:
        {"device_id":0, "component":kernel, "version":x.x.x.x.x}
        {"device_id":0, "component":dtb, "version":x.x.x.x.x}
        {"device_id":0, "component":tee, "version":x.x.x.x.x}
```

组件存放在flash和SD/EMMC/M.2分区，其中SD/EMMC/M.2包括主备分区。

- SD/EMMC/M.2中的kernel、DTB、tee组件分为主备分区，正常启动时使用主区，启动成功后会同步这些组件到备区。当主区启动异常时，BIOS在5分钟后自动重启，重启4次后从备区启动，启动成功后会自动同步备区组件到主区。因此调试DTB和kernel组件时，建议只升级主区。
- 根据组件类型选择不同命令升级。
  - 升级hboot1a、hboot1b、hboot2、ddr、hsm、hlink、Sys_Base_Config、atf、Usr_Base_Config组件

    命令：**/var/davinci/driver/upgrade-tool --device_index** *<dev_id>* **--component** *<type>* **--path** *<firmware_path>*


版权所有 © 华为技术有限公司

示例：**/var/davinci/driver/upgrade-tool --device_index 0 --component hboot1a --path ./AS310B_HBOOT1_a.bin**

- 升级kernel、DTB、tee组件

  命令：**/var/davinci/driver/upgrade-tool --device_index** *<dev_id>* **--component** *<type>* **--media --mmc --***<main/backup>* **--path** <firmware_path>

  示例：**/var/davinci/driver/upgrade-tool --device_index 0 --component kernel --media --mmc --main --path ./Image**

  组件与文件名及驱动和固件包中的相对路径对应关系如**表A-7**所示。

**表 A-7** 组件与文件名、路径对应关系表

| 组件类型 | 文件名全称 | 驱动包相对路径 | 固件包相对路径 |
| --- | --- | --- | --- |
| hboot1a | AS310B_HBOOT1_a.bin | ./firmware/ | ./firmware/image/ |
| hboot1b | AS310B_HBOOT1_b.bin | | |
| hboot2 | AS310B_HBOOT2_UEFI.fd | | |
| ddr | lpddr_mcu.bin | | |
| hsm | microwatt.bin | | |
| hlink | hilink25_fw.bin | | |
| Sys_Base_Config | sysBaseConfig.bin | | |
| atf | TrustedFirmware.bin | | |
| Usr_Base_Config | userBaseConfig.bin | | |
| kernel | Image | | - |
| DTB | dt.img | | |
| tee | itrustee.img | | |

# A.6 定制文件系统

## A.6.1 定制 OpenEuler 22.03 文件系统

由于openeuler22.03公共镜像源的最小文件系统initrd没有自带dnf下载命令，无法通过挂载本地源方式进行文件系统配置，需要制作带dnf下载命令的小文件系统，定制Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img或Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img。


版权所有 © 华为技术有限公司

## 前提条件

- 制作环境为Arm架构的openEuler 22.03操作系统的服务器。
- 从官方网站获取openEuler 22.03 OS镜像“openEuler-22.03-LTS-aarch64-dvd.iso”（若有软实时OS需求，请获取“openEuler-22.03-LTS-SP1-aarch64-dvd.iso”）。

  **说明**

  - 若用户无Arm架构的openEuler 22.03操作系统的服务器，可访问**Atlas 200I A2 加速模块软件包**，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img或Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img。
  - 社区版软件不需要申请下载权限可以直接下载，但软件不能用于商业用途， 详细请参见**社区华为软件许可协议**；如果您需要用于商业用途，参见本章节的操作步骤进行制作。因此如果您下载软件仅用于调试验证，推荐下载社区版软件。

## 操作步骤

**步骤1** 登录Linux服务器。

**步骤2** 执行如下命令，切换至root用户。

**su - root**

**步骤3** 挂载iso镜像。

1. 执行如下命令，创建临时目录。例如创建目录为“/opt/tmp/iso_mount”。

   **mkdir -p** */opt/tmp/iso_mount*
2. 执行如下命令，挂载iso镜像。

   **mount -o loop** *iso文件路径 /opt/tmp/iso_mount*
3. 执行如下命令，修改镜像源配置。

   **mv /etc/yum.repos.d/openEuler.repo /etc/yum.repos.d/openEuler.repo.bak**

   **vim /etc/yum.repos.d/openEuler.repo**
4. 写入本地镜像源配置。

   ```
   [openEuler-local]
   name=openEuler
   baseurl=file:///opt/tmp/iso_mount
   enable=1
   gpgcheck=0
   ```
5. 按“Esc”键，再执行如下命令，保存修改并按“Enter”键退出。

   **:wq!**

**步骤4** 拷贝initrd.img。

执行如下命令，将initrd.img从挂载路径拷贝至任意目录下，例如“/opt/tmp”。

**cp** */opt/tmp/iso_mount/images/pxeboot/initrd.img /opt/tmp/*

**步骤5** 执行如下命令，进入“tmp”目录。

**cd** */opt/tmp*

**步骤6** 解压initrd.img。

1. 执行如下命令，重命名initrd.img。


版权所有 © 华为技术有限公司

**mv initrd.img initrd.img.xz**

**说明**

initrd.img是xz压缩格式，可通过执行**file initrd.img**命令查看，所以解压前需要先重命名为.xz结尾的文件。

2. 执行如下命令，解压initrd.img.xz。解压后initrd.img名称不变。

   **xz -d initrd.img.xz**

3. 执行如下命令，新创建“initrd”目录，例如“/opt/tmp/initrd”。

   **mkdir -p** */opt/tmp/initrd*

4. 执行如下命令，进入创建的“initrd”目录。

   **cd** */opt/tmp/initrd*

5. 执行如下命令，解压initrd.img。

   **cpio -idmv <** */opt/tmp/initrd.img*

6. 删除当前目录下ppc文件夹。

   **rm -rf usr/lib/kbd/keymaps/legacy/ppc**

**步骤7** 安装dnf。

执行如下命令，将dnf安装至initrd.img的解压目录下，例如“/opt/tmp/initrd”。

**dnf install --installroot=***/opt/tmp/initrd* **dnf -y**

打印信息中显示如下：表示dnf安装成功。

```
Complete!
```

**步骤8** 重新制作initrd.img。

1. 执行如下命令，创建新镜像的存放目录，例如“/opt/tmp/new_initrd”。

   **mkdir -p** */opt/tmp/new_initrd*

2. 执行如下命令，压缩生成新的initrd.img，并将其命名为Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img（若有软实时OS需求，则命名为Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img）。

   **cd** */opt/tmp/initrd*

   **find . | cpio -o | gzip >** */opt/tmp/new_initrd/Sample-root-filesystem-soc_openEuler-22.03-LTS-aarch64.img*

   若有软实时OS需求，该命令为**find . | cpio -o | gzip >** */opt/tmp/new_initrd/Sample-root-filesystem-soc_openEuler-22.03-LTS-SP1-aarch64.img*。

**----结束**

# A.6.2 定制 Ubuntu 22.04 文件系统

由于当前Ubuntu 22.04的公共镜像源没有自带常用工具包，无法通过挂载本地源方式进行文件系统配置，所以需要通过配置网络源手动下载常用Linux组件，定制Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img。

## 前提条件

- 制作环境为Arm架构的Ubuntu操作系统的服务器。
- 已获取Ubuntu 22.04镜像“ubuntu-22.04-live-server-arm64.iso”，并上传到部署环境的任意目录。


版权所有 © 华为技术有限公司

**说明**

- 镜像获取方式：**https://old-releases.ubuntu.com/releases/22.04.1/ubuntu-22.04-live-server-arm64.iso**
- 若用户无Arm架构的Ubuntu操作系统的服务器，可访问**Atlas 200I A2 加速模块软件包**，下载并解压Ascend-hdk-310b-npu-sample-root-filesystem-soc_23.0.rc1_linux-aarch64.zip，直接获取Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img。
- 社区版软件不需要申请下载权限可以直接下载，但软件不能用于商业用途， 详细请参见**社区华为软件许可协议**；如果您需要用于商业用途，参见本章节的操作步骤进行制作。因此如果您下载软件仅用于调试验证，推荐下载社区版软件。

## 操作步骤

**步骤1** 登录制作Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img的服务器，详细操作请参见**14.1 使用PuTTY登录设备（网口方式）**。

**步骤2** 获取最小文件系统。

1. 在Ubuntu 22.04镜像所在目录下执行以下命令，挂载镜像到“/mnt”目录。
   **mount ubuntu-22.04-live-server-arm64.iso /mnt**
2. 执行以下命令复制ubuntu-server-minimal.squashfs到当前目录。
   **cp /mnt/casper/ubuntu-server-minimal.squashfs ./**
3. 执行以下命令，解挂载“/mnt”目录。
   **umount /mnt**

**步骤3** 执行以下命令，解压镜像，生成“squashfs-root”目录。

**unsquashfs ubuntu-server-minimal.squashfs**

**步骤4** 配置Ubuntu的网络源。

执行以下命令，在当前目录创建sources.list文件。

**vim sources.list**

网络源配置参考如下。

```
deb https://repo.huaweicloud.com/ubuntu-ports/ bionic main restricted universe multiverse
deb-src https://repo.huaweicloud.com/ubuntu-ports/ bionic main restricted universe multiverse
deb https://repo.huaweicloud.com/ubuntu-ports/ bionic-security main restricted universe multiverse
deb-src https://repo.huaweicloud.com/ubuntu-ports/ bionic-security main restricted universe multiverse
deb https://repo.huaweicloud.com/ubuntu-ports/ bionic-updates main restricted universe multiverse
deb-src https://repo.huaweicloud.com/ubuntu-ports/ bionic-updates main restricted universe multiverse
deb https://repo.huaweicloud.com/ubuntu-ports/ bionic-backports main restricted universe multiverse
deb-src https://repo.huaweicloud.com/ubuntu-ports/ bionic-backports main restricted universe multiverse
## Not recommended
# deb https://repo.huaweicloud.com/ubuntu-ports/ bionic-proposed main restricted universe multiverse
# deb-src https://repo.huaweicloud.com/ubuntu-ports/ bionic-proposed main restricted universe multiverse
```

保存配置文件并退出。

执行以下命令，将sources.list文件复制到“squashfs-root/etc/apt”目录下。

**cp sources.list squashfs-root/etc/apt/**

**步骤5** 切换根系统。

执行如下命令，切换根系统。

**chroot squashfs-root**


版权所有 © 华为技术有限公司

**步骤6** 执行以下命令，更新源。

```
apt-get update
```

**说明**

若执行命令后报错“The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 3B4FE6ACC0B21F32”，请参见**15.4 更新网络源时签名不可用**。

**步骤7** 执行以下命令，安装Linux组件。

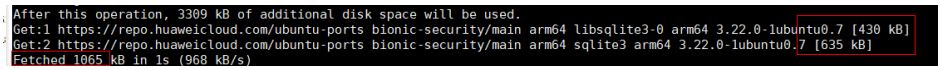
```
apt-get install unzip -y
apt-get install vim -y
apt-get install sysstat -y
apt-get install libnuma1 -y
apt-get install dmidecode -y
apt-get install rsync -y
apt-get install net-tools -y
apt-get install psmisc -y
apt-get install parted -y
apt-get install arping -y
apt-get install ntpdate -y
apt-get install iproute2 -y
apt-get install iputils-ping -y
apt-get install mawk -y
apt-get install cracklib-runtime -y
apt-get install ethtool -y
apt-get install ntp -y
apt-get install fdisk -y
apt-get install libssl-dev -y
apt-get install libpam-cracklib -y
apt-get install logrotate -y
```


版权所有 © 华为技术有限公司

**说明**

- 执行apt-get install命令后，回显中出现的需要获取的包与取得的包大小一致，则下载成功，如图A-1所示。

  **图 A-1** 完成下载

- 执行apt-get install命令后，若出现“perl：warning：*xxx*”相关报错、“debconf：*xxx*”相关报错、“Failed to retrieve available kernel versions.”以及“Failed to check for processor microcode upgrades.”报错，不影响Linux组件下载。

**步骤8** 执行以下命令退出根系统。

**exit**

**步骤9** 执行以下命令进入“squashfs-root”目录。

**cd squashfs-root**

**步骤10** 执行以下命令，重新生成Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img。

**find . | cpio -o -H newc | gzip > ../Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img**

**说明**

Sample-root-filesystem-soc_ubuntu-22.04-aarch64.img在Ubuntu 22.04镜像所在目录下。

**----结束**

# A.7 术语&缩略语

## B

| | | |
|---|---|---|
| **BSBC** | BootROM Secure Boot Code | 安全启动代码，是CPU内部的BOOTROM，是安全可信启动整个信任链的信任根或信任起点； BSBC是CPU芯片厂商事先固化到CPU内部的ROM上，不能修改。 |

## C

| | | |
|---|---|---|
| **CMS** | Cryptographic Message Syntax | 加密消息语法，是IETF加密保护消息的标准。它可用于对任何形式的数字数据进行数字签名、摘要、身份验证或加密。 |
| **CAN** | Controller Area Network | 控制器局域网。 |


版权所有 © 华为技术有限公司

## D

| | | |
|---|---|---|
| **DTB** | Device Tree Bolb | 设备树二进制文件，是由dts/dtsi被编译后生成的二进制文件，可以被内核解析，会被烧录到系统中，由UBOOT指定的地址供内核去读取。 |
| **DTS** | Device Tree Source | 设备树源码，Device Tree是一种描述硬件的数据结构。 |
| **DNF** | Dandified YUM | 新一代软件包管理器，用于安装、更新、卸载Linux系统的软件。 |

## E

| | | |
|---|---|---|
| **eMMC** | Embedded Multimedia Card | 嵌入式多媒体卡。 |
| **eFuse** | Electronic Fuse | 电子熔断器。是一种可编程电子保险丝，用于存储信息和保护芯片的非易失性存储器件。 |

## G

| | | |
|---|---|---|
| **GPIO** | General Purpose Input/Output | 通用输入输出接口。 |
| **GMAC** | Gigabit Ethernet Media Access Controller | 千兆网媒体访问控制器。 |

## H

| | | |
|---|---|---|
| **HBOOT2** | High-Level Bootloader2 | HBOOT2是一种引导程序，它负责加载操作系统和其他必要的组件，以确保设备能够正常启动。 |
| **HDMI** | High Definition Multimedia Interface | 高清晰多媒体接口。 |

## I

| | | |
|---|---|---|
| **$I^2C$** | Inter-integrated Circuit | 内部整合电路。 |


版权所有 © 华为技术有限公司

## M

| | | |
|---|---|---|
| **MDIO** | Management Data Input/ Output | 管理数据输入输出。 |
| **MIPI-CSI** | Mobile Industry Processor Interface-Camera Serial Interface | MIPI-CSI是一种用于移动设备和嵌入式系统的串行接口标准，用于连接图像传感器和图像处理器之间的接口。 |
| **MIPI-DSI** | Mobile Industry Processor Interface-<br>display serial interface | 显示器串行接口。 |
| **MAC** | Medium Access Control | 媒体接入控制。 |

## N

| | | |
|---|---|---|
| **NPU** | Neural-Network Processing Unit | 神经网络处理器单元。 |

## O

| | | |
|---|---|---|
| **OS** | Operating System | 操作系统。 |

## P

| | | |
|---|---|---|
| **PWM** | Pulse Width Modulation | 脉冲宽度调制。 |
| **PMBus** | power management bus | 电源管理总线。 |
| **PCIe** | Peripheral Component Interconnect Express | 快捷外围部件互连标准。 |


版权所有 © 华为技术有限公司

## R

| | | |
|---|---|---|
| **RSA** | Ron Rivest, Adi Shamir, Leonard Adleman | 非对称密钥算法。 |

## S

| | | |
|---|---|---|
| **SD卡** | Secure Digital Memory Card | 安全数码卡。 |
| **SerDes** | serializing/ deserializing circuitry | 串行/并行电路。 |
| **SDIO** | secure digital input/output | 安全数字输入输出接口。 |
| **SPI** | Serial Peripheral Interface | 串行外设接口。 |
| **SFC** | SPI flash controller | SPI flash控制器。 |
| **SATA** | Serial Advanced Technology Attachment | 串行高级技术附件。 |

## T

| | | |
|---|---|---|
| **TOPS** | trillion operations per second | 每秒万亿次的运算。 |

## U

| | | |
|---|---|---|
| **USB** | Universal Serial Bus | 通用串行总线。 |
| **UART** | Universal Asynchronous Receiver/ transmitter | 通用异步收发传输器。 |


版权所有 © 华为技术有限公司